Verifikacija softvera - Vežbe

Ivan Ristović

2026

Kolekcija materijala sa vežbi za kurs Verifikacija softvera na Matematičkom fakultetu, Univerziteta u Beogradu.

Materijali prate praktičan tok rada: od dijagnoziranja simptoma i debagovanja, preko testiranja i profajliranja, do simboličkog izvršavanja, proveravanja modela i formalne verifikacije. Svaka tema je organizovana kroz kratko objašnjenje, komande, očekivane nalaze i male primere koji se nalaze uz odgovarajuće poglavlje.

Pokrivene su sledeće celine:

- **Dijagnoziranje problema**: osnovni Linux alati, posmatranje procesa i analiza ponašanja programa metodom crne kutije.
- **Debagovanje**: `gdb`, QtCreator, core dump analiza, watchpoint-i, niti, prikačivanje na proces, Java JDWP, agenti i native/AOT izvršive verzije.
- **Refaktorisanje**: prepoznavanje lošeg dizajna, izdvajanje odgovornosti i primeri kroz C++, Java i C# kod.
- **Testiranje jedinica koda**: QtTest, JUnit, xUnit, NUnit, pokrivenost koda, `lcov`, JaCoCo i .NET alati za pokrivenost.
- **Testiranje pomoću objekata imitatora**: ručni imitatori, Mockito, Moq i testiranje koda koji zavisi od servisa, vremena ili baze podataka.
- **Fuzz testiranje**: libFuzzer, tmux tokovi rada, Jazzer za Java kod i AFL++ na realnijem C/C++ primeru.
- **Profajliranje instrumentacijom**: Valgrind alati kao što su Memcheck, Massif, DHAT, Cachegrind, Callgrind, Helgrind i DRD, kao i `gprof` i LLVM XRay.
- **Profajliranje uzorkovanjem**: `perf`, flame graph tok rada, Intel VTune Profiler i Java async-profiler.
- **eBPF**: model izvršavanja, verifier, mape, `bpftrace`, BCC alati, sistemsko profajliranje i ograničenja dozvola.
- **KLEE**: simboličko izvršavanje, generisanje testova, pronalaženje grešaka, rad sa bibliotekama i heuristike pretrage.
- **CBMC**: proveravanje svojstava C programa, odmotavanje petlji, pretpostavke, postuslovi, harness funkcije i konkurentni primeri.
- **Clang alati**: Clang statički analizator, `clang-tidy`, sanitizeri, pojedinačni primeri grešaka i analiza projekata.
- **Dafny**: specifikacije, pre/postuslovi, invarijante petlji i dokazi korektnosti za male algoritme.
- **Instalacije**: kratke beleške za alate i pakete koji su potrebni za lokalno pokretanje vežbi.

Contents

1 Dijagnoziranje problema **7**

1.1 Alati za analizu performansi . . . 7

1.1.1 Kako birati sledeći alat . . . 7

1.1.2 Obrazac laboratorijske vežbe . . . 8

1.1.3 `uptime` . . . 9

1.1.4 `top` / `htop` . . . 10

1.1.5 `ps` . . . 10

1.1.6 `vmstat` . . . 10

1.1.7 `iostat` . . . 11

1.1.8 `swapon` . . . 11

1.1.9 `mpstat` . . . 12

1.1.10 `lsof` . . . 12

1.1.11 `pidstat` . . . 12

1.1.12 `strace` . . . 13

1.1.13 Analiza mreže . . . 15

1.1.14 System Activity Reporter (`sar`) . . . 17

1.2 Laboratorijski primeri . . . 17

1.2.1 Pregled primera . . . 18

1.2.2 `lab001`: CPU petlja . . . 18

1.2.3 `lab002`: čekanje na konekciju . . . 19

1.2.4 `lab003`: čitanje nula bajtova . . . 21

1.2.5 `lab004`: kratkotrajni child procesi . . . 23

1.2.6 `lab005`: sinhroni upisi na disk . . . 24

1.2.7 `lab006`: više CPU vrućih funkcija . . . 25

1.2.8 `lab007`: proces uglavnom spava . . . 26

1.2.9 `lab008`: disk latencije sa izuzecima . . . 27

1.2.10 `lab013`: rekurzija i stek . . . 29

2 Debagovanje **31**

2.1 Okruženje izvršavanja i strategija prevođenja . . . 31

2.2 Kada koristiti debager . . . 32

2.3 gdb . . . 32

2.3.1 Buffer Overflow primer (gdb) . . . 32

2.4 QtCreator debager . . . 36

2.4.1 Debagovanje spoljašnje aplikacije kroz QtCreator . . . 36

2.4.2 Buffer Overflow primer (QtCreator) . . . 37

2.4.3 Raspored struktura i padding . . . 37

2.4.4 Memorijski mapovan uređaj i optimizacije . . . 40

2.4.5 Postmortem analiza iz core dump-a . . . 42

2.4.6 Watchpoint za pronalaženje korupcije memorije . . . 44

2.4.7 Zaglavljene niti i deadlock . . . 45

2.4.8 Prikačivanje na pokrenut proces . . . 47

2.5 Debagovanje Java aplikacija . . . 48

2.5.1 Primer: Sudoku aplikacija . . . 48
2.6 Instrumentovanje Java bajtkoda - Java agenti . . . 50
2.6.1 Primer . . . 51

3 Pisanje testabilnog koda **52**
3.1 C++ primer - kalkulator . . . 52
3.2 Java OOP primer - Game of Life . . . 52
3.3 C# - REST API klijent primer . . . 53

4 Testiranje jedinica koda metodom bele kutije **55**
4.1 C++ - QtTest, gcov, lcov . . . 55
4.1.1 Pisanje testova jedinica koda pomoću QtTest radnog okvira . . . 55
4.1.2 Pisanje testova . . . 56
4.1.3 Analiza pokrivenosti . . . 56
4.1.4 Kreiranje izveštaja uz pomoć skripte . . . 57
4.1.5 Integracija pokrivenosti u QtCreator . . . 58
4.2 Java - JUnit, JaCoCo . . . 58
4.3 C# (.NET) - xUnit, NUnit . . . 62
4.3.1 xUnit . . . 62
4.3.2 NUnit . . . 64
4.3.3 `dotnet-coverage` . . . 64

5 Testiranje pomoću objekata imitatora (engl. Mock testing) **69**
5.1 C++ - ručno kreiranje objekata imitatora . . . 70
5.2 C# - Moq . . . 70
5.3 C# - ”imitatori” baza podataka . . . 72

6 ”Fuzz” testiranje **77**
6.1 Korpus . . . 78
6.2 Primer rada libFuzzer-a . . . 79
6.3 tmux fuzz testiranje . . . 81
6.4 Jazzer (JVM fuzzer) . . . 82
6.5 Korišćenje AFL++ fuzzer-a nad libxml2 . . . 84

7 Profajliranje instrumentacijom **87**
7.1 Izbor instrumentacionog alata . . . 87
7.2 Valgrind . . . 88
7.2.1 Struktura i upotreba Valgrind alata . . . 88
7.2.2 Memcheck . . . 89
7.2.3 Massif . . . 103
7.2.4 DHAT . . . 104
7.2.5 Cachegrind . . . 106
7.2.6 Callgrind . . . 113
7.2.7 Helgrind / DRD . . . 116
7.2.8 gprof . . . 125
7.2.9 LLVM XRay . . . 128

8 Profajliranje uzorkovanjem **131**
8.1 Kada koristiti uzorkovanje . . . 131
8.2 Perf . . . 131
8.2.1 Događaji . . . 133
8.2.2 Brojanje događaja . . . 135
8.2.3 Uzorkovanje . . . 136
8.3 Intel VTune Profiler . . . 137
8.3.1 Česte analize . . . 138
8.3.2 Komandni tok rada . . . 138
8.3.3 Kako čitati rezultat . . . 139
8.3.4 Ograničenja i zamke . . . 139
8.3.5 Odnos prema alatu `perf` . . . 139
8.4 Java async-profiler . . . 139

9 eBPF **143**
9.1 Model izvršavanja . . . 143
9.2 bpftrace . . . 143
9.3 BCC . . . 144
9.4 Profajliranje pomoću eBPF-a . . . 145
9.5 Dozvole i zamke . . . 145

10 Simboličko izvršavanje (alat KLEE) **146**
10.1 Primer upotrebe KLEE alata . . . 146
10.2 Primer semantičke greške . . . 150
10.3 Primer greške u radu sa pokazivačima . . . 151
10.4 Korišćenje C bibliotečkih funkcija i simboličkog okruženja . . . 152
10.4.1 Primer sa simboličkim argumentom komandne linije . . . 152
10.5 KLEE heuristike . . . 153
10.5.1 Korišćenje različitih heuristika za pretragu . . . 154
10.5.2 Testiranje heuristika . . . 155
10.5.3 Kreiranje nove heuristike za KLEE . . . 156

11 Ograničena provera modela (alat CBMC) **157**
11.1 Primer sa mikrotalasnom rernom . . . 157
11.2 CBMC . . . 159
11.2.1 Primer sa argumentima komandne linije . . . 160
11.2.2 Primer odvojenog testiranja funkcija . . . 163
11.2.3 Primer neuspelog odmotavanja petlji . . . 164
11.2.4 Primer sa neograničenim petljama . . . 165
11.2.5 Odmotavanje zasnovano na broju iteracija . . . 166
11.2.6 Zadavanje ograničenja odmotavanja za pojedinačne petlje . . . 168
11.2.7 Odsecanje petlji . . . 170
11.2.8 Odmotavanje zasnovano na dubini (Depth-based Unwinding) . . . 171
11.2.9 Dekompozicija koraka analize . . . 171
11.2.10 Uprošćavanje verifikacione formule . . . 172
11.2.11 Binarna reprezentacija grafa kontrole toka . . . 172
11.2.12 Modelovanje problema . . . 173

11.2.13 Višenitni programi . . . 174

12 Clang alati za analizu C/C++ programa **176**
12.1 Kada koristiti koji Clang alat . . . 176
12.2 Clang statički analizator . . . 176
12.2.1 `clang-tidy` . . . 179
12.2.2 Primeri . . . 180
12.3 Sanitizeri u Clang-u . . . 180

13 Jezici za programiranje i formalnu verifikaciju (Dafny) **186**
13.1 Pisanje Dafny programa . . . 186
13.1.1 Polja . . . 186
13.1.2 Metode . . . 186
13.1.3 Funkcije . . . 187
13.1.4 Induktivni tipovi . . . 188
13.1.5 Naredbe . . . 188
13.1.6 Izrazi . . . 189
13.2 Primer: Faktorijel . . . 189
13.3 Primer: Maksimum sekvence . . . 190

14 Instalacije **192**
14.1 Alati za debagovanje i razvojna okruženja . . . 192
14.1.1 gdb . . . 192
14.1.2 QtCreator . . . 192
14.1.3 Java agenti . . . 192
14.2 Alati/Biblioteke za testiranje jedinica koda i pokrivenosti koda . . . 193
14.2.1 gcov, lcov . . . 193
14.2.2 Gradle . . . 193
14.2.3 xUnit / NUnit . . . 193
14.3 Alati/Biblioteke za Mock testiranje . . . 193
14.3.1 Moq . . . 193
14.4 Profajliranje instrumentacijom . . . 194
14.4.1 Valgrind . . . 194
14.4.2 gprof . . . 194
14.4.3 LLVM XRay . . . 194
14.5 Profajliranje uzorkovanjem . . . 194
14.5.1 Perf . . . 194
14.5.2 Intel VTune Profiler . . . 195
14.5.3 async-profiler . . . 195
14.6 eBPF alati . . . 195
14.7 Simboličko izvršavanje . . . 196
14.7.1 KLEE . . . 196
14.8 Provera modela . . . 196
14.8.1 CBMC . . . 196
14.9 Clang alati . . . 196
14.9.1 Clang . . . 196
14.10 Jezici za programiranje i formalnu verifikaciju . . . 197

14.10.1 Dafny 197

1 Dijagnoziranje problema

Ciljevi:

- Posmatrati program kao crnu kutiju (često ne znamo koji se kod izvršava)
- Analizirati ponašanje kako bi se pronašli uzroci problema
- Proširiti znanje o osnovnim alatima za merenje performansi
- Napraviti temelj za metodičnu analizu sistema - ne možemo raditi kvalitetnu analizu ako ne znamo šta i kako možemo analizirati

1.1 Alati za analizu performansi

Prvi korak u dijagnozi problema je određivanje opsega: da li je problem na nivou celog sistema, jednog procesa, jedne niti, diska, mreže ili spoljnog servisa. Alati iz ovog poglavlja ne traže poznavanje izvornog koda programa. Njihov cilj je da daju merenja i tragove koji usmeravaju dalju analizu.

Tipičan redosled rada je:

1. zabeležiti vreme, komandu, ulaz i simptome problema;
2. proveriti opšte stanje sistema pomoću `uptime`, `top`, `vmstat` ili `sar`;
3. suziti problem na proces ili nit pomoću `ps`, `pidstat`, `lsof` i `ss`;
4. proveriti konkretan resurs: CPU pomoću `mpstat`, disk pomoću `iostat`, mrežu pomoću `ss` i `tcpdump`;
5. ako je potrebno, analizirati sistemske pozive pomoću `strace`.

1.1.1 Kako birati sledeći alat

U praksi je najvažnije brzo preći sa opšteg simptoma na alat koji meri bliži nivo sistema. Sledeća tabela daje početni izbor alata. Nije zamena za detaljnu analizu, već mapa za prvi sledeći korak.

Simptom	Prvi alati	Sledeći korak
Visok CPU na sistemu	`uptime`, `top`, `mpstat`	Ako je kriv jedan proces, preći na `pidstat`, debager ili uzorkujući profajler.
Proces troši CPU	`top -H`, `pidstat`, `strace -c`	Ako nema mnogo sistemskih poziva, uzorkovati stek pomoću `perf`-a ili otvoriti proces u debageru.
Proces spava ili deluje zaglavljeno	`ps`, `pidstat -w`, `strace -tt -T`	Tražiti sistemski poziv u kome proces čeka: `accept`, `read`, `futex`, disk I/O ili mrežu.
Disk je spor ili zauzet	`iostat -x`, `pidstat -d`, `lsof`	Povezati uređaj, proces i datoteku, pa proveriti obrazac čitanja ili upisa.

Simptom	Prvi alati	Sledeći korak
Mrežna konekcija ne radi	`ss`, `lsof -i`, `tcpdump`	Prvo potvrditi da proces sluša, zatim da paket stiže do mašine i da odgovor izlazi.
Proces koristi previše memorije	`top`, `ps`, `pmap`	Za C/C++ programe preći na Massif, DHAT, Memcheck ili sanitizer; za JVM/.NET koristiti odgovarajući runtime alat.

1.1.2 Obrazac laboratorijske vežbe

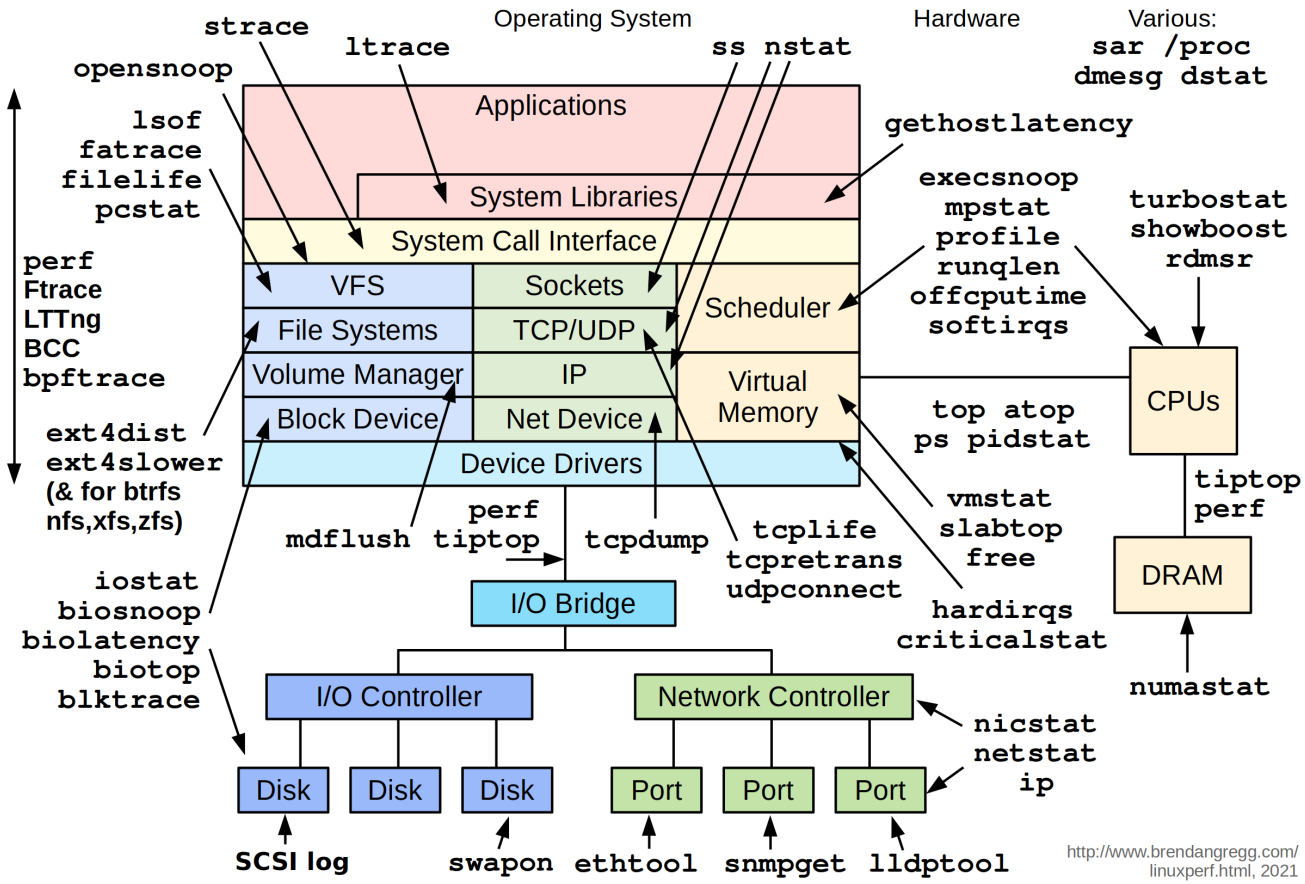
Svaku laboratorijsku vežbu treba dokumentovati istim redosledom:

1. **Simptom**: šta se vidi spolja, bez čitanja izvornog koda.
2. **Reprodukcija**: tačna komanda, ulaz i vreme pokretanja.
3. **Prvo merenje**: sistemski pregled koji potvrđuje da se simptom zaista dešava.
4. **Sužavanje**: proces, nit, resurs ili sistemski poziv koji objašnjava simptom.
5. **Potvrda**: alat bliži uzroku, na primer debager, profajler, sanitizer, Valgrind, KLEE ili CBMC.
6. **Tumačenje**: šta iz izlaza alata dokazuje uzrok i šta bi bio sledeći inženjerski korak.

Korisno je znati gde Linux čuva sirove podatke o procesima, jezgrima, uređajima i sistemskoj konfiguraciji:

- `/proc` sadrži informacije o procesima, memoriji, CPU jezgrima, montiranim sistemima datoteka i parametrima kernela.
- `/sys` prikazuje model uređaja, drajvere, klase uređaja i deo konfiguracije hardvera.

Mnogi alati prikazani u nastavku čitaju podatke iz ovih direktorijuma i formatiraju ih u oblik pogodan za brzu analizu.



Širi pregled Linux observability alata može se naći na linku.

1.1.3 `uptime`

`uptime` daje najbrži odgovor na pitanje da li je sistem opterećen. Prikazuje trenutno vreme, koliko dugo sistem radi, broj korisnika i prosečno opterećenje u poslednjih 1, 5 i 15 minuta.

Alat je koristan kada treba brzo proveriti:

- da li je problem kratkotrajan ili traje duže;
- da li se opterećenje povećava ili smanjuje;
- da li ima smisla odmah preći na detaljnije alate kao što su `top`, `vmstat` ili `pidstat`.

```
$ uptime
14:17:27 up  5:43,  1 user,  load average: 0.41, 0.93, 1.06
```

Prosek opterećenja dat je u intervalima od približno 1, 5 i 15 minuta. Vrednost predstavlja broj zadataka koji su spremni za izvršavanje ili čekaju neprekidiv I/O događaj. Zato visok load ne znači uvek da je CPU zauzet; uzrok može biti i spor disk ili mrežni sistem datoteka.

1.1.4 `top` / `htop`

`top` i `htop` prikazuju interaktivni pregled sistema i procesa. Koriste se kada treba brzo videti koji proces troši CPU, memoriju ili veliki broj niti, kao i da li se stanje menja tokom vremena.

Najčešće se posmatraju:

- opšti CPU i memorijski redovi na vrhu prikaza;
- kolone `%CPU`, `%MEM`, `RES`, `S`, `TIME+` i `COMMAND`;
- procesi koji često menjaju stanje ili zauzimaju jedno CPU jezgro;
- niti procesa, ako se prikaz proširi na thread-ove.

Ovi alati su dobri za interaktivnu analizu dugotrajnog problema. Manje su pogodni za procese koji traju kratko, jer uzorkuju stanje u intervalima. Za takve slučajeve korisniji su `pidstat`, `sar` ili alat koji čuva istoriju, kao što je `atop`. `top` i `htop` takođe troše resurse, pa ih treba pažljivo koristiti na već opterećenom sistemu.

1.1.5 `ps`

`ps` prikazuje trenutni snimak procesa. Za razliku od `top`, ne prati promene tokom vremena. Koristan je kada treba sačuvati stanje sistema u izveštaju, proveriti hijerarhiju procesa, argumente kojima je proces pokrenut ili stanje procesa.

`ps -ef f` prikazuje roditelj-dete odnose u obliku stabla. To je korisno kod servera koji pokreću radne procese, shell skripti koje pokreću druge programe i aplikacija koje ostavljaju procese nakon završetka roditelja.

```
$ ps -ef f
[...]
root        1044       1  0 08:34 ?        Ssl    0:00 /usr/bin/lightdm
root        1056    1044  7 08:34 tty7     Ssl+  26:47  \_ /usr/lib/Xorg :0 -seat seat0 -auth
↪  /run/lightdm/root/:0
root        1088    1044  0 08:34 ?        Sl     0:00  \_ lightdm --session-child 13 16
ivan        1118    1088  0 08:34 ?        Ss     0:08      \_ i3
root        1050       1  0 08:34 ?        Ssl    0:00 /usr/lib/accounts-daemon
ivan        1095       1  0 08:34 ?        Ss     0:00 /usr/lib/systemd/systemd --user
ivan        1096    1095  0 08:34 ?        S      0:00  \_ (sd-pam)
[...]
```

1.1.6 `vmstat`

`vmstat` daje sistemski pregled procesa, memorije, swap-a, ulaza/izlaza, prekida, promene konteksta i CPU vremena. Nema detalje po procesu, ali je veoma koristan za prvi pregled jer pokriva više klasa problema u jednom prikazu.

Tipična upotreba je pokretanje sa intervalom od jedne sekunde dok se problem reprodukuje. Prvi red često predstavlja prosek od pokretanja sistema, pa se za aktivnu dijagnozu obično gledaju redovi koji dolaze posle njega.

```
$ vmstat -Sm 1
procs -----------memory---------- ---swap-- -----io---- -system-- -------cpu-------
 r  b   swpd   free   buff  cache   si   so    bi    bo   in   cs us sy id wa st gu
```

```
 2  0     46    782    251   7993    0    0  3406   312 6224   35 12  4 83  1  0  0
 1  0     46    806    251   7986    0    0     0     0 2564 6940  4  3 93  0  0  0
 1  0     46    802    251   7986    0    0     0     0 1713 4499  3  2 95  0  0  0
 0  0     46    801    251   7986    0    0     0     0 1730 4536  2  2 96  0  0  0
^C
```

Važne kolone:

- `r` označava broj procesa spremnih za izvršavanje; trajno visoka vrednost ukazuje na CPU čekanje.
- `b` označava procese blokirane na I/O operacijama.
- `si` i `so` pokazuju čitanje iz swap-a i pisanje u swap; nenulte vrednosti tokom problema često ukazuju na memorijski pritisak.
- `bi` i `bo` pokazuju ulaz i izlaz prema blok uređajima.
- `us` je korisničko CPU vreme, odnosno vreme aplikacije, JVM-a ili runtime-a.
- `sy` je sistemsko CPU vreme, odnosno vreme provedeno u kernelu.
- `wa` je čekanje na I/O; visoka vrednost upućuje na disk, mrežni storage ili blokirajući uređaj.

1.1.7 `iostat`

`iostat` analizira opterećenje blok uređaja. Koristi se kada `vmstat` ili `top` pokažu čekanje na I/O, kada aplikacija sporo čita ili piše podatke, ili kada treba uporediti više diskova i particija.

Opcija `-x` daje proširenu statistiku, `-m` prikazuje protok u MB/s, `-d` ograničava prikaz na uređaje, a `-z` izostavlja uređaje bez aktivnosti.

```
$ iostat -xmdz 1
Linux [...]   09/23/2023      _x86_64_        (8 CPU)

Device    r/s     rMB/s   rrqm/s  %rrqm r_await rareq-sz     w/s     wMB/s  ...
dm-0      8.97      0.31     0.00   0.00    0.45    35.22   22.48      0.23  ...
dm-1      0.06      0.00     0.00   0.00    0.26     5.58    0.07      0.00  ...
loop0     0.00      0.00     0.00   0.00    0.00     1.27    0.00      0.00  ...
nvme0n1   6.49      0.31     2.55  28.24    0.27    48.76   12.25      0.23  ...
sda      25.68      3.00     0.46   1.78    2.95   119.49    0.52      0.08  ...

^C
```

Važno je posmatrati i protok i kašnjenje. Velik `rMB/s` ili `wMB/s` znači da uređaj radi mnogo posla, dok veliki `await` znači da operacije čekaju. Ako je `%util` blizu 100% i `await` raste, disk ili storage sloj je verovatan uzrok usporenja.

1.1.8 `swapon`

`swapon` prikazuje aktivne swap prostore. Samo postojanje swap-a nije problem; problem je kada sistem tokom reprodukcije aktivno prebacuje stranice memorije između RAM-a i swap-a. Zato se `swapon` obično koristi zajedno sa kolonama `si` i `so` iz `vmstat`-a ili memorijskim izveštajem iz `sar`-a.

```
$ swapon -s
Filename    Type            Size            Used            Priority
/dev/dm-1   partition       17898688        3991268         -2
```

1.1.9 `mpstat`

`mpstat` prikazuje CPU statistiku po jezgru. Koristi se kada prosečna zauzetost CPU-a ne objašnjava problem, na primer kada je samo jedno jezgro stalno zauzeto, a ostatak sistema deluje neiskorišćeno.

Opcija `-P ALL` prikazuje sva jezgra. To pomaže kod aplikacija koje imaju usko grlo u jednoj niti, kod procesa vezanih za određeni CPU, kod softirq opterećenja i kod sistema na kojima raspodela posla nije ravnomerna.

```
$ mpstat -P ALL 1
Linux [...]   09/23/2023      _x86_64_        (8 CPU)

CPU    %usr   %nice    %sys %iowait    %irq   %soft  %steal  %guest  %gnice   %idle
all    4.14    0.00    2.01    0.50    0.38    0.25    0.00    0.00    0.00   92.73
  0    3.03    0.00    1.01    0.00    0.00    1.01    0.00    0.00    0.00   94.95
  1    5.94    0.00    2.97    0.00    0.00    0.00    0.00    0.00    0.00   91.09
  2    3.96    0.00    1.98    0.00    0.99    0.99    0.00    0.00    0.00   92.08
  3    4.04    0.00    2.02    0.00    1.01    0.00    0.00    0.00    0.00   92.93
  4    5.00    0.00    1.00    0.00    1.00    0.00    0.00    0.00    0.00   93.00
  5    4.00    0.00    1.00    0.00    0.00    0.00    0.00    0.00    0.00   95.00
  6    3.06    0.00    3.06    4.08    0.00    0.00    0.00    0.00    0.00   89.80
  7    4.00    0.00    3.00    0.00    0.00    0.00    0.00    0.00    0.00   93.00
^C
```

1.1.10 `lsof`

`lsof` prikazuje otvorene datotečne deskriptore. Na Linux sistemu to obuhvata regularne datoteke, direktorijume, biblioteke, pipe-ove, socket-e, terminale i anoniman kernel objekat izložen kao datotečni deskriptor.

Alat je koristan kada treba odgovoriti na pitanja:

- koji proces drži otvorenu datoteku, port ili socket;
- zašto se direktorijum, disk ili mount tačka ne mogu demontirati;
- da li proces curi datotečne deskriptore;
- da li proces i dalje drži obrisanu datoteku, pa prostor na disku nije oslobođen.

```
$ lsof -p <PID>
$ lsof -iTCP -sTCP:LISTEN -P -n
$ lsof +L1
```

Opcije `-P` i `-n` izbegavaju prevođenje portova i adresa u imena. To ubrzava prikaz i uklanja zavisnost od DNS-a tokom dijagnoze.

1.1.11 `pidstat`

`pidstat` meri potrošnju resursa po procesu ili niti kroz vreme. Koristi se kada `top` pokaže sumnjiv proces, ali je potrebno sačuvati precizniji vremenski trag ili razdvojiti CPU, memoriju, disk i promene konteksta.

Korisne varijante su:

- `pidstat -p <PID> 1` za praćenje jednog procesa;
- `pidstat -t -p <PID> 1` za praćenje niti unutar procesa;
- `pidstat -d 1` za I/O po procesu;
- `pidstat -r 1` za memoriju po procesu;
- `pidstat -w 1` za dobrovoljne i nedobrovoljne promene konteksta.

```
$ pidstat -t 1
11:04:46 AM   UID       PID    %usr %system  %guest   %wait    %CPU   CPU  Command
11:04:46 AM     0         1    0.00    0.00    0.00    0.00    0.01     1  systemd
11:04:46 AM     0         2    0.00    0.00    0.00    0.00    0.00     3  kthreadd
11:04:46 AM     0        15    0.00    0.00    0.00    0.00    0.00     0  ksoftirqd/0
11:04:46 AM     0        16    0.00    0.02    0.00    0.01    0.02     4  rcu_preempt
11:04:46 AM     0        18    0.00    0.00    0.00    0.00    0.00     0  migration/0
11:04:46 AM     0        24    0.00    0.00    0.00    0.00    0.00     1  migration/1
11:04:46 AM     0        25    0.00    0.00    0.00    0.00    0.00     1  ksoftirqd/1
11:04:46 AM     0        30    0.00    0.00    0.00    0.00    0.00     2  migration/2
11:04:46 AM     0        31    0.00    0.01    0.00    0.00    0.01     2  ksoftirqd/2
11:04:46 AM     0        36    0.00    0.00    0.00    0.00    0.00     3  migration/3
11:04:46 AM     0        37    0.00    0.00    0.00    0.00    0.00     3  ksoftirqd/3
^C
```

1.1.12 `strace`

`strace` prati sistemske pozive procesa. Koristi se kada se problem vidi kao interakcija programa sa operativnim sistemom: otvaranje datoteka, mrežne konekcije, procesi, signali, čekanje na lock ili čitanje i pisanje.

Tipični slučajevi upotrebe su:

- program ne nalazi konfiguracionu datoteku ili biblioteku;
- program dobija `EACCES`, `ENOENT`, `ECONNREFUSED` ili sličnu grešku;
- proces je blokiran, pa treba videti da li čeka na `read`, `futex`, `poll`, `connect` ili drugi sistemski poziv;
- potrebno je proveriti koje child procese program pokreće;
- potrebno je dobiti sažetak najčešćih sistemskih poziva.

`strace` je baziran na `ptrace` mehanizmu i može značajno usporiti program. Zbog toga se na produkcionim sistemima najpre ograničava opseg: bira se konkretan PID, grupa sistemskih poziva ili kratko vreme reprodukcije.

```
$ strace ls
execve("/usr/bin/ls", ["ls"], 0x7fffc329f270 /* 131 vars */) = 0
brk(NULL)                               = 0x55e00371d000

[...]

close(3)                                = 0
openat(AT_FDCWD, "/usr/lib/libcap.so.2", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0 s\0\0\0\0\0\0"..., 832) = 832
newfstatat(3, "", {st_mode=S_IFREG|0755, st_size=42992, ...}, AT_EMPTY_PATH) = 0
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7f76305ec000
```

```
mmap(NULL, 45128, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7f76305e0000
mmap(0x7f76305e3000, 20480, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x3000) =
↪  0x7f76305e3000
mmap(0x7f76305e8000, 8192, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x8000) =
↪  0x7f76305e8000
mmap(0x7f76305ea000, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x9000) =
↪  0x7f76305ea000
close(3)

[...]
```

```
$ strace -e close ls
close(3)                                = 0
close(3)                                = 0
close(3)                                = 0
close(3)                                = 0
close(3)                                = 0
INDEX     lab002.c  lab004.c  lab006.c  lab008.c  Makefile
lab001.c  lab003.c  lab005.c  lab007.c  lab013.c  README.md
close(1)                                = 0
close(2)                                = 0
+++ exited with 0 +++
```

Opcija `-e` ograničava praćenje na izabrane sistemske pozive ili grupe poziva. Na primer, `-e trace=file` prati pristup datotekama, `-e trace=network` mrežne pozive, a `-e trace=process` kreiranje i završavanje procesa.

```
$ strace -t -r -p <PID> -o <REPORT_OUTPUT>
# -t - timestamp
# -r - relative time for each syscall
# -p - PID
# -o - output path
```

Kada se `strace` priključi na postojeći proces, korisno je zapisati izlaz u datoteku. Opcija `-t` dodaje vreme događaja, a `-r` vreme proteklo od prethodnog sistemskog poziva. To pomaže da se pronađu dugi zastoji.

```
$ strace -c ls
INDEX     lab002.c  lab004.c  lab006.c  lab008.c  Makefile
lab001.c  lab003.c  lab005.c  lab007.c  lab013.c  README.md
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
  0.00    0.000000           0         2           read
  0.00    0.000000           0         2           write
  0.00    0.000000           0         7           close
  0.00    0.000000           0        13           mmap
  0.00    0.000000           0         4           mprotect
  0.00    0.000000           0         1           munmap
  0.00    0.000000           0         3           brk
  0.00    0.000000           0         2           ioctl
  0.00    0.000000           0         2           pread64
```

```
  0.00    0.000000           0         1         1 access
  0.00    0.000000           0         1           execve
  0.00    0.000000           0         6         4 prctl
  0.00    0.000000           0         2         1 arch_prctl
  0.00    0.000000           0         2           getdents64
  0.00    0.000000           0         1           set_tid_address
  0.00    0.000000           0         5           openat
  0.00    0.000000           0         6           newfstatat
  0.00    0.000000           0         1           set_robust_list
  0.00    0.000000           0         1           prlimit64
  0.00    0.000000           0         1           getrandom
  0.00    0.000000           0         1           rseq
------ ----------- ----------- --------- --------- ----------------
100.00    0.000000           0        64         6 total
```

Opcija `-c` daje agregirani izveštaj. Ona je korisna kada ne treba čitati svaki poziv, već samo proceniti gde proces provodi vreme i koji pozivi se ponavljaju. Za programe koji pokreću child procese često se dodaje i `-f`, kako bi se pratio ceo procesni skup.

1.1.13 Analiza mreže

Mrežni problem se često prvo vidi kao spor odgovor aplikacije, odbijena konekcija, zauzet port ili timeout. Pre detaljne analize protokola korisno je proveriti da li proces sluša na očekivanom portu, da li postoji konekcija ka udaljenom sistemu i da li paketi zaista prolaze kroz interfejs.

Za duboku analizu protokola koristi se alat kao što je Wireshark. U komandnoj liniji se najčešće kombinuju `ss`, `lsof`, `strace` i `tcpdump`.

`tcpdump` `tcpdump` hvata pakete na mrežnom interfejsu. Koristi se kada treba dokazati da paket napušta mašinu, da odgovor stiže, da se koristi pogrešan port ili da mrežni tok izgleda drugačije od očekivanog.

Alat često zahteva administratorske privilegije. Na zauzetim sistemima treba koristiti filtere, jer sirov mrežni saobraćaj brzo postaje prevelik za čitanje.

```
$ sudo tcpdump dst port 123
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on eth0, link-type EN10MB (Ethernet), capture size 65535 bytes
21:02:19.112502 IP test33.ntp > 199.30.140.74.ntp: NTPv4, Client, length 48
21:02:19.113888 IP 216.239.35.0.ntp > test33.ntp: NTPv4, Server, length 48
21:02:20.150347 IP test33.ntp > 216.239.35.0.ntp: NTPv4, Client, length 48
21:02:20.150991 IP 216.239.35.0.ntp > test33.ntp: NTPv4, Server, length 48
```

```
$ sudo tcpdump -nn -i any host <HOST> and port <PORT>
$ sudo tcpdump -nn -i any -w trace.pcap
```

Opcija `-nn` isključuje razrešavanje imena i portova. Opcija `-i any` hvata saobraćaj sa svih interfejsa, što je korisno kada nije jasno kojim putem paket ide. Zapis u `pcap` datoteku se kasnije može otvoriti u Wireshark-u.

`netstat` / `ss` `netstat` i `ss` prikazuju socket-e, konekcije, portove i procese koji ih koriste. `netstat` je stariji alat, dok je `ss` deo modernog `iproute2` skupa i obično daje brži i potpuniji prikaz.

Ovi alati se koriste kada treba proveriti:

- da li servis sluša na očekivanom portu;
- koji proces je zauzeo port;
- da li su konekcije u stanjima `LISTEN`, `ESTABLISHED`, `TIME-WAIT` ili `CLOSE-WAIT`;
- da li redovi `Recv-Q` i `Send-Q` rastu, što može ukazati na zagušenje ili aplikaciju koja ne čita podatke.

```
$ netstat -tulnp
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 127.0.0.1:57343         0.0.0.0:*               LISTEN      3501/steam
tcp        0      0 0.0.0.0:27036           0.0.0.0:*               LISTEN      3501/steam
tcp        0      0 127.0.0.1:33095         0.0.0.0:*               LISTEN      3501/steam
tcp        0      0 127.0.0.1:35831         0.0.0.0:*               LISTEN      3501/steam
tcp        0      0 127.0.0.1:43221         0.0.0.0:*               LISTEN      -
tcp        0      0 127.0.0.1:27060         0.0.0.0:*               LISTEN      3501/steam
tcp6       0      0 ::1:631                 :::*                    LISTEN      -
tcp6       0      0 :::6600                 :::*                    LISTEN      1135/mpd
udp        0      0 0.0.0.0:27036           0.0.0.0:*                           3501/steam
udp        0      0 0.0.0.0:5353            0.0.0.0:*                           -
udp        0      0 0.0.0.0:46653           0.0.0.0:*                           -
udp        0      0 0.0.0.0:38865           0.0.0.0:*                           -
udp6       0      0 :::44350                :::*                                -
udp6       0      0 :::5353                 :::*                                -
udp6       0      0 :::46653                :::*                                -
```

`ss` je preporučeni izbor za nove analize:

```
$ ss -tunpl
Netid State  Recv-Q Send-Q  Local Address:Port    Peer Address:Port Process
udp   UNCONN 0      0             0.0.0.0:27036        0.0.0.0:*
↪  users:(("steam",pid=3501,fd=123))
udp   UNCONN 0      0             0.0.0.0:5353         0.0.0.0:*
udp   UNCONN 0      0             0.0.0.0:46653        0.0.0.0:*
udp   UNCONN 0      0             0.0.0.0:38865        0.0.0.0:*
udp   UNCONN 0      0                [::]:44350           [::]:*
udp   UNCONN 0      0                [::]:5353            [::]:*
udp   UNCONN 0      0                [::]:46653           [::]:*
tcp   LISTEN 0      128         127.0.0.1:57343        0.0.0.0:*     users:(("steam",pid=3501,fd=48))
tcp   LISTEN 0      128           0.0.0.0:27036        0.0.0.0:*
↪  users:(("steam",pid=3501,fd=124))
tcp   LISTEN 0      4096        127.0.0.1:631          0.0.0.0:*
tcp   LISTEN 0      128         127.0.0.1:33095        0.0.0.0:*     users:(("steam",pid=3501,fd=53))
tcp   LISTEN 0      128         127.0.0.1:35831        0.0.0.0:*     users:(("steam",pid=3501,fd=83))
tcp   LISTEN 0      4096        127.0.0.1:43221        0.0.0.0:*
tcp   LISTEN 0      128         127.0.0.1:27060        0.0.0.0:*     users:(("steam",pid=3501,fd=84))
tcp   LISTEN 0      4096            [::1]:631             [::]:*
tcp   LISTEN 0      5                   *:6600                *:*     users:(("mpd",pid=1135,fd=4))
```

Opcije u primeru znače: `-t` za TCP, `-u` za UDP, `-n` za numerički prikaz adresa i portova, `-p` za proces, a `-l` za socket-e koji slušaju.

1.1.14 System Activity Reporter (`sar`)

`sar` prikuplja i prikazuje sistemske metrike kroz vreme. Pripada paketu `sysstat`. Koristan je kada problem nije moguće uhvatiti interaktivno, jer se na mnogim sistemima podaci periodično čuvaju i mogu se analizirati naknadno.

`sar` može da prati CPU, memoriju, swap, mrežne uređaje, diskove i druge sistemske metrike. U nastavku su tri česta preseka: CPU, memorija i mreža.

```
$ sar -u 1 3
11:12:22 AM     CPU     %user     %nice   %system   %iowait    %steal     %idle
11:12:23 AM     all      3.88      0.00      1.88      0.25      0.00     93.99
11:12:24 AM     all      4.45      0.00      1.11      0.12      0.00     94.31
11:12:25 AM     all      3.98      0.00      1.99      0.25      0.00     93.78
Average:        all      4.10      0.00      1.66      0.21      0.00     94.03
```

```
$ sar -r 1 3
11:13:26 AM kbmemfree   kbavail kbmemused  %memused kbbuffers  kbcached  kbcommit   %commit
↪  kbactive   kbinact   kbdirty
11:13:27 AM   5841564   7857352   7545328     46.37     31224   2286720  21332312     62.43
↪  6722340   1418056      1004
11:13:28 AM   5841060   7856848   7545928     46.38     31224   2286720  21328940     62.42
↪  6722596   1418048         0
11:13:29 AM   5837028   7852948   7549820     46.40     31232   2286852  21332780     62.43
↪  6725632   1418188        76
Average:      5839884   7855716   7547025     46.38     31227   2286764  21331344     62.43
↪  6723523   1418097       360
```

```
$ sar -n DEV 1 1
11:14:55 AM     IFACE   rxpck/s   txpck/s    rxkB/s    txkB/s   rxcmp/s   txcmp/s  rxmcst/s   %ifutil
11:14:56 AM        lo      9.00      9.00      0.74      0.74      0.00      0.00      0.00      0.00
11:14:56 AM    enp3s0      0.00      0.00      0.00      0.00      0.00      0.00      0.00      0.00
11:14:56 AM    wlp2s0      1.00      2.00      0.38      1.20      0.00      0.00      0.00      0.00
11:14:56 AM   docker0      0.00      0.00      0.00      0.00      0.00      0.00      0.00      0.00
11:14:56 AM  nordlynx      1.00      2.00      0.34      1.08      0.00      0.00      0.00      0.00

Average:        IFACE   rxpck/s   txpck/s    rxkB/s    txkB/s   rxcmp/s   txcmp/s  rxmcst/s   %ifutil
Average:           lo      9.00      9.00      0.74      0.74      0.00      0.00      0.00      0.00
Average:       enp3s0      0.00      0.00      0.00      0.00      0.00      0.00      0.00      0.00
Average:       wlp2s0      1.00      2.00      0.38      1.20      0.00      0.00      0.00      0.00
Average:      docker0      0.00      0.00      0.00      0.00      0.00      0.00      0.00      0.00
Average:     nordlynx      1.00      2.00      0.34      1.08      0.00      0.00      0.00      0.00
```

Kada je prikupljanje istorije uključeno, `sar` se koristi za analizu problema koji se desio ranije. Na primer, može se proveriti da li je u vreme incidenta porastao `%iowait`, potrošnja memorije, swap aktivnost ili mrežni protok. Time se dijagnoza ne oslanja samo na trenutno stanje sistema.

1.2 Laboratorijski primeri

Primeri su preuzeti iz perf-labs repozitorijuma Brendana Gregg-a, korišćenog tokom prezentacije o alatima za merenje performansi Linux kernel-a. Svaki program namerno pravi jedan simptom.

Cilj vežbe nije odmah otvoriti izvorni kod, već prvo posmatrati proces kao crnu kutiju i zatim potvrditi uzrok.

Pre pokretanja primera treba ih prevesti:

```
$ make all
```

Pokretati jedan primer po jedan, u posebnom terminalu. Posle analize zaustaviti proces pomoću `Ctrl+C` ili komandom:

```
$ pkill lab001
```

Tok dijagnoze za svaki primer je isti:

1. proveriti opšte stanje sistema;
2. pronaći proces koji pravi simptom;
3. odrediti resurs koji je usko grlo;
4. potvrditi uzrok alatom koji posmatra bliži nivo sistema.

1.2.1 Pregled primera

Primer	Simptom	Prvi alati
`lab001`	CPU petlja u jednoj funkciji	`top`, `pidstat`, `strace`
`lab002`	proces čeka konekciju	`ss`, `strace`, `tcpdump`
`lab003`	veliki broj sistemskih poziva	`vmstat`, `pidstat`, `strace`
`lab004`	kratkotrajni child procesi	`pidstat`, `ps`, `strace -f`
`lab005`	sinhroni upisi na disk	`vmstat`, `iostat`, `pidstat -d`
`lab006`	CPU rad raspodeljen kroz više funkcija	`top`, `mpstat`, `pidstat`
`lab007`	proces uglavnom spava	`top`, `pidstat -w`, `strace`
`lab008`	upisi na disk sa povremenim kašnjenjem	`iostat`, `pidstat -d`, `strace -T`
`lab013`	iscrpljivanje steka rekurzijom	`ulimit`, `dmesg`, `sar -r`

1.2.2 `lab001`: CPU petlja

Ovaj program zauzima CPU bez sistemskih poziva i bez I/O aktivnosti. Kod je mali, ali je koristan za vežbanje razlike između procesa koji radi u korisničkom prostoru i procesa koji čeka kernel ili uređaj.

```c
/*
 * lab001 - CPU hot stack.
 *
 * 21-May-2015    Brendan Gregg    Created this.
 */

void func_c() {
    for (;;) {
```

```
    }
}

void func_b() { func_c(); }

void func_a() { func_b(); }

int main(int argc, char *argv[]) {
    func_a();
    return (0);
}
```

Pokrenuti program i posmatrati ga iz drugog terminala:

```
$ ./lab001 &
$ top -p $(pgrep lab001)
$ pidstat -p $(pgrep lab001) 1
$ strace -p $(pgrep lab001)
```

Očekivano je da `top` i `pidstat` pokažu visoku CPU potrošnju. `strace` neće prikazivati stalne sistemske pozive, jer se program vrti u funkciji `func_c`. Time se problem svodi na CPU-bound kod u korisničkom prostoru. Za tačnu lokaciju vrućeg steka koriste se debager ili profajler.

1.2.3 `lab002`: čekanje na konekciju

Ovaj program otvara TCP port i zatim čeka klijentsku konekciju. Simptom je proces koji postoji, ali troši malo CPU vremena i deluje neaktivan.

```
/*
 * lab002 - waiting on accept().
 *
 * 21-May-2015    Brendan Gregg    Created this.
 */

#include <netinet/in.h>
#include <stdio.h>
#include <stdlib.h>
#include <strings.h>
#include <sys/socket.h>
#include <sys/types.h>
#include <unistd.h>

#define BUFSIZE 256
#define PORT 12340

void connection_handler(int connection_fd) {
    int nbytes;
    char buffer[BUFSIZE];

    nbytes = read(connection_fd, buffer, BUFSIZE);
    buffer[nbytes] = 0;
```

```
    nbytes = sprintf(buffer, "Shhh... I'm sleeping.\n");
    write(connection_fd, buffer, nbytes);
}

void wait_for_work(int socket_fd) {
    int connection_fd;
    struct sockaddr_in client;
    socklen_t length;

    length = sizeof(client);

    while ((connection_fd =
                accept(socket_fd, (struct sockaddr *)&client, &length)) > -1) {
        connection_handler(connection_fd);
        close(connection_fd);
    }
}

int main(int argc, char *argv[]) {
    struct sockaddr_in address;
    int socket_fd, connection_fd;
    size_t address_length;
    pid_t child;
    struct hostent *server;

    if ((socket_fd = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
        printf("ERROR: socket() failed\n");
        exit(1);
    }

    bzero((char *)&address, sizeof(address));
    address.sin_family = AF_INET;
    address.sin_port = htons(PORT);
    address.sin_addr.s_addr = INADDR_ANY;

    if (bind(socket_fd, (struct sockaddr *)&address, sizeof(address)) < 0) {
        printf("ERROR: bind() failed");
        exit(2);
    }

    if (listen(socket_fd, 5) != 0) {
        printf("ERROR: listen() failed\n");
        exit(3);
    }

    wait_for_work(socket_fd);

    close(socket_fd);
    return (0);
}
```

Prvo proveriti da li proces sluša na očekivanom portu:

```
$ ./lab002 &
```

```
$ ss -ltnp 'sport = :12340'
$ lsof -iTCP:12340 -sTCP:LISTEN -P -n
```

Zatim proveriti na kom sistemskom pozivu proces čeka:

```
$ strace -tt -T -p $(pgrep lab002)
```

Očekivani trag pokazuje da proces čeka u `accept`. Kada se klijent poveže, `accept` se završava, proces čita podatke i šalje odgovor.

```
$ nc 127.0.0.1 12340
```

Ako klijent ne može da se poveže, proveriti port pomoću `ss`, lokalni firewall i paketni trag pomoću `tcpdump`. Ako port nije otvoren, problem je pre mrežnog toka: proces nije pokrenut, nije uspeo `bind`, ili sluša na drugoj adresi.

1.2.4 `lab003`: čitanje nula bajtova

Ovaj program pravi datoteku, otvara je za čitanje i zatim u beskonačnoj petlji poziva `read` sa veličinom 0. Poziv uspeva, ali ne prenosi podatke. Simptom je neobično velika aktivnost sistemskih poziva bez stvarnog disk I/O rada.

```c
/*
 * lab003 - 0 byte successful reads.
 *
 * 21-May-2015    Brendan Gregg    Created this.
 */

#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <unistd.h>

const char *datafile = "lab003.data";

#define BUFSIZE (8 * 1024)
#define FILESIZE (10 * 1024 * 1024)

void init_data() {
    int fd;
    long long i;
    char buf[BUFSIZE];

    if ((fd = open(datafile, O_CREAT | O_WRONLY, 0644)) < 0) {
        printf("ERROR: Can't open datafile: %s\n", datafile);
        exit(1);
    }
```

```
    for (i = 0; i < FILESIZE / BUFSIZE; i++) {
        if (write(fd, buf, BUFSIZE) < 0) {
            printf("ERROR: write error.\n");
            exit(2);
        }
    }

    close(fd);
}

void load_data() {
    int fd;
    char buf[1];

    if ((fd = open(datafile, O_RDONLY)) < 0) {
        printf("ERROR: Can't open database: %s\n", datafile);
        exit(2);
    }

    for (;;) {
        read(fd, &buf, 0);
    }

    (void)close(fd);
}

void import_database() { load_data(); }

int main() {
    init_data();

    for (;;) {
        import_database();
    }

    return (0);
}
```

Krenuti od opšteg pregleda:

```
$ ./lab003 &
$ vmstat 1
$ pidstat -p $(pgrep lab003) 1
$ iostat -x 1
$ sar -n DEV 1
```

Ako CPU raste, a disk i mreža ne pokazuju odgovarajući promet, suziti analizu na sistemske pozive:

```
$ strace -tp $(pgrep lab003) 2>&1 | head -n 20
$ strace -c -p $(pgrep lab003)
```

Očekivani nalaz je ponavljanje poziva nalik na `read(fd, ..., 0) = 0`. To objašnjava zašto proces troši CPU i sistemsko vreme, a ne proizvodi korisno čitanje podataka.

1.2.5 `lab004`: kratkotrajni child procesi

Ovaj program u petlji pokreće komandu `cksum` nad datotekom. Problem može lako promaći u `top`-u, jer child procesi kratko traju.

```c
/*
 * lab004 - short lived cksum processes.
 *
 * 21-May-2015    Brendan Gregg    Created this.
 */

#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <unistd.h>

const char *datafile = "lab004.data";

#define BUFSIZE (8 * 1024)
#define FILESIZE (10 * 1024 * 1024)

void init_data() {
    int fd;
    long long i;
    char buf[BUFSIZE];

    if ((fd = open(datafile, O_CREAT | O_WRONLY, 0644)) < 0) {
        printf("ERROR: Can't open datafile: %s\n", datafile);
        exit(1);
    }

    for (i = 0; i < FILESIZE / BUFSIZE; i++) {
        if (write(fd, buf, BUFSIZE) < 0) {
            printf("ERROR: write error.\n");
            exit(2);
        }
    }

    close(fd);
}

void check_log() {
    char cmd[256];
    snprintf(cmd, sizeof(cmd), "cksum %s >/dev/null 2>&1", datafile);

    system(cmd);
}

int main() {
    init_data();

    for (;;) {
        check_log();
```

```
    }

    return (0);
}
```

Početi od procesa roditelja, a zatim pratiti child procese:

```
$ ./lab004 &
$ pidstat -p $(pgrep lab004) 1
$ ps -ef f | grep -E 'lab004|cksum'
$ strace -f -e trace=process -p $(pgrep lab004)
```

Opcija `-f` je bitna jer prati child procese. Očekivani trag pokazuje ponavljanje kreiranja procesa, izvršavanje `cksum` i čekanje da child proces završi. Ako se gleda samo roditeljski proces, CPU ili I/O slika može izgledati nepotpuno.

Za proveru disk aktivnosti koristiti:

```
$ pidstat -d 1
$ iostat -x 1
```

1.2.6 `lab005`: sinhroni upisi na disk

Ovaj program otvara datoteku sa `O_SYNC` i stalno piše u nju. Svaki upis čeka da podaci budu sinhronizovani sa storage slojem. Simptom može biti visok `iowait`, zauzet disk i spor odziv drugih procesa.

```
/*
 * lab005 - sync disk writes to a file.
 *
 * 21-May-2015    Brendan Gregg    Created this.
 */

#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <unistd.h>

const char *datafile = "lab005.data";

#define BUFSIZE (8 * 1024)
#define FILESIZE (10 * 1024 * 1024)

void write_log(int fd) {
    char buf[BUFSIZE];
    int i;

    for (;;) {
        for (i = 0; i < FILESIZE / BUFSIZE; i++) {
```

```
            if (write(fd, buf, BUFSIZE) < 0) {
                printf("ERROR: write error.\n");
                exit(2);
            }
        }

        if (lseek(fd, 0, SEEK_SET) < 0) {
            printf("ERROR: seek() failed.\n");
            exit(3);
        }
    }
}

int main() {
    int fd;

    if ((fd = open(datafile, O_CREAT | O_WRONLY | O_SYNC, 0644)) < 0) {
        printf("ERROR: writing to %s\n", datafile);
        exit(1);
    }

    write_log(fd);

    return (0);
}
```

Za ovaj primer je zgodna USE metodologija: proveriti iskorišćenost, zasićenje i greške za CPU, memoriju, disk i mrežu.

```
$ ./lab005 &
$ htop
$ mpstat -P ALL 1
$ vmstat 1
$ iostat -x 1
$ sar -n DEV 1
```

Ako CPU nije potpuno zauzet, nema swap aktivnosti, a `iostat` pokazuje veliki `%util` ili rast `await`, uzrok je najverovatnije disk. Zatim proveriti procesni I/O i otvorenu datoteku:

```
$ pidstat -d -p $(pgrep lab005) 1
$ lsof -p $(pgrep lab005) | grep lab005.data
```

Bitna veza sa kodom je opcija `O_SYNC`. Ona menja cenu svakog `write` poziva, pa problem nije samo u količini podataka već i u načinu pisanja.

1.2.7 `lab006`: više CPU vrućih funkcija

Ovaj program troši CPU u dve funkcije. Za razliku od `lab001`, vruće vreme nije vezano za jednu jedinu petlju. Primer pokazuje granicu sistemske dijagnoze: alati kao `top` i `pidstat` mogu da potvrde CPU problem, ali ne mogu sami da raspodele vreme po funkcijama.

```c
/*
 * lab006 - CPU hot stacks.
 *
 * 21-May-2015    Brendan Gregg    Created this.
 */

#define SPINS (1000 * 1000) /* tune as desired */

void func_c() {
    int i;
    for (i = 0; i < SPINS; i++) {
    }
}

void func_b() { func_c(); }

void func_e() {
    int i;
    for (i = 0; i < 5 * SPINS; i++) {
    }
}

void func_a() {
    for (;;) {
        func_b();
        func_e();
    }
}

int main(int argc, char *argv[]) {
    func_a();
    return (0);
}
```

Prvo potvrditi da je problem CPU-bound:

```
$ ./lab006 &
$ top -p $(pgrep lab006)
$ mpstat -P ALL 1
$ pidstat -p $(pgrep lab006) 1
$ strace -p $(pgrep lab006)
```

Ako `strace` ne prikazuje stalne sistemske pozive, a CPU potrošnja je visoka, proces radi u korisničkom prostoru. Sledeći korak je uzorkovanje steka pomoću profajlera ili analiza procesa u debageru. Ovaj primer zato povezuje poglavlje o dijagnozi sa poglavljima o debagovanju i profajliranju.

1.2.8 `lab007`: proces uglavnom spava

Ovaj program periodično spava, pa zatim kratko radi u petlji. Simptom je proces koji je prisutan, ali troši malo CPU vremena. Takav proces može izgledati nebitno u `top`-u, iako je važan za razumevanje kašnjenja.

```c
/*
 * lab007 - asleep in function call.
 *
 * 21-May-2015    Brendan Gregg    Created this.
 */

#include <unistd.h>

#define SPINS (10 * 1000 * 1000) /* tune to ~1% CPU */

void func_l() {
    int i, j;

    for (;;) {
        usleep(1000 * 1000);
        for (i = 0, j = 0; i < SPINS; i++) {
            j++;
        }
    }
}

void func_k() { func_l(); }

void func_j() { func_k(); }

int main(int argc, char *argv[]) {
    func_j();
    return (0);
}
```

Posmatrati CPU vreme i promene konteksta:

```
$ ./lab007 &
$ top -p $(pgrep lab007)
$ pidstat -w -p $(pgrep lab007) 1
$ strace -tt -T -p $(pgrep lab007)
```

Očekivani trag pokazuje periodične pozive za spavanje, najčešće kroz `nanosleep` ili srodan sistemski poziv. `pidstat -w` pokazuje promene konteksta, jer proces dobrovoljno prepušta CPU dok spava.

1.2.9 `lab008`: disk latencije sa izuzecima

Ovaj program takođe koristi `O_SYNC`, ali povremeno piše veći blok podataka. Simptom nije samo zauzet disk, već i povremeni skokovi latencije pojedinačnih `write` poziva.

```c
/*
 * lab008 - sync disk writes to a file, with (syscall) latency outliers.
 *
 * 21-May-2015    Brendan Gregg    Created this.
 */
```

```c
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <strings.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <unistd.h>

const char *datafile = "lab008.data";

#define BUFSIZE (8 * 1024)
#define BIGSIZE (10 * 1024 * 1024)
#define FILESIZE (10 * 1024 * 1024)

void write_log(int fd) {
    char *buf, *big;
    long long i;
    int ret, j;

    buf = malloc(BUFSIZE);
    big = malloc(BIGSIZE);
    if (buf == NULL || big == NULL) {
        printf("ERROR: malloc buffers.\n");
        exit(1);
    }
    bzero(buf, BUFSIZE);
    bzero(big, BIGSIZE);

    for (;;) {
        for (i = 0, j = 0; i < FILESIZE;) {
            if ((j++ % 100) == 0) {
                ret = write(fd, big, BIGSIZE);
                i += BIGSIZE;
            } else {
                ret = write(fd, buf, BUFSIZE);
                i += BUFSIZE;
            }

            if (ret < 0) {
                printf("ERROR: write error.\n");
                exit(2);
            }
        }

        if (lseek(fd, 0, SEEK_SET) < 0) {
            printf("ERROR: seek() failed.\n");
            exit(3);
        }
    }

    free(buf);
    free(big);
}

int main() {
```

```
    int fd;

    if ((fd = open(datafile, O_CREAT | O_WRONLY | O_SYNC, 0644)) < 0) {
        printf("ERROR: writing to %s\n", datafile);
        exit(1);
    }

    write_log(fd);

    return (0);
}
```

Prvo potvrditi da je usko grlo disk:

```
$ ./lab008 &
$ iostat -x 1
$ pidstat -d -p $(pgrep lab008) 1
```

Zatim izmeriti trajanje sistemskih poziva:

```
$ strace -tt -T -e trace=write,lseek -p $(pgrep lab008)
```

Opcija `-T` dodaje trajanje svakog sistemskog poziva. Očekivano je da se vide obični upisi i povremeni znatno skuplji upisi. U kodu se to poklapa sa granom koja periodično šalje veći bafer.

1.2.10 `lab013`: rekurzija i stek

Ovaj program beskonačno rekurzivno poziva istu funkciju. Simptom može izgledati kao memorijski problem, ali uzrok nije heap alokacija već rast steka do ograničenja procesa.

```
/*
 * lab013 - out of memory by recursion. tricky.
 *
 * 21-May-2015    Brendan Gregg    Created this.
 */

void func_r() {
    int i;
    for (i = 0; i < 100; i++) {
    }
    func_r();
}

int main(int argc, char *argv[]) {
    func_r();
    return (0);
}
```

Pošto proces može brzo da se završi, prvo proveriti ograničenje steka i poruke kernela posle pada:

```
$ ulimit -s
$ ./lab013
$ dmesg -T | tail -n 20
```

Ako sistem generiše core dump, sledeći korak je analiza core datoteke u debageru. Ako se želi posmatranje dok proces radi, može se privremeno povećati ograničenje steka u shell-u iz kog se pokreće primer:

```
$ ulimit -s 65536
$ ./lab013 &
$ pidstat -r -p $(pgrep lab013) 1
```

Ovaj primer je dobar podsetnik da globalna memorijska slika ne mora odmah da pokaže uzrok. Lokalna ograničenja procesa, kao što je veličina steka, mogu biti dovoljna da program padne.

2 Debagovanje

Debager se koristi kada treba posmatrati konkretno izvršavanje programa: vrednosti promenljivih, stek poziva, registre, niti, memoriju i tačku na kojoj se ponašanje odvaja od očekivanog. Pre debagovanja treba imati reprodukciju problema i, ako je moguće, izvršivu verziju prevedenu sa simbolima za debagovanje.

2.1 Okruženje izvršavanja i strategija prevođenja

Pre izbora debagerskog pristupa treba znati kako program nastaje i gde se izvršava. Interpreter, JIT (Just-In-Time) i AOT (Ahead-Of-Time) okruženja različito mapiraju izvorni kod, međukod i mašinski kod. To utiče na breakpoint-e, simbole, stack trace, profajliranje, instrumentaciju i reprodukciju grešaka.
Zeleno označava povoljan aspekt, žuto kompromis ili oprez, a crveno nepovoljan aspekt za debagovanje, profajliranje ili isporuku.

Interpreter	JIT	AOT
Tumači kod tokom izvršavanja.	Tumači, zatim prevodi vruće putanje.	Prevodi pre pokretanja.
Brz razvojni ciklus.	Brz razvojni ciklus.	Sporiji ciklus zbog punog prevođenja.
Izvršava samo posećene putanje.	Kreće od bajtkoda ili međukoda.	Optimizuje ceo poznati program.
Greške se vide u runtime-u.	Runtime menja kod tokom rada.	Optimizacije mogu zakloniti uzrok.
Prenosiv uz isti interpreter.	Prenosiv uz isti runtime.	Vezan za ciljnu platformu.
Sporije pokretanje.	Sporije pokretanje i zagrevanje.	Brzo pokretanje.
Niže vršne performanse.	Visoke posle zagrevanja.	Visoke uz dobre optimizacije.
Profajl meri i interpreter.	Runtime često ima dobru podršku.	Teže zbog inlining-a i simbola.
Laka instrumentacija.	Runtime hook-ovi pomažu.	Instrumentacija pre prevođenja ili binarno.
Dinamičke osobine su prirodne.	Dinamičke osobine su prirodne.	Dinamika je obično ograničena.
Veći memorijski otisak.	Runtime, JIT kod i metapodaci.	Manji memorijski otisak.
Potreban interpreter/runtime.	Potreban runtime.	Runtime nije nužan kod statičkog povezivanja.

2.2 Kada koristiti debager

Situacija	Šta debager daje	Kada izabrati drugi alat
Pad programa koji se ponavlja	Stek poziva, argumente funkcija i stanje memorije u trenutku pada.	Ako se pad ne ponavlja lokalno, prvo prikupiti core dump ili logove.
Pogrešna vrednost u toku izvršavanja	Watchpoint, breakpoint i korak-po-korak praćenje mesta gde se vrednost menja.	Ako postoji sumnja na prekoračenje memorije, prvo uključiti ASan ili Memcheck.
Zaglavljen proces	Stekove svih niti i lokaciju čekanja, na primer muteks, `futex`, I/O ili beskonačnu petlju.	Ako treba profil vremena čekanja kroz duži period, koristiti `perf`, eBPF ili runtime profajler.
Postmortem analiza	Analizu core dump-a bez ponovnog pokretanja programa.	Ako nema simbola i tačne binarne verzije, prvo obezbediti artefakte za reprodukciju.
Interakcija sa hardverom ili mapiranom memorijom	Registre, memorijske adrese, asemblerski prikaz i uticaj optimizacija.	Ako se problem vidi samo kao sistemski događaj, prvo koristiti `strace`, `dmesg` ili eBPF.

Tipičan tok rada je: pokrenuti program sa istim ulazom, zaustaviti se pre sumnjivog mesta, proveriti pretpostavke o stanju, izvršiti sledećih nekoliko koraka i uporediti očekivano stanje sa stvarnim. Debager ne dokazuje da greška ne postoji; on objašnjava jedno konkretno izvršavanje.

2.3 gdb

GNU Debugger (gdb) je debager koji se koristi za debagovanje (najčešće) C/C++ programa. Preko gdb je moguće pokrenuti program sa proizvoljnim argumentima komandne linije, posmatrati stanje promenljivih ili registara procesora, pratiti izvršavanje kroz naredbe originalnog ili asembliranog koda, postavljanje bezuslovnih ili uslovnih tačaka prekida i sl.

Više informacija o gdb debageru se može pročitati na ovom linku.

Iako se gdb-om najčešće debaguju C/C++ programi, nije nemoguće gdb koristiti i u druge svrhe. Moguće je gdb prikačiti na bilo koji pokrenuti proces ili debagovati proizvoljne programe. To uključuje debagovanje `native` funkcija Java virtualne mašine, ili unapred (ahead of time) prevedenih Java/C# programa. Korisno debagovanje zahteva dostupne simbole za debagovanje (npr. opcija `-g` je prosleđena `gcc` kompilatoru prilikom kompilacije).

2.3.1 Buffer Overflow primer (gdb)

Program od korisnika očekuje lozinku i, ukoliko je ona ispravna, daje privilegije korisniku i pristup podacima. Bez gledanja u izvorni kod programa, probajmo da ”pogodimo” šifru, ili, još gore, da dobijemo privilegije bez pogađanja šifre.

Prvi pokušaj:

```
$ ./sample
```

```
Input password:
www
Wrong password.
$ strings sample
...
xxxxxxxxxxxxxxx
SECRET
ThisIsASecretPassword
DEBUG
DEBUG:
...
$  ./sample
Input password:
ThisIsASecretPassword
Correct password.
Password check verification passed!
Sensitive data: xxxxxxxxxxxxxxx
cat: /etc/shadow: Permission denied
```

Naravno, za svrhe ovog primera, lozinka je hardkodirana u samoj izvršnoj datoteci. Dodatno, ukoliko bismo programu dali privilegije, ispisao bi i sadržaj datoteke `/etc/shadow`.

Pošto program ima polje za unos teksta, očekujemo da u programu postoji niz karaktera koji će čuvati uneti tekst. Taj niz ima neku predefinisanu dužinu - šta ako je na ulazu više karaktera nego što može da stane u taj niz? Probajmo da unesemo predugačku lozinku:

```
$ ./sample
Input password:
wwww
```

```
Input password:
wwww
DEBUG: User entered: wwww
DEBUG: Password check var value: 0
Wrong password.
```

Vidimo da, ukoliko se unese pogrešna lozinka, promenljiva za proveru lozinke koja se koristi u uslovu nije ispravno postavljena:

```
$ DEBUG= ./sample
DEBUG: Address of user input array:     0x7ffd1183dda0
DEBUG: Address of password check var:   0x7ffd1183ddbc
DEBUG: Address of stored secret array:  0x564acefaa023
DEBUG: Current secret: ThisIsASecretPassword
Input password:
wwwwwwwwwwwwwwwwwwwwwwwwwwwwwwwwww
DEBUG: User entered: wwwwwwwwwwwwwwwwwwwwwwwwwwwwwwwwww
DEBUG: Password check var value: 2004318071
Wrong password.
Password check verification passed!
Sensitive data:
cat: /etc/shadow: Permission denied
```

Na osnovu ispisanih adresa, vidimo da se ta promenljiva nalazi odmah posle niza za unos. Ukoliko prekoračimo niz, prepisaćemo vrednost te promenljive. Iskoristimo `gdb` da bismo potvrdili da se to zapravo dešava (pritom sada pokrećemo program bez `DEBUG` promenljive):

```
$ gdb ./sample
```

Spisak komandi koje gdb pruža možemo dobiti komandom `help`. Za sve ove naredbe postoje i skraćene varijante - npr. za `running` možemo kucati `run` ili samo `r`. Prazna komanda ponavlja prethodno izvršenu komandu.

```
(gdb) help
List of classes of commands:

aliases -- User-defined aliases of other commands.
breakpoints -- Making program stop at certain points.
data -- Examining data.
files -- Specifying and examining files.
internals -- Maintenance commands.
obscure -- Obscure features.
running -- Running the program.
stack -- Examining the stack.
status -- Status inquiries.
support -- Support facilities.
text-user-interface -- TUI is the GDB text based interface.
tracepoints -- Tracing of program execution without stopping the program.
user-defined -- User-defined commands.
```

Komandom `info` možemo dobiti informacije o lokalnim promenljivim, registrima itd. (npr. `info locals` tj. `info registers`). Komandom `info breakpoints` možemo videti spisak tačaka prekida.

Alternativno, moguće je iskoristiti drugačiji korisnički interfejs, komandom `tui` npr. `tui enable`. Tako dobijamo pogled na izvorni kod na jednoj polovini korisničkog interfejsa. Komandom `tui reg` možemo dobiti i prikaz registara sa `tui reg all`.

Tačke prekida možemo postaviti komandom `breakpoints`, aktiviramo/deaktiviramo komandama `enable`/`disable`, a brišemo komandom `delete`. Komandom `continue` nastavljamo rad programa do naredne tačke prekida. Tačke prekida mogu biti linije, npr. `main.c:15` ali i funkcije, npr. `main` ili `main.c:main`. Koristeći gdb možemo dodati i uslovne tačke prekida, npr.:

```
(gdb) b main.c:36 if privileged != 0
```

Postavimo tačku prekida na funkciju `main`, i kontrolišimo izvršavanje programa naredbama `step` (nalik na *step into*) i `next` (nalik na *step over*):

```
(gdb) b main.c:main
Breakpoint 1 at 0x1192: file main.c, line 14.
(gdb) r
Starting program: ... /sample
Breakpoint 1, main () at main.c:14
14      int main() {
(gdb) s
15          char pwd[PWD_SIZE];
(gdb) s
16          int privileged = 0;
(gdb) s
17          char *sensitive_data = "xxxxxxxxxxxxxxx";
(gdb) s
19          char *secret = getenv("SECRET");
(gdb) n
20          if (secret == NULL) {
(gdb) n
21              secret = "ThisIsASecretPassword";
(gdb) n
24          if (getenv("DEBUG") != NULL) {
(gdb) n
28          dbg("Address of user input array    : %p\n", pwd);
(gdb) n
29          dbg("Address of password check var  : %p\n", &privileged);
(gdb) n
30          dbg("Address of stored secret array : %p\n", secret);
(gdb) n
31          dbg("Current secret: %s\n", secret);
(gdb) n
33          printf("Input password:\n");
(gdb) n
Input password:
34          scanf("%s", pwd);
(gdb) n
wwwwwwwwwwwwwwwwwwwwwwwwwwwwwwwwwwww
36          dbg("User entered: %s\n", pwd);
```

```
(gdb) info locals
pwd = 'w' <repeats 16 times>
privileged = 2004318071
sensitive_data = 0x555555550077 <error: Cannot access memory at address 0x555555550077>
secret = 0x555555556023 "ThisIsASecretPassword"
(gdb) p pwd
$1 = 'w' <repeats 16 times>
(gdb) p privileged
$2 = 2004318071
(gdb) p &pwd
$1 = (char (*)[16]) 0x7fffffffce80
(gdb) p &privileged
$2 = (int *) 0x7fffffffce9c
(gdb) p pwd[17]
$6 = 119 'w'
(gdb) p/x privileged
$1 = 0x77777777
(gdb) finish
```

Karakter `'w'` ima heksadecimalni kod 77, i vidimo da promenljiva `privileged` takođe ima vrednost `0x77777777` - dakle naš unos je pregazio memoriju promenljive. Jasno je da smo mogli da pažljivo odaberemo unos tako da promenljivu postavimo na koju god vrednost želimo!

Da bismo sprečili napade prekoračenjem bafera savet je da se primenjuju dobre programerske prakse i da se:

- preoverava upravljanje memorijom tokom programa koristeci neki od alata poput `valgrind memcheck`
- upotrebljava `fgets()` funkcije umesto `gets()` ili `scanf()` koje ne vrše provere granica promenljivih.
- upotrebljava `strncmp()` umesto `strcmp()`, `strncpy()` umesto `strcpy()`, itd.

2.4 QtCreator debager

QtCreator dolazi sa debagerom koji predstavlja interfejs između QtCreator-a i native debagera (gdb, CDB, LLDB, ...). Moguće je debagovati Qt aplikacije, native C/C++ aplikacije, kačiti se na već pokrenute procese i formirati sesije za debagovanje.

Više informacija je moguće pronaći u Qt dokumentaciji.

2.4.1 Debagovanje spoljašnje aplikacije kroz QtCreator

Ukoliko pokrenemo prethodni primer izvan QtCreator-a, to ne znači da ne možemo debagovati taj program. Moguće je QtCreator debager zakačiti za bilo koji program koji se izvršava ili će se tek izvršiti. Iz menija `Debug` biramo `Start Debugging`. Na raspolaganju su nam opcije:

- *Attach debugger to started application* - da bismo debagovali aplikaciju koja je već pokrenuta.
- *Attach debugger to unstarted application* - da bismo debagovali aplikaciju koja će biti pokrenuta. Zadaje se putanja do izvršne verzije programa. Debager će se aktivirati kada aplikacija bude pokrenuta.

- *Start and Debug External Application* - slično kao prethodno samo što će se program pokrenuti odmah. Možemo čekirati opciju da se doda tačka prekida na početak `main` funkcije.

Napomena: Ako QtCreator ne uspe da se poveže na proces, moguće je da treba dozvoliti povezivanje na procese tako što se upiše `0` u datoteku `/proc/sys/kernel/yama/ptrace_scope`.

Da bismo imali informacije o linijama izvornog koda koji odgovara programu koji se debaguje, potrebno je da taj program bude preveden sa simbolima za debagovanje (za `gcc` to je opcija `-g`). Inače, debager će nam prikazivati memoriju kojoj pristupa program.

2.4.2 Buffer Overflow primer (QtCreator)

Ukoliko debagujemo Qt aplikacije, dovoljno je pokrenuti aplikaciju u Debug modu. Pokrenuti QtCreator i otvoriti `BufferOverflow.pro` projekat. Prilikom otvaranja projekta, QtCreator će otvoriti konfiguracioni dijalog - konfigurisati projekat sa podrazumevanim podešavanjima.

Napomena: QtCreator podrazumevano uključuje `clang` statički analizator ukoliko je on dostupan. Iako nije neophodno, ukoliko želite da sami pronađete probleme u izvornom kodu a tek potom proverite svoje pronalaske koristeći `clang`, isključite `clang` analizator u podešavanjima (`Edit -> Preferences -> Analyzer`).

Unutar QtCreator-a vidimo nekoliko pogleda (`Edit`, `Design`, `Debug` itd.). Trebalo bi da je `Edit` pogled već selektovan. Otvoriti datoteku `main.c` i prodiskutovati o logici programa i njegovim slabim tačkama.

Unutar `Project` prozora možemo dodavati ili menjati postojeće konfiguracije za kompilaciju i pokretanje programa. To uključuje i dodavanje argumenata komandne linije (`Run` odeljak).

Pokrenuti sesiju za debagovanje. Isprobati akcije za kontrolisanje izvršavanja programa (`Debug` meni u glavnom meniju):

- `Interrupt`
- `Continue`
- `Step over`
- `Step into`
- `Step out`
- `Set/Remove breakpoint`

2.4.3 Raspored struktura i padding

C kompajler ne mora da smesti polja strukture jedno za drugim bez praznog prostora. Zbog poravnanja podataka ubacuje *padding* bajtove između polja i na kraj strukture. To je čest izvor grešaka kada se strukture upisuju u datoteku, šalju preko mreže ili mapiraju preko memorije koju nije napravio isti program.

Primer prikazuje tri strukture sa istim poljima u različitom redosledu:

```
#include <stdio.h>
#include <stdlib.h>

/*
 * Rules:
    char and char[] are special, could be any memory address, so they don't need
```

```c
 padding before them. E.g., on many systems, if struct's largest member is int
 then by divisible by 4, if short then by 2.
 */
int main() {
    struct A {
        // Before each individual member, there will
        // be padding so that to make it start at an
        // address that is divisible by its alignment
        // requirement (except char and char[]).

        int x; // sizeof(int) = 4

        // 4 byte padding
        double z; // sizeof(double) = 8

        short int y; // sizeof(short int) = 2

        // The size of whole struct will be padded to
        // a size divisible by strictest alignment
        // requirement of any of its members.

        // 4 + 4 + 8 + 2 = 18
        // strictest alignment is double (8)
        // so next number divisible by 8 is 24
        // hence:

        // 6 byte padding
    };
    struct A a = {.x = 10, .z = 20.5, .y = 30};
    printf("Size of A: %ld\n", sizeof(struct A));
    printf("A layout: &a=%p &a.x=%p &a.z=%p &a.y=%p\n", (void *)&a,
           (void *)&a.x, (void *)&a.z, (void *)&a.y);

    struct B {
        double z; // sizeof(double) = 8

        int x; // sizeof(int) = 4

        short int y; // sizeof(short int) = 2
                     // 2 byte padding
    };
    struct B b = {.z = 20.5, .x = 10, .y = 30};
    printf("Size of B: %ld\n", sizeof(struct B));
    printf("B layout: &b=%p &b.z=%p &b.x=%p &b.y=%p\n", (void *)&b,
           (void *)&b.z, (void *)&b.x, (void *)&b.y);

    struct C {
        double z; // sizeof(double) = 8

        short int y; // sizeof(short int) = 2
                     // 2 byte padding
        int x;       // sizeof(int) = 4
    };
    struct C c = {.z = 20.5, .y = 30, .x = 10};
    printf("Size of C: %ld\n", sizeof(struct C));
    printf("C layout: &c=%p &c.z=%p &c.y=%p &c.x=%p\n", (void *)&c,
```

```
           (void *)&c.z, (void *)&c.y, (void *)&c.x);

    // Note: It is possible to disable padding, via
    // a process called "packing", which has to be
    // explicitly requested, e.g., with GCC attrs:
    // struct __attribute__((__packed__)) mystruct_A {
    //     char a;
    //     int b;
    //     char c;
    // };
    // ^ this struct would have size 6

    return 0;
}
```

Prevedimo i pokrenimo program:

```
$ make
$ ./sample
```

Tipičan rezultat je da `struct A` zauzima više bajtova od struktura u kojima su veća polja stavljena pre manjih polja.

U gdb-u možemo posmatrati i tip i konkretne adrese polja:

```
(gdb) break main
(gdb) run
(gdb) next
(gdb) next
(gdb) ptype struct A
(gdb) p sizeof(struct A)
(gdb) p &a
(gdb) p &a.x
(gdb) p &a.z
(gdb) p &a.y
```

Komanda `ptype` prikazuje definiciju tipa koju je `gdb` pročitao iz simbola za debagovanje. Komande `p &a.x`, `p &a.z` i `p &a.y` pokazuju stvarne adrese polja u jednoj instanci. Razlika između adresa objašnjava gde su prazni bajtovi.

Za binarni prikaz memorije koristimo komandu `x`:

```
(gdb) x/24xb &a
(gdb) x/24xb &b
(gdb) x/24xb &c
```

Format `x/24xb` znači: ispiši 24 jedinice, u heksadecimalnom formatu, po jedan bajt. Kod malih celih vrednosti lako se vidi gde se nalaze polja, a gde su bajtovi koje program nije eksplicitno postavio.

Zaključak vežbe nije da uvek treba ručno prepakovati strukture. Redosled polja utiče na veličinu objekata i lokalnost podataka, ali ABI, kompatibilnost formata i čitljivost koda mogu biti važniji od nekoliko bajtova. `gdb` nam ovde služi kao alat za proveru pretpostavke o stvarnom rasporedu podataka.

2.4.4 Memorijski mapovan uređaj i optimizacije

Neke greške se vide tek kada program komunicira sa memorijom čiju vrednost može da promeni neko van običnog toka C programa: hardver, kernel, drugi proces ili signal handler. Primer ispod simulira uređaj kroz datoteku `dev.bin` mapiranu u memoriju.

```c
#include <assert.h>
#include <fcntl.h>
#include <stddef.h>
#include <stdint.h>
#include <stdio.h>
#include <sys/mman.h>

typedef struct {
    int32_t command;
    int32_t data;
    int8_t isBusy;
} MyHardwareGadget;

void SendCommand(MyHardwareGadget *gadget, int command, int data) {
    // wait while the gadget is busy:
    while (gadget->isBusy) {
    }

    // set data first before starting the action
    gadget->data = data;

    // writing the command starts the action
    gadget->command = command;
}

int main() {
    int fd = open("dev.bin", O_RDWR);
    assert(fd >= 0);

    MyHardwareGadget *gadget =
        (MyHardwareGadget *)mmap(NULL, sizeof(MyHardwareGadget),
                                 PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
    assert(gadget != MAP_FAILED);

    printf("dev->cmd : %d\ndev->data: %d\ndev->busy: %d\n", gadget->command,
           gadget->data, gadget->isBusy);

    SendCommand(gadget, 0x41, 0x42);

    munmap(gadget, sizeof(MyHardwareGadget));
}
```

Program čeka da uređaj više nije zauzet, zatim upisuje podatak i komandu. Ako kompajler zaključi da se polje `isBusy` ne menja unutar petlje, optimizovana verzija može da ukloni ponovno čitanje tog polja. Popravljena verzija označava pokazivač kao `volatile`:

```c
#include <assert.h>
```

```c
#include <fcntl.h>
#include <stddef.h>
#include <stdint.h>
#include <stdio.h>
#include <sys/mman.h>

typedef struct {
    int32_t command;
    int32_t data;
    int8_t isBusy;
} MyHardwareGadget;

// declare `gadget` as volatile
void SendCommand(volatile MyHardwareGadget *gadget, int command, int data) {
    // now this loop will not be optimized-out!
    while (gadget->isBusy) {
    }

    // the order of these statements is guaranteed!
    gadget->data = data;
    gadget->command = command;
}

int main() {
    int fd = open("dev.bin", O_RDWR);
    assert(fd >= 0);

    MyHardwareGadget *gadget =
        (MyHardwareGadget *)mmap(NULL, sizeof(MyHardwareGadget),
                                 PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
    assert(gadget != MAP_FAILED);

    printf("dev->cmd : %d\ndev->data: %d\ndev->busy: %d\n", gadget->command,
           gadget->data, gadget->isBusy);

    SendCommand(gadget, 0x41, 0x42);

    munmap(gadget, sizeof(MyHardwareGadget));
}
```

Prevedimo obe verzije:

```
$ make
```

Ako je datoteka uređaja u stanju “zauzet”, pokretanje loše verzije može izgledati kao običan beskonačan rad:

```
$ ./make_device_ready
$ ./sample
```

U drugom terminalu prikačimo `gdb` na proces ili ga pokrenimo direktno:

```
(gdb) break SendCommand
(gdb) run
(gdb) disassemble /m SendCommand
(gdb) info registers
```

Komanda `disassemble /m` spaja izvorni kod i instrukcije. Kod optimizovanog programa to je često važnije od linijskog koraka kroz C kod: linije mogu biti preuređene, spojene ili uklonjene.

Zatim uporedimo popravljenu verziju:

```
(gdb) file ./sample-fixed
(gdb) break SendCommand
(gdb) run
(gdb) disassemble /m SendCommand
```

U popravljanoj verziji očekujemo da se polje `isBusy` ponovo čita iz memorije tokom petlje. To ne znači da `volatile` rešava sinhronizaciju između niti. Ovde se koristi za komunikaciju sa memorijom čija promena nije vidljiva kroz obična pravila optimizacije C programa.

Realna upotreba ovog obrasca je debagovanje drajvera u korisničkom prostoru, rada sa `mmap` interfejsom, deljene memorije i grešaka koje se pojavljuju samo u optimizovanim buildovima. U takvim slučajevima treba gledati i C promenljive i generisane instrukcije.

2.4.5 Postmortem analiza iz core dump-a

Kada program padne van debagera, nije uvek moguće ponoviti problem odmah. *Core dump* čuva sliku procesa u trenutku pada: stekove niti, registre i delove memorije. To je osnovni format za postmortem analizu crash-eva.

Primer ima namernu grešku: funkcija `find_user` može da vrati `NULL`, ali pozivalac to ne proverava.

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct User {
    int id;
    char name[32];
    struct User *next;
} User;

static User *make_user(int id, const char *name, User *next) {
    User *user = malloc(sizeof(*user));
    if (user == NULL) {
        perror("malloc");
        exit(EXIT_FAILURE);
    }

    user->id = id;
    snprintf(user->name, sizeof(user->name), "%s", name);
    user->next = next;
```

```
    return user;
}

static User *find_user(User *head, int id) {
    for (User *it = head; it != NULL; it = it->next) {
        if (it->id == id) {
            return it;
        }
    }

    return NULL;
}

static void print_badge(User *user) {
    printf("user #%d: %s\n", user->id, user->name);
}

int main(int argc, char **argv) {
    int wanted = 2;
    if (argc > 1) {
        wanted = atoi(argv[1]);
    }

    User *users = make_user(1, "ana", make_user(2, "marko", NULL));
    User *selected = find_user(users, wanted);

    print_badge(selected);

    free(users->next);
    free(users);
    return 0;
}
```

Prevodimo program sa simbolima za debagovanje:

```
$ make
```

Na sistemima koji dozvoljavaju core datoteke, jedan tok rada je:

```
$ ulimit -c unlimited
$ ./crash 42
Segmentation fault (core dumped)
$ gdb ./crash core
```

Na distribucijama sa `systemd-coredump`, core datoteka često ne ostaje u radnom direktorijumu. Tada koristimo:

```
$ coredumpctl list crash
$ coredumpctl gdb crash
```

U `gdb`-u prvo tražimo gde se program zaustavio:

```
(gdb) bt
(gdb) bt full
(gdb) frame 0
(gdb) info locals
(gdb) p user
(gdb) frame 1
(gdb) p selected
(gdb) p wanted
```

`bt` prikazuje stek poziva, a `bt full` dodaje lokalne promenljive za svaki okvir. U ovom primeru je neposredni pad u `print_badge`, ali stvarni uzrok je prethodni okvir: `selected` je `NULL` jer traženi korisnik ne postoji.

Postmortem analiza dobro odgovara realnim incidentima: servis padne, sistem sačuva core, a analiza se radi kasnije na razvojnoj mašini sa istom binarnom verzijom i simbolima za debagovanje. Ključna disciplina je da se čuva tačna verzija izvršne datoteke, biblioteka i simbola koji odgovaraju core datoteci.

2.4.6 Watchpoint za pronalaženje korupcije memorije

Tačka prekida zaustavlja program kada izvršavanje dođe do određene linije ili funkcije. *Watchpoint* zaustavlja program kada se promeni određena memorijska lokacija. To je korisno kada znamo koja promenljiva dobija pogrešnu vrednost, ali ne znamo ko ju je prepisao.

Primer ima ručno napisano kopiranje stringa bez provere granice:

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    char name[8];
    int is_admin;
    int balance;
} Account;

static void copy_name(Account *account, const char *input) {
    for (int i = 0; input[i] != '\0'; i++) {
        account->name[i] = input[i];
    }
}

static void print_account(const Account *account) {
    printf("name=%s admin=%d balance=%d\n", account->name, account->is_admin,
           account->balance);
}

int main(int argc, char **argv) {
    const char *name = argc > 1 ? argv[1] : "petar";
    Account account = {.name = "", .is_admin = 0, .balance = 100};

    copy_name(&account, name);
    print_account(&account);
```

```
    if (account.is_admin) {
        puts("administrative path enabled");
    }

    return 0;
}
```

Prevedimo program:

```
$ make
$ ./watchpoint_demo petar
$ ./watchpoint_demo AAAAAAAAAAAA
```

Drugo pokretanje može promeniti polje `is_admin`, jer se ulaz upisuje preko kraja niza `name`. U `gdb`-u prvo zaustavljamo program posle inicijalizacije strukture, a zatim postavljamo watchpoint:

```
(gdb) break watchpoint_demo.c:23
(gdb) run AAAAAAAAAAAA
(gdb) watch account.is_admin
(gdb) continue
```

Kada se polje promeni, `gdb` ispisuje staru i novu vrednost i zaustavlja program na instrukciji koja je izvršila upis. Zatim proveravamo kontekst:

```
(gdb) bt
(gdb) frame 0
(gdb) info locals
(gdb) p i
(gdb) p account
```

Očekivani zaključak je da problem nije u kodu koji čita `is_admin`, nego u funkciji `copy_name`. Kada brojač `i` pređe veličinu niza, sledeći upisi prelaze u naredna polja strukture.

Hardverski watchpoint-i su ograničen resurs. Procesor obično može da prati mali broj adresa istovremeno. Ipak, kada je simptom korupcija jedne promenljive, ovo je često brže od ručnog pregleda celog toka programa.

2.4.7 Zaglavljene niti i deadlock

Kod višedretvenih programa simptom često nije pad, već proces koji ne završava. `gdb` je tada koristan jer možemo zaustaviti proces i pogledati gde čeka svaka nit.

Primer namerno zaključava dva muteksa u različitom redosledu:

```
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>

static pthread_mutex_t left = PTHREAD_MUTEX_INITIALIZER;
static pthread_mutex_t right = PTHREAD_MUTEX_INITIALIZER;
```

```c
static void *worker_a(void *arg) {
    (void)arg;
    pthread_mutex_lock(&left);
    sleep(1);
    pthread_mutex_lock(&right);
    puts("worker_a entered critical section");
    pthread_mutex_unlock(&right);
    pthread_mutex_unlock(&left);
    return NULL;
}

static void *worker_b(void *arg) {
    (void)arg;
    pthread_mutex_lock(&right);
    sleep(1);
    pthread_mutex_lock(&left);
    puts("worker_b entered critical section");
    pthread_mutex_unlock(&left);
    pthread_mutex_unlock(&right);
    return NULL;
}

int main(void) {
    pthread_t a;
    pthread_t b;

    pthread_create(&a, NULL, worker_a, NULL);
    pthread_create(&b, NULL, worker_b, NULL);

    pthread_join(a, NULL);
    pthread_join(b, NULL);
    return 0;
}
```

Pokrenimo program iz gdb-a:

```
$ make
$ gdb ./deadlock
```

Program će se najčešće zaglaviti posle kratkog čekanja. U gdb-u ga prekidamo kombinacijom `Ctrl-C`, zatim pregledamo niti:

```
(gdb) run
^C
(gdb) info threads
(gdb) thread apply all bt
```

Komanda `thread apply all bt` prikazuje stek svake niti. U ovom primeru jedna nit čeka zaključavanje `right`, a druga čeka zaključavanje `left`. Pošto svaka već drži drugi muteks, nijedna ne može da nastavi.

Za detaljniju analizu biramo konkretne niti:

```
(gdb) thread 2
(gdb) bt
(gdb) frame 1
(gdb) info locals
(gdb) thread 3
(gdb) bt
```

Kod realnih programa stekovi obično vode do viših funkcija aplikacije, ne samo do `pthread_mutex_lock`. Zato se zaključak izvodi iz kombinacije: koji resurs se čeka, koja nit ga drži i kojim redosledom se resursi zaključavaju.

Popravka je dizajnersko pravilo, ne samo promena u debageru: svi delovi programa moraju zaključavati deljene resurse u istom globalnom redosledu ili koristiti viši mehanizam koji taj redosled nameće.

2.4.8 Prikačivanje na pokrenut proces

U realnom radu problem se često pojavi u procesu koji je već pokrenut. `gdb` može da se prikači na takav proces, zaustavi ga, pročita stanje i zatim se odvoji bez namernog gašenja procesa.

Primer simulira mali servis koji periodično obrađuje zahteve:

```c
#include <signal.h>
#include <stdio.h>
#include <unistd.h>

static volatile sig_atomic_t running = 1;
static unsigned long requests = 0;
static unsigned long slow_path = 0;

static void stop(int signal) {
    (void)signal;
    running = 0;
}

static void handle_request(void) {
    requests++;
    if (requests % 10 == 0) {
        slow_path++;
        usleep(250000);
    } else {
        usleep(50000);
    }
}

int main(void) {
    signal(SIGINT, stop);
    signal(SIGTERM, stop);

    printf("pid=%ld\n", (long)getpid());
    fflush(stdout);

    while (running) {
```

```
        handle_request();
    }

    printf("requests=%lu slow_path=%lu\n", requests, slow_path);
    return 0;
}
```

U prvom terminalu pokrenemo servis:

```
$ make
$ ./service
pid=12345
```

U drugom terminalu se prikačimo na PID koji je program ispisao:

```
$ gdb -p 12345
```

Proces se zaustavlja dok je `gdb` prikačen. Tada možemo pročitati stek i globalno stanje:

```
(gdb) bt
(gdb) p requests
(gdb) p slow_path
(gdb) info threads
(gdb) continue
^C
(gdb) p requests
(gdb) detach
(gdb) quit
```

Ovaj tok je koristan za procese koji troše CPU, vise u čekanju ili imaju stanje koje je teško reprodukovati od početka. Ipak, prikačivanje nije neutralno: proces je zaustavljen dok ga pregledamo, što može izazvati timeout-e ili promeniti ponašanje sistema.

Na Linux sistemima prikačivanje može biti ograničeno podešavanjem `/proc/sys/kernel/yama/ptrace_scope`. Ako se dobije greška `Operation not permitted`, proveriti vlasništvo procesa, capability-je i to podešavanje. Na produkcionim sistemima odluka da se prikači debager mora biti deo operativne procedure, jer debager ima pun pristup memoriji procesa.

2.5 Debagovanje Java aplikacija

2.5.1 Primer: Sudoku aplikacija

Ovaj primer sadrži konzolnu Java aplikaciju za igru Sudoku i jedan JUnit test koji zamenjuje standardni ulaz i izlaz. Primer je pogodan za vežbanje debagovanja aplikacije koja čita komande od korisnika, jer se ista putanja može pokrenuti ručno ili kroz test.

Ulazna tačka aplikacije je klasa `com.company.Main`. Metod `play` prikazuje meni, učitava izbor korisnika i poziva klase koje menjaju stanje table. Za debagovanje je korisno postaviti tačke prekida na mestima gde se učitava izbor težine, unosi polje i proverava rezultat poteza.

JUnit test priprema veštački ulaz preko `ByteArrayInputStream` i hvata izlaz preko `ByteArrayOutputStream`. Time se interaktivna aplikacija pretvara u ponovljiv test slučaj.

```java
package test;

import com.company.Main;
import com.company.SudokuWrapper;
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

import java.io.ByteArrayInputStream;
import java.io.ByteArrayOutputStream;
import java.io.InputStream;
import java.io.PrintStream;

import static com.company.UserChoosingDifficulty.difficulty;
import static org.junit.jupiter.api.Assertions.*;

class MainTest {
    private final InputStream originalSystemIn = System.in;
    private final PrintStream originalSystemOut = System.out;
    private ByteArrayInputStream testIn;
    private ByteArrayOutputStream testOut;

    @BeforeEach
    public void setUp() {
        testOut = new ByteArrayOutputStream();
        System.setOut(new PrintStream(testOut));
    }

    @AfterEach
    public void restoreSystem() {
        System.setIn(originalSystemIn);
        System.setOut(originalSystemOut);
    }

    public void setUpInput(String input) {
        testIn = new ByteArrayInputStream(input.getBytes());
        System.setIn(testIn);
    }

    @Test
    public void testDifficultyEasy() {
        // Setup fake input
        setUpInput("1\n");

        // Act
        int result = difficulty(new SudokuWrapper(null));

        // Assert
        assertEquals(1, result);
    }

    @Test
    public void testPlay(){
        // Setup fake input
```

```
        String simulatedInput="1\r\n1\r\nA9\r\n7\r\n";
        setUpInput(simulatedInput);

        // Act
        Main.main(null);

        // Assert
        assertTrue(testOut.toString().contains("Successfully added 7 to A9"));
    }

}
```

Tipičan tok vežbe:

- pokrenuti test `testPlay` iz IDE okruženja u debug režimu;
- pratiti vrednosti `usrChoice`, `location` i `parseAddReturn`;
- ući u poziv `parseAdd` i proveriti kako se tekstualna oznaka polja prevodi u koordinate table;
- promeniti simulirani ulaz u testu i proveriti kako aplikacija reaguje na neispravnu komandu.

Pošto test menja globalne tokove `System.in` i `System.out`, metode `setUp` i `restoreSystem` moraju da vrate originalno stanje posle svakog testa. U suprotnom se greška može preneti na sledeći test.

2.6 Instrumentovanje Java bajtkoda - Java agenti

Java agenti su specijalni tip klase koji koristi Java Instrumentation API da modifikuje bajtkod aplikacija koje rade na JVM. Java agenti postoje od ranih verzija Java jezika (tačnije, od verzije 5). Instrumentovanje koda je korisno jer ne zahteva modifikovanje originalnog koda, stoga se agenti mogu "uključiti" u već postojeće aplikacije - to se radi zadavanjem odgovarajućih argumenata u komandnoj liniji prilikom pozivanja aplikacije. Jedan primer agenta može biti i debager - na kraju krajeva, debager treba da posmatra bajtkod i unosi instrukcije za praćenje rada aplikacije, za šta su agenti i namenjeni. Drugi primer upotrebe agenta može biti praćenje pokrivenosti, što je upravo način rada popularnih biblioteka za praćenje pokrivenosti kao npr. JaCoCo.

Agent je zapravo normalna Java klasa koja ima specijalni metod `premain`, sa jednim od potpisa:

```
public static void premain(String agentArgs, Instrumentation inst)
public static void premain(String agentArgs)
```

Kada se JVM inicijalizuje, poziva `premain` metod svakog priključenog agenta. Nakon što se svi `premain` metodi izvrše, poziva se `main` metod same aplikacije.

Agenti alternativno mogu da imaju metod `agentmain` umesto metoda `premain`, sa jednim od potpisa:

```
public static void agentmain(String agentArgs, Instrumentation inst)
public static void agentmain(String agentArgs)
```

Metod `agentmain` se poziva nakon JVM inicijalizacije, i *nakon* pokretanja originalne aplikacije. Agent može imati i `premain` i `agentmain`, ali u tom slučaju će se pozvati samo `premain`.

2.6.1 Primer

Kao primer, data je jednostavna Maven aplikacija. Pokrenimo je:

```
$ mvn package
[...]
$ java -jar ./prog-test/target/prog-test-0.1-SNAPSHOT.jar
Original method
```

U primeru je dat agent koji modifikuje bajtkod jedne metode date aplikacije. Da bismo priključili agenta, pozovimo opet aplikaciju, ali ovaj put sa dodatnim JVM argumentom `-javaagent`:

```
-javaagent:<putanja_do_agenta>[=<argumenti_agenta>]`
```

Kao putanju prosleđujemo generisani JAR agenta, koji je dobijen kao artefakt izgradnje aplikacije Maven-om. Kao argument prosleđujemo putanju do trenutnog direktorijuma pošto agentu treba lokacija klase kako bi promenio njen bajtkod.

```
$ java -javaagent:./agent/target/agent-0.1-SNAPSHOT.jar=$PWD -jar
↪  ./prog-test/target/prog-test-0.1-SNAPSHOT.jar
Method hacked
```

Vidimo da je agent uspešno modifikovao bajtkod metoda pre izvršavanja aplikacije. Modifikacija je urađena korišćenjem biblioteke Javassist.

Agent takođe ima i metod `agentmain`, ali on se ne izvršava jer agent ima i `premain` koji ima veći prioritet.

3 Pisanje testabilnog koda

Da bismo efikasno pisali testove jedinica koda, neophodno je da kod pišemo tako da bude pogodan za testiranje, ali i da softver razvijamo tako da je moguće paralelno pisati testove. Jedan način da se obezbedi takav način rada je razvoj vođen testovima (engl. *Test-Driven-Development*, skr. *TDD*). U situacijama kada već imamo dostupan izvorni kod i treba da napišemo testove jedinica koda, treba znati kako pristupiti pisanju testova i kako refaktorisati kod tako da bude pogodan za testiranje. Nekada funkcije koje treba da testiramo nemaju deterministički izlaz tako da ih je nemoguće testirati u izvornom obliku.

U ovom poglavlju će biti reči o tehnikama za pisanje testabilnog i skalabilnog koda. Primeri na kojima će ove tehnike biti prikazane su pisani u raznim programskim jezicima ali su koncepti koje prikazuju univerzalni, kao npr. inverzija zavisnosti, zamena implementacije itd.

3.1 C++ primer - kalkulator

Naš kod za kalkulator je nepodesan za jedinično testiranje jer se mnogo posla zapravo obavlja u main funkciji. Da bismo sve delove testirali moramo kod izdeliti na funkcije koje obavljaju po jednu celinu. Već postoje funkcije za svaku pojedinačnu operaciju koje imaju dva argumenta, njih nećemo dirati.

U `main` funkciji test podaci se učitavaju sa standardnog ulaza i sve se ispisuje na standardni izlaz. Da bismo mogli u jediničnim testovima da zadajemo svoje ulaze, promenićemo da funkcije ne učitavaju sa `std::cin` već sa proizvoljnog toka tipa `std::istream`, i da ispisuju ne na `std::cout`, već na proizvoljni tok tipa `std::ostream`. Na taj način ćemo izbeći učitavanje sa standardnog ulaza, a moći ćemo da unapred zadamo ulaz na kom se testira.

Iz tog razloga već postojećoj funkciji `showChoices` treba dodati argument `std::ostream & ostr` na koji će ispisivati ponudene opcije umesto na `std::cout`. Vidimo da se u `main` funkciji nakon prikaza opcije očekuje unos izbora operacije i proverava ispravosti unetog podatka. Taj deo objedinjen izdvajamo u posebnu funkciju `readChoice` koja će pozivati funkciju `showChoices` i učitavati opciju sa ulaznog toka sve dok se ne unese jedna od ispravnih cifara.

Nakon toga bi trebalo da se unesu dva operanda nad kojima će se primeniti izabrana operacija. To se može izdvojiti u posebnu funkciju `readOperands` koja će učitati dva realna broja sa ulaznog toka. Primenu odabrane operacije na unete operande na osnovu izabrane opcije izdvajamo u posebnu funkciju `calculate`. U njoj ćemo proveriti da li je opcija validna, primeniti operaciju i vratiti rezultat. Ukoliko se ne pošalje dobra vrednost argumenta `choice` funkcija bi, npr., mogla da baci izuzetak tipa `std::invalid_argument` ili vrati vrednost `false`. Proveru možemo uraditi i sa `assert(choice >= 1 && choice <= 4);` iz `cassert` zaglavlja. Ukoliko se ne pošalje očekivana vrednost za argument `choice` program će biti prekinut. Ovo nam je način da nametnemo važenje preduslova za funkciju. Istu proveru bismo mogli da stavimo i u `printResults` funkciju. Dobili smo funkcije koje rade nezavisne poslove. Time su potencijalno upotrebljive i za neku dalju upotrebu ili pisanje kompleksnijeg kalkulatora. Sada i `main` funkcija može izgledati jednostavno kao niz poziva ovih funkcija.

3.2 Java OOP primer - Game of Life

Program predstavlja implementaciju poznate ćelijske automatizacije pod imenom Game of Life. Početna konfiguracija igre (početno stanje table kao i veličina mreže) su konfigurabilni. Stanje

igre se ispisuje na standardni izlaz nakon svake iteracije.

Da bismo pisali testove jedinica koda, moramo razumeti odnose klasa u okviru aplikacije:

- paket `app` sadrži implementaciju `GameOfLife` igre
- paket `model` sadrži implementacije klasa `Cell` i `Grid`, koje se koriste za reprezentaciju stanja igre
    - paket `model.conf` sadrži implementacije početnih stanja igre (nasumičnu i jednu unapred kreiranu sa specifičnim osobinama, pod nazivom `Glider`)

Testovi koje bismo voleli da napišemo bi testirali:

- da li se mreža ispravno kreira na osnovu konfiguracije
- da li se pravila igre ispravno primenjuju iz generacije u generaciju
- da li se stanje igre ispravno ispisuje na standardni izlaz

Testove da li se mreža ispravno kreira na osnovu konfiguracije možemo lako napraviti. `Grid` klasa već implementira sve neophodno. Jedini problem predstavlja to što nemamo način da ručno postavimo konfiguraciju koja će se koristiti kao početna. Možemo testirati nasumičnu konfiguraciju međutim radije bismo da imamo deterministične testove. Takođe, naši testovi bi idealno testirali konfiguraciju u specijalnim slučajevima, npr. ivice mreže, što ne možemo kontrolisati nasumičnom konfiguracijom. Možemo ručno promeniti stanje klase `Grid` nakon kreiranja ali testovi **nikada** ne treba da zalaze u detalje implementacije klasa, tj. njihova stanja. Kreiranje ručnih determinističkih konfiguracija nam omogućava i testiranje pravila igre - možemo kreirati `Grid` objekat sa specifičnom konfiguracijom namenjenom da testira određeno pravilo igre ili kombinaciju više pravila, pozivajući metod `Grid.advance()` i testirati da li su se ćelije promenile onako kako bi trebalo. Zatim možemo testirati i specijalne slučajeve kao što su ivice mreže kao i situacije u kojima se na jednu ćeliju primenjuje više pravila.

Da bismo implementirali ručne konfiguracije, primetimo da je `model.conf.GliderConfiguration` jedna implementacija ručne konfiguracije. Možemo da implementiramo naše specifične konfiguracije na sličan način, međutim imali bismo ponavljanje koda pošto bi se jedino menjao konstruktor klase. Apstrahujmo `GliderConfiguration` implementaciju - dodajemo klasu `ManualConfiguration` i menjamo klasu `GliderConfiguration` da nasleđuje klasu `ManualConfiguration`. Sada naše test konfiguracije mogu da instanciraju `ManualConfiguration` sa odgovarajućom konfiguracijom mreže, a takođe aplikacija može da se proširi dodavanjem novih predefinisanih konfiguracija.

Slično kao u `02_refactoring/01_cpp_calculator` primeru, treba apstrahovati rad sa standardnim izlazom kako bismo mogli da testiramo prikaz stanja igre. Prikaz stanja se trenutno vrši u glavnoj klasi aplikacije, što takođe nije optimalno. Dodajmo `views` paket sa implementacijom `View` interfejsa koji predstavlja apstraktnu implementaciju prikaza aplikacije. Sada možemo kreirati implementaciju koja ispisuje stanje igre na proizvoljni izlazni tok (`PrintStreamView`) odnosno `System.out` ukoliko izlazni tok nije naveden. Logiku ispisa stanja igre pomeramo iz `app.GameOfLife` u `view.ConsoleView`.

3.3 C# - REST API klijent primer

Aplikacija je primer RESTful API klijenta koji prikazuje trenutnu temperaturu i dnevnu prognozu tako što kontaktira servis OpenWeather, tačnije njegov API server (*Napomena: pokretanje*

primera zahteva ključ koji aplikacija traži u datoteci `key.txt`). Detalji funkcionalnosti ovog servisa nisu od značaja za razumevanje ovog primera. Pojednostavljeno, klijent će serveru poslati HTTP zahtev za odgovarajućim resursom (trenutna temperatura, prognoza, i sl.) i server će poslati objekat sa odgovarajućim informacijama serijalizovanim u JSON. Pogledajmo implementaciju:

- prostor imena `Common` sadrži klase koje se koriste za deserijalizaciju odgovora servera
- prostor imena `Services` sadrži klasu `WeatherService` koja će biti meta naših testova
- glavni prostor imena koristi `WeatherService` da prikaže trenutnu temperaturu i prognozu za odgovarajući upit

Da bismo pisali testove za klasu `WeatherService`, pogledajmo njen javni interfejs (to što su neke funkcije asinhrone nema uticaj na suštinu primera):

```
bool IsDisabled();
async Task<CompleteWeatherData?> GetCurrentDataAsync(string query);
async Task<Forecast?> GetForecastAsync(string query);
```

Metod `IsDisabled` može lako da se testira kreiranjem servisa bez ključa. Druge metode su teže za testiranje jer u sebi rade više poslova: kreiraju HTTP zahtev, šalju zahtev, primaju odgovor i deserijalizuju odgovor. Direktni testovi bi slali zahteve stvarnom OpenWeather servisu, pa bi bili spori i nedeterministički. Takvi testovi ne bi mogli pouzdano da provere ni ponašanje u slučaju nevalidnog ili nekompletnog odgovora servera. Lokalna instanca OpenWeather servera mogla bi da reši deo problema, ali bi testovi i dalje zavisili od dodatnog procesa. Zato ćemo odgovor servera ubrizgati kroz apstrakciju za HTTP komunikaciju. Trenutna implementacija to ne dozvoljava, pa je potrebno da je refaktorišemo.

Pre svega, preimenujmo `WeatherService` u `OpenWeatherService` i apstrahujmo interfejs ove klase `IWeatherService`. Kontaktiranje servera možemo apstrahovati u logiku klase `WeatherHttpService`. U našim testovima, možemo koristiti implementaciju nalik na onu u klasi `TestWeatherHttpService`. Ne želimo da `OpenWeatherService` direktno koristi `WeatherHttpService` pošto nam to ne omogućava zamenu implementacije klase `WeatherHttpService` klasom `TestWeatherHttpService` u našim testovima. Stoga kreirajmo interfejs `IWeatherHttpService` koji će implementirati klase `WeatherHttpService` i `TestWeatherHttpService`. Sada klasa `OpenWeatherService` može da ima zavisnost na interfejs u konstruktoru (što dodatno omogućava druge povoljnosti kao što je ubrizgavanje zavisnosti).

4 Testiranje jedinica koda metodom bele kutije

Jedinični testovi (engl. *Unit tests*) treba da budu kreirani za sve **javne** metode klasa, uključujući konstruktore i operatore. Trebalo bi da pokriju sve glavne putanje kroz funkcije, uključujuću različite grane uslova, petlji itd. Jedinični testovi bi trebali da pokriju i trivijalne i granične slučajeve, kao i situacije izvršavanja metoda nad pogrešnim podacima da bi se testiralo i reagovanje na greške.

4.1 C++ - QtTest, gcov, lcov

4.1.1 Pisanje testova jedinica koda pomoću QtTest radnog okvira

QtCreator kao razvojno okruženje pruža mogućnost pisanja jediničnih testova. To je moguće učiniti pisanjem projekta tipa `Qt Unit Test` na više načina, u zavisnosti od toga kako želimo da organizujemo stvarni i test projekat.

Uključivanjem biblioteke Prvi način kako možemo kreirati test projekat koji testira već postojeći projekat je tako što kreiramo novi projekat tipa `Qt Unit Test`:

```
New Project -> Other Project -> Qt Unit Test
```

Pošto želimo da pišemo testove za već postojeći projekat, treba da dodamo lokaciju projekta opcijom `Add Existing Directory`.

Unutar test projekta će se kreirati klasa koja nasleduje `QObject`. Ispod modifikatora `private Q_SLOTS` pišemo bar jednu test funkciju. Trebalo bi da su nam već ponuđene funkcije npr. `testCase1()`.

Dodatno, postoje i četiri privatne metode koje se ne tretiraju kao test funkcije, ali će ih test radni okvir izvršavati bilo kada inicijalizuje ili čisti za celim testom ili trenutnom test funkcijom:

- `initTestCase()` će biti pozvana pre izvršavanja prve test funkcije
- `cleanupTestCase()` će biti pozvana nakon izvršavanja poslednje test funkcije
- `init()` će biti pozvana pre svakog poziva test funkcije
- `cleanup()` će biti pozvana nakon svakog izvršavanja test funkcije

Ukoliko se `initTestCase()` ne izvrši uspešno, nijedna test funkcija neće biti izvršavana. Ako `init()` funkcija ne prođe, njena prateća test funkcija se neće izvršiti, ali će se nastaviti sa sledećom.

Qt Subdirs projekat Drugi način da testiramo projekat je da kreiramo nov `Qt Subdirs` projekat u koji ćemo uključiti postojeći i novi `Qt Unit Test` projekat. Ovaj način organizacije je koristan ukoliko pišemo projekat od nule i želimo da usput pišemo i testove.

4.1.2 Pisanje testova

QtTest radni okvir pruža makroe za testiranje, i neke od njih ćemo koristiti da testiramo naš kalkulator:

- `QCOMPARE`
- `QVERIFY`
- `QVERIFY_EXCEPTION_THROWN`

4.1.3 Analiza pokrivenosti

Uz `gcc` kompajler dolazi i `gcov` alat za određivanje pokrivenosti koda prilikom izvršavanja programa (engl. *code coverage*). Koristi se zajedno sa `gcc` kompajlerom da bi se analizirao program i utvrdilo kako se može kreirati efikasniji, brži kod i da bi se testovima pokrili delovi programa.

Alat `gcov` se može koristiti kao alat za profajliranje u cilju otkrivanja dela koda čija bi optimizacija najviše doprinela efikasnosti programa. Korišćenjem `gcov`-a možemo saznati koje su naredbe, linije, grane, funkcije itd. izvršene i koliko puta. Zarad lepše reprezentacije rezultata detekcije pokrivenosti koda izvršavanjem test primera, koristimo alat `lcov`.

Prilikom kompilacije neophodno je koristiti dodatne opcije kompajlera koje omogućavaju snimanje koliko je puta koja linija, grana i funkcija izvršena. Ti podaci se čuvaju u datotekama ekstenzije `.gcno` za svaku datoteku sa izvornim kodom. One će kasnije biti korišćene za kreiranje izveštaja o pokrivenosti koda.

```
$ g++ -g -Wall -fprofile-arcs -ftest-coverage -O0 main.cpp -o test
```

Alternativno:

```
g++ -g -Wall --coverage -O0 main.cpp -o test
```

Nakon izvršavanja test programa, informacije o pokrivenosti biće sačuvane u datoteci tipa `.gcda`, ponovo za svaku datoteku sa izvornim kodom. Pokrenimo alat `lcov` da bismo dobili čitljiviji prikaz rezultata:

```
$ lcov --rc lcov_branch_coverage=1 -c -d . -o coverage-test.info
```

Opcija:

- `--rc lcov_branch_coverage=1` uključuje određivanje pokrivenosti grana, koje podrazumevano nije uključeno
- `-c` kreiranje pokrivenosti
- `-d .` koristi tekući direktorijum, jer u našem slučaju on sadrži potrebne `.gcda` i `.gcno` datoteke
- `-o coverage-test.info` zadaje naziv izlazne datoteke sa izveštajem koji treba da ima ekstenziju `.info`

Možemo neke datoteke isključiti iz analize pokrivenosti. Na primer, biblioteke jezika koje ne testiramo mogu nam samo zamagliti pokrivenost koja nas zanima - pokrivenost funkcionalnosti koje testiramo.

```
$ lcov --rc lcov_branch_coverage=1 \
    -r coverage.info '/usr/*' '/opt/*' '*.moc' \
    -o coverage-filtered.info
```

Opcija `-r coverage.info` uklanja iz prethodno dobijenog izveštaja `coverage.info` izvorne datoteke koje odgovaraju nekom od šablona koji su navedeni kao argumenti opcije.

Alat `lcov` ima podalat `genhtml` koji na osnovu prethodno generisanog izveštaja pravi `.html` datoteke za jednostavniji pregled. Potrebno je izvršiti naredbu:

```
$ genhtml --rc lcov_branch_coverage=1 -o Reports coverage-filtered.info
```

Opcija `-o Reports` određuje naziv direktorijuma koji će biti kreiran i popunjen generisanim `.html` dokumentima.

Izveštaj možemo otvoriti u Web pretraživaču, npr.

```
$ firefox Reports/index.html
```

Ukoliko ne obrišemo `.gcda` datoteke od prethodnih pokretanja programa, prikaz pokrivenosti će uključiti sve, zbirno.

4.1.4 Kreiranje izveštaja uz pomoć skripte

Postupak kreiranja izveštaja nakon kompilacije programa može se automatizovati pokretanjem bash skripte `generateCodeCoverageReport.sh`:

```
$ ./generateCodeCoverageReport.sh . test data
```

Argumenti:

- `.` - Direktorijum u kom se nalaze potrebne `.gcda` i `.gcno` datoteke i izvršni program. U našem slučaju to je tekući direktorijum. Inače bismo navodili relativnu ili apsolutnu putanju do potrebnog direktorijuma.
- `test` - Drugi argument treba da je naziv izvršne verzije programa koja će se pokretati.
- `data` - Naziv direktorijuma u koji će alati `lcov` i `genhtml` upisivati svoje rezultate. Ukoliko se ne navede, sve će se upisivati u tekući direktorijum skripte. Ukoliko navedeni direktorijum ne postoji, biće kreiran.

Skript briše `.gcda` datoteke od prethodnih pokretanja programa. Prikazana pokrivenost je samo za poslednje pokretanje programa.

4.1.5 Integracija pokrivenosti u QtCreator

Ukoliko želimo da kreiramo pokrivenost koda prilikom izvršavanja postojećeg Qt projekta, potrebno je da definiciji projekta (datoteci tipa `.pro`) dopišemo:

```
QMAKE_CXXFLAGS += -g -Wall -fprofile-arcs -ftest-coverage -O0
QMAKE_LFLAGS += -g -Wall -fprofile-arcs -ftest-coverage -O0

LIBS += \
    -lgcov
```

Opcija kompajlera `--coverage` menja opcije `-fprofile-arcs -ftest-coverage` i linkovanje sa `-lgcov`. Nakon pokretanja projekta, `.gcda` i `.gcno` datoteke i izvršivi program biće u direktorijumu gde se nalaze ostali artefakti prevođenja (podrazumevano u direktorijumu sa prefiksom `build_`). Izveštaj možemo potom napraviti prema ranije prikazanom postupku.

4.2 Java - JUnit, JaCoCo

JUnit je jedan od najpopularnijih radnih okvira za testiranje jedinica koda u programskom jeziku Java. Neke od osobina JUnit radnog okvira su jednostavnost pisanja testova uz bogat skup anotacija koje opisuju testove, kao i veoma velika podrška za najčešće situacije u procesu pisanja testova kao što su uslovno uključivanje/isključivanje testova na osnovu promenljivih iz okruženja, operativnog sistema, proizvoljnih predikata itd.

JUnit svoje artefakte isporučuje na Maven Central i moguće ga je uključiti u Maven (i Maven-kompatibilne) alate za prevođenje. U ovom primeru ćemo koristiti drugi popularni alat za prevođenje pod imenom Gradle. Gradle koristi datoteku `build.gradle` (slično kao što Maven koristi `pom.xml`) za definiciju projekta, zavisnosti i podešavanja dodataka.

Kreirajmo novi projekat:

```
$ gradle init
```

Možemo u `build.gradle` dodati zavisnost za JUnit:

```
  | dependencies {
  |     ...
+ |     testImplementation "junit:junit:4.13"
  | }
```

Za praćenje pokrivenosti koda, koristićemo JaCoCo. JaCoCo možemo lako uključiti u projekat dodavanjem niske `'jacoco'` u spisak dodataka za projekat i dodatno konfigurisati izveštaj koji JaCoCo pravi:

```
  | plugins {
  |     id 'java'
+ |     id 'jacoco'
  | }
  |
```

```
  | ...
  |
+ | jacocoTestReport {
+ |    reports {
+ |        xml.required = false
+ |        csv.required = false
+ |        html.outputLocation = layout.buildDirectory.dir('jacocoHtml')
+ |    }
+ | }
+ |
+ | check.dependsOn jacocoTestReport
```

Nakon ovoga možemo iskoristiti sledeće komande:

- `gradle build` - prevođenje projekta
- `gradle test` - pokretanje testova jedinica koda
- `gradle jacocoTestReport` - kreiranje izveštaja o pokrivenosti

Izveštaji o pokrenutim testovima i statusu izvršavanja mogu se naći u direktorijumu `build/reports/tests/` u HTML formatu. JaCoCo izveštaj o pokrivenosti može se naći u direktorijumu koji smo naveli u datoteci `build.gradle` - `build/jacocoHtml`, takođe u HTML formatu.

Pregled nekih od korisnih anotacija JUnit radnog okvira:

- `@Test` - Radni okvir će pokrenuti ovaj metod automatski prilikom pokretanja testova.
- `@TestFactory` - Metod koji generiše testove u vremenu izvršavanja. Najčešće se koristi da pokrene nasumične testove ili testove bazirane na spoljnim podacima.
- `@DisplayName` - Čini izveštaje čitljivijim tako što testovima daje navedeno ime.
- `@BeforeAll`/`@BeforeEach` - Izvršava metod pre svih odnosno svakog testa.
- `@AfterAll`/`@AfterEach` - Izvršava metod posle svih odnosno svakog testa.
- `@Tag` - Dodaje oznaku testu radi kategorisanja testova u svite, npr. `@Tag("fast")` dodaje test u svitu sa oznakom `"fast"`.
- `@Disabled` - Isključuje test metod iz radnog okvira.
- `@Nested` - Koristi se u unutrašnjim klasama najčešće radi definisanja redosleda kojim se pokreću testovi.

Primeri korišćenja JUnit anotacija, test metoda i metoda pretpostavki:

```java
import org.junit.jupiter.api.*;
public class AppTest {
    @BeforeAll
    static void setup(){
        System.out.println("Executes a method Before all tests");
    }
    @BeforeEach
    void setupThis(){
        System.out.println("Executed Before each @Test method " +
                           "in the current test class");
    }
    @AfterEach
    void tearThis(){
```

```
        System.out.println("Executed After each @Test method " +
                           "in the current test class");
    }
    @AfterAll
    static void tear(){
        System.out.println("Executes a method After all tests");
    }
}
```

```
Assertions.assertAll("heading",
    () -> assertTrue(true),
    () -> assertEquals("expected", objectUnderTest.getSomething()
);
```

```
@TestFactory
Stream dynamicTests(MyContext ctx) {
    // Generates tests for every line in the file
    return Files.lines(ctx.testDataFilePath)
                .map(l -> dynamicTest("Test: " + l,
                                      () -> assertTrue(runTest(l))
                );
}
```

```
@Test
void exampleTest() {
    Assertions.assertTrue(trueBool);
    Assertions.assertFalse(falseBool);
    Assertions.assertNotNull(notNullString);
    Assertions.assertNull(notNullString);
    Assertions.assertNotSame(originalObject, otherObject);
    Assertions.assertEquals(4, 4);
    Assertions.assertNotEquals(3, 2);
    Assertions.assertArrayEquals(
        new int[] { 1, 2, 3 },
        new int[] { 1, 2, 3 },
        "Array Equal Test"
    );
    Iterable<Integer> listOne = new ArrayList<>(Arrays.asList(1,2,3,4));
    Iterable<Integer> listTwo = new ArrayList<>(Arrays.asList(1,2,3,4));
    Assertions.assertIterableEquals(listOne, listTwo);
    Assertions.assertTimeout(Duration.ofMillis(100), () -> {
        Thread.sleep(50);
        return "result";
    });
    Throwable exception = Assertions.assertThrows(
        IllegalArgumentException.class,
        () -> throw new IllegalArgumentException("error message");
    );
    Assertions.fail("not found good reason to pass");
}
```

```
@Test
```

```
void testAssumption() {
    System.setProperty("prop", "foo");
    Assumptions.assumeTrue("foo".equals(System.getProperty("prop")));
}
```

```
@Test
@EnabledForJreRange(min = JRE.JAVA_8, max = JRE.JAVA_11)
public void test1()
{
    System.out.println("Will run only on JRE between 8 and 11");
}

@Test
@EnabledOnJre({JRE.JAVA_8, JRE.JAVA_11})
public void test2()
{
    System.out.println("Will run only on JRE 8 and 11");
}

@Test
@DisabledForJreRange(min = JRE.JAVA_8, max = JRE.JAVA_11)
public void test3()
{
    System.out.println("Will NOT run on JRE between 8 and 11");
}

@Test
@DisabledOnJre({JRE.JAVA_8, JRE.JAVA_11})
public void test4()
{
    System.out.println("Will NOT run on JRE 8 and 11");
}
```

```
@Test
@EnabledOnOs({OS.LINUX, OS.WINDOWS})
void onLinuxOrWindows() {
    System.out.println("Will run on Linux or Windows.");
}

@Test
@DisabledOnOs({OS.WINDOWS, OS.SOLARIS, OS.MAC})
void notOnWindowsOrSolarisOrMac() {
    System.out.println("Won't run on Windows, Solaris or MAC!");
}
```

```
@Test
@EnabledIf("myCustomPredicate")
void enabled() {
    assertTrue(true);
}

@Test
@DisabledIf("myCustomPredicate")
```

```
void disabled() {
    assertTrue(true);
}

boolean myCustomPredicate() {
    return true;
}
```

```
@Test
@EnabledIfEnvironmentVariable(named = "ENV", matches = ".*oracle.*")
public void executeOnlyInDevEnvironment() {
    return true;
}

@Test
@DisabledIfEnvironmentVariable(named = "ENV", matches = ".*mysql.*")
public void disabledOnProdEnvironment() {
    return true;
}

@Test
@EnabledIfSystemProperty(named = "my.property", matches = "prod*")
public void onlyIfMyPropertyStartsWithProd() {
    return true;
}
```

4.3 C# (.NET) - xUnit, NUnit

U okviru .NET ekosistema postoji bogat skup radnih okvira za testiranje jedinica koda. Neki od njih olakšavaju repetitivno pisanje testova ubrizgavanjem vrednosti u šablone testova (tzv. *teorije*), ili pružaju interfejs za pisanje testova navođenjem ograničenja. Primeri ovakvih radnih okvira koje ćemo razmatrati su xUnit i NUnit. Oba radna okvira podržavaju sve jezike u okviru .NET ekosistema (C#, F#, VB.NET, ...).

4.3.1 xUnit

Osim jednostavnog interfejsa za pisanje testova nalik na JUnit u programskom jeziku Java, gde se testovi markiraju odgovarajućim anotacijama (što je i slučaj u xUnit radnom okviru markiranjem metoda atributom `[Fact]` [1]), jedna od najpopularnijih osobina xUnit radnog okvira je mogućnost pisanja *teorija* - šablona za testove. Umesto da pišemo isti skup pod-testova iznova i iznova za različite podatke (ili umesto da ih izdvajamo u funkcije), možemo zakačiti atribut [2] `[Theory]`, a unutar atributa `[InlineData]` definisati podatke koji će biti ulaz za test:

```
public class ParameterizedTests
{
```

[1] Atributi u programskom jeziku C# su donekle ekvivalentni anotacijama u programskom jeziku Java. Za razumevanje primera nije neophodno duboko poznavanje koncepta atributa.

[2] Atributi u programskom jeziku C# su donekle ekvivalentni anotacijama u programskom jeziku Java. Za razumevanje primera nije neophodno duboko poznavanje koncepta atributa.

```
    public bool SampleAssert1(int a, int b, int c, int d)
    {
        return (a + b) == (c + d);
    }

    public bool SampleAssert2(int a, int b, int c, int d)
    {
        return (a + c) == (b + d);
    }

    // Regular xUnit test case
    // Sub-optimal (repeated asserts)
    [Fact]
    public void SampleFact()
    {
        Assert.True(SampleAssert1(4, 4, 4, 4));
        Assert.True(SampleAssert2(4, 4, 4, 4));

        Assert.True(SampleAssert1(3, 2, 2, 3));
        Assert.True(SampleAssert2(3, 2, 2, 3));

        Assert.True(SampleAssert1(7, 0, 0, 7));
        Assert.True(SampleAssert2(7, 0, 0, 7));

        Assert.True(SampleAssert1(0, 7, 7, 0));
        Assert.True(SampleAssert2(0, 7, 7, 0));
    }

    // Regular xUnit test case
    // No repeated asserts but requires a local method
    [Fact]
    public void SampleFact()
    {
        PerformAsserts(4, 4, 4, 4);
        PerformAsserts(3, 2, 2, 3);
        PerformAsserts(7, 0, 0, 7);
        PerformAsserts(0, 7, 7, 0);

        void PerformAsserts(int a, int b, int c, int d)
        {
            Assert.True(SampleAssert1(a, b, c, d));
            Assert.True(SampleAssert2(a, b, c, d));
        }
    }

    // Using Theory and InlineData
    // Optimal solution, replaces above patterns
    [Theory]
    [InlineData(4, 4, 4, 4)]
    [InlineData(3, 2, 2, 3)]
    [InlineData(7, 0, 0, 7)]
    [InlineData(0, 7, 7, 0)]
    public void SampleTheory(int a, int b, int c, int d)
    {
        Assert.True(SampleAssert1(a, b, c, d));
        Assert.True(SampleAssert2(a, b, c, d));
```

```
    }

    // There exist special "InlineData" variants, for example "SqlServerData"
    [Theory]
    [SqlServerData("(local)",
                   "TestDatabase",
                   "select FirstName, LastName from Users")]
    public void SqlServerTests(string FirstName, string LastName)
    {
        Assert.Equal("Peter Beardsley", $"{FirstName} {LastName}");
    }
}
```

Druga popularna odlika xUnit radnog okvira je jednostavna izolacija test metoda. To se postiže kreiranjem zasebne instance test klase za svaki test metod. Za razliku od drugih popularnih radnih okvira, xUnit ne daje interfejs za markiranje metoda sa ciljem pokretanja tog metoda pre ili posle jednog ili svih testova, već se na osnovu izolacije testova po instanci klase, piše čitljiviji kod koji u konstruktoru i destruktoru klase vrši odgovarajuću pripremu odnosno čišćenje pre odnosno posle pokretanja testova.

Primeri korišćenja xUnit radnog okvira su preuzeti iz zvaničnog repozitorijuma sa primerima i mogu se naći kao git podmodul u okviru repozitorijuma sa materijalima.

4.3.2 NUnit

Glavna odlika NUnit radnog okvira je model ograničenja (constraint model). Takav model u radnom okviru pruža samo jedan metod za implementaciju testova. Logika potrebna za testiranje se kodira u objektu ograničenja koji se prosleđuje toj metodi:

```
Assert.That(myString, Is.EqualTo("Hello"));
Assert.That(myString, Is.Not.EqualTo("Bello"));
```

Primeri korišćenja NUnit radnog okvira su preuzeti iz zvaničnog repozitorijuma sa primerima i mogu se naći kao git podmodul u okviru repozitorijuma sa materijalima.

4.3.3 `dotnet-coverage`

Pokrivenost koda meri koji delovi programa su izvršeni tokom pokretanja testova. To nije dokaz ispravnosti programa. Korisna je kao povratna informacija: pokazuje grane koje testovi ne dodiruju i pomaže da se razgovor o nedostajućim testovima veže za konkretan kod.

Alat `dotnet-coverage` sakuplja pokrivenost za .NET proces koji pokreće zadatu komandu. U ovoj vežbi ga koristimo kao lokalni .NET alat i izvozimo rezultat u Cobertura XML format. Cobertura je pogodan za pregled u alatima za izveštavanje i CI sistemima, ali u ovoj vežbi ne postavljamo prag koji ruši proveru.

Manifest alata je deo direktorijuma vežbe:

```
{
  "version": 1,
  "isRoot": true,
```

```
  "tools": {
    "dotnet-coverage": {
      "version": "18.6.2",
      "commands": [
        "dotnet-coverage"
      ]
    }
  }
}
```

Primer koristi malu biblioteku sa granama u logici popusta. Cilj nije domen prodaje, već kratak kod nad kojim se jasno vidi razlika između prolaznih testova i dobrih testova.

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <TargetFramework>net10.0</TargetFramework>
    <LangVersion>default</LangVersion>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>

</Project>
```

```
namespace CoverageLab;

public enum CustomerKind
{
    Regular,
    Student,
    Partner
}
```

```
namespace CoverageLab;

public static class DiscountPolicy
{
    public static decimal Calculate(decimal subtotal, CustomerKind kind)
    {
        if (subtotal < 0m)
        {
            throw new ArgumentOutOfRangeException(nameof(subtotal), "Subtotal must be
            ↪  non-negative.");
        }

        decimal discountRate = kind switch
        {
            CustomerKind.Student => 0.10m,
            CustomerKind.Partner => 0.15m,
            _ => 0m
        };
```

```
        if (subtotal >= 200m)
        {
            discountRate += 0.05m;
        }

        decimal discount = decimal.Round(subtotal * discountRate, 2, MidpointRounding.AwayFromZero);
        return subtotal - discount;
    }
}
```

Prvi skup testova proverava deo ponašanja. Svi testovi prolaze, ali pokrivenost pokazuje da nisu provereni partnerski popust i obrada neispravnog međuzbira.

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <LangVersion>default</LangVersion>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>

  <ItemGroup>
    <ProjectReference Include="..\CoverageLab\CoverageLab.csproj" />
    <PackageReference Include="xunit.v3" Version="3.2.2" />
  </ItemGroup>

</Project>
```

```
using Xunit;

namespace CoverageLab.Initial.Tests;

public sealed class DiscountPolicyTests
{
    [Theory]
    [InlineData(100, CustomerKind.Regular, 100)]
    [InlineData(100, CustomerKind.Student, 90)]
    [InlineData(250, CustomerKind.Student, 212.50)]
    public void Calculate_returns_discounted_total(decimal subtotal, CustomerKind kind, decimal
    ↪  expected)
    {
        decimal actual = DiscountPolicy.Calculate(subtotal, kind);

        Assert.Equal(expected, actual);
    }
}
```

Drugi skup testova dodaje propuštene slučajeve. On ne menja produkcioni kod, već pokazuje kako ista biblioteka dobija bolju pokrivenost kada testovi eksplicitno pokriju odluke u metodi.

```xml
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <LangVersion>default</LangVersion>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>

  <ItemGroup>
    <ProjectReference Include="..\CoverageLab\CoverageLab.csproj" />
    <PackageReference Include="xunit.v3" Version="3.2.2" />
  </ItemGroup>

</Project>
```

```csharp
using Xunit;

namespace CoverageLab.Complete.Tests;

public sealed class DiscountPolicyTests
{
    [Theory]
    [InlineData(100, CustomerKind.Regular, 100)]
    [InlineData(100, CustomerKind.Student, 90)]
    [InlineData(100, CustomerKind.Partner, 85)]
    [InlineData(250, CustomerKind.Regular, 237.50)]
    [InlineData(250, CustomerKind.Student, 212.50)]
    [InlineData(250, CustomerKind.Partner, 200)]
    public void Calculate_returns_discounted_total(decimal subtotal, CustomerKind kind, decimal
    ↪  expected)
    {
        decimal actual = DiscountPolicy.Calculate(subtotal, kind);

        Assert.Equal(expected, actual);
    }

    [Fact]
    public void Calculate_rejects_negative_subtotal()
    {
        Assert.Throws<ArgumentOutOfRangeException>(() =>
            DiscountPolicy.Calculate(-1m, CustomerKind.Regular));
    }
}
```

Vežba se pokreće iz direktorijuma `04_dotnet_coverage`:

```
$ dotnet tool restore
$ dotnet restore CoverageLab.sln
$ dotnet test --solution CoverageLab.sln --no-restore
```

Komande za pokrivenost ispod koriste statičku instrumentaciju preko opcije `-if`. Instrumentiše se biblioteka koja je kopirana u izlazni direktorijum konkretnog test projekta, pa pre ovih komandi treba jednom pokrenuti `dotnet test`.

Pokrivenost početnog skupa testova:

```
$ dotnet tool run dotnet-coverage -- collect \
    "dotnet CoverageLab.Initial.Tests/bin/Debug/net10.0/CoverageLab.Initial.Tests.dll" \
    -if CoverageLab.Initial.Tests/bin/Debug/net10.0/CoverageLab.dll \
    -f cobertura \
    -o coverage-initial.cobertura.xml
```

Pokrivenost dopunjenog skupa testova:

```
$ dotnet tool run dotnet-coverage -- collect \
    "dotnet CoverageLab.Complete.Tests/bin/Debug/net10.0/CoverageLab.Complete.Tests.dll" \
    -if CoverageLab.Complete.Tests/bin/Debug/net10.0/CoverageLab.dll \
    -f cobertura \
    -o coverage-complete.cobertura.xml
```

U dobijenim XML datotekama treba uporediti vrednosti atributa `line-rate` i `branch-rate`. Bolji rezultat nije cilj sam po sebi. Ako test samo izvrši liniju bez smislenog tvrđenja, pokrivenost raste, ali kvalitet provere ne raste. Zato izveštaj treba čitati zajedno sa test kodom.

`Coverlet` je česta alternativa kada je potrebno da sakupljanje pokrivenosti bude direktno vezano za `dotnet test` i test adaptere. U ovoj vežbi koristimo `dotnet-coverage`, jer isti tok rada može da obuhvati bilo koju .NET komandu ili proces koji se pokreće iz komandne linije.

5 Testiranje pomoću objekata imitatora (engl. Mock testing)

Prilikom pisanja jediničnih testova fokusiramo se samo na jednu funkciju i ispitujemo njeno ponašanje u kontrolisanom okruženju. Sadržaj testa je uvek skup inicijalizacija okruženja, pokretanje funkcije koju testiramo i zatim poređenje dobijenog i očekivanog rezultata. Nekada je kod takav da je nemoguće napraviti kontrolisano okruženje za testiranje, npr. korišćenje sistemskih poziva, podataka iz baze podataka, mrežna komunikacija i sl. U tim situacijama pribegava se pisanju klasa koje imitiraju realne objekte u svrhu testiranja.

Na primer, funkcija može da obrađuje podatke iz datoteke i uzima naziv datoteke kao ulazni parametar. Bolje rešenje je prepraviti funkciju tako da radi sa ulaznim tokom koji treba da obradi. Funkcija potom neće raditi sve stvari kao ranije - otvarati datoteku i obrađivati podatke.

U jediničnom testu, objekti imitatori mogu da imitiraju ponašanje kompleksnog stvarnog objekta. Korisni su u situacijama kada stvarni objekat nije praktično ili je nemoguće uklopiti u jedinični test. Objekat imitator se obično koristi ukoliko stvarni objekat ima neku od narednih karakteristika:

- obezbeđuje nedeterministički rezultat (npr. trenutno vreme, trenutnu temperaturu, ...)
- ima stanja koja je teško kreirati ili reprodukovati, (npr. greška u mrežnoj komunikaciji)
- spor je (npr. baza podataka, koja bi pre svakog testa morala biti inicijalizovana)
- ne postoji još uvek, ili može promeniti ponašanje u budućnosti
- morao bi da dobije nove informacije i metode da bismo mogli da ga koristimo za testiranje, a inače mu nisu potrebne

Objekti imitatori treba da imaju isti interfejs kao stvarni objekti koje imitiraju. Tako omogućavaju da objekat koji ih koristi ne pravi razliku između stvarnog objekta i imitatora. Mnogi radni okviri za objekte imitatore omogućavaju da se navede objekat koje klase se imitira, a zatim da programer zada koji metodi se pozivaju na objektu imitatoru, kojim redom i sa kojim parametrima, kao i koja vrednost se očekuje kao povratna. Na taj način se mogu imitirati ponašanja kompleksnih objekata (npr. socket) i omogućiti da programer testira da li se objekat ponaša korektno sa svim različitim stanjima. To je jednostavniji postupak nego izazivanje svih situacija na stvarnom objektu.

Rad sa objektima imitatorima obično obuhvata sledeće korake:

- kreiranje interfejsa za klasu koju bi trebalo testirati
- kreiranje klase imitatora ručno ili pomoću nekog radnog okvira:
  - C++ - `FakeIt`, `CppUMock` (unutar `CppUnit`), `GoogleMock`
  - Java - `Mockito`, `JMock`, `EasyMock`, `PowerMock`
  - .NET - `Moq`
- priprema koda koji će se testirati na objektu imitatoru;
- pisanje testa koji će koristiti objekat imitator umesto stvarnog objekta

Unutar testa je potrebno: – kreirati instancu objekta klase imitatora – podesiti ponašanje i očekivanja od objekta imitatora – pokrenuti kod koji će koristiti objekat imitator – po izvršavanju, porediti dobijene i očekivane vrednosti (ovaj korak obično izvršava radni okvir prilikom uništavanja objekta imitatora)

5.1 C++ - ručno kreiranje objekata imitatora

Dobili smo zadatak da u igru koju razvijamo dodamo novu funkcionalnost koja meri koliko je igrač aktivno igrao igru. Vreme provedeno u glavnom meniju i pauze ne treba da se uključe u vreme igranja. U tu svrhu kreiramo jednostavnu klasu `play_time` koja ima metode za pokretanje i zaustavljanje sesije i vraća ukupno vreme igranja.

Testiranje ove jednostavne klase zahteva par koraka:

1. kreiranje instance klase `play_time`
2. započinjanje sesije
3. uspavljivanje programa na neko vreme
4. zaustavljanje sesije
5. pozivanje metoda za dobijanje ukupno vreme igranja
6. poređenje dobijene vrednosti sa vremenom uspavljivanja programa

Problem je u tome što program treba da spava neko vreme. Nije test sam po sebi problem, klasa `play_time` koja zavisi od sistemskog sata. Rešenje je da generalizujemo konstruktor klase i da eksplicitno naglasimo zavisnost klase od sistemskog sata. Sve dok `play_time` dobija trenutno vreme nekako, pravi izvor nam nije presudno bitan.

Kreirajmo interfejs `second_clock`. Menjamo konstruktor klase `play_time` tako da kao argument dobija instancu `second_clock` interfejsa. Time je svakom jasno da naša klasa zavisi od sata. Menjamo i metode za pokretanje i zaustavljanje sesije, jer sada treba da zavise od parametra klase, sata.

Kreirajmo klasu `system_clock` koja će implementirati već kreiran interfejs `second_clock`. Kada želimo da objekat klase `play_time` koristi sistemski sat, konstruktoru ćemo slati objekat klase `system_clock`.

Kreirajmo sada klasu imitatora `mock_clock` koja će da odgovara ponašanju sata bez baterija. Uvek će pokazivati podešeno vreme. Implementiramo konstruktor, metode `get` i `set` za postavljanje vremena.

Prilikom testiranja, koristimo instancu `mock_clock` prilikom konstrukcije instance klase `play_time`. Umesto da uspavamo program, pomerićemo vreme na satu za neki interval i očekujemo da isti interval vrati i metoda `played_time`.

5.2 C# - Moq

Moq (izgovara se *Mock-you* ili jednostavno *Mok*) je vodeći radni okvir za pisanje objekata imitatora u .NET ekosistemu. Dizajniran je da bude veoma praktičan i bezbedan. Neke od osobina:

- Jako tipiziran (ne koriste se niske za definisanje očekivanja, povratne vrednosti metoda su specifični tipovi a ne opšti `object` tip)
- Jednostavni idiomi - konstrukcija imitatora, podešavanje ponašanja imitatora, očekivanja
- Granularna kontrola ponašanja imitatora
- Imitira i interfejse i klase
- Presretanje događaja nad imitatorima

U datoteci `MockExamples.cs` imamo definicije nekoliko interfejsa i klasa:

- `IBookService` - predstavlja interfejs servisa koji koristimo da dovučemo informacije o knjigama na osnovu kategorije ili ISBN
- `IEmailSender` - predstavlja interfejs servisa koji koristimo da pošaljemo e-mail
- `AccountsService` - koristi navedene servise
- `SampleAccountsServiceTests` - testovi za `AccountsService`

Pošto `IBookService` može na proizvoljan način da dovlači informacije o knjigama (REST-ful API, baza podataka itd.), jasno je da ne želimo da je testiramo direktno. Štaviše, naši testovi se tiču `AccountsService` klase, a ne klasa koje ona koristi, dakle podrazumevamo da se implementacije `IBookService` i `IEmailSender` ispravno ponašaju. Tu pretpostavku implementiramo pomoću objekata imitatora.

Posmatrajmo klasu `SampleAccountsServiceTests`. Naredni primer prikazuje jednostavan idiom za korišćenje Moq radnog okvira:

```
public void GetAllBooksForCategory_returns_list_of_available_books()
{
    // 1
    var bookServiceStub = new Mock<IBookService>();

    // 2
    bookServiceStub
        .Setup(x => x.GetBooksForCategory("UnitTesting"))
        .Returns(new List<string> {
            "The Art of Unit Testing",
            "Test-Driven Development",
            "Working Effectively with Legacy Code"
        });

    // 3
    var accountService = new AccountService(bookServiceStub.Object, null);
    var result = accountService.GetAllBooksForCategory("UnitTesting");

    // 4
    Assert.Equal(3, result.Count());
}
```

Sličan imitator možemo napisati i za `IEmailService`, ukoliko želimo da testiramo metod `SendEmail`:

```
public void SendEmail_sends_email_with_correct_content()
{
    // 1
    var emailSenderMock = new Mock<IEmailSender>();

    // 2
    var accountService = new AccountService(null, emailSenderMock.Object);
    // 3

    accountService.SendEmail("test@gmail.com", "Test - Driven Development");
    // 4
    emailSenderMock.Verify(
        x => x.SendEmail(
```

```
            "test@gmail.com",
            "Awesome Book",
            $"Hi,\n\nThis book is awesome: Test - Driven Development"
        ),
        Times.Once
    );
}
```

Moguće je modelirati višestruke pozive. U primeru koji sledi zadajemo povratne vrednosti za prva četiri poziva metoda `GetISBNFor`, poziv metoda `GetBooksForCategory` podešavamo tako da baca izuzetak:

```
bookServiceStub
    .SetupSequence(x => x.GetISBNFor(It.IsAny<string>()))
    .Returns("0-9020-7656-6")   //returned on 1st call
    .Returns("0-9180-6396-5")   //returned on 2nd call
    .Returns("0-3860-1173-7")   //returned on 3rd call
    .Returns("0-5570-1450-6")   //returned on 4th call
    ;

bookServiceStub
    .Setup(x => x.GetBooksForCategory(It.IsAny<string>()))
    .Throws<InvalidOperationException>();
```

Za poznavaoce asinhronog šablona zasnovanog na zadacima (engl. Task-Based Asynchronous Pattern, skr. TAP) u programskom jeziku C#, mogu od značaja biti i asinhroni primeri korišćenja Moq radnog okvira:

```
httpClientMock
    .Setup(x => x.GetAsync(It.IsAny<string>()))
    .ReturnsAsync(true);
```

5.3 C# - ”imitatori” baza podataka

Objekti imitatori, iako korisni, ne mogu zameniti ponašanje pravih sistema za upravljanje bazama podataka. Iako je testove potrebno izvršiti nad produkcijskom bazom podataka pre isporučivanja aplikacije, neefikasno je te testove pokretati tokom razvoja aplikacije. Želeli bismo da testove pokrenemo u okruženju što sličnijem pravoj bazi podataka, ali ne bismo da gubimo na efikasnosti - potrebno je bazu pripremiti za svaki test posebno, što uključuje veliki broj upita. S druge strane, možda smo već napisali imitatore i zadovoljni smo ponašanjem naših servisa, ali bismo da testiramo logiku ostvarivanja odnosno raskidanja veze sa bazom. U ovakvim slučajevima, ali i mnogim drugim, izvršavanje testova nad ”pravom” bazom podataka je nešto što bismo voleli da imamo. Međutim, voleli bismo da to sve bude transparentno i da ne zahteva nikakve izmene u kodu, ali i takođe dovoljno efikasno tako da ne odlazi dosta vremena na izvršavanje testova.

Objektno-relacioni maperi (engl. Object Relational Mappers, skr. ORM) često se koriste radi apstrahovanja specifičnosti konkretnog sistema za upravljanje bazama podataka. Neki poznati ORM radni okviri su Hibernate (Java), Entity Framework (.NET). U ovom primeru ćemo

iskoristiti pogodnosti koje Entity Framework i SUBP SQLite pružaju, kako bismo testove izvršili nad privremenom bazom podataka u radnoj memoriji.

Postoje sistemi za upravljanje bazama podataka koji mogu da rade sa privremenim bazama podataka skladištenim u radnoj memoriji. Jedan od takvih SUBP je SQLite, koji pruža tzv. *in-memory database provider* upravo za ove svrhe. SQLite čuva baze podataka u datotekama na sistemu datoteka. Da bismo kreirali bazu podataka u memoriji, potrebno je da u nisku za konekciju na bazu ubacimo `:memory:` na mesto gde bi inače išla putanja do datoteke na sistemu datoteka. Privremena baza živi u radnoj memoriji sve dok postoji otvorena veza ka njoj. Raskid veze zato briše bazu podataka, što odgovara testovima koji treba da se pokreću u izolovanom okruženju.

Klasa `SampleDbContext` predstavlja kontekst veze s bazom podataka. Taj kontekst u sebi ima svojstva tipa `DbSet<T>` koja će se mapirati u odgovarajuće tabele. Pošto koristimo Entity Framework, možemo ga konfigurisati tako da, na osnovu konfiguracije koju korisnik navodi, koristimo odgovarajući zadnji deo (engl. back-end) koji će komunicirati sa odgovarajućim SUBP. Recimo da želimo da podržimo naredne SUBP:

```
public enum Provider
{
    Sqlite = 0,
    PostgreSql = 1,
    SqlServer = 2,
    SqliteInMemory = 3
}
```

Prilikom konfigurisanja EF radnog okvira, možemo odabrati odgovarajući zadnji deo:

```
switch (this.Provider) {
    case Provider.PostgreSql:
        optionsBuilder.UseNpgsql(this.ConnectionString);
        break;
    case Provider.Sqlite:
    case Provider.SqliteInMemory:
        optionsBuilder.UseSqlite(this.ConnectionString);
        break;
    case Provider.SqlServer:
        optionsBuilder.UseSqlServer(this.ConnectionString);
        break;
    default:
        throw new NotSupportedException("Provider not supported!");
}
```

Možemo implementirati provajder baze za testove tako što prosledimo odgovarajuću nisku za konekciju ka bazi podataka:

```
public class TestDbProvider
{
    public readonly string ConnectionString { get; }
    public readonly SqliteConnection DatabaseConnection { get; }
```

```
    public TestDbProvider()
    {
        ConnectionString = "DataSource=:memory:;foreign keys=true;";
        DatabaseConnection = new SqliteConnection(ConnectionString);
    }

    private void CreateContext()
    {
        var options = new DbContextOptionsBuilder<SampleDbContext>()
            .UseSqlite(DatabaseConnection)
            .Options;

        return new SampleDbContext(
            SampleDbContext.Provider.SqliteInMemory,
            ConnectionString,
            options
        );
    }

    // ...
}
```

Za izvršavanje testova je neophodno da:

- ostvarimo konekciju ka bazi
- ubacimo podatke u bazu (pošto se baza uvek briše nakon raskidanja konekcije)
- odradimo logiku koju test treba da proveri
- proverimo rezultat
- raskinemo vezu sa bazom

Da bismo smanjili ponavljanje koda, dodaćemo metod `SetupAlterAndVerify` koji će da primi funkcije:

- `void Setup(SampleDbContext ctx)` - priprema bazu podataka za test
- `void Alter(SampleDbContext ctx)` - izvršava logiku koju test treba da proveri
- `void Verify(SampleDbContext ctx)` - testira rezultujuće stanje baze podataka

```
public void SetupAlterAndVerify(
    Action<SampleDbContext>? setup,
    Action<SampleDbContext>? alter,
    Action<SampleDbContext>? verify)
{
    DatabaseConnection.Open();
    try {
        this.CreateDatabase();
        this.SeedDatabase();

        if (setup is not null) {
            using SampleDbContext context = this.CreateContext();
            setup(context);
            context.SaveChanges();
```

```
        }

        if (alter is not null) {
            using SampleDbContext context = this.CreateContext();
            alter(context);
            context.SaveChanges();
        }

        if (verify is not null) {
            using SampleDbContext context = this.CreateContext();
            verify(context);
        }
    } finally {
        DatabaseConnection.Close();
    }
}
```

Primetimo da, iako kontekst kreiramo više puta, veza ka bazi i dalje ostaje aktivna dok se ne pozove `DatabaseConnection.Close()` metod. Kreiranje zasebnih konteksta je poželjno pošto bismo da sačuvamo stanje baze nakon svakog koraka (setup, alter, verify). Tip `Action<T1, T2, ..., Tn>` u programskom jeziku C# predstavlja funkciju: `void f(T1, T2, ... Tn)`. Oznaka `?` je skraćenica za tip `Nullable<T>` koji predstavlja opcioni tip. Drugim rečima, metodi `SetupAlterAndVerify` prosleđujemo opcione akcije, i možemo da odlučimo da naš test ne mora da ima neku od njih (tako što prosledimo `null`, stoga provere u telu funkcije pre poziva funkcija `setup`, `alter` i `verify`). Ključna reč `using` je deo upravljanja resursa nad objektima koji implementiraju `IDisposable` interfejs u programskom jeziku C#, sa ciljem da se automatski počisti objekat nakon što kontrola toka izađe iz opsega u kojem je vidljiv (nešto nalik na *try-with-resources* šablon u programskom jeziku Java). Drugim rečima, automatski će se pozvati metod `IDisposable.Dispose()` nad kontekstom koji je definisan naredbom koja je kvalifikovana ključnom rečju `using` (interfejs `IDisposable` je implementiran u natklasi klase `SampleDbContext` koja dolazi iz EF radnog okvira).

Testove onda možemo veoma jednostavno pisati:

```
public class SampleDbTests
{
    public TestDbProvider DbProvider { get; }
    public IServiceToTest Service { get; set; }

    public SampleDbTests(IServiceToTest service)
    {
        this.DbProvider = new TestDbProvider();
        this.Service = service;
    }

    [Test]
    public void SampleTest()
    {
        this.DbProvider.SetupAlterAndVerify(
            setup:  ctx => ctx.Students.Clear(),
            alter:  ctx => Service.PerformLogic(ctx.Students),
            verify: ctx => Assert.That(ctx.Students, Is.Not.Empty)
```

```
        );
    }
}
```

Za poznavaoce asinhronog šablona zasnovanog na zadacima (engl. Task-Based Asynchronous Pattern, skr. TAP) u programskom jeziku C#, mogu od značaja biti i asinhrona varijanta metoda `SetupAlterAndVerify`:

```
public async Task SetupAlterAndVerifyAsync(
    Func<SampleDbContext, Task>? setup,
    Func<SampleDbContext, Task>? alter,
    Func<SampleDbContext, Task>? verify)
{
    DatabaseConnection.Open();
    try {
        this.CreateDatabase();
        this.SeedDatabase();

        if (setup is not null) {
            await using SampleDbContext context = this.CreateContext();
            await setup(context);
            await context.SaveChangesAsync();
        }

        if (alter is not null) {
            await using SampleDbContext context = this.CreateContext();
            await alter(context);
            await context.SaveChangesAsync();
        }

        if (verify is not null) {
            await using SampleDbContext context = this.CreateContext();
            await verify(context);
        }
    } finally {
        DatabaseConnection.Close();
    }
}
```

6 ”Fuzz” testiranje

Jedna od tehnika pronalaženja grešaka u programu koja se pokazala korisnom u praksi je *fuzz testiranje* (eng. *fuzzing*). Prilikom takvog testiranja, programu se šalju neočekivani i loši ulazni podaci, u cilju otkrivanja ulaza koji izazivaju greške. Standardni fuzzer alati mutiraju postojeće testove ili generišu testove na osnovu definisane gramatike ili skupa pravila. Fuzz testiranje vođeno pokrivenošću kôda (eng. *coverage guided fuzzing*) prati putanje izvršavanja i daje prednost ulazima koji povećavaju pokrivenost. Tako rade postojeći alati za fuzz testiranje, kao što su American Fuzzy Lop (AFL), LLVM libFuzzer i HonggFuzz.

U najjednostavnijim slučajevima potrebno je da izolujemo funkcionalnost koju bismo da testiramo, napišemo nekoliko pratećih linija kôda, kompajliramo primer i pokrenemo fuzzer. Fuzzer će izvršiti hiljade ili desetine hiljada testova po sekundi i sačuvaće one interesantne ulaze koji povećavaju pokrivenost ili izazivaju određeno ponašanje.

Prvi korak u upotrebi *libFuzzer*-a nad bibliotekom je kreiranje funkcije koja se u literaturi zove *fuzz target*. Ta funkcija prihvata niz bajtova i na njih primenjuje funkcije iz biblioteke koju testiramo.

```
extern "C" int LLVMFuzzerTestOneInput(const uint8_t *Data, size_t Size) {
    DoSomethingInterestingWithMyAPI(Data, Size);
    return 0;  // vrednosti osim 0 i -1 su rezervisane
}
```

Ova funkcija ne zavisi ni u kojoj meri od *libFuzzer*-a, pa ju je moguće i poželjno koristiti i sa drugim alatima za fuzz testiranje npr. AFL i/ili Radamsa.

Važne činjenice o fuzz funkciji (fuzz target):

- Alat za fuzz testiranje će izvršiti ovu funkciju mnogo puta sa različitim ulazima u okviru istog procesa.
- Mora tolerisati sve vrste ulaznih podataka (prazne, velike, loše formatirane, itd.)
- Ni za jedan ulaz se ne sme pozivati `exit()`.
- Može koristiti niti, ali idealno bi bilo da sve niti budu spojene na kraju funkcije.
- Mora biti deterministička, koliko god je moguće. Nedeterminizam, kao što su nasumične odluke nezavisne od ulaznih bajtova, učiniće fuzz testiranje neefikasnim.
- Mora biti brza. Pokušajte zaobići kubne i veće složenosti, logovanje ili preveliko trošenje memorije.
- Idealno bi bilo da ne modifikuje nijedno globalno stanje (iako to nije strogo neophodno).
- Uža fuzz funkcija je obično bolja. Ako funkcija može da parsira više formata ulaza, treba je podeliti na nekoliko fuzz funkcija, po jednu za svaki format.

LibFuzzer koristi sanitizer za pokrivenost kôda kojim se instrumentuje kôd da bi fuzzer dobijao informacije o pokrivenosti koje usmeravaju dalje fuzz testiranje. Podešava se opcijama iz grupe:

```
-fsanitize-coverage=...
```

Tokom fuzz testiranja treba uključiti i sanitizer koji prijavljuje greške tokom izvršavanja:

- **AddressSanitizer (ASAN)** detektuje greške sa memorijom. Uključuje se opcijom:

  ```
  -fsanitize=address
  ```

- **UndefinedBehaviorSanitizer (UBSAN)** detektuje upotrebu C/C++ konstrukcija koje uzrokuju nedefinisano ponašanje. Tipično se pokreće tako da se program zaustavi posle prijave:

  ```
  -fsanitize=undefined -fno-sanitize-recover=undefined
  ```

  Može se uključiti i samo pojedinačna UBSAN provera, na primer:

  ```
  -fsanitize=signed-integer-overflow -fno-sanitize-recover=undefined
  ```

  UBSAN se može kombinovati sa ASAN-om u istom prevođenju.

- **MemorySanitizer (MSAN)** detektuje upotrebu neinicijalizovanih vrednosti, odnosno kôd čije ponašanje zavisi od sadržaja neinicijalizovane memorijske lokacije. Uključuje se opcijom:

  ```
  -fsanitize=memory
  ```

  MSAN se ne sme kombinovati sa drugim sanitizerima i treba ga koristiti u zasebnom prevođenju.

Za svaku prijavu treba sačuvati ulaz koji je izazvao grešku, komandu kojom je fuzzer pokrenut i prvi relevantan okvir steka iz sanitizer izveštaja. Time fuzz nalaz postaje običan regresioni test.

6.1 Korpus

Fuzzeri, kao *libFuzzer*, koji koriste pokrivenost kôda za generisanje testova, oslanjaju se na korpus prethodno datih ulaza. Korpus treba da bude odmerena kolekcija validnih i nevalidnih ulaznih podataka. Fuzzer nasumičnim mutacijama generiše nove testove iz trenutnog korpusa. Ako mutacija aktivira izvršavanje prethodno nepokrivene putanje u kôdu koji se testira, tada se mutacija čuva kao nov test u korpusu i služi za buduće varijacije.

LibFuzzer može da radi bez inicijalnog test primera, ali će biti manje efikasan ako biblioteka koja se testira prihvata kompleksne i strukturirane ulaze. Korpus može da posluži i za proveru da li svi ranije pronađeni testovi i dalje rade bez problema kada se propuste kroz kôd.

Ako imamo veliki korpus, bilo da je dobijen fuzz testiranjem ili drugačije, možemo ga smanjiti upotrebom opcije `-merge=1` tako da zadržimo pokrivenost.

```
$ mkdir NEW_CORPUS_DIR  # Store minimized corpus here.
$ ./my_fuzzer -merge=1 NEW_CORPUS_DIR FULL_CORPUS_DIR
```

Ista opcija se može koristiti da se doda još testova u postojeći korpus. Samo oni testovi koji povećavaju pokrivenost će biti dodati u prvi navedeni korpus.

```
$ ./my_fuzzer -merge=1 CURRENT_CORPUS_DIR NEW_POTENTIALLY_INTERESTING_INPUTS_DIR
```

Da bi se pokrenuo fuzzer sa korpusom testova, prvo treba kreirati direktorijum sa inicijalnim test primerima i potom ga pokrenuti:

```
$ mkdir CORPUS_DIR
$ cp /some/input/samples/* CORPUS_DIR
$ ./my_fuzzer CORPUS_DIR  # -max_len=1000 -jobs=20 ...
```

Novi testovi biće dodavani u direktorijum korpusa.

Podrazumevano ponašanje je da fuzzer nastavlja da radi beskonačno ili dok ne pronađe grešku. Svaki pad ili greška koju detektuje sanitizer biće prijavljeni, fuzzer zaustavljen, a test primer koji je prouzrokovao grešku biće sačuvan u datoteku (obično, `crash-<sha1>`, `leak-<sha1>` ili `timeout-<sha1>`).

6.2 Primer rada libFuzzer-a

Primer `fuzz_me.cpp` prevodimo korišćenjem *clang*-a:

```
$ clang++  -fsanitize=fuzzer,address fuzz_me.cpp -o fuzz_me
```

Kreiramo direktorijum u koji će nam se čuvati generisani testovi i pokrećemo program:

```
$ mkdir testovi
$ ./fuzz_me testovi/
```

Testovi se generišu variranjem postojećih testova iz direktorijuma `testovi`, ukoliko ih ima. Novi testovi se dodaju u korpus samo ako povećavaju pokrivenost kôda u odnosu na već generisane testove. Ako želimo da se generišu testovi samo od ASCII karaktera, prilikom pozivanja programa treba proslediti i opciju `-only_ascii=1`; `0` je podrazumevana vrednost.

```
$ ./fuzz_me -only_ascii=1 testovi/
```

Testovi se generišu u direktorijum, a nama se na ekranu prikazuje sledeći izlaz:

```
INFO: Seed: 2050712183
INFO: Loaded 1 modules (7 guards): [0x77be60, 0x77be7c),
INFO: -max_len is not provided; libFuzzer will not generate inputs larger than 4096 bytes
INFO: A corpus is not provided, starting from an empty corpus
#0      READ units: 1
#2      INITED cov: 3 ft: 3 corp: 1/1b exec/s: 0 rss: 30Mb
#5      NEW    cov: 4 ft: 4 corp: 2/4b exec/s: 0 rss: 31Mb L: 3 MS: 3 CopyPart-ChangeBit-InsertByte-
#1114   NEW    cov: 5 ft: 5 corp: 3/122b exec/s: 0 rss: 33Mb L: 118 MS: 2
↪  ShuffleBytes-InsertRepeatedBytes-
#176228 NEW    cov: 6 ft: 6 corp: 4/242b exec/s: 0 rss: 235Mb L: 120 MS: 1 CMP- DE: "U\x00"-
#555573 NEW    cov: 7 ft: 7 corp: 5/366b exec/s: 555573 rss: 401Mb L: 124 MS: 1 CMP- DE:
↪  "Z\x00\x00\x00"-
#1048576        pulse  cov: 7 ft: 7 corp: 5/366b exec/s: 524288 rss: 409Mb
#2097152        pulse  cov: 7 ft: 7 corp: 5/366b exec/s: 419430 rss: 409Mb
==================================================================
==26069==ERROR: AddressSanitizer: heap-buffer-overflow on address 0x602000023bb3 at pc 0x000000539e73
↪  bp 0x7fff6c301bc0 sp 0x7fff6c301bb8
READ of size 1 at 0x602000023bb3 thread T0
.....
==26069==ABORTING
MS: 2 ChangeBinInt-CrossOver-; base unit: 1162a64ba4eb59dab49cfb3a8bb87abe708c3bb1
0x46,0x55,0x5a,
FUZ
artifact_prefix='./'; Test unit written to ./crash-0eb8e4ed029b774d80f2b66408203801cb982a60
Base64: RlVa
```

Opcijom `-max-len=N` možemo postaviti maksimalnu dužinu ulaza na neku drugu veličinu. Poruka `INFO: A corpus is not provided, starting from an empty corpus` označava da smo počeli sa praznim korpusom. `READ` prati broj pročitanih testova iz korpusa, a svaki novi test obeležen je sa `NEW`. Broj na početku linije predstavlja redni broj generisanog testa. Ako je `NEW` u nastavku, test je dodat u korpus. Ako stoji `pulse`, taj red je ispisan samo da obavesti programera da fuzzer i dalje radi. Vidimo da je fuzzer generisao 2097152 testova pre nego što je naišao na grešku koja mu je prekinula rad. Dogodilo se prekoračenje memorije na hipu i generisan je izveštaj o prekidu rada `./crash-0eb8e4ed029b774d80f2b66408203801cb982a60` sa ulazom koji je grešku izazvao: `FUZ`.

Pojašnjenje kôdova događaja i prateće statistike:

- `READ` - Fuzzer je pročitao sve od datih testova iz direktorijuma sa korpusima.
- `INITED` - Fuzzer je završio inicijalizaciju, što uključuje izvršavanje svakog inicijalnog testa kroz kôd koji se testira.
- `NEW` - Fuzzer je kreirao nov test koji pokriva do sada nepokrivene delove kôda koji se testira. Ovaj test će biti sačuvan u direktorijumu glavnog korpusa.
- `REDUCE` - Fuzzer je pronašao bolji (kraći) test koji izaziva prethodno otkriveno ponašanje. Može se isključiti opcijom `-reduce_inputs=0`.
- `pulse` - Fuzzer je generisao `2n` testova. Poruka se generiše periodično da bi se korisnik uverio da fuzzer i dalje radi.

- `DONE` - Fuzzer je završio sa radom jer je došao do limita broja iteracija (zadaje se opcijom `-runs`) ili vremenskog limita (zadaje se opcijom `-max_total_time`).
- `RELOAD` - Fuzzer periodično ponovo učitava testove iz direktorijuma sa korpusom. Ovo omogućava fuzzeru da uzme u obzir testove koji su otkriveni drugim procesima fuzzera, prilikom paralelnog fuzz testiranja.

Svaka izlazna linija sadrži i izveštaj sa sledećim statistikama, kada nisu 0:

- `cov:` Ukupan broj blokova kôda ili ivica pokrivenih izvršavanjem trenutnog korpusa.
- `ft:` *libFuzzer* koristi različite signale da proceni pokrivenost kôda: pokrivenost ivica, brojače ivica, vrednosne profile, indirektne parove pozivalac/pozvani, itd. Ovi signali su kombinovani i obeleženi sa `(ft:)`.
- `corp:` Broj testova u trenutnom korpusu u memoriji i njegova veličina u B.
- `lim:` Trenutni limit u dužini novih ulaznih podataka u korpusu. Povećava se tokom vremena, dok ne dostigne maksimalnu dužinu. Ona se može zadati opcijom `-max_len`.
- `exec/s:` Broj iteracija fuzzera u sekundi.
- `rss:` Trenutno iskorišćenje memorije.

Za nove (NEW) događaje, izlazna linija sadrži i informacije:

- `L:` Veličina novog testa u bajtovima.
- `MS: <n> <ops>` Broj i lista operacija mutacija koje su korišćene da bi se generisao nov ulaz.

6.3 tmux fuzz testiranje

Projekat `tmux` je dobar primer primene fuzz testiranja nad programom koji obrađuje dosta tekstualnih protokola i terminalskih sekvenci. Fuzz testovi su smešteni u direktorijumu `fuzz` u uzvodnom repozitorijumu.

Ovi testovi su korisni za traženje grešaka u parserima i kodu koji obrađuje ulaz iz terminala, konfiguracionih datoteka i mrežnih veza. Takav kod često ima veliki broj grana i mnogo kratkih, nevalidnih ulaza koji se retko pokriju ručno pisanim testovima.

Prilikom čitanja ovog primera obratiti pažnju na tri stvari:

- fuzz target izoluje deo programa koji obrađuje jedan format ulaza;
- ulaz fuzzera se tretira kao niz bajtova, bez pretpostavke da je sintaksno ispravan;
- pad programa, prekoračenje memorije ili beskonačno izvršavanje predstavljaju koristan nalaz.

Ako se ovakav pristup prenosi na sopstveni projekat, prvo treba izabrati malu funkciju koja parsira ulaz i napraviti omotač koji samo priprema stanje, poziva parser i oslobađa resurse. Fuzzer tada može da izvrši veliki broj kratkih testova bez pokretanja cele aplikacije.

6.4 Jazzer (JVM fuzzer)

Jazzer omogućava fuzz testiranje Java i drugih JVM programa. U JUnit načinu rada fuzz test se piše kao običan test metod označen anotacijom `@FuzzTest`. Bez dodatnog podešavanja takav test može da se izvršava kao regresioni test nad sačuvanim ulazima. Kada se postavi promenljiva okruženja `JAZZER_FUZZ=1`, Jazzer pokreće generisanje novih ulaza.

Minimalna Maven konfiguracija za primer u ovom direktorijumu koristi `jazzer-junit` verziju `0.30.0`.

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         ↪  https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>org.example</groupId>
    <artifactId>jazzer-example</artifactId>
    <version>1.0-SNAPSHOT</version>

    <properties>
        <maven.compiler.release>17</maven.compiler.release>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
        <jazzer.version>0.30.0</jazzer.version>
        <surefire.version>3.2.5</surefire.version>
    </properties>

    <dependencies>
        <dependency>
            <groupId>com.code-intelligence</groupId>
            <artifactId>jazzer-junit</artifactId>
            <version>${jazzer.version}</version>
            <scope>test</scope>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-surefire-plugin</artifactId>
                <version>${surefire.version}</version>
            </plugin>
        </plugins>
    </build>
</project>
```

Primer ispod proverava jednostavnu šemu kodiranja. Jedinični test proverava jedan poznat ulaz, dok fuzz test proverava isto svojstvo nad velikim brojem ulaza koje bira Jazzer. Anotacije `@NotNull`, `@WithUtf8Length` i `@InRange` ograničavaju prostor ulaza kada je to potrebno za smislen test.

```
package org.example;
```

```
public final class SomeScheme {
    private SomeScheme() {
    }

    public static String encode(String input) {
        StringBuilder result = new StringBuilder(input.length());
        for (int i = 0; i < input.length(); i++) {
            char c = input.charAt(i);
            if (c == '\\') {
                result.append("\\\\");
            } else if (c == ':') {
                result.append("\\c");
            } else if (c == '\n') {
                result.append("\\n");
            } else {
                result.append(c);
            }
        }
        return result.toString();
    }

    public static String decode(String input) {
        StringBuilder result = new StringBuilder(input.length());
        boolean escaped = false;
        for (int i = 0; i < input.length(); i++) {
            char c = input.charAt(i);
            if (!escaped && c == '\\') {
                escaped = true;
                continue;
            }
            if (escaped) {
                if (c == 'c') {
                    result.append(':');
                } else if (c == 'n') {
                    result.append('\n');
                } else {
                    result.append(c);
                }
                escaped = false;
            } else {
                result.append(c);
            }
        }
        if (escaped) {
            result.append('\\');
        }
        return result.toString();
    }
}
```

```
package org.example;

import com.code_intelligence.jazzer.junit.FuzzTest;
import com.code_intelligence.jazzer.mutation.annotation.InRange;
import com.code_intelligence.jazzer.mutation.annotation.NotNull;
```

```java
import com.code_intelligence.jazzer.mutation.annotation.WithUtf8Length;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertTrue;

class ParserTests {
    @Test
    void unitTest() {
        assertEquals("foo:bar", SomeScheme.decode(SomeScheme.encode("foo:bar")));
    }

    @FuzzTest
    void fuzzTestRoundTrip(@NotNull String input) {
        assertEquals(input, SomeScheme.decode(SomeScheme.encode(input)));
    }

    @FuzzTest
    void fuzzTestConstrainedInput(
            @NotNull @WithUtf8Length(min = 1, max = 200) String input,
            @InRange(min = -10, max = 10) int shift) {
        assertEquals(input, SomeScheme.decode(SomeScheme.encode(input)));
        assertTrue(shift >= -10 && shift <= 10);
    }
}
```

Testovi se mogu pokrenuti kao obični JUnit testovi:

```
$ mvn test -Dtest=ParserTests
```

Za fuzz režim postavlja se promenljiva okruženja:

```
$ JAZZER_FUZZ=1 mvn test -Dtest=ParserTests
```

Kada Jazzer pronađe ulaz koji ruši proces ili narušava tvrđenje, taj ulaz treba sačuvati kao regresioni test. Na taj način se pronađena greška proverava i posle ispravke.

6.5 Korišćenje AFL++ fuzzer-a nad libxml2

AFL++ je fuzzer vođen pokrivenošću koda. Za C i C++ projekte najčešći tok rada je da se program prevede AFL++ kompajlerskim omotačima, pripremi početni korpus ulaza i zatim pokrene `afl-fuzz` nad instrumentovanom izvršnom datotekom.

Primer u ovom direktorijumu koristi alat `xmllint` iz projekta `libxml2`. Ulazi su XML datoteke iz test direktorijuma projekta, a fuzzer menja te ulaze i prati koje mutacije vode ka novim putanjama u kodu.

```
#!/usr/bin/env bash

set -euo pipefail
```

```
set -x

workdir="${AFL_WORKDIR:-${TMPDIR:-/tmp}/afl-libxml2}"
run_system_config="${AFL_RUN_SYSTEM_CONFIG:-0}"

mkdir -p "$workdir"
cd "$workdir"

if [ ! -d AFLplusplus ]; then
    git clone https://github.com/AFLplusplus/AFLplusplus
    make -C AFLplusplus all
fi

if [ ! -d libxml2 ]; then
    git clone https://gitlab.gnome.org/GNOME/libxml2.git
fi

cd libxml2

if [ ! -x configure ]; then
    ./autogen.sh
fi

./configure --enable-shared=no

export AFL_USE_UBSAN="${AFL_USE_UBSAN:-1}"
export AFL_USE_ASAN="${AFL_USE_ASAN:-1}"

make clean || true
make CC="$workdir"/AFLplusplus/afl-clang-fast CXX="$workdir"/AFLplusplus/afl-clang-fast++
↪  LD="$workdir"/AFLplusplus/afl-clang-fast

mkdir -p fuzz
cp xmllint fuzz/xmllint_cov

mkdir -p fuzz/in
cp test/*.xml fuzz/in/

cd fuzz

if [ "$run_system_config" = "1" ]; then
    sudo "$workdir"/AFLplusplus/afl-system-config
fi

exec "$workdir"/AFLplusplus/afl-fuzz -i in -o out -- ./xmllint_cov @@
```

Skripta koristi sledeće korake:

- preuzima i gradi AFL++ ako nije već prisutan u radnom direktorijumu;
- preuzima `libxml2`, pokreće konfiguraciju i prevodi ga pomoću `afl-clang-fast`;
- uključuje AddressSanitizer i UndefinedBehaviorSanitizer preko promenljivih `AFL_USE_ASAN` i `AFL_USE_UBSAN`;
- kopira `xmllint` i početne XML ulaze u direktorijum `fuzz`;
- pokreće `afl-fuzz -i in -o out -- ./xmllint_cov @@`.

Oznaka `@@` u komandi označava mesto na kome AFL++ prosleđuje ime trenutne test datoteke programu koji se testira. Direktorijum `in` sadrži početni korpus, a `out` sadrži redove čekanja, padove programa i druge nalaze.

Podrazumevani radni direktorijum skripte je `/tmp/afl-libxml2`. Može se promeniti promenljivom `AFL_WORKDIR`. Sistemsko podešavanje preko `afl-system-config` nije uključeno podrazumevano, jer zahteva `sudo`; uključuje se sa `AFL_RUN_SYSTEM_CONFIG=1`.

```
$ AFL_WORKDIR=$HOME/afl-libxml2 ./run
$ AFL_RUN_SYSTEM_CONFIG=1 ./run
```

7 Profajliranje instrumentacijom

Profajliranje je vrsta dinamičke analize programa (program se analizira tokom izvršavanja) koja se sprovodi kako bi se izmerila, npr. količina memorije koju program zauzima, vreme koje program provodi u određenim funkcijama, iskorišćenost keša itd. Programi koji vrše profajliranje se zovu *profajleri*. Na ovom kursu će biti reči o popularnim profajlerima, njihovim prednostima i manama, uz primere upotrebe.

Profajliranje bi trebalo da nam da jasnu informaciju o tome da li imamo značajna uska grla u kodu. Ako primetimo da neka funkcija uzima 60% vremena izvršavanja, onda je ona glavni kandidat za optimizaciju. Ako nijedna funkcija ne troši više od nekoliko procenata vremena, treba proveriti druge pristupe poboljšanja performansi aplikacije: brži hardver, bolju arhitekturu, bolju paralelizaciju ili promenu algoritma.

Profajleri mogu obezbediti informaciju o tome koliko je vremena potrošeno u svakoj funkciji i u pozivima drugih funkcija, pa i koliko je potrošeno u svakoj liniji koda. Te informacije se, za neke alate kao što su Cachegrind i Callgrind koji generišu sličan izlaz, mogu prikazati kroz alate koje Valgrind pruža ili kroz specijalizovane alate kao što je *KCachegrind*. Pored Valgrind alata, ovo poglavlje uvodi `gprof` i LLVM XRay. Clang sanitizeri su obrađeni u poglavlju 12.3, pošto pripadaju Clang/LLVM toku prevođenja i izvršavanja.

7.1 Izbor instrumentacionog alata

Alat	Koristiti kada	Glavno ograničenje
Memcheck	Treba pronaći neispravne pristupe memoriji, curenje memorije ili upotrebu neinicijalizovanih vrednosti bez posebnog prevođenja.	Izvršavanje je znatno sporije; rezultat zavisi od putanja koje su zaista izvršene.
Massif	Treba razumeti rast memorije kroz vreme i mesto najveće potrošnje hipa.	Ne objašnjava svaku alokaciju detaljno kao alat usmeren na obrasce alokacije.
DHAT	Treba analizirati životni vek, broj pristupa i korisnost alokacija.	Izveštaj može biti obiman i zahteva reprezentativan ulaz.
Cachegrind	Treba porediti broj instrukcija, promašaje keša i uticaj lokalnosti podataka.	Model keša je simulacija i ne zamenjuje merenje na stvarnom hardveru.
Callgrind	Treba razumeti graf poziva i inkluzivnu cenu funkcija.	Instrumentacija menja vreme izvršavanja, pa rezultat čitati kao relativni profil.
Helgrind/DRD	Treba pronaći trke za podacima, pogrešnu upotrebu muteksa ili problematičan redosled zaključavanja.	Prijave treba proveriti u odnosu na stvarnu sinhronizacionu nameru programa.
`gprof`	Treba jednostavan profil za mali C/C++ program preveden sa podrškom za `gprof`.	Zahteva posebno prevođenje i slabije pokriva moderne, višedretvene i kratkotrajne programe.

Alat	Koristiti kada	Glavno ograničenje
LLVM XRay	Treba instrumentacija funkcija u Clang/LLVM toku rada uz kontrolu nad time šta se prati.	Zahteva odgovarajuće prevođenje i pripremu izlaza za analizu.

7.2 Valgrind

Valgrind je platforma za pravljenje alata za dinamičku analizu mašinskog koda, snimljenog ili kao objektni modul (nepovezan) ili kao izvršivi program (povezan). Postoje Valgrind alati koji mogu automatski da detektuju probleme sa memorijom i procesima.

Valgrind se može koristiti i kao alat za pravljenje novih alata. Valgrind distribucija, između ostalih, uključuje sledeće alate: detektor memorijskih grešaka (Memcheck), detektor grešaka u višenitnim programima (Helgrind i DRD), profiler keš memorije i grana (Cachegrind), profiler grafa poziva (Callgrind) i profiler korišćenja dinamičke memorije (Massif).

7.2.1 Struktura i upotreba Valgrind alata

Alat Valgrind se sastoji od alata za dinamičku analizu koda koji se kreira kao dodatak pisan u C programskom jeziku na jezgro Valgrinda. Jezgro Valgrinda omogućuje izvršavanje klijentskog programa, kao i snimanje izveštaja koji su nastali prilikom analize samog programa. Alati Valgrinda koriste metodu bojenja vrednosti. Oni svaki registar i memorijsku vrednost boje (zamenjuju) sa vrednošću koja govori nešto dodatno o originalnoj vrednosti. Proces rada svakog alata Valgrinda je u osnovi isti.

Valgrind deli originalni kod u sekvence koje se nazivaju osnovni blokovi. Osnovni blok je pravolinijska sekvenca mašinskog koda, na čiji se početak skače, a koja se završava skokom, pozivom funkcije ili povratkom u funkciju pozivaoca. Svaki kod programa koji se analizira ponovo se prevodi na zahtev, pojedinačno po osnovnim blokovima, neposredno pre izvršavanja samog bloka. Veličina osnovnog bloka je ograničena na maksimalno šezdeset mašinskih instrukcija.

Alat analizira dobijen kod i vrši translaciju - proces koji se sastoji od sledećih koraka:

1. Disasembliranje (razgradnja) - prevođenje mašinskog koda u ekvivalentni interni skup instrukcija koje se nazivaju međukod instrukcije. U ovoj fazi međukod je predstavljen stablom. Ova faza je zavisna od arhitekture na kojoj se program izvršava.
2. Optimizacija 1 - prva faza optimizacije linearizuje prethodno izgrađeni međukod. Primenjuju se neke standardne optimizacije programskih prevodilaca kao što su uklanjanje redundantnog koda, eliminacija podizraza itd.
3. Instrumentacija - Blok međukoda se prosleduje alatu, koji može proizvoljno da ga transformiše. Prilikom instrumentacije alat u zadati blok dodaje dodatne međukod operacije, kojima proverava ispravnost rada programa. Treba napomenuti da ubačene instrukcije ne narušavaju konzistentno izvršavanje originalnog koda.
4. Optimizacija 2 - jednostavnija faza optimizacije od prve. Uključuje izračunavanje matematičkih izraza koji se mogu izvršiti pre faze izvršavanja i uklanjanje mrtvog koda.
5. Izgradnja stabla - linearizovani međukod se konvertuje natrag u stablo radi lakšeg izbora instrukcija.
6. Odabir instrukcija - stablo međukoda se konvertuje u listu instrukcija koje koriste virtuelne registre. Ova faza se takođe razlikuje u zavisnosti od arhitekture na kojoj se izvršava.

7. Alokacija registara - zamena virtuelnih registara stvarnim. Po potrebi se uvode prebacivanja u memoriju. Ne zavisi od platforme. Koristi se poziv funkcija koje pronalaze iz kojih se registara vrši čitanje i u koje se vrši upis.
8. Asembliranje - kodiranje izabranih instrukcija na odgovarajući način i smeštanje u blok memorije. Ova faza se takođe razlikuje u zavisnosti od arhitekture na kojoj se izvršava.

Jezgro Valgrinda troši najviše vremena na sam proces pravljenja, pronalaženja i izvršavanja translacije (originalni kod se nikad ne izvršava). Treba napomenuti da sve ove korake osim instrumentacije izvršava jezgro Valgrinda, dok samu instrumentaciju izvršava određeni alat koji smo koristili za analizu izvornog koda.

Sve međukod instrukcije, originalne i dodate translacijom, prevode se u mašinske reči ciljne platforme i snimaju u prevedeni osnovni blok. Alat u originalni kod umeće operacije za instrumentaciju, zatim se takav kod prevodi.

Prilikom analize programa alatom Valgrind izvršavanje programa traje 20-100 puta duže nego inače. Analiza prevedenog programa Valgrindom, vrši sledećom naredbom:

```
valgrind --tool=alat [argumenti alata] ./a.out [argumenti za a.out]
```

ili pokretanjem Valgrind memory analizer-a iz QtCreator-a za aktivan projekat.

Ukoliko se ne zada vrednost argumenta `--tool` podrazumeva se `memcheck`.

Prve tri linije izlazne poruke štampaju se prilikom pokretanja bilo kog alata koji je u sklopu Valgrinda. U nastavku se prikazuju poruke o greškama koje je alat pronašao u programu. Zatim sledi izlaz samog programa, praćen sumiranim izveštajem o greškama.

Nekada informacija koja se dobije o grešci nije dovoljno detaljna da se u hiljadama linija koda nađe pravo mesto. Da bismo u okviru poruke o grešci imali i informaciju o liniji koda u kojoj je detektovana, potrebno je da program prevedemo sa simbolima za debagovanje (opcija `-g` za `gcc`). Da se ne bi dogodilo da se ne prijavljuje tačna linija u kojoj je detektovana greška, preporučuje se da se isključe optimizacije (opcija `-O0` za `gcc`). Nepotpuni stek okviri se manifestuju linijama koje sadrže nepoznate simbole (`???`), u kom slučaju se zarad njihovog poboljšanja, osim kompajlerskih opcija `-g` i `-O` mogu koristiti i neke Valgrind opcije.

7.2.2 Memcheck

Memcheck detektuje memorijske greške korisničkog programa. Kako ne vrši analizu izvornog koda već mašinskog, Memcheck ima mogućnost analize programa pisanom u bilo kom programskom jeziku. Za programe pisane u jezicima C i C++ detektuje sledeće probleme:

- Korišćenje nedefinisanih vrednosti, vrednosti koje nisu inicijalizovane ili koje su izvedene od drugih nedefinisanih. vrednosti. Problem se detektuje tek kada su upotrebljene.
- Čitanje ili pisanje u nedopuštenu memoriju na hipu, steku, bilo da je potkoračenje ili prekoračenje dozvoljene memorije ili pristupanje već oslobodenoj memoriji.
- Neispravno oslobadanje memorije na hipu, npr. duplo oslobadanje memorije na hipu ili neupareno korišćenje funkcija `malloc/new/new[]` i `free/delete/delete[]`.
- Poklapanje argumenata `src` i `dest` funkcije `memcpy` i njoj sličnim.
- Prosledivanje loših vrednosti za veličinu memorijskog prostora funkcijama za alokaciju memorije, npr. negativnih.
- Curenje memorije, npr. gubitak pokazivača na alociran prostor.

Korišćenje nedefinisanih vrednosti Program `01_uninitialized.c` koristi nedefinisanu promenljivu `x`. Prevedimo kod i pokrenimo `memcheck`:

```
$ gcc -g -O0 -Wall 01_uninitialized.c -o 1
$ valgrind ./1
```

Nedefinisana promenljiva može više puta da se kopira. Memcheck prati i beleži podatke o tome, ali ne prijavljuje grešku. U slučaju da se nedefinisane vrednosti koriste tako da od te vrednosti zavisi dalji tok programa ili ako je potrebno prikazati vrednosti nedefinisane promeljive, Memcheck prijavljuje grešku.

```
==11003== Memcheck, a memory error detector
==11003== Copyright (C) 2002-2017, and GNU GPL'd, by Julian Seward et al.
==11003== Using Valgrind-3.14.0 and LibVEX; rerun with -h for copyright info
==11003== Command: ./1
==11003==
==11003== Conditional jump or move depends on uninitialised value(s)
==11003== at 0x48DEE40: __vfprintf_internal (vfprintf-internal.c:1644)
==11003== by 0x48C98D7: printf (printf.c:33)
==11003== by 0x109162: main (01_uninitialized.c:7)
==11003==
==11003== Use of uninitialised value of size 8
==11003== at 0x48C332E: _itoa_word (_itoa.c:179)
==11003== by 0x48DE9EF: __vfprintf_internal (vfprintf-internal.c:1644)
==11003== by 0x48C98D7: printf (printf.c:33)
==11003== by 0x109162: main (01_uninitialized.c:7)
==11003==
==11003== Conditional jump or move depends on uninitialised value(s)
==11003== at 0x48C3339: _itoa_word (_itoa.c:179)
==11003== by 0x48DE9EF: __vfprintf_internal (vfprintf-internal.c:1644)
==11003== by 0x48C98D7: printf (printf.c:33)
==11003== by 0x109162: main (01_uninitialized.c:7)
==11003==
==11003== Conditional jump or move depends on uninitialised value(s)
==11003== at 0x48DF48B: __vfprintf_internal (vfprintf-internal.c:1644)
==11003== by 0x48C98D7: printf (printf.c:33)
==11003== by 0x109162: main (01_uninitialized.c:7)
==11003==
==11003== Conditional jump or move depends on uninitialised value(s)
==11003== at 0x48DEB5A: __vfprintf_internal (vfprintf-internal.c:1644)
==11003== by 0x48C98D7: printf (printf.c:33)
==11003== by 0x109162: main (01_uninitialized.c:7)
==11003==
x = -16778112
==11003== Conditional jump or move depends on uninitialised value(s)
==11003== at 0x48DEE40: __vfprintf_internal (vfprintf-internal.c:1644)
==11003== by 0x48C98D7: printf (printf.c:33)
==11003== by 0x109189: main (01_uninitialized.c:10)
==11003==
==11003== Use of uninitialised value of size 8
==11003== at 0x48C332E: _itoa_word (_itoa.c:179)
3
==11003== by 0x48DE9EF: __vfprintf_internal (vfprintf-internal.c:1644)
```

```
==11003== by 0x48C98D7: printf (printf.c:33)
==11003== by 0x109189: main (01_uninitialized.c:10)
==11003==
==11003== Conditional jump or move depends on uninitialised value(s)
==11003== at 0x48C3339: _itoa_word (_itoa.c:179)
==11003== by 0x48DE9EF: __vfprintf_internal (vfprintf-internal.c:1644)
==11003== by 0x48C98D7: printf (printf.c:33)
==11003== by 0x109189: main (01_uninitialized.c:10)
==11003==
==11003== Conditional jump or move depends on uninitialised value(s)
==11003== at 0x48DF48B: __vfprintf_internal (vfprintf-internal.c:1644)
==11003== by 0x48C98D7: printf (printf.c:33)
==11003== by 0x109189: main (01_uninitialized.c:10)
==11003==
==11003== Conditional jump or move depends on uninitialised value(s)
==11003== at 0x48DEB5A: __vfprintf_internal (vfprintf-internal.c:1644)
==11003== by 0x48C98D7: printf (printf.c:33)
==11003== by 0x109189: main (01_uninitialized.c:10)
==11003==
t = 0
==11003==
==11003== HEAP SUMMARY:
==11003== in use at exit: 4 bytes in 1 blocks
==11003== total heap usage: 2 allocs, 1 frees, 1,028 bytes allocated
==11003==
==11003== LEAK SUMMARY:
==11003== definitely lost: 4 bytes in 1 blocks
==11003== indirectly lost: 0 bytes in 0 blocks
==11003== possibly lost: 0 bytes in 0 blocks
==11003== still reachable: 0 bytes in 0 blocks
==11003== suppressed: 0 bytes in 0 blocks
==11003== Rerun with --leak-check=full to see details of leaked memory
==11003==
==11003== For counts of detected and suppressed errors, rerun with: -v
==11003== Use --track-origins=yes to see where uninitialised values come from
==11003== ERROR SUMMARY: 24 errors from 10 contexts (suppressed: 0 from 0)
```

Da bi nam bilo lakše da pronađemo glavni izvor greške sa korišćenjem nedefinisanih promenljivih koristimo opciju `--track-origins=yes`.

```
$ valgrind --track-origins=yes ./1
```

Tada uz poruku o upotrebi neinicijalizovane promenljive dobijamo i informaciju o liniji u kojoj je deklarisana:

```
==18060== Conditional jump or move depends on uninitialised value(s)
==18060== at 0x48DEE40: __vfprintf_internal (vfprintf-internal.c:1644)
==18060== by 0x48C98D7: printf (printf.c:33)
==18060== by 0x109162: main (01_uninitialized.c:7)
==18060== Uninitialised value was created by a stack allocation
==18060== at 0x109145: main (01_uninitialized.c:5)
==18060==
```

```
==18060== Use of uninitialised value of size 8
==18060== at 0x48C332E: _itoa_word (_itoa.c:179)
==18060== by 0x48DE9EF: __vfprintf_internal (vfprintf-internal.c:1644)
==18060== by 0x48C98D7: printf (printf.c:33)
==18060== by 0x109162: main (01_uninitialized.c:7)
==18060== Uninitialised value was created by a stack allocation
==18060== at 0x109145: main (01_uninitialized.c:5)
```

Primetimo da nemamo grešku da je promenljiva `y` inicijalizovana neinicijalizovanom promenljivom `x`. Tada se samo obeležava da ni `y` nije inicijalizovana. Tek prilikom prve upotrebe promenljive `y` biće detektovana greška, čiji uzrok je neinicijalizovano `x`.

Prosleđivanje sistemskim pozivima neinicijalizovane ili neadresirane vrednosti Memcheck prati sve parametre sistemskih poziva. Proverava svaki pojedinačno, bez obzira da li je inicijalizovan ili ne. Ukoliko sistemski poziv treba da čita iz prosleđenog bafera, Memcheck proverava da li je ceo bafer adresiran i inicijalizovan. Ako sistemski poziv treba da piše u memoriju, proverava se da li je adresirana. Posle sistemskog poziva Memcheck ažurira svoje informacije o praćenju stanja memorije tako da one precizno opisuju promene koje su nastale izvršavanjem sistemskog poziva.

Program `02_undefined.c` sadrži dva sistemska poziva sa neinicijalizovanim parametrima. Memcheck je detektovao prvu grešku u prosleđivanju neinicijalizovanog parametra `arr` sistemskom pozivu `write()`. Druga je u tome što sistemski poziv `read()` dobija neadresiran prostor. Treća greška je u tome što se sistemskom pozivu `exit()` prosleđuje nedefinisan argument. Prikazane su nam i linije u samom programu koje sadrže detektovane greške.

```
$ valgrind --track-origins=yes ./02_undefined.out
```

```
==3422== Memcheck, a memory error detector
==3422== Copyright (C) 2002-2017, and GNU GPL'd, by Julian Seward et al.
==3422== Using Valgrind-3.14.0 and LibVEX; rerun with -h for copyright info
==3422== Command: ./a.out
==3422==
==3422== Syscall param write(buf) points to uninitialised byte(s)
==3422== at 0x4978024: write (write.c:26)
==3422== by 0x10919E: main (02_undefined.c:9)
==3422== Address 0x4a59040 is 0 bytes inside a block of size 10 alloc'd
==3422== at 0x483874F: malloc (in
↪  /usr/lib/x86_64-linux-gnu/valgrind/vgpreload_memcheck-amd64-linux.so)
==3422== by 0x109176: main (02_undefined.c:6)
==3422== Uninitialised value was created by a heap allocation
==3422== at 0x483874F: malloc (in
↪  /usr/lib/x86_64-linux-gnu/valgrind/vgpreload_memcheck-amd64-linux.so)
==3422== by 0x109176: main (02_undefined.c:6)
==3422==
==3422== Syscall param read(buf) contains uninitialised byte(s)
==3422== at 0x4977F81: read (read.c:26)
==3422== by 0x1091B4: main (02_undefined.c:10)
==3422== Uninitialised value was created by a stack allocation
==3422== at 0x109165: main (02_undefined.c:5)
==3422==
```

```
==3422== Syscall param read(buf) points to unaddressable byte(s)
==3422== at 0x4977F81: read (read.c:26)
==3422== by 0x1091B4: main (02_undefined.c:10)
==3422== Address 0x0 is not stack'd, malloc'd or (recently) free'd
==3422==
==3422== Syscall param exit_group(status) contains uninitialised byte(s)
==3422== at 0x494C926: _Exit (_exit.c:31)
==3422== by 0x48B23A9: __run_exit_handlers (exit.c:132)
==3422== by 0x48B23D9: exit (exit.c:139)
==3422== by 0x1091C1: main (02_undefined.c:11)
==3422== Uninitialised value was created by a heap allocation
==3422== at 0x483874F: malloc (in
↪  /usr/lib/x86_64-linux-gnu/valgrind/vgpreload_memcheck-amd64-linux.so)
==3422== by 0x109184: main (02_undefined.c:8)
5
==3422==
==3422==
==3422== HEAP SUMMARY:
==3422== in use at exit: 14 bytes in 2 blocks
==3422== total heap usage: 2 allocs, 0 frees, 14 bytes allocated
==3422==
==3422== LEAK SUMMARY:
==3422== definitely lost: 0 bytes in 0 blocks
==3422== indirectly lost: 0 bytes in 0 blocks
==3422== possibly lost: 0 bytes in 0 blocks
==3422== still reachable: 14 bytes in 2 blocks
==3422== suppressed: 0 bytes in 0 blocks
==3422== Rerun with --leak-check=full to see details of leaked memory
==3422==
==3422== For counts of detected and suppressed errors, rerun with: -v
==3422== ERROR SUMMARY: 4 errors from 4 contexts (suppressed: 0 from 0)
```

Takođe, Memcheck prilikom izvršavanja beleži podatke o svim dinamički alociranim blokovima memorija. Po završetku programa, ima sve informacije o neoslobodenim memorijskim blokovima. Opcijom `--leak-check=yes` za svaki neosloboden blok se odreduje da li mu je moguće pristupiti preko pokazivača (still reachable) ili ne (definitely lost).

Opcijama `--leak-check=full --show-leak-kinds=all` tražimo da nam se prikaže detaljan izveštaj o svakom definitivno ili potencijalno izgubljenom bloku, kao i o tome gde je alociran u delu sa izveštajem sa hipa - `HEAP SUMMARY`.

```
==3439== HEAP SUMMARY:
==3439== in use at exit: 14 bytes in 2 blocks
==3439== total heap usage: 2 allocs, 0 frees, 14 bytes allocated
==3439==
==3439== 4 bytes in 1 blocks are still reachable in loss record 1 of 2
==3439== at 0x483874F: malloc (in
↪  /usr/lib/x86_64-linux-gnu/valgrind/vgpreload_memcheck-amd64-linux.so)
==3439== by 0x109184: main (02_undefined.c:8)
==3439==
==3439== 10 bytes in 1 blocks are still reachable in loss record 2 of 2
==3439== at 0x483874F: malloc (in
↪  /usr/lib/x86_64-linux-gnu/valgrind/vgpreload_memcheck-amd64-linux.so)
==3439== by 0x109176: main (02_undefined.c:6)
```

```
==3439==
==3439== LEAK SUMMARY:
==3439== definitely lost: 0 bytes in 0 blocks
==3439== indirectly lost: 0 bytes in 0 blocks
==3439== possibly lost: 0 bytes in 0 blocks
==3439== still reachable: 14 bytes in 2 blocks
==3439== suppressed: 0 bytes in 0 blocks
```

Nedopušteno oslobadanje memorije Memcheck prati svaku alokaciju memorije pozivom funkcija kao što su `malloc` i `calloc`, ali i alokacije uzrokovane konstrukcijom objekata (`new`). Iz tog razloga tačno zna da li je argument funkcije `free`, odnosno `delete`, ispravan ili ne. U sledećem programu isti blok dinamički alocirane memorije se oslobada dva puta. Memcheck prijavljuje tu grešku a potom detektuje i drugu grešku prilikom pokušaja oslobadanje bloka memorije sa adrese koja nije na hipu.

```
$ valgrind --show-leak-kinds=all --leak-check=full --track-origins=yes ./03_malloc.out
```

```
==3914== Memcheck, a memory error detector
==3914== Copyright (C) 2002-2017, and GNU GPL'd, by Julian Seward et al.
==3914== Using Valgrind-3.14.0 and LibVEX; rerun with -h for copyright info
==3914== Command: ./3
==3914==
==3914== Invalid free() / delete / delete[] / realloc()
==3914== at 0x483997B: free (in /usr/lib/x86_64-linux-gnu/valgrind/vgpreload_memcheck-amd64-linux.so)
==3914== by 0x10919F: main (03_malloc.c:11)
==3914== Address 0x4a590a0 is 0 bytes inside a block of size 12 free'd
==3914== at 0x483997B: free (in /usr/lib/x86_64-linux-gnu/valgrind/vgpreload_memcheck-amd64-linux.so)
==3914== by 0x109193: main (03_malloc.c:10)
==3914== Block was alloc'd at
==3914== at 0x483874F: malloc (in
↪  /usr/lib/x86_64-linux-gnu/valgrind/vgpreload_memcheck-amd64-linux.so)
==3914== by 0x109183: main (03_malloc.c:9)
==3914==
==3914== Invalid free() / delete / delete[] / realloc()
==3914== at 0x483997B: free (in /usr/lib/x86_64-linux-gnu/valgrind/vgpreload_memcheck-amd64-linux.so)
==3914== by 0x1091B3: main (03_malloc.c:14)
==3914== Address 0x1ffefffd0f is on thread 1's stack
==3914== in frame #1, created by main (03_malloc.c:5)
```

I u ovom programu imamo i pored svega curenje memorije jer 19B bivaju alocirani i potom pokazivač `p` dobije vrednost adrese novog prostora. Pokretanjem sa opcijama `--leak-check=full` i `--show-leak-kinds=all` dobijamo informaciju da imamo 19B koji su sasvim izgubljeni jer smo izgubili adresu alociranog bloka na hipu.

```
==3914== HEAP SUMMARY:
==3914== in use at exit: 19 bytes in 1 blocks
==3914== total heap usage: 2 allocs, 3 frees, 31 bytes allocated
==3914==
==3914== 19 bytes in 1 blocks are definitely lost in loss record 1 of 1
```

```
==3914== at 0x483874F: malloc (in
↪  /usr/lib/x86_64-linux-gnu/valgrind/vgpreload_memcheck-amd64-linux.so)
==3914== by 0x109175: main (03_malloc.c:8)
==3914==
==3914== LEAK SUMMARY:
==3914== definitely lost: 19 bytes in 1 blocks
==3914== indirectly lost: 0 bytes in 0 blocks
==3914== possibly lost: 0 bytes in 0 blocks
==3914== still reachable: 0 bytes in 0 blocks
==3914== suppressed: 0 bytes in 0 blocks
==3914==
==3914== For counts of detected and suppressed errors, rerun with: -v
==3914== ERROR SUMMARY: 3 errors from 3 contexts (suppressed: 0 from 0)
```

Nekorektno oslobađanje memorije U primeru `04_new_delete.cpp` se ne upotrebljavaju odgovaraju´ce funkcije za oslobadanje dinami˘cki alocirane memorije. Pokrenimo Memcheck:

```
$ valgrind --track-origins=yes ./04_new_delete.out
```

```
==4011== Memcheck, a memory error detector
==4011== Copyright (C) 2002-2017, and GNU GPL'd, by Julian Seward et al.
==4011== Using Valgrind-3.14.0 and LibVEX; rerun with -h for copyright info
==4011== Command: ./4
==4011==
==4011== Invalid free() / delete / delete[] / realloc()
==4011== at 0x483997B: free (in /usr/lib/x86_64-linux-gnu/valgrind/vgpreload_memcheck-amd64-linux.so)
==4011== by 0x1091AD: main (04_new_delete.cpp:10)
==4011== Address 0x4db6c88 is 8 bytes inside a block of size 168 alloc'd
==4011== at 0x48394DF: operator new[](unsigned long) (in
↪  /usr/lib/x86_64-linux-gnu/valgrind/vgpreload_m
==4011== by 0x10916B: main (04_new_delete.cpp:8)
==4011==
==4011==
==4011== HEAP SUMMARY:
==4011== in use at exit: 168 bytes in 1 blocks
==4011== total heap usage: 2 allocs, 2 frees, 72,872 bytes allocated
==4011==
==4011== LEAK SUMMARY:
==4011== definitely lost: 168 bytes in 1 blocks
==4011== indirectly lost: 0 bytes in 0 blocks
==4011== possibly lost: 0 bytes in 0 blocks
==4011== still reachable: 0 bytes in 0 blocks
==4011== suppressed: 0 bytes in 0 blocks
==4011== Rerun with --leak-check=full to see details of leaked memory
==4011==
==4011== For counts of detected and suppressed errors, rerun with: -v
==4011== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
```

Delimično preklapanje izvorne i ciljne memorije Program `05_overlap.c` prepisuje nisku u lokaciju koja se preklapa sa onom odakle se prepisuje. Ako unesemo reč `Da`, neće se detektovati gredška, jer prilikom izvrdšavanja za unetu reč neće biti preklapanja. Ponovo pokrenimo program

i unesimo reč `dobro`. Memcheck detektuje problem preklapanja memorijskih lokacija sa koje se kopira i one na koju se kopira.

```
==16178== Source and destination overlap in memcpy(0x5204041, 0x5204040, 5)
==16178== at 0x4C32513: memcpy@@GLIBC_2.14 (in /usr/lib/valgrind/vgpreload_memcheck-amd64-linux.so)
==16178== by 0x400617: main (05_overlap.c:9)
```

Ukoliko se unese kraća reč od 3 slova ovo će biti jedina gredška koju imamo. Ukoliko unesemo dužu reč na ulazu, `printf` će nam prijaviti pristup neinicilizovanoj vrednosti.

Detekcija neispravnog argumenta pri alokaciji memorije Memcheck može da detekuje grešku slanja negativnog argumenta za veličinu alociranog prostora. Program `06_fishy_arguments.c` ilustruje tu grešku. Ukoliko se program pokrene za negativne brojeve neće se ništa desiti jer petlja u tom slučaju neće imati ni jednu iteraciju, pa neće biti ni poziva `malloc` funkcije. Pozivi sa `n < 27` neće biti problematični - sve će se lepo alocirati i osloboditi. Imaćemo samo prijavljeno upozorenje:

```
(Warning: set address range perms: large range)
```

Ovo znači da se velikom bloku memorije menjaju prava pristupa. To upozorenje je namenjeno zapravo najviše Valgrind developerima i možemo ga ignorisati jer će memorija biti alocirana. Ukoliko pokrenemo ponovo Valgrind i za `n` unesemo `28`, dobićemo grešku jer smo prekoračili opseg `int` domena.

```
sada je i = 26 allocated = 1073741824
==5105== Warning: set address range perms: large range [0x59c89028, 0x99c89058) (noaccess)
sada je i = 27 allocated = -2147483648B
==5105== Argument 'size' of function malloc has a fishy (possibly negative) value: -2147483648
==5105== at 0x483874F: malloc (in
↪  /usr/lib/x86_64-linux-gnu/valgrind/vgpreload_memcheck-amd64-linux.so)
==5105== by 0x109219: main (06_fishy_arguments.c:20)
==5105==
Realloc failed!
```

Dodatni Memcheck primeri Ovaj direktorijum sadrži dodatne parove programa za kratke laboratorijske vežbe. Svaki par ima lošu verziju i popravljenu verziju. Cilj je da se isti test ponovi nad obe verzije i da se razlikuju stvarne greške od poruka koje su posledica ranije greške.

```
$ make
$ valgrind --leak-check=full ./01_invalid_read.out
$ valgrind --leak-check=full ./01_invalid_read_fixed.out
```

- `01_invalid_read.c`: čitanje jednog bajta van alociranog bloka; očekuje se `Invalid read`.
- `02_invalid_write.c`: upis nul karaktera van alociranog bloka; očekuje se `Invalid write`.
- `03_watch_your_pointers.c`: izgubljen rezultat `realloc`; očekuju se greške pri oslobađanju i curenje.
- `04_test01.cpp`: prekoračenje niza i pogrešno oslobađanje; očekuju se `Invalid write` i `Mismatched free/delete`.
- `05_test02.cpp`: neinicijalizovani elementi niza; očekuje se prijava za `uninitialised`.

Neispravno čitanje

```c
#include <stdint.h>
#include <stdlib.h>

int main() {
    char *destination = calloc(27, sizeof(char));
    char *source = malloc(26 * sizeof(char));

    for (int i = 0; i < 27; i++) {
        *(destination + i) = *(source + i);
    }

    free(destination);
    free(source);
    return 0;
}
```

```c
#include <stdint.h>
#include <stdlib.h>

int main() {
    char *destination = calloc(27, sizeof(char));
    char *source = malloc(27 * sizeof(char));

    for (int i = 0; i < 27; i++) {
        *(destination + i) = *(source + i);
    }

    free(destination);
    free(source);
    return 0;
}
```

Neispravan upis

```c
#include <stdint.h>
#include <stdlib.h>

int main() {
    char *alphabet = calloc(26, sizeof(char));

    for (int i = 0; i < 26; i++) {
        *(alphabet + i) = 'A' + i;
    }
    *(alphabet + 26) = '\0';

    free(alphabet);
    return 0;
}
```

```c
#include <stdint.h>
```

```c
#include <stdlib.h>

int main() {
    char *alphabet = calloc(27, sizeof(char));

    for (int i = 0; i < 26; i++) {
        *(alphabet + i) = 'A' + i;
    }
    *(alphabet + 26) = '\0';

    free(alphabet);
    return 0;
}
```

Sačuvati rezultat realokacije

```c
#include <stdint.h>
#include <stdlib.h>

struct _List {
    int *data;
    int length;
};
typedef struct _List List;

List *resizeArray(List *array) {
    int *dPtr = array->data;
    dPtr = realloc(dPtr, 15 * sizeof(int));
    return array;
}

int main() {
    List *array = calloc(1, sizeof(List));
    array->data = calloc(10, sizeof(int));
    array = resizeArray(array);

    free(array->data);
    free(array);
    return 0;
}
```

```c
#include <stdint.h>
#include <stdlib.h>

struct _List {
    int *data;
    int length;
};
typedef struct _List List;

List *resizeArray(List *array) {
    int *dPtr = array->data;
    dPtr = realloc(dPtr, 15 * sizeof(int));
```

```
    array->data = dPtr;
    return array;
}

int main() {
    List *array = calloc(1, sizeof(List));
    array->data = calloc(10, sizeof(int));
    array = resizeArray(array);

    free(array->data);
    free(array);
    return 0;
}
```

C++ niz i oslobađanje memorije

```
#include <cstdlib>
#include <iostream>

using namespace std;

int main();
void f(int n);

int main()
{
    int n = 10;

    cout << "\n";
    cout << "TEST01\n";
    cout << "  C++ version.\n";
    cout << "  A sample code for analysis by VALGRIND.\n";

    f(n);
    //
    //  Terminate.
    //
    cout << "\n";
    cout << "TEST01\n";
    cout << "  Normal end of execution.\n";

    return 0;
}

void f(int n)
{
    int i;
    int *x;

    x = (int *)malloc(n * sizeof(int));

    x[0] = 1;
    cout << "  " << 0 << "  " << x[0] << "\n";
```

```
    x[1] = 1;
    cout << "  " << 1 << "  " << x[1] << "\n";

    for (i = 2; i <= n; i++) {
        x[i] = x[i - 1] + x[i - 2];
        cout << "  " << i << "  " << x[i] << "\n";
    }

    delete[] x;

    return;
}
```

```
#include <cstdlib>
#include <iostream>

using namespace std;

int main();
void f(int n);

int main()
{
    int n = 10;

    cout << "\n";
    cout << "TEST01\n";
    cout << "  C++ version.\n";
    cout << "  A sample code for analysis by VALGRIND.\n";

    f(n);
    //
    //  Terminate.
    //
    cout << "\n";
    cout << "TEST01\n";
    cout << "  Normal end of execution.\n";

    return 0;
}

void f(int n)
{
    int i;
    int *x;

    x = (int *)malloc(n * sizeof(int));

    x[0] = 1;
    cout << "  " << 0 << "  " << x[0] << "\n";

    x[1] = 1;
    cout << "  " << 1 << "  " << x[1] << "\n";

    for (i = 2; i < n; i++) {
```

```
        x[i] = x[i - 1] + x[i - 2];
        cout << "  " << i << "  " << x[i] << "\n";
    }

    free(x);

    return;
}
```

Neinicijalizovani elementi niza

```
#include <cstdlib>
#include <iostream>

using namespace std;

void junk_data();
int main();

int main()
{
    cout << "\n";
    cout << "TEST02:\n";
    cout << "  C++ version\n";
    cout << "  A sample code for analysis by VALGRIND.\n";

    junk_data();
    //
    //  Terminate.
    //
    cout << "\n";
    cout << "TEST02\n";
    cout << "  Normal end of execution.\n";

    return 0;
}

void junk_data()
{
    int i;
    int *x;

    x = new int[10];

    for (i = 0; i < 5; i++) {
        x[i] = i;
    }

    x[2] = x[7];
    x[5] = x[6];

    for (i = 0; i < 10; i++) {
        x[i] = 2 * x[i];
    }
```

```
    for (i = 0; i < 10; i++) {
        cout << "  " << i << "  " << x[i] << "\n";
    }

    delete[] x;

    return;
}
```

```
#include <cstdlib>
#include <iostream>

using namespace std;

void junk_data();
int main();

int main()
{
    cout << "\n";
    cout << "TEST02:\n";
    cout << "  C++ version\n";
    cout << "  A sample code for analysis by VALGRIND.\n";

    junk_data();
    //
    //  Terminate.
    //
    cout << "\n";
    cout << "TEST02\n";
    cout << "  Normal end of execution.\n";

    return 0;
}

void junk_data()
{
    int i;
    int *x;

    x = new int[10];

    for (i = 0; i < 5; i++) {
        x[i] = i;
    }

    x[2] = x[7];
    x[5] = x[6];

    for (i = 0; i < 10; i++) {
        x[i] = 2 * x[i / 2];
    }

    for (i = 0; i < 10; i++) {
```

```
        cout << "  " << i << "  " << x[i] << "\n";
    }

    delete[] x;

    return;
}
```

7.2.3 Massif

Massif je profajler hip memorije - beleži iskorišćen prostor i dodatne bajtove koji se zauzimaju radi poravnanja i vođenja evidencije bajtova u upotrebi. Može da meri i zauzeće memorije na steku, ali to ne radi bez uključivanja dodatne opcije (`--stacks=yes`) jer dosta usporava rad alata.

Profajliranje hip memorije nam može pomoći da smanjimo korišćenu memoriju i otkrijemo neka curenja memorije koja se ne mogu prepoznati Memcheck-om. To je korisno kada memorija nije izgubljena, jer pokazivač i dalje postoji, ali se alocirani blok više ne koristi. Kod takvih programa nepotrebno se povećava memorija koja se koristi tokom vremena. Massif nam može reći koliko memorije na hipu program koristi i koja linija koda je odgovorna za alokaciju.

Kao i pri upotrebi drugih Valgrind alata, program prevodimo sa simbolima za debagovanje (opcija `-g`). Optimizacija neće uticati na količinu upotrebljene memorije. Massif svoj izveštaj upisuje u datoteku `massif.out.<pid>`, gde je `<pid>` ID procesa. Ukoliko želimo da se upiše u drugu datoteku, koristimo opciju `--massif-out-file` i navedemo naziv izlazne datoteke. Informacije iz izveštaja prikazujemo programom `ms_print` prosledujući mu datoteku sa izveštajem.

`ms_print` proizvodi graf koji prikazuje zauzeće memorije tokom izvršavanja programa, kao i detaljnije informacije o različitim tačkama programa koje su odgovorne za alokaciju/dealokaciju memorije. Vrednosti na y osi predstavljaju presek stanja iskorišćenosti memorije u određenom vremenskom trenutku. Na x osi, Massif podrazumevano koristi broj izvršenih instrukcija kao jedinicu vremena. To se može promeniti opcijom `--time-unit=B` da jedinica vremena bude broj bajtova alociran/dealociran na hipu.

Massif pravi preseke stanja iskorišćenosti memorije za svaku alokaciju i dealokaciju hipa, ali ako se program duže izvršava Massif ređe pravi preseke. Kada dostigne maksimalni broj preseka, odbaci oko pola ranijih preseka. Podrazumevan broj preseka koje čuva je 100, ali se to može promeniti opcijom `--max-snapshots`. Detaljnije obrađeni preseci su na grafu predstavljeni simbolom `@`. Podrazumevano detaljnije obrađuje svaki deseti presek, ali se i to može promeniti opcijom `--detailed-freq`.

```
$ valgrind --tool=massif ./massif_example.out
```

Na grafu je simbolom # predstavljen još jedan detaljan presek koji je obrađen i ujedno je presek sa najvećim iskorišćenjem memorije. Određivanje preseka sa najvećim iskorišćenjem memorije nije uvek tačno jer Massif uzima u obzir samo preseke kod kojih se desila dealokacija. Time se izbegava mnogo nepotrebnih preseka za najveću iskorišćenost memorije, npr. u slučaju da program iz više navrata alocira dosta memorije, i svaki put to je nova najveća iskorišćenost memorije. Takođe, ako program nikada ne dealocira memoriju, nećemo ni imati ovakav presek. Takođe ako program i dealocira memoriju ali kasnije alocira još veći blok koji kasnije ne oslobodi, imaćemo presek sa najvećom iskorišćenosti memorije, ali će on biti dosta niži od stvarnog.

Alat Massif meri samo hip memoriju, tj. onu koju smo alocirali funkcijama `malloc`, `calloc`, `realloc`, `memalign`, `new`, `new[]`. Ne meri memoriju alociranu sistemskim pozivima kao što su `mmap`, `mremap`, `brk`. Ukoliko nam je od značaja merenje celokupne alocirane memorije, potrebno je uključiti opciju `--pages-as-heap=yes`. Sa ovom opcijom Massif neće profajlirati hip memoriju, već stranice u memoriji.

7.2.4 DHAT

DHAT je Valgrind alat za analizu ponašanja hip alokacija. Koristan je kada program nema očigledno curenje memorije, ali pravi previše kratkoživećih alokacija, često realocira iste podatke ili koristi mali deo alociranog bloka.

DHAT ne zamenjuje Memcheck. Memcheck prvo treba da potvrdi da program nema neispravna čitanja, pisanja i curenja. Nakon toga DHAT prikazuje kako se memorija koristi: koliko blokova je alocirano, koliko bajtova prolazi kroz alokator, koliko dugo blokovi žive i koliko se često čitaju ili pišu.

Primer u datoteci `dhat_churn_bad.c` pravi veliki broj malih privremenih stringova. Svaki prolaz alocira niz pokazivača, zatim za svaki zapis alocira zaseban string i odmah ga oslobađa na kraju prolaza.

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

enum { RECORDS = 5000, PASSES = 20, WIDTH = 64 };

static char *make_record(int id, int pass) {
    char buffer[WIDTH];
    int written =
        snprintf(buffer, sizeof(buffer), "student:%05d:pass:%02d:verification",
                 id, pass);
    if (written < 0) {
        exit(2);
    }

    char *record = malloc((size_t)written + 1);
    if (record == NULL) {
        exit(1);
    }

    memcpy(record, buffer, (size_t)written + 1);
    return record;
}

int main(void) {
    unsigned long checksum = 0;

    for (int pass = 0; pass < PASSES; pass++) {
        char **records = calloc(RECORDS, sizeof(*records));
        if (records == NULL) {
            return 1;
        }
```

```
        for (int i = 0; i < RECORDS; i++) {
            records[i] = make_record(i, pass);
            checksum += (unsigned char)records[i][0];
            checksum += (unsigned char)records[i][(i + pass) % 16];
        }

        for (int i = 0; i < RECORDS; i++) {
            free(records[i]);
        }
        free(records);
    }

    printf("checksum=%lu\n", checksum);
    return 0;
}
```

Popravljena verzija zadržava isti izlazni radni tok, ali koristi jedan radni bafer za sve zapise.

```
#include <stdio.h>
#include <stdlib.h>

enum { RECORDS = 5000, PASSES = 20, WIDTH = 64 };

int main(void) {
    unsigned long checksum = 0;
    char(*records)[WIDTH] = malloc(RECORDS * sizeof(*records));
    if (records == NULL) {
        return 1;
    }

    for (int pass = 0; pass < PASSES; pass++) {
        for (int i = 0; i < RECORDS; i++) {
            int written =
                snprintf(records[i], WIDTH,
                         "student:%05d:pass:%02d:verification", i, pass);
            if (written < 0 || written >= WIDTH) {
                free(records);
                return 2;
            }

            checksum += (unsigned char)records[i][0];
            checksum += (unsigned char)records[i][(i + pass) % 16];
        }
    }

    free(records);
    printf("checksum=%lu\n", checksum);
    return 0;
}
```

Programi se prevode i pokreću ovako:

```
$ make
```

```
$ valgrind --tool=dhat ./dhat_churn_bad.out
$ valgrind --tool=dhat ./dhat_churn_fixed.out
```

DHAT pravi datoteku oblika `dhat.out.<pid>`. Izveštaj se otvara kroz HTML prikazivač koji dolazi uz Valgrind:

```
$ firefox /usr/lib/valgrind/dh_view.html
```

Putanja do `dh_view.html` zavisi od distribucije. Ako komanda iznad ne postoji, pronađite je komandom:

```
$ find /usr -name dh_view.html 2>/dev/null
```

U poređenju ova dva programa obratiti pažnju na sledeće stavke u DHAT izveštaju:

- **Total blocks**: loša verzija pravi mnogo više alokacija.
- **Total bytes**: loša verzija propušta više bajtova kroz alokator.
- **At t-gmax**: pokazuje mesto najveće zauzetosti hipa.
- **Reads/Writes**: pomaže da se vidi da li je alocirani blok zaista korišćen ili je samo rezervisan.

Za vežbu prvo proveriti obe verzije Memcheck-om, zatim uporediti DHAT izveštaje. Očekivani zaključak je da popravljena verzija ne menja semantiku programa, ali značajno smanjuje broj hip alokacija i ukupan promet kroz alokator.

7.2.5 Cachegrind

Merenje performansi keša je postalo važno jer se eksponencijalno povećava razlika u brzini RAM memorije i performansi procesora. Uloga keša je da premosti tu razliku u brzini. Da bismo utvrdili koliko je keš sposoban da to učini prate se pogodci (*hits*) i promašaji (*misses*) keša. Jedan od koraka za poboljšanje je da se smanji broj promašaja na najvišim nivoima keša.

Cachegrind je alat koji omogućava softversko profajliranje keš memorije tako što simulira i prati pristup keš memoriji mašine na kojoj se program izvršava. Može se koristiti i za profajliranje izvršavanja grana, korišćenjem opcije `--branch-sim=yes`. Cachegrind simulira memoriju mašine koja ima prvi nivo keš memorije podeljen u dve odvojene sekcije: **I1** - sekcija keš memorije u koju se smeštaju instrukcije i **D1** - sekcija keš memorije u koju se smeštaju podaci.

Drugi nivo keš memorije koju Cachegrind simulira je objedinjen - **LL**, skraćeno od eng. *last level*. Ovaj način konfiguracije odgovara mnogim modernim mašinama. Postoje mašine koje imaju i tri ili četiri nivoa keš memorije. U tom slučaju, Cachegrind simulira pristup prvom i poslednjem nivou. Generalno gledano, Cachegrind simulira I1, D1 i LL (poslednji nivo keš memorije). Cachegrind prikuplja sledeće statističke podatke o programu koji analizira:

- O čitanjima instrukcija iz keš memorije:
  - **Ir** - ukupan broj izvršenih instrukcija
  - **I1mr** - broj promašaja čitanja instrukcija iz keš memorije nivoa I1

    - **ILmr** - broj promašaja čitanja instrukcija iz keš memorije nivoa LL

- O čitanjima brze memorije:
    - **Dr** - ukupan broj čitanja memorije
    - **D1mr** - broj promašaja čitanja nivoa keš memorije D1
    - **DLmr** - broj promašaja čitanja nivoa keš memorije LL

- O pisanjima u brzu memoriju:
    - **Dw** - ukupan broj pisanja u memoriji
    - **D1mw** - broj promašaja pisanja u nivo keš memorije D1
    - **DLmw** - broj promašaja pisanja u nivo keš memorije LL

- O izvršavanju grana:
    - **Bc** - broj uslovno izvršenih grana
    - **Bcm** - broj promašaja uslovno izvršenih grana
    - **Bi** - broj indirektno izvršenih grana
    - **Bim** - broj promašaja indirektno izvršenih grana

Broj pristupa D1 delu keš memorije jednak je zbiru statistika D1mr i D1mw. Ukupan broj pristupa LL nivou jednak je zbiru ILmr, DLmr i DLmw. Primeri u C/C++ koji proizvode indirektno grananje su pozivi funkcija preko pokazivača na funkcije ili pozivi virtuelnih funkcija i `switch` naredbe. Uslovno grananje se generiše `if` naredbama i uslovnim ternarnim operatorom `?:`.

Statistika se prikuplja na nivou celog programa, kao i pojedinačno na nivou funkcija. Na modernim mašinama L1 promašaj košta oko 10 procesorskih ciklusa, LL promašaj košta oko 200 procesorskih ciklusa, a promašaji uslovno i indirektno izvršene grane od 10 do 30 procesorskih ciklusa. Detaljno profajliranje upotrebe keš memorije može pomoći u poboljšanju performansi programa. Izvršavanjem komande `lscpu` na Linux sistemima, dobićemo informacije o veličini keša na mašini na kojoj radimo.

Program koji želimo da analiziramo propuštamo kroz Cachegrind navodeći opciju `--tool=cachegrind`. Za razliku od ostalih Valgrind alata želimo uključenu optimizaciju, tako da ne koristimo opciju `-O0` prilikom prevođenja programa. Kompilacija treba da bude sa optimizacijom, jer nema smisla ovako profajlirati kod koji je drugačiji od onoga koji će se normalno izvršavati. Na standardni izlaz se ispisuju sumarne informacije na nivou celog programa, dok se detaljne informacije upisuju u `cachegrind.out.<pid>` datoteku, gde `<pid>` predstavlja ID procesa. Alat grupiše sve troškove po datotekama i funkcijama kojima ti troškovi pripadaju.

Detaljniji izveštaj možemo videti iz iste datoteke korišćenjem alata `cg_annotate`. Ukoliko imamo izveštaje iz više pokretanja Cachegrind-a za isti program, možemo ih sumirati u jednu datoteku korišćenjem alata `cg_merge` i njegov izlaz kasnije pregledati alatom `cg_annotate`. Moguće je i praviti razliku između više izveštaja Cachegrind-a pomoću alata `cg_diff` i njegov izlaz, slično, otvoriti pomoću `cg_annotate`. Ukoliko modifikujemo kod, to nam može koristiti da pratimo kako modifikacija utiče na performanse programa. Programom `cg_annotate` podrazumevano se prikazuje izveštaj sumiran po funkcijama.

Ako kolona sadrži samo tačkicu, to označava da funkcija ne sadrži instrukcije koje bi prouzrokovale taj događaj. Ukoliko u koloni za ime funkcije stoji `???`, to znači da nije bilo moguće odrediti ime na osnovu simbola za debagovanje. Ukoliko većina redova sadrži `???` za ime funkcije program verovatno nije preveden sa simbolima za debagovanje (opcija `-g`).

Ukoliko želimo da vidimo izveštaj o broju promašaja po linijama koda, potrebno je da prosledimo izvorne datoteke programu `cg_annotate` ili da uključimo opciju `--auto=yes` kada će se anotirati svaki izvorni kod koji se može naći.

Preglednije je gledati izveštaj pomoću alata KCachegrind.

Cachegrind nam može otkriti gde imamo usko grlo u programu, ali nam ne može reći kako da ga popravimo. Potrebno je napomenuti da su rezultati vrlo osetljivi. Promena veličine izvršnog programa ili neke od deljenih biblioteka koje koristi, pa čak i promena dužine imena datoteka, može izmeniti rezultate. Varijacije će biti male, ali svakako ne treba očekivati potpuno ponovljive rezultate ako se program promeni.

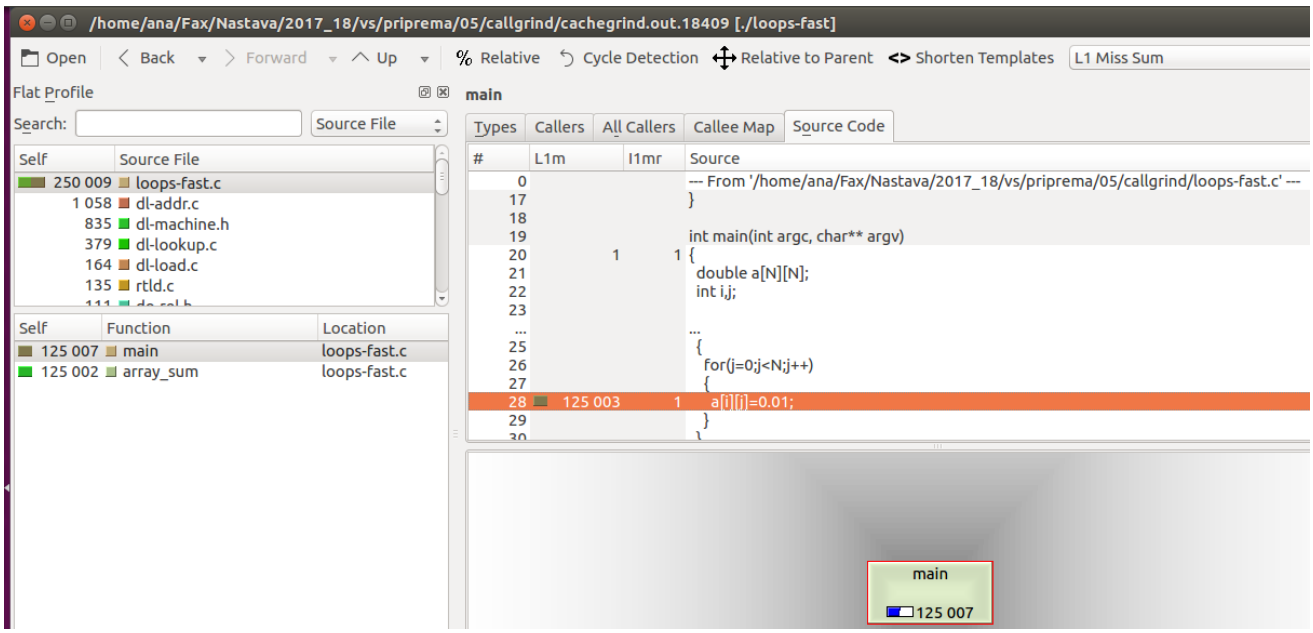
Na novijim GNU/Linux distribucijama se prilikom ponovnih pokretanja istog programa deljene biblioteke učitavaju na različite lokacije, iz bezbednosnih razloga. To takođe može uticati na varijacije u rezultatima različitog profajliranja istog programa.

Dakle, ne treba se uzdati da su podaci veoma precizni, ali su svakako korisni.

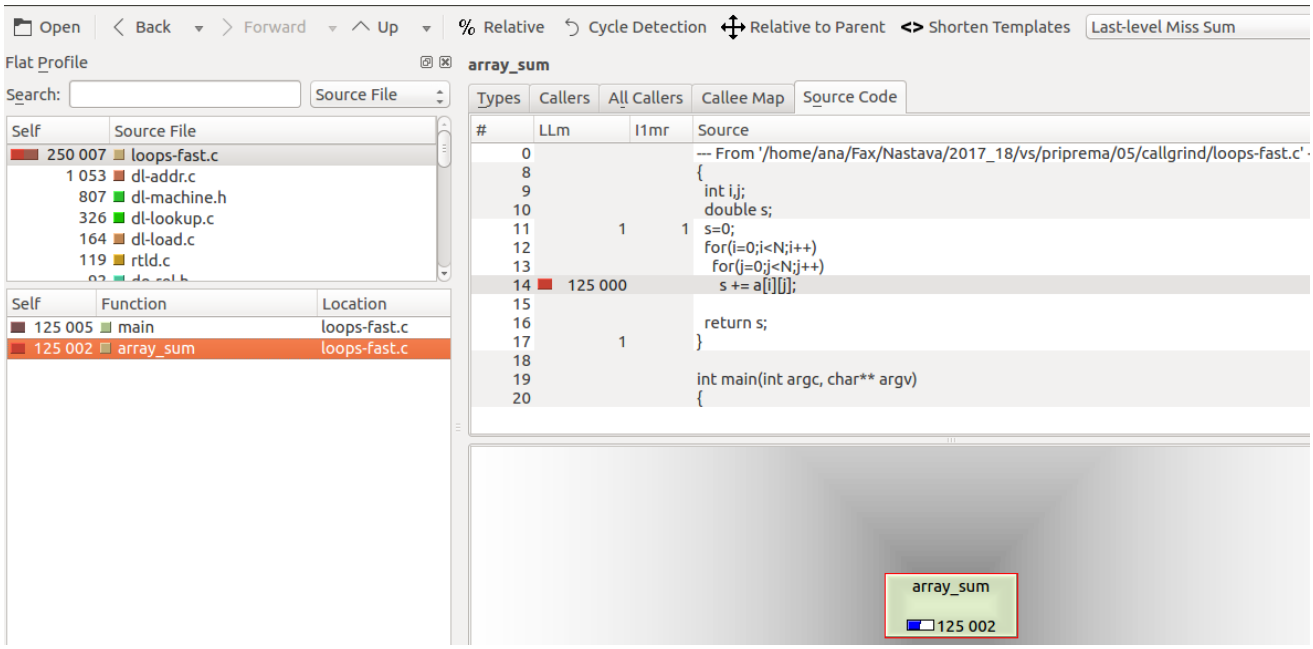
Primer: Petlje Matrica se popunjava u `main` funkciji, a čita u funkciji `array_sum`. Iz konzole izvršavamo komandu:

```
$ valgrind --tool=cachegrind --cache-sim=yes ./loops-fast
$ kcachegrind cachegrind.out.*
```

Na ekranu vidimo izveštaj na nivou celog programa i vidimo da program ima preko 125 000 promašaja prilikom čitanja podataka sa D1 i LL nivoa keša.



Istaknute su nam konkretne linije u kojima je bilo promašaja, zajedno sa brojem promašaja. Uočavamo da imamo po 125 000 promašaja obe petlje, i u main i u array sum funkciji.



Cela matrica ima `1000 x 1000 x 8B = 8MB`. Veličina keša [3], npr:

```
L1d cache:                32K
L1i cache:                32K
L2 cache:                256K
L3 cache:               6144K   ---> LL
```

```
$ getconf -a | grep CACHE
```

Svakako premali za celu matricu, pa se zbog toga i događaju promašaji. Ukoliko bi LL keš bio dovoljno veliki (>8MB) u funkciji array sum ne bismo imali promašaje na LL nivou, jer se cela matrica učita u keš pre poziva funkcije u petlji u main funkciji.

Ponovo pokrećemo program, ali ovaj put Cachegrind -u zadajemo 16MB za veličinu LL keša:

```
$ valgrind --tool=cachegrind --cache-sim=yes --LL=16777216,8,64 ./loops-fast
```

Izveštaj možemo iz konzole čitati komandom

```
$ cg_annotate --auto=yes cachegrind.out.*
```

```
--------------------------------------------------------------------------------
I1 cache:         32768 B, 64 B, 8-way associative
D1 cache:         32768 B, 64 B, 8-way associative
LL cache:         16777216 B, 64 B, 2-way associative
Command:          ./loops-fast
Data file:        cachegrind.out.26469
```

[3] Veličina keša i linije na konkretnom procesoru se može saznati izvršavanjem:

```
Events recorded:  Ir I1mr ILmr Dr D1mr DLmr Dw D1mw DLmw
Events shown:     Ir I1mr ILmr Dr D1mr DLmr Dw D1mw DLmw
Event sort order: Ir I1mr ILmr Dr D1mr DLmr Dw D1mw DLmw
Thresholds:       0.1 100 100 100 100 100 100 100 100
Include dirs:
User annotated:
Auto-annotation:  on

--------------------------------------------------------------------------------
       Ir  I1mr  ILmr        Dr    D1mr  DLmr        Dw    D1mw    DLmw
--------------------------------------------------------------------------------
9,170,730 1,068 1,053 2,040,968 127,541 1,976 1,012,294 125,603 125,487  PROGRAM TOTALS

--------------------------------------------------------------------------------
       Ir I1mr ILmr        Dr    D1mr DLmr        Dw    D1mw    DLmw  file:function
--------------------------------------------------------------------------------
5,004,015    2    2 1,000,004       3    1 1,000,002 125,002 124,928  loops-fast.c:main
4,004,004    1    1 1,000,001 125,001    0         0       0       0  loops-fast.c:array_sum
   56,470    9    9    13,757   1,047  868        17       2       1
   ↪  /build/glibc-Cl5G7W/glibc-2.23/elf/dl-addr.c:_dl_addr
   21,444   20   20     7,663     187  133     3,329      11       6
   ↪  /build/glibc-Cl5G7W/glibc-2.23/elf/dl-lookup.c:do_lookup_x
   17,173   10   10     3,626      92   80     1,993      12       8
   ↪  /build/glibc-Cl5G7W/glibc-2.23/elf/dl-lookup.c:_dl_lookup_symbol_x
   16,311   20   20     3,949     555  535     2,048     243     235
   ↪  /build/glibc-Cl5G7W/glibc-2.23/elf/../sysdeps/x86_64/dl-machine.h:_dl_relocate_object

--------------------------------------------------------------------------------
-- Auto-annotated source: loops-fast.c
--------------------------------------------------------------------------------
       Ir I1mr ILmr        Dr    D1mr DLmr        Dw    D1mw    DLmw

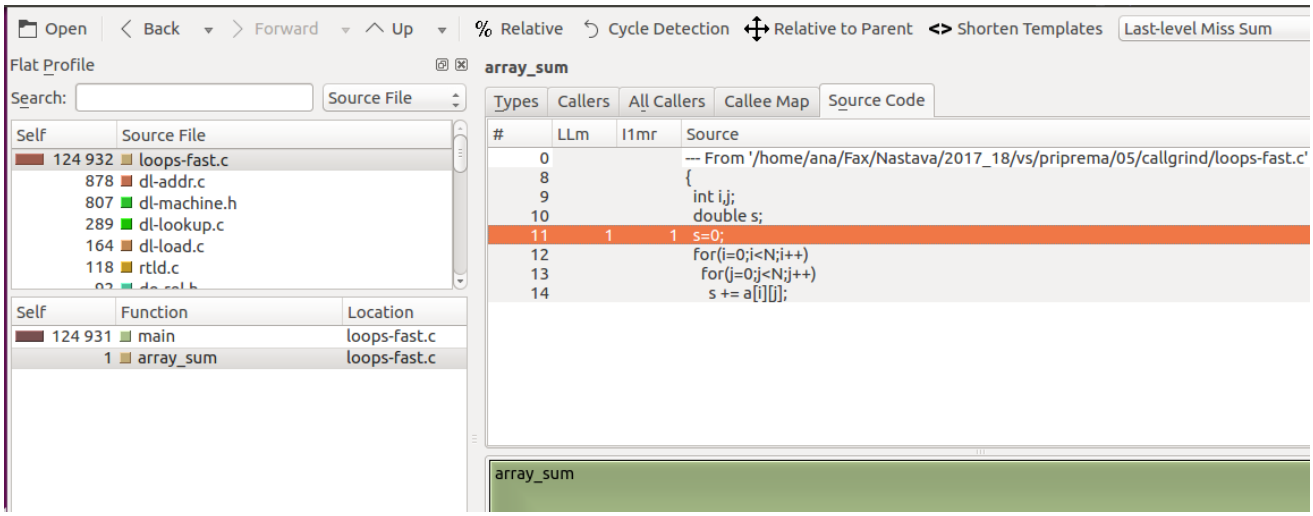
-- line 3 ----------------------------------------
        .    .    .         .       .    .         .       .       .  #define N 1000
        .    .    .         .       .    .         .       .       .
        .    .    .         .       .    .         .       .       .  double array_sum(double[N][N]);
        .    .    .         .       .    .         .       .       .
        .    .    .         .       .    .         .       .       .  double array_sum(double
        ↪  a[N][N])
        .    .    .         .       .    .         .       .       .  {
        .    .    .         .       .    .         .       .       .    int i,j;
        .    .    .         .       .    .         .       .       .    double s;
    2,003    1    1         0       0    0         0       0       0    s=0;
    2,000    0    0         0       0    0         0       0       0    for(i=0;i<N;i++)
2,000,000    0    0         0       0    0         0       0       0      for(j=0;j<N;j++)
2,000,000    0    0 1,000,000 125,000    0         0       0       0        s += a[i][j];
        .    .    .         .       .    .         .       .       .
        .    .    .         .       .    .         .       .       .    return s;
        1    0    0         1       1    0         0       0       0  }
        .    .    .         .       .    .         .       .       .
        .    .    .         .       .    .         .       .       .  int main(int argc, char** argv)
    2,007    1    1         1       0    0         1       0       0  {
        .    .    .         .       .    .         .       .       .    double a[N][N];
        .    .    .         .       .    .         .       .       .    int i,j;
        .    .    .         .       .    .         .       .       .
    2,000    0    0         0       0    0         0       0       0    for(i=0;i<N;i++)
```

```
        .    .    .          .       .    .          .       .       .    {
2,000,000    0    0          0       0    0          0       0       0      for(j=0;j<N;j++)
        .    .    .          .       .    .          .       .       .      {
3,000,000    1    1 1,000,000       1    1 1,000,000 125,001 124,928        a[i][j]=0.01;
        .    .    .          .       .    .          .       .       .      }
        .    .    .          .       .    .          .       .       .    }
        .    .    .          .       .    .          .       .       .
        .    .    .          .       .    .          .       .       .    /* this is just to prevent
        ↪  the compiler
        .    .    .          .       .    .          .       .       .       from optimizing the upper
        ↪  loop away */
        2    0    0          0       0    0          1       1       0    printf("sum =
        ↪  %10.3f\n",array_sum(a));
        .    .    .          .       .    .          .       .       .
        .    .    .          .       .    .          .       .       .    return 0;
        6    0    0          3       2    0          0       0       0  }

--------------------------------------------------------------------------------
The following files chosen for auto-annotation could not be found:
--------------------------------------------------------------------------------
  /build/glibc-Cl5G7W/glibc-2.23/elf/dl-lookup.c
  /build/glibc-Cl5G7W/glibc-2.23/elf/../sysdeps/x86_64/dl-machine.h
  /build/glibc-Cl5G7W/glibc-2.23/elf/dl-addr.c

--------------------------------------------------------------------------------
Ir I1mr ILmr Dr D1mr DLmr Dw D1mw DLmw
--------------------------------------------------------------------------------
98    0    0 98   98    0 99  100  100  percentage of events annotated
```

ili otvaranjem u KCachegrind -u.

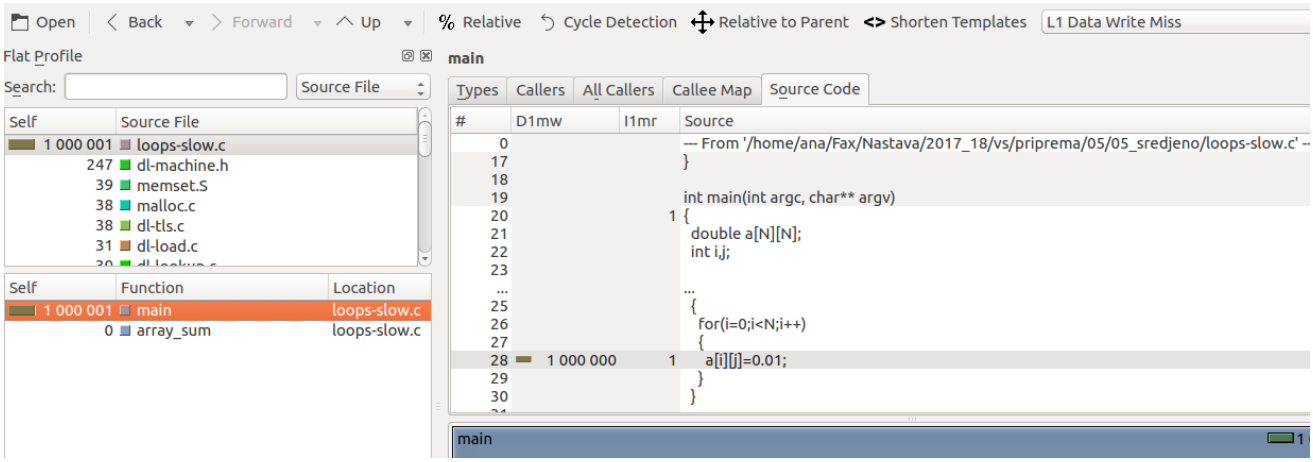


U programu `loops_slow` se elementima matrice ne pristupa po vrstama već po kolonama.

```
$ valgrind --tool=cachegrind --cache-sim=yes ./loops-slow
```

Prilikom učitavanja podataka u keš, učitavaju se zajedno podaci koji su blizu u memoriji pod pretpostavkom da će biti uskoro upotrebljeni. Tako se učitava pored jednog elementa vrste i oni

koji su za njim u vrsti jer nam je matrica tako alocirana. U petlji u main funkciji pristupamo jednom elementu, a potom narednom elementu iz iste kolone. Ta dva elementa, za dovoljno veliku matricu ne moraju biti zajedno učitani u keš. Ukoliko je D1 keš veličine 32KB i ako je veličina linije keša 64B, tj. 8 double vrednosti, imamo 512 linija keša. Prilikom pristupanja elementu `a[0][0]`, potrebno ga je dovući u keš D1. Sa njim će u D1 biti učitano i narednih 7, do `a[0][7]`. U narednom koraku nam treba element `a[1][0]` i njega nema u kešu, imamo još jedan promašaj. Možemo imati najviše 512 linija istovremeno prisutnih u kešu stoga posle prvog elementa 512. vrste, moraće da se izbacuju najranije učitane linije, da bi se napravio prostor za nove podatke [4]. Iz tog razloga imamo 1000000 promašaja na D1 prilikom pisanja u matricu. Nijedan element matrice nije u D1 kešu u trenutku kada je potreban.



Ukoliko povećamo D1 keš na 64KB, imaćemo sličnu situaciju kao sa prethodnim programom sa oko 125000 promašaja prilikom pisanja na kešu D1.

```
$ valgrind --tool=cachegrind --cache-sim=yes --D1=65536,8,64 ./loops-slow
```

Sa toliko velikim D1 kešom možemo imati 1024 linije u jednom trenutku, pa je onda nakon prvih 1000 promašaja za prvi element svake vrste, istovremeno u D1 keš učitano prvih 8 kolona matrice, pa za narednih 7 kolona nećemo imati promašaje. Dakle, otprilike svaka 8 kolona će generisati novih 1000 promašaja. Iz tog razloga imamo oko 125000 promašaja prilikom pisanja na D1. Programu `loops_fast` je dovoljan duplo manji D1 keš za iste performanse.

Primer: Množenje matrica Programi: `matrix_slowest`, `matrix_slower`, `matrix_medium` i `matrix_mediumish`, implementiraju množenje matrica.

`matrix_slowest` program rešava problem naivno primenjujući formulu. Uzrok sporosti izvršavanja tog programa leži u velikom broju promašaja u kešu. Oni nastaju kao posledica toga što je potrebno `i`-tu vrstu matrice `A` pomnožiti skalarno sa `j`-tom kolonom matrice `B`, a elementi kolone se ne nalaze na sukcesivnim adresama u memoriji.

Podesimo `N` na `10` i profajliramo Cachegrind-om. Primećujemo da imamo malo promašaja, najviše na D1.

[4] Pretpostavljamo da je politika zamene linija keša takva da se najranije ubačene linije izbace prve.

```
==20967== Cachegrind, a cache and branch-prediction profiler
==20967== Copyright (C) 2002-2015, and GNU GPL'd, by Nicholas Nethercote et al.
==20967== Using Valgrind-3.11.0 and LibVEX; rerun with -h for copyright info
==20967== Command: ./matrix_slow
==20967==
--20967-- warning: L3 cache found, using its data for the LL simulation.
==20967==
==20967== I refs: 2,186,832
==20967== I1 misses: 1,530
==20967== LLi misses: 1,455
==20967== I1 miss rate: 0.07%
==20967== LLi miss rate: 0.07%
==20967==
==20967== D refs: 732,348 (537,801 rd + 194,547 wr)
==20967== D1 misses: 15,790 ( 13,572 rd + 2,218 wr)
==20967== LLd misses: 9,191 ( 7,761 rd + 1,430 wr)
==20967== D1 miss rate: 2.2% ( 2.5% + 1.1% )
==20967== LLd miss rate: 1.3% ( 1.4% + 0.7% )
==20967==
==20967== LL refs: 17,320 ( 15,102 rd + 2,218 wr)
==20967== LL misses: 10,646 ( 9,216 rd + 1,430 wr)
==20967== LL miss rate: 0.4% ( 0.3% + 0.7% )
```

Probajmo da profajliramo program za daleko veće `N`, tako da matrica ne može stati u keš. Tri matrice 1000x1000 `float` brojeva su dovoljno velike da ne bi bile zajedno u kešu.

Broj promašaja u `main` funkciji bi mogao da se smanji razdvajanjem obrade svake matrice u zasebnu petlju, a ne da se sve 3 matrice inicijalizuju u jednoj dvostrukoj petlji.

Izvršavati svaki program i meriti vreme:

```
$ time ./matrix_slowest
$ time valgrind --tool=cachegrind --cache-sim=yes ./matrix_slowest
```

i primetiti koliko puta Valgrind usporava rad programa.

Probati da se zamene neki redosledi petlji, npr, po `col` i `row`, i ispratiti broj promašaja u kešu.

7.2.6 Callgrind

Callgrind je alat koji generiše listu poziva funkcija korisničkog programa u vidu grafa. U osnovnim podešavanjima sakupljeni podaci obuhvataju broj izvršenih instrukcija, njihov odnos sa linijom u izvršnom kodu, odnos pozivalac/pozvani između funkcija, kao i broj takvih poziva.

Podaci koji se analiziraju nakon završetka rada programa i alata zapisuju se u datoteku `callgrind.out.<pid>`, gde `<pid>` predstavlja identifikator procesa. Program `callgrind_annotate` na osnovu generisanog izveštaja prikazuje listu funkcija. Za grafičku vizuelizaciju preporučeno je koristiti alate kao npr. KCachegrind, koji olakšava navigaciju ukoliko Callgrind napravi veliku količinu podataka.

Program `callgrind_control` omogućava interaktivnu kontrolu i nadgledanje programa prilikom izvršavanja. Mogu se dobiti informacije o stanju na steku, a može se i u svakom trenutku generisati profil.

Alat Cachegrind sakuplja podatke, odnosno broji događaje koji se dešavaju direktno u jednoj funkciji. Ovaj mehanizam sakupljanja podataka se naziva ekskluzivnim. Callgrind ovu funkcionalnost proširuje tako što propagira cenu funkcije preko njenih granica. Na primer, ako funkcija `foo` poziva funkciju `bar`, cena funkcije `bar` se dodaje funkciji `foo`. Kada se ovaj mehanizam primeni na celu funkciju, dobija se slika takozvanih inkluzivnih poziva, gde cena svake funkcije uključuje i cene svih funkcija koje ona poziva, direktno ili indirektno.

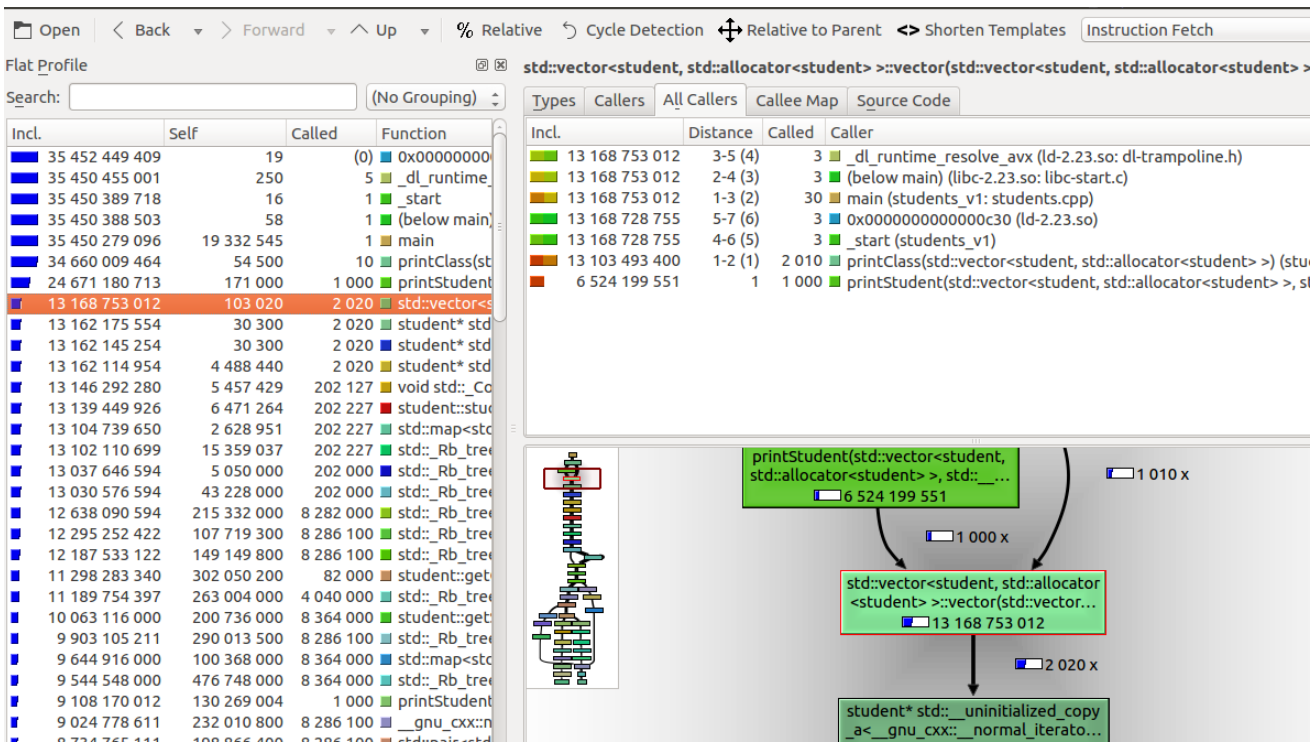
Zahvaljujući grafu poziva, može da se odredi, počevši od `main` funkcije, koja funkcija ima najveću cenu poziva. Odnos pozivalac/pozvani koristan je za profajliranje funkcija koje imaju više poziva iz raznih funkcija. Tada možemo optimizovati program promenom koda u funkciji koja poziva, odnosno smanjenjem broja poziva.

Dodatno može da vrši analizu upotrebe keš memorije i profajliranje grana programa slično kao kod Cachegrind-a. Za takvo profajliranje potrebno je uključiti opcije `--cache-sim=yes` i `--branch-sim=yes`. Stoga Callgrind možemo smatrati i proširenjem Cachegrind-a.

Primer: Studenti Program učitava imena i prezimena studenata iz datoteke `students.txt` i predmete iz datoteke `subjects.txt` i potom ispisuje informacije o studentima i prosečnim ocenama, po predmetu i u globalu. Ukupno 100 studenata sa po 100 ocena iz svakog od 40 predmeta. Spiskovi sa studentskim rezultatima ispisuju se 10 puta. Analiziramo program Callgrind-om:

```
$ valgrind --tool=callgrind --cache-sim=yes ./students.out
```

Otvaramo izveštaj u KCachegrind-u. Gledamo sa leve strane broj pozivanja svake funkcije i broj instrukcija koji je zahtevalo njeno izvršavanje, samostalno i uključujući izvršavanja drugih funkcija koje je pozivala. Sa desne strane možemo izabrati prikaz mape poziva i sa izborom *All Callers*, videćemo koje su sve funkcije pozivale funkciju od interesa.



Zanima nas zašto se funkcija iz standardne biblioteke poziva 2020 puta. Ako iz alata pogledamo izvorni kod, videćemo da se pri samom početku funkcije `printClass` konstruktor kopije za `std::vector` poziva 10 puta. To je broj generisanja izveštaja i broj poziva ove funkcije. Problem je što se ceo vektor studenata u funkciju prenosi po vrednosti, a ne po referenci. Zato se prilikom svakog poziva kreira kopija za konkretan poziv. Izmenimo kod i primetno ćemo ubrzati izvršavanje programa.

Primer: Prosti brojevi Program koristi ne tako jeftinu bibliotečku funkciju `sqrt`.

- Izmeniti funkciju tako da se ne koristi promenljiva limit već da se u uslov for petlje stavi poziv funkcije `sqrt`. Kako to utiče na broj instrukcija programa?
- Izmenom programa da umesto upotrebe `sqrt` imamo u petlji izlazak ukoliko `i*i > x` stvari se ne popravljaju. Funkcija se poziva 1000000 puta i pre svake naredne iteracije petlje proverava se uslov ostanka u petlji. Koliko god da je brže množenje od korenovanja, pomnoženo sa nekoliko miliona, čak ima i veći broj instrukcija nego funkcija koja koristi korenovanje.
- Izmeniti uslov petlje na `i < x/2`. Izmeniti petlju tako da se pre petlje proverava za parne brojeve, a posle u petlji da se iterira samo po neparnim brojevima.
- Sklopiti najefikasniju implementaciju.

Primer: Pretraga niski Program pretražuje nisku `txt` tražeći podnisku `pat`. U programu se 10 puta vrši pretraga iste niske i iste podniske koristeći 3 različita algoritma. Pomoću Callgrind-a možemo utvrditi koji od algoritama najviše instrukcija izvršava.

Izvorni kod je u datoteci `pattern_search.c`. Primer sadrži naivnu pretragu, Boyer-Moore pretragu sa heuristikom lošeg karaktera i Knuth-Morris-Pratt pretragu. Pošto se isti ulaz obrađuje više puta, Callgrind jasno prikazuje cenu svake implementacije.

Primer se prevodi i profajlira iz direktorijuma primera:

```
$ make
$ valgrind --tool=callgrind ./pattern_search.out
$ callgrind_annotate callgrind.out.*
```

U izveštaju treba porediti broj instrukcija u funkcijama `search_naive`, `search_boyer` i `search_kmp`. Dobar signal za optimizaciju nije samo ukupna cena funkcije, već i broj poziva pomoćnih funkcija, na primer računanje tabele prefiksa kod KMP algoritma ili popunjavanje tabele loših karaktera kod Boyer-Moore algoritma.

Za grafički pregled može se otvoriti isti izveštaj u alatu KCachegrind. Tada je korisno krenuti od `main` funkcije i pratiti inkluzivnu cenu ka funkcijama za pretragu.

Primer: Sortiranje Program izvršava funkcije koje implementiraju različite algoritme sortiranja, na rastuće ili opadajuće sortiranom nizu ili nasumice generisanom nizu celih brojeva. U zavisnosti od izbora ulaznog niza neki algoritmi se pokazuju daleko bolje od očekivanog.

Izvorni kod je podeljen na `main.c`, `sort.c` i `sort.h`. Program generiše niz, pravi kopiju za svaki algoritam i zatim redom pokreće selection sort, insertion sort, bubble sort, shell sort, merge sort i quick sort.

Primer se može prevesti ručno:

```
$ gcc -O2 -g -fno-inline main.c sort.c -o sort.out
```

Pokretanje sa nasumičnim nizom:

```
$ valgrind --tool=callgrind ./sort.out 10000
```

Pokretanje sa rastuće i opadajuće sortiranim nizom:

```
$ valgrind --tool=callgrind ./sort.out -r 10000
$ valgrind --tool=callgrind ./sort.out -o 10000
```

U izveštaju treba porediti funkcije `selection_sort`, `insertion_sort`, `bubble_sort`, `shell_sort`, `merge_sort` i `quick_sort`. Posebno je korisno uporediti kako se broj instrukcija menja kada je ulaz već sortiran ili sortiran opadajuće. Na tim ulazima neki algoritmi imaju najbolji slučaj, a neki najgori slučaj.

Ako je izveštaj prevelik, smanjiti dimenziju niza. Cilj primera nije apsolutno merenje vremena, već poređenje oblika troška između algoritama.

7.2.7 Helgrind / DRD

Helgrind je alat u sklopu programskog paketa Valgrind koji otkriva greške sinhronizacije prilikom upotrebe modela niti `POSIX`.

Glavne apstrakcije modela `POSIX` niti su: grupa niti koji deli zajednički adresni prostor, formiranje niti, čekanje na završetak izvršavanja funkcije niti, izlaz iz funkcije niti, muteks objekti, uslovne promenljive, čitaj-piši zaključavanje i semafori.

Problemi poput ovih imaju kao rezultat nereproduktivne, vremenski zavisne padove, mrtve petlje i druga loša ponašanja programa koja se mogu teško otkriti drugim sredstvima. *Helgrind* je svestan svih apstrakcija niti i prati njihove efekte. Najbolje radi ukoliko program koristi isključivo `POSIX` niti. Moguće ga je koristiti i za programe koji koriste druge standarde za niti, ali je neophodno opisati *Helgrind*-u njihovo ponašanje korišćenjem `ANNOTATE_*` makroa definisanih u `helgrind.h` zaglavlju).

DRD je *Valgrind*-ov alat za detekciju grešaka u *C* i *C++* programima koji koriste više niti. Alat radi za svaki program koji koristi niti POSIX standarda ili koji koriste koncepte koji su nadograđeni na ovaj standard.

	Helgrind	DRD
Mrtve petlje	DA	NE
Trka za podacima	DA	DA
Nepravilno korišćenje API-ja	DA	DA
Zadržavanje podataka	NE	DA

Alati *DRD* i *Helgrind* ne koriste iste algoritme za otkrivanje grešaka, samim tim ne otkrivaju iste tipove grešaka, iako imaju veliki broj poklapanja. Helgrind proizvodi izlaze koji su lakši za interpretaciju, dok *DRD* ima bolje performanse.

Loša upotreba interfejsa za programiranje `POSIX` niti Mnoge implementacije interfejsa su optimizovane radi kraćeg vremena izvršavanja. Takve implementacije se neće buniti na određene greške (ako muteks otključa neka druga nit, a ne ona koja ga je zaključala).

Greške koje Helgrind i DRD pronalaze su:

- Greške u otključavanju muteksa - kada je muteks nevažeći, nije zaključan ili je zaključan od strane druge niti;
- Greške u radu sa zaključanim muteksom - uništavanje nevažećeg ili zaključanog muteksa, rekurzivno zaključavanje nerekurzivnog muteksa, dealokacija memorije koja sadrži zaključan muteks;
- Prosleđivanje muteksa kao argumenta funkcije koja očekuje kao argument `reader-writer lock` i obrnuto;
- Greške sa `pthread barrier` - nevažeća ili dupla inicijalizacija, uništavanje `pthread barrier` koji nikada nije inicijalizovan ili koga niti čekaju ili čekanje na objekat koji nije nikada inicijalizovan;
- Greške prilikom korišćenja funkcije `pthread_cond_wait` - prosleđivanje nezaključanog, nevažećeg ili muteksa koga je zaključala druga nit;
- `Pthread` funkcija vrati kôd greške koji je potrebno dodatno obraditi;
- Greška kada se nit uništi, a još drži zaključanu promenljivu;
- Nekonzistentne veze između uslovnih promenljivih i njihovih odgovarajućih muteksa.

Ovakve greške mogu da dovedu do nedefinisanog ponašanja programa i do pojave grešaka u programima koje je kasnije veoma teško otkriti. *Helgrind* presreće pozive ka funkcijama biblioteke `pthread`, i zbog toga je u mogućnosti da otkrije veliki broj grešaka. Za sve `pthread` funkcije koje *Helgrind* presreće, generiše se podatak o grešci ako funkcija vrati kôd greške, iako *Helgrind* nije našao greške u kôdu. Provere koje se odnose na mutekse se takođe primenjuju i na `reader-writer lock`. Prijavljena greška prikazuje i primarno stanje steka koje pokazuje gde je detektovana greška. Takođe, ukoliko je moguće ispisuje se i broj linije u samom kôdu gde se greška nalazi. Ukoliko se greška odnosi na muteks, *Helgrind* će prikazati i gde je prvi put detektovao problematični muteks.

Primer `01_bad.unlock.c` prikazuje ovakav tip grešaka. Prevodimo program sa opcijom `-pthread` pošto koristimo POSIX niti i pokrećemo Helgrind:

```
$ valgrind --tool=helgrind -v --log-file=bl.log ./bad_unlock.out
```

Opcijom `--log-file=bl.log` zadajemo da nam se izveštaj upiše u datoteku `bl.log`. Vidimo da nam je prijavljen veliki broj grešaka:

```
==24477== ---Thread-Announcement------------------------------------------
==24477==
==24477== Thread #1 is the program's root thread
==24477==
```

```
==24477== ----------------------------------------------------------------
==24477==
==24477== Thread #1 unlocked a not-locked lock at 0xFFEFFF720
==24477==    at 0x4C326B4: ??? (in /usr/lib/valgrind/vgpreload_helgrind-amd64-linux.so)
==24477==    by 0x4008DE: nearly_main (01_bad_unlock.c:27)
==24477==    by 0x40096D: main (01_bad_unlock.c:49)
==24477==  Lock at 0xFFEFFF720 was first observed
==24477==    at 0x4C360BA: pthread_mutex_init (in
↪  /usr/lib/valgrind/vgpreload_helgrind-amd64-linux.so)
==24477==    by 0x4008B1: nearly_main (01_bad_unlock.c:23)
==24477==    by 0x40096D: main (01_bad_unlock.c:49)
==24477==  Location 0xffefff720 is 0 bytes inside local var "mx1"
==24477==  declared at 01_bad_unlock.c:18, in frame #1 of thread 1
```

Prijavljena nam je greška jer se u 27. liniji otključava nezaključani muteks. Poruka *was first observed* dodatno objašnjava gde je *Helgrind* prvi put video taj muteks.

```
==24477== Thread #2 unlocked lock at 0xFFEFFF750 currently held by thread #1
==24477==    at 0x4C326B4: ??? (in /usr/lib/valgrind/vgpreload_helgrind-amd64-linux.so)
==24477==    by 0x40084D: child_fn (01_bad_unlock.c:11)
==24477==    by 0x4C34DB6: ??? (in /usr/lib/valgrind/vgpreload_helgrind-amd64-linux.so)
==24477==    by 0x4E476B9: start_thread (pthread_create.c:333)
==24477==  Lock at 0xFFEFFF750 was first observed
==24477==    at 0x4C360BA: pthread_mutex_init (in
↪  /usr/lib/valgrind/vgpreload_helgrind-amd64-linux.so)
==24477==    by 0x4008F2: nearly_main (01_bad_unlock.c:31)
==24477==    by 0x40096D: main (01_bad_unlock.c:49)
==24477==  Location 0xffefff750 is 0 bytes inside local var "mx2"
==24477==  declared at 01_bad_unlock.c:18, in frame #2 of thread 1
```

Prijavljen nam je pokušaj da nit koja nije zaključala muteks pokušava da ga otključa naredbom u liniji 11.

```
==24477== Thread #1 unlocked an invalid lock at 0xFFEFFF848
==24477==    at 0x4C326B4: ??? (in /usr/lib/valgrind/vgpreload_helgrind-amd64-linux.so)
==24477==    by 0x40094D: nearly_main (01_bad_unlock.c:41)
==24477==    by 0x40096D: main (01_bad_unlock.c:49)
==24477==
==24477== ----------------------------------------------------------------
==24477==
==24477== Thread #1's call to pthread_mutex_unlock failed
==24477==    with error code 22 (EINVAL: Invalid argument)
==24477==    at 0x4C327DA: ??? (in /usr/lib/valgrind/vgpreload_helgrind-amd64-linux.so)
==24477==    by 0x40094D: nearly_main (01_bad_unlock.c:41)
==24477==    by 0x40096D: main (01_bad_unlock.c:49)
```

Greška je u pokušaju otključavanja promenljive koja nije muteks.

```
==24477== Thread #1 unlocked a not-locked lock at 0xFFEFFF720
==24477==    at 0x4C326B4: ??? (in /usr/lib/valgrind/vgpreload_helgrind-amd64-linux.so)
==24477==    by 0x4008DE: nearly_main (01_bad_unlock.c:27)
```

```
==24477==    by 0x400990: main (01_bad_unlock.c:50)
==24477==  Lock at 0xFFEFFF720 was first observed
==24477==    at 0x4C360BA: pthread_mutex_init (in
↪  /usr/lib/valgrind/vgpreload_helgrind-amd64-linux.so)
==24477==    by 0x4008B1: nearly_main (01_bad_unlock.c:23)
==24477==    by 0x40096D: main (01_bad_unlock.c:49)
==24477==  Location 0xffefff720 is 0 bytes inside local var "mx1"
==24477==  declared at 01_bad_unlock.c:18, in frame #1 of thread 1
```

Greška je u tome što se u liniji 27 otključava već otključani muteks.

```
==24477== Thread #1: Attempt to re-lock a non-recursive lock I already hold
==24477==    at 0x4C320F4: ??? (in /usr/lib/valgrind/vgpreload_helgrind-amd64-linux.so)
==24477==    by 0x400901: nearly_main (01_bad_unlock.c:32)
==24477==    by 0x400990: main (01_bad_unlock.c:50)
==24477==  Lock was previously acquired
==24477==    at 0x4C321BC: ??? (in /usr/lib/valgrind/vgpreload_helgrind-amd64-linux.so)
==24477==    by 0x400901: nearly_main (01_bad_unlock.c:32)
==24477==    by 0x40096D: main (01_bad_unlock.c:49)
```

Prijavljuje nam da ponovo zaključavamo muteks koji je ista nit već zaključala u 32. liniji funkcije `nearly_main`, pri pozivu iz 49. linije funkcije `main`. Do ponovnog zaključavanja dolazi u istom redu funkcije `nearly_main`, pri pozivu iz 50. linije funkcije `main`, jer se tada funkcija `nearly_main` poziva drugi put. Problematični muteks je ostao zaključan iz prethodnog izvršavanja funkcije. Ako na kraju funkcije otključamo muteks `mx2`, rešićemo i ovaj i naredni problem.

```
==24477== Thread #1: Exiting thread still holds 1 lock
==24477==    at 0x5129748: _Exit (_exit.c:31)
==24477==    by 0x5096FAA: __run_exit_handlers (exit.c:97)
==24477==    by 0x5097044: exit (exit.c:104)
==24477==    by 0x507D836: (below main) (libc-start.c:325)
```

Greška je što nit završava sa radom, ali nije otključala sve katance koje je držala.

Ispravimo greške jednu po jednu sve dok nam *Helgrind* prijavljuje nešto. Sada pokrećemo program kroz *Valgrind* sa alatom *drd*.

```
$ valgrind --tool=drd ./bad_unlock.out
```

```
==8394== drd, a thread error detector
==8394== Copyright (C) 2006-2015, and GNU GPL'd, by Bart Van Assche.
==8394== Using Valgrind-3.11.0 and LibVEX; rerun with -h for copyright info
==8394== Command: ./bad_unlock.out
==8394==
----------------------
==8394== Mutex reinitialization: mutex 0xffefff570, recursion count 0, owner 1.
==8394==    at 0x4C36719: pthread_mutex_init (in /usr/lib/valgrind/vgpreload_drd-amd64-linux.so)
==8394==    by 0x4008A1: nearly_main (01_bad_unlock_fixed.c:23)
==8394==    by 0x40096B: main (01_bad_unlock_fixed.c:51)
==8394== mutex 0xffefff570 was first observed at:
```

```
==8394==    at 0x4C36719: pthread_mutex_init (in /usr/lib/valgrind/vgpreload_drd-amd64-linux.so)
==8394==    by 0x4008A1: nearly_main (01_bad_unlock_fixed.c:23)
==8394==    by 0x400948: main (01_bad_unlock_fixed.c:50)
==8394==
==8394== Mutex reinitialization: mutex 0xffefff5a0, recursion count 0, owner 1.
==8394==    at 0x4C36719: pthread_mutex_init (in /usr/lib/valgrind/vgpreload_drd-amd64-linux.so)
==8394==    by 0x4008D3: nearly_main (01_bad_unlock_fixed.c:31)
==8394==    by 0x40096B: main (01_bad_unlock_fixed.c:51)
==8394== mutex 0xffefff5a0 was first observed at:
==8394==    at 0x4C36719: pthread_mutex_init (in /usr/lib/valgrind/vgpreload_drd-amd64-linux.so)
==8394==    by 0x4008D3: nearly_main (01_bad_unlock_fixed.c:31)
==8394==    by 0x400948: main (01_bad_unlock_fixed.c:50)
==8394==
==8394==
==8394== For counts of detected and suppressed errors, rerun with: -v
==8394== ERROR SUMMARY: 2 errors from 2 contexts (suppressed: 14 from 6)
```

Prijavljuje nam da imamo reinicijalizovane mutekse. Prema dokumentaciji reinicijalizovanje inicijalizovanog muteksa ima nedefinisano ponašanje i stoga nam se prijavljuje greška. Muteksi iako su deklarisani u telu funkcije, ne ponašaju se kao obične lokalne promenljive. Dodajemo na kraj funkcije `nearly_main` sledeće naredbe:

```
pthread_mutex_destroy(&mx1);
pthread_mutex_destroy(&mx2);
```

Pokretanje program kroz *Valgrind* sa alatom *drd* ovaj put prolazi bez greške.

Potencijalno blokiranje niti *Helgrind* prati redosled kojim niti zaključavaju promenljive. Tako detektuje potencijalne delove kôda koji mogu dovesti do blokiranja niti. Na ovaj način je moguće otkriti greške koje se nisu javile tokom testiranja programa, ali mogu da se pojave kasnije tokom korišćenja.

Ilustracija ovakvog problema:

- Pretpostavimo da je potrebno zaključati dve promenljive `M1` i `M2` da bi se pristupilo deljenom objektu `O`.
- Dve niti `T1` i `T2` žele da pristupe deljenoj promenljivoj `O`. Do blokiranja niti dolazi kada nit `T1` zaključa `M1`, a u istom trenutku `T2` zaključa `M2`. Nakon toga nit `T1` ostaje blokirana jer čeka da se otključa `M2`, a nit `T2` ostaje blokirana jer čeka da se otključa `M1`.

Helgrind kreira graf koji predstavlja sve promenljive koje se mogu zaključati, a koje je ranije otkrio. Kada nit naiđe na novu promenljivu koju zaključava, graf se osvežava i proverava se da li sadrži krug sa zaključanim promenljivama. Postojanje takvog kruga je znak da se niti mogu blokirati tokom izvršavanja. Ako u krugu postoje više od dve zaključane promenljive, problem je još ozbiljniji. Alat *DRD* ne otkriva ovaj tip grešaka.

Primer `02_dining_philosophers_deadlock` ilustruje deadlock scenario:

```
==15457== Thread #6: lock order "0x6010E0 before 0x6012C0" violated
==15457==
==15457== Observed (incorrect) order is: acquisition of lock at 0x6012C0
==15457==    at 0x4C321BC: ??? (in /usr/lib/valgrind/vgpreload_helgrind-amd64-linux.so)
==15457==    by 0x40090D: dine(void*) (02_dinning_philosophers_deadlock.cpp:26)
==15457==    by 0x4C34DB6: ??? (in /usr/lib/valgrind/vgpreload_helgrind-amd64-linux.so)
==15457==    by 0x4E476B9: start_thread (pthread_create.c:333)
==15457==
==15457==  followed by a later acquisition of lock at 0x6010E0
==15457==    at 0x4C321BC: ??? (in /usr/lib/valgrind/vgpreload_helgrind-amd64-linux.so)
==15457==    by 0x400937: dine(void*) (02_dinning_philosophers_deadlock.cpp:27)
==15457==    by 0x4C34DB6: ??? (in /usr/lib/valgrind/vgpreload_helgrind-amd64-linux.so)
==15457==    by 0x4E476B9: start_thread (pthread_create.c:333)
==15457==
==15457==  Lock at 0x6010E0 was first observed
==15457==    at 0x4C360BA: pthread_mutex_init (in
↪  /usr/lib/valgrind/vgpreload_helgrind-amd64-linux.so)
==15457==    by 0x400A0E: main (02_dinning_philosophers_deadlock.cpp:48)
==15457==  Address 0x6010e0 is 0 bytes inside data symbol "_ZL4chop"
==15457==
==15457==  Lock at 0x6012C0 was first observed
==15457==    at 0x4C360BA: pthread_mutex_init (in
↪  /usr/lib/valgrind/vgpreload_helgrind-amd64-linux.so)
==15457==    by 0x400A0E: main (02_dinning_philosophers_deadlock.cpp:48)
==15457==  Address 0x6012c0 is 480 bytes inside data symbol "_ZL4chop"
```

Ne treba ignorisati činjenicu da *Helgrind* prijavljuje narušavanje relacije, čak i ako se program uspešno završi. Ako svaki filozof samo jednom ruča, može se dogoditi da nema blokiranja. Ako povećamo broj iteracija petlje i svaki filozof pokuša da ruča 1000 puta, problem postaje vidljiviji.

Problem se rešava uvođenjem uređenja nad izborom viljuške, jer se tada nameće i relacija *„desilo se pre"*. Filozof treba uvek da bira prvo manju viljušku, pa zatim veću po rednom broju. Kako se sada ponaša program?

Program `03_deadlock.c` ostaje zaglavljen u mrtvoj petlji i ne završava se. Otvorimo kôd i primetimo da se u funkcijama `run1` i `run2` program uspavljuje na 10ms između dva zaključavanja muteksa, `mx1` i `mx2`, i pre otključavanja. Te redove možemo privremeno staviti pod komentare. Nakon toga, pokretanje programa se uspešno završava. Ako sada pokrenemo program kroz *Valgrind* sa alatom *Helgrind*, dobićemo poruku o narušenom redosledu zaključavanja muteksa i upozorenje da postoji mogućnost mrtve petlje.

Trka za podacima Trka za podacima *(eng. Data race)* može da se javi usled nedostatka adekvatnog zaključavanja ili sinhronizacije. Pristup podacima bez adekvatnog zaključavanja ili sinhronizacije se odnosi na problem kada dve ili više niti pristupaju deljenom podatku bez sinhronizacije. Na ovaj način je moguće da dve ili više niti u istom trenutku pristupe deljenom objektu.

Algoritam detekcije pristupa promenljivoj bez sinhronizacije u okviru alata *Helgrind* implementira *„desilo se pre"* relaciju *(eng. „happens-before" relation)*. Na primer, kao u gore pokazanom programu, nit roditelj kreira nit dete. Zatim obe menjaju vrednost promenljive `var`, a zatim nit roditelja čeka da nit deteta izvrši svoju funkciju. Ovaj program nije dobro napisan jer ne možemo sa sigurnošću da znamo koja je vrednost promenljive `var` prilikom štampanja iste.

Ako je nit roditelja brža od niti deteta, onda će biti štampana vrednost 10, u suprotnom će biti 20. Brzina izvršavanja niti roditelja i deteta je nešto na šta programer nema uticaja. Rešenje ovog problema je u zaključavanju promenljive `var`. Na primer, možemo da pošaljemo poruku iz niti roditelj nakon što ona promeni vrednost promenljive `var`, a nit dete neće promeniti vrednost promenljive `var` dok ne dobije poruku. Na ovaj način smo sigurni da će program ispisati vrednost 10. Razmena poruka kreira *„desilo se pre"* zavisnost između dve dodele vrednost: `var = 20;` se događa pre `var = 10;`. Takođe, sada više nemamo pristup promenljivoj bez sinhronizacije. Nije obavezno da šaljemo poruku iz niti roditelj. Možemo poslati poruku iz niti dete nakon što ona izvrši svoju dodelu. Na ovaj način smo sigurni da će se ispisati vrednost 20.

Alat *Helgrind* radi na istom ovom principu. On prati svaki pristup memorijskoj lokaciji. Ako se lokacija, u ovom primeru `var`, pristupa iz dve niti, Helgrind proverava da li su ti pristupi povezani sa *„desilo se pre"* vezom. Ako nisu, alat prijavljuje grešku o pristupu promenljivoj bez sinhronizacije. Ako je pristup deljenoj promenljivoj iz dve ili više programerske niti povezan sa *„desilo se pre"* relacijom, znači da postoji sinhronizacioni lanac između programskih niti koje obezbeđuje da se sam pristup odvija po tačno određenom redosledu, bez obzira na stvarne stope napretka pojedinačnih niti.

Standardne primitive `pthread` niti kreiraju *„desilo se pre"* relaciju:

- Ako je muteks otključan od strane niti `T1`, a kasnije ili odmah zaključan od strane niti `T2`, onda se sav pristup memoriji iz niti `T1` pre otključavanja muteksa dešava pre pristupa iz niti `T2` nakon njenog zaključavanja muteksa.
- Ista ideja se odnosi i na `reader-writer` zaključavanje promenljivih.
- Ako je kondiciona promenljiva signalizirana u funkciji niti `T1` i ako druga nit `T2` čeka na taj signal, da bi nastavila sa radom, onda se memorijski pristup u `T1` dešava pre signalizacije, dok nit `T2` vrši pristup memoriji nakon što izađe iz stanja čekanja na signal koji šalje nit `T1`.
- Ako nit `T2` nastavlja sa izvršavanjem nakon što nit `T1` oslobodi semafor, onda kažemo da postoji *„desilo se pre"* relacija između programskih niti `T1` i `T2`.

Helgrind presreće sve gore navedene događaje i kreira graf koji predstavlja sve *„desilo se pre"* relacije u programu. Takođe, on prati sve pristupe memoriji u programu. Ako postoji pristup nekoj memorijskoj lokaciji u programu od strane dve niti i *Helgrind* ne može da nađe putanju kroz graf od jednog pristupa do drugog, generiše podatak o grešci u programu koji analizira. *Helgrind* ne proverava da li postoji pristup memorijskoj lokaciji bez sinhronizacije ukoliko se svi pristupi toj lokaciji odnose na čitanje sadržaja te lokacije. Dva pristupa memorijskoj lokaciji su u *„desilo se pre"* relaciji, i ako postoji proizvoljno dugačak lanac sinhronizacije događaja između ta dva pristupa. Ako nit `T1` pristupa lokaciji `M`, zatim signalizira nit `T2` , koja kasnije signalizira nit `T3` koja pristupa lokaciji `M`, kažemo da su ova dva pristupa između niti `T1` i `T3` u *„desilo se pre"* relaciji, iako između njih ne postoji direktna veza.

Pokrenimo Helgrind:

```
==972== Thread #1 is the program's root thread
==972==
==972== ---Thread-Announcement------------------------------------------
==972==
```

```
==972== Thread #2 was created
==972==    at 0x51643DE: clone (clone.S:74)
==972==    by 0x4E46149: create_thread (createthread.c:102)
==972==    by 0x4E47E83: pthread_create@@GLIBC_2.2.5 (pthread_create.c:679)
==972==    by 0x4C34BB7: ??? (in /usr/lib/valgrind/vgpreload_helgrind-amd64-linux.so)
==972==    by 0x400705: main (simpleDataRace.c:12)
==972==
==972== ----------------------------------------------------------------
==972==
==972== Possible data race during read of size 4 at 0x60104C by thread #1
==972== Locks held: none
==972==    at 0x400706: main (simpleDataRace.c:13)
==972==
==972== This conflicts with a previous write of size 4 by thread #2
==972== Locks held: none
==972==    at 0x4006C7: child_fn (simpleDataRace.c:6)
==972==    by 0x4C34DB6: ??? (in /usr/lib/valgrind/vgpreload_helgrind-amd64-linux.so)
==972==    by 0x4E476B9: start_thread (pthread_create.c:333)
==972==  Location 0x60104c is 0 bytes inside global var "var"
==972==  declared at simpleDataRace.c:3
==972==
--972-- warning: evaluate_Dwarf3_Expr: unhandled DW_OP_ 0xf2
==972== ----------------------------------------------------------------
==972==
==972== Possible data race during write of size 4 at 0x60104C by thread #1
==972== Locks held: none
==972==    at 0x40070F: main (simpleDataRace.c:13)
==972==
==972== This conflicts with a previous write of size 4 by thread #2
==972== Locks held: none
==972==    at 0x4006C7: child_fn (simpleDataRace.c:6)
==972==    by 0x4C34DB6: ??? (in /usr/lib/valgrind/vgpreload_helgrind-amd64-linux.so)
==972==    by 0x4E476B9: start_thread (pthread_create.c:333)
==972==  Location 0x60104c is 0 bytes inside global var "var"
==972==  declared at simpleDataRace.c:3
```

Helgrind najpre ispisuje podatke o tome gde su napravljene niti koje učestvuju u grešci. Glavni podatak o grešci počinje porukom *„Possible data race during read"*. Zatim se ispisuje adresa na kojoj se dešava nesinhronizovan pristup memoriji, kao i veličina memorije kojoj se pristupa. Prikazuje se stek za obe niti, roditeljsku i dečju. U nastavku *Helgrind* ispisuje gde druga nit pristupa istoj lokaciji. Kada se pokrene sa opcijom `--read-var-info=yes`, *Helgrind* ispisuje i ime promenljive kojoj se pristupa, kao i mesto deklaracije. Na obe lokacije pristupa promenljivoj ispisuje i sva zaključavanja koja postoje u tom trenutku. U ovom primeru ih nema ni na jednom mestu. Takva informacija je korisna za zaključivanje koja nit nije uspela da zaključa željeni objekat.

Kada imamo dva poziva na steku, kako nalazimo uzrok trke za podacima?

Prvo treba ispitati lokacije na koje se referiše u svakom steku. Oba steka bi trebalo da pokazuju na pristup istoj lokaciji, odnosno promenljivoj. Potom treba utvrditi kako tu lokaciju zaštititi. Ako je već bilo planirano da bude zaštićena muteksom, treba zaključati i otključati muteks na obe tačke pristupa, čak i ako je na jednom mestu planirano samo čitanje. *Helgrind* u momentu pristupa ispisuje i sve katance koje u tom trenutku drži nit koja pristupa lokaciji. Ako smo zaboravili da zaključamo, tako možemo primetiti.

Možda je namera bila da se lokacija zaštiti nekim drugim postupkom, na primer signalizacijom uslovnih promenljivih. U tom slučaju treba pokušati da se pronađe dokaz da je raniji pristup u relaciji *„desilo se pre"* sa kasnijim pristupom. *Helgrind* prijavljuje trku za podacima ako nije uočio *„desilo se pre"* relaciju između dva pristupa. Ako je u pravu, onda ni detaljnim ispitivanjem izvornog kôda ne bi trebalo da možemo da nađemo takvu relaciju. Ispitivanjem kôda ćemo uočiti i gde bi operacije za sinhronizaciju trebalo da stoje.
DRD ispisuje poruku svaki put kada otkrije da je došlo do trke za podacima u programu. Treba imati u vidu par sledećih stvari prilikom tumačenja ispisa koji nam alat *DRD* daje. Prvo, *DRD* dodeljuje svakoj niti jedinstveni broj `ID`. Brojevi koji se dodeljuju nitima kreću od jedan i nikada se ne koristi isti broj za više niti. Drugo, termin segment se odnosi na sekvencu uzastopnih operacija čuvanja, čitanja i sinhronizacije koje se izvršavaju u jednoj niti. Segment uvek počinje i završava se operacijom sinhronizacije. Analiza trke za podacima se izvršava između dva segmenta umesto između pojedinačnih operacija čitanja i čuvanja podataka, isključivo zbog učinka. Na kraju, uvek postoje bar dva pristupa memoriji prilikom trke za podacima. *DRD* štampa izveštaj o svakom pristupu memoriji koje je dovelo do trke za podacima.

Pokrenućemo za prethodni primer sada alat *DRD*:

```
==24411== Conflicting load by thread 1 at 0x0060104c size 4
==24411==    at 0x400706: main (simpleDataRace.c:13)
==24411== Allocation context: BSS section of /home/student/vs/06/helgrind/a.out
==24411== Other segment start (thread 2)
==24411==    at 0x51703E1: clone (clone.S:81)
==24411== Other segment end (thread 2)
==24411==    at 0x516A807: madvise (syscall-template.S:84)
==24411==    by 0x4E5394C: start_thread (pthread_create.c:432)
==24411==
==24411== Conflicting store by thread 1 at 0x0060104c size 4
==24411==    at 0x40070F: main (simpleDataRace.c:13)
==24411== Allocation context: BSS section of /home/student/vs/06/helgrind/a.out
==24411== Other segment start (thread 2)
==24411==    at 0x51703E1: clone (clone.S:81)
==24411== Other segment end (thread 2)
==24411==    at 0x516A807: madvise (syscall-template.S:84)
==24411==    by 0x4E5394C: start_thread (pthread_create.c:432)
```

Pristupi memoriji koji učestvuju u trci za podacima označeni su kao *konfliktni* sa prethodnim pristupom. Mora ih biti bar dva, od kojih je bar jedan upis u memorijsku lokaciju. Za jedan od pristupa prikazuje se potpun stek poziva, a za ostale delimični stekovi poziva. Oni uglavnom uključuju početak i kraj segmenta pristupa. Ove informacije se mogu protumačiti na sledeći način:

- Krenuti od dna oba steka poziva, brojati stek okvire sa istim imenima funkcije, izvorne datoteke i broja linije.
- Naredni viši okvir na steku u oba steka poziva govori između kojih delova izvornog kôda se desio pristup memorijskoj lokaciji.

Problem u ovom primeru je što nemamo sinhronizaciju pristupa promenljivoj `var`. Uvođenje muteksa i njegovo zaključavanje pri pristupu promenljivoj rešava problem.

Zadržavanje katanaca Prilikom rada niti može doći do pojave zadržavanja katanca *(eng. lock contention)*, pri kojoj jedna nit ne može da radi zbog blokiranja drugih niti. Dešava se da nit mora da čeka da muteks ili sinhronizacioni *reader-writer* objekat bude otključan od strane druge niti. Ovakva pojava je nepoželjna u višenitnim sistemima, a alat *DRD* otkriva ovaj tip problema. Zadržavanje katanaca stvara kašnjenja koja bi trebalo da budu što kraća. Opcije `--exclusive-threshold=<n>` i `--shared-threshold=<n>` omogućavaju da DRD otkrije preterano zadržavanje katanca, tako što će prijaviti svako zadržavanje katanca koje je duže od zadatog praga. Alat *Helgrind* ne otkriva ovakav tip grešaka.

Pokrenimo DRD za primer `hold_lock.c`. Program dobija preko argumenata komandne linije `-i 500` da bi između zaključavanja i otključavanja muteksa bio uspavan 500ms.

```
$ valgrind --tool=drd --exclusive-threshold=10 ./hard_lock -i 500
```

DRD nam generiše izveštaj o nitima koje su prekoračile vreme od 10ms koje smo zadali opcijom `--exclusive-threshold=10`.

```
Locking mutex ...
==25008== Acquired at:
==25008==    at 0x4C3725B: pthread_mutex_lock (in /usr/lib/valgrind/vgpreload_drd-amd64-linux.so)
==25008==    by 0x400E1C: main (07_hold_lock.c:51)
==25008== Lock on mutex 0xffefff8f0 was held during 505 ms (threshold: 10 ms).
==25008==    at 0x4C3818C: pthread_mutex_unlock (in /usr/lib/valgrind/vgpreload_drd-amd64-linux.so)
==25008==    by 0x400E4D: main (07_hold_lock.c:55)
==25008== mutex 0xffefff8f0 was first observed at:
==25008==    at 0x4C36719: pthread_mutex_init (in /usr/lib/valgrind/vgpreload_drd-amd64-linux.so)
==25008==    by 0x400E04: main (07_hold_lock.c:49)
==25008==
Locking rwlock exclusively ...
==25008== Acquired at:
==25008==    at 0x4C3F484: pthread_rwlock_wrlock (in /usr/lib/valgrind/vgpreload_drd-amd64-linux.so)
==25008==    by 0x400E94: main (07_hold_lock.c:61)
==25008== Lock on rwlock 0xffefff920 was held during 502 ms (threshold: 10 ms).
==25008==    at 0x4C40955: pthread_rwlock_unlock (in /usr/lib/valgrind/vgpreload_drd-amd64-linux.so)
==25008==    by 0x400EAD: main (07_hold_lock.c:63)
==25008== rwlock 0xffefff920 was first observed at:
==25008==    at 0x4C3E705: pthread_rwlock_init (in /usr/lib/valgrind/vgpreload_drd-amd64-linux.so)
==25008==    by 0x400E88: main (07_hold_lock.c:60)
==25008==
Locking rwlock shared ...
Done.
```

Ovako dolazimo do saznanja u kojoj liniji je zaključan neki od katanaca i koliko se dugo držao dok nije otključan.

7.2.8 gprof

gprof je klasičan GNU alat za profajliranje programa prevedenih sa opcijom `-pg`. Kompajler i linker ubacuju dodatnu instrumentaciju koja broji pozive funkcija i omogućava da se posle izvršavanja napravi datoteka `gmon.out`.

Ovaj model je koristan kao uvod u instrumentaciono profajliranje: program se mora posebno prevesti, pokrenuti do normalnog završetka, a izveštaj se pravi naknadno. Cena merenja i sam način instrumentacije mogu promeniti ponašanje programa, pa rezultate treba čitati kao trag za dalju analizu, ne kao apsolutnu istinu.

Primer poredi namerno spor `bubble_sort` i bibliotekarsko sortiranje:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define COUNT 6000

static void fill_input(int *values, int count) {
    unsigned int x = 0x12345678u;
    for (int i = 0; i < count; i++) {
        x = x * 1103515245u + 12345u;
        values[i] = (int)(x % 100000u);
    }
}

static long checksum(const int *values, int count) {
    long sum = 0;
    for (int i = 0; i < count; i++) {
        sum += values[i] * (i + 1L);
    }
    return sum;
}

static void bubble_sort(int *values, int count) {
    for (int i = 0; i < count; i++) {
        for (int j = 0; j + 1 < count - i; j++) {
            if (values[j] > values[j + 1]) {
                int tmp = values[j];
                values[j] = values[j + 1];
                values[j + 1] = tmp;
            }
        }
    }
}

static int compare_ints(const void *left, const void *right) {
    int a = *(const int *)left;
    int b = *(const int *)right;
    return (a > b) - (a < b);
}

static void library_sort(int *values, int count) {
    qsort(values, (size_t)count, sizeof(values[0]), compare_ints);
}

static void run_bubble(const int *input) {
    int *copy = malloc(sizeof(int) * COUNT);
    if (copy == NULL) {
        perror("malloc");
        exit(EXIT_FAILURE);
```

```
    }

    memcpy(copy, input, sizeof(int) * COUNT);
    bubble_sort(copy, COUNT);
    printf("bubble checksum=%ld\n", checksum(copy, COUNT));
    free(copy);
}

static void run_library(const int *input) {
    int *copy = malloc(sizeof(int) * COUNT);
    if (copy == NULL) {
        perror("malloc");
        exit(EXIT_FAILURE);
    }

    memcpy(copy, input, sizeof(int) * COUNT);
    library_sort(copy, COUNT);
    printf("qsort checksum=%ld\n", checksum(copy, COUNT));
    free(copy);
}

int main(void) {
    int *input = malloc(sizeof(int) * COUNT);
    if (input == NULL) {
        perror("malloc");
        return EXIT_FAILURE;
    }

    fill_input(input, COUNT);
    run_bubble(input);
    run_library(input);

    free(input);
    return 0;
}
```

Prevodimo program sa `-pg`:

```
$ make
```

Prvo pokretanje pravi `gmon.out`:

```
$ ./sort_workload
$ ls gmon.out
```

Zatim pravimo tekstualni izveštaj:

```
$ gprof ./sort_workload gmon.out > gprof.txt
$ less gprof.txt
```

Flat profile Prvi deo izveštaja je *flat profile*. On grupiše uzorke po funkciji:

```
  %   cumulative   self              self     total
 time   seconds   seconds    calls  ms/call  ms/call  name
  ...
         ...       ...          1             bubble_sort
```

Za ovaj primer očekujemo da se `bubble_sort` izdvoji kao dominantna funkcija. Ako se u izveštaju ne vidi očekivana razlika, povećati veličinu ulaza ili broj ponavljanja. Kratki programi daju premalo uzoraka.

Call graph Drugi deo izveštaja je graf poziva. On povezuje vreme funkcije sa njenim pozivaocima i pozvanim funkcijama:

```
index % time    self  children    called     name
                ...       ...       1/1          run_bubble
[1]     ...     ...       ...       1          bubble_sort
```

Graf poziva pomaže kada ista funkcija ima više pozivalaca. Tada nije dovoljno znati da je funkcija skupa; treba znati koji tok programa je najviše koristi.

Ograničenja

- Program treba da završi normalno da bi se `gmon.out` upisao.
- Optimizacije, inlining i kratko izvršavanje mogu sakriti funkcije ili dati nestabilne procente.
- Višedretveni programi i programi koji mnogo čekaju na I/O zahtevaju pažljivo tumačenje.
- `gprof` ne zamenjuje moderne uzorkujuće alate, ali je dobar za razumevanje razlike između instrumentacije poziva i statističkog uzorkovanja.

7.2.9 LLVM XRay

LLVM XRay je instrumentacija koju Clang može da ubaci u funkcije programa. Za razliku od `gprof`-a, XRay je projektovan za selektivno uključivanje i detaljnije tragove ulaska i izlaska iz funkcija. Koristan je kada želimo da vidimo vremenski trag poziva, latencije funkcija ili kada želimo da izdvojimo skupe putanje kroz program.

Primer ima tri tipa posla: CPU petlju, sortiranje i povremeno čekanje:

```cpp
#include <algorithm>
#include <chrono>
#include <iostream>
#include <numeric>
#include <thread>
#include <vector>

[[clang::xray_always_instrument]] static long busy_work(int rounds) {
    long value = 0;
    for (int i = 0; i < rounds; i++) {
        value += (i * 17) % 31;
```

```
    }
    return value;
}

[[clang::xray_always_instrument]] static long sort_work() {
    std::vector<int> values(2000);
    std::iota(values.rbegin(), values.rend(), 1);
    std::sort(values.begin(), values.end());
    return values[values.size() / 2];
}

[[clang::xray_always_instrument]] static void wait_work() {
    std::this_thread::sleep_for(std::chrono::milliseconds(20));
}

[[clang::xray_always_instrument]] static long handle_request(int id) {
    long result = busy_work(20000 + id * 100);
    result += sort_work();
    if (id % 5 == 0) {
        wait_work();
    }
    return result;
}

int main() {
    long total = 0;
    for (int i = 0; i < 30; i++) {
        total += handle_request(i);
    }

    std::cout << "total=" << total << "\n";
    return 0;
}
```

Prevodimo ga Clang-om sa XRay instrumentacijom:

```
$ make
```

XRay se uključuje pri pokretanju kroz promenljivu okruženja:

```
$ XRAY_OPTIONS="patch_premain=true xray_mode=xray-basic verbosity=1" ./xray_workload
```

Pokretanje pravi datoteku oblika `xray-log.xray_workload.<pid>`. Tekstualni sažetak se može dobiti alatom `llvm-xray`:

```
$ llvm-xray account xray-log.xray_workload.* > xray-account.txt
$ less xray-account.txt
```

U sažetku tražimo funkcije sa najvećim ukupnim ili prosečnim vremenom. U ovom primeru `wait_work` ima veliko zidno vreme po pozivu, dok `busy_work` i `sort_work` pokazuju aktivan rad programa.

Za trag poziva možemo koristiti `stack` analizu:

```
$ llvm-xray stack xray-log.xray_workload.* > xray-stacks.txt
```

Kada koristiti XRay XRay je dobar kada je pitanje vezano za funkcijske pozive i latenciju putanja kroz program: koji handler je spor, koliko traje određena faza, koliko često se ulazi u skupu funkciju. Pošto je instrumentacija deo binarne verzije, treba praviti posebnu konfiguraciju za merenje i jasno je odvojiti od produkcione verzije.

Ograničenja

- Potrebni su Clang, XRay runtime i `llvm-xray`; dostupnost zavisi od distribucije i arhitekture.
- Instrumentacija menja binarnu datoteku i uvodi dodatni trošak.
- Veoma česte male funkcije mogu napraviti veliki log. U praksi treba ograničiti šta se instrumentira ili koristiti pragove.
- XRay meri funkcijske događaje; za hardverske događaje, keš i mikroarhitekturu treba koristiti alate kao što su `perf` ili VTune.

8 Profajliranje uzorkovanjem

Profajleri u ovom poglavlju periodično uzorkuju stanje procesa ili koriste hardverske brojače. Dobijeni profil je statistički prikaz izvršavanja: pokazuje gde program provodi vreme i koji hardverski događaji prate to izvršavanje. Takav rezultat treba tumačiti uz reprezentativan ulaz i dovoljno dugo pokretanje.

`perf` je osnovni Linux alat za ovaj tip analize. Intel VTune Profiler dodaje gotove analize za niti, memoriju i mikroarhitekturu, uz veći instalacioni i operativni trošak. Za Java programe, async-profiler daje praktičan put do profila Java metoda, alokacija i zaključavanja bez prevođenja posebne verzije aplikacije.

8.1 Kada koristiti uzorkovanje

Pitanje	Alat	Napomena
Gde program provodi CPU vreme?	`perf record/report`, flame graph, async-profiler za JVM	Uzorak mora trajati dovoljno dugo i koristiti reprezentativan ulaz.
Da li problem dolazi od hardverskih događaja?	`perf stat`, `perf record -e`, VTune	Simboli i odgovarajuće dozvole često određuju koliko će rezultat biti čitljiv.
Zašto niti čekaju?	VTune threading analiza, async-profiler lock/wall, eBPF alati	Za čekanje van CPU-a treba meriti događaje čekanja, ne samo CPU uzorke.
Šta se dešava u JVM procesu?	async-profiler, JFR izvoz	Običan `perf` može videti proces, ali bez dodatne podrške rezultat često ne mapira dobro Java metode.
Treba sistemski pogled kroz procese i kernel	`perf`, eBPF, BCC alati	Početi od sistema, zatim suziti na proces, nit, događaj ili stek.

8.2 Perf

Perf je alat za profajliranje koji pruža jednostavan interfejs preko komandne linije. Perf je zasnovan na `perf_events` interfejsu koji daju novije verzije Linux kernela.

```
$ perf
```

```
usage: perf [--version] [--help] COMMAND [ARGS]

The most commonly used perf commands are:
 annotate        Read perf.data (created by perf record) and display annotated code
 archive         Create archive with object files with build-ids found in perf.data file
 bench           General framework for benchmark suites
 buildid-cache   Manage <tt>build-id</tt> cache.
```

```
  buildid-list    List the buildids in a perf.data file
  diff            Read two perf.data files and display the differential profile
  inject          Filter to augment the events stream with additional information
  kmem            Tool to trace/measure kernel memory(slab) properties
  kvm             Tool to trace/measure kvm guest os
  list            List all symbolic event types
  lock            Analyze lock events
  probe           Define new dynamic tracepoints
  record          Run a command and record its profile into perf.data
  report          Read perf.data (created by perf record) and display the profile
  sched           Tool to trace/measure scheduler properties (latencies)
  script          Read perf.data (created by perf record) and display trace output
  stat            Run a command and gather performance counter statistics
  test            Runs sanity tests.
  timechart       Tool to visualize total system behavior during a workload
  top             System profiling tool.

 See 'perf help COMMAND' for more information on a specific command.
```

Neke komande mogu da budu nedostupne pošto zahtevaju specijalnu kernel podršku. Dodatni argumenti komandi se mogu videti prosleđivanjem opcije `-h` odgovarajućoj komandi:

```
$ perf stat -h
```

```
 usage: perf stat [<options>] [<command>]

    -e, --event <event>   event selector. use 'perf list' to list available events
    -i, --no-inherit      child tasks do not inherit counters
    -p, --pid <n>         stat events on existing process id
    -t, --tid <n>         stat events on existing thread id
    -a, --all-cpus        system-wide collection from all CPUs
    -c, --scale           scale/normalize counters
    -v, --verbose         be more verbose (show counter open errors, etc)
    -r, --repeat <n>      repeat command and print average + stddev (max: 100)
    -n, --null            null run - dont start any counters
    -B, --big-num         print large numbers with thousands' separators
```

Kontrola dozvola pristupa odgovarajućim događajima može se izmeniti kroz datoteku `/proc/sys/kernel/perf_event_paranoid`. Opis podešavanja je u dokumentaciji kernela:

```
perf_event_paranoid:

Controls use of the performance events system by unprivileged
users (without CAP_SYS_ADMIN).  The default value is 2.

 -1: Allow use of (almost) all events by all users
     Ignore mlock limit after perf_event_mlock_kb without CAP_IPC_LOCK
>=0: Disallow ftrace function tracepoint by users without CAP_SYS_ADMIN
     Disallow raw tracepoint access by users without CAP_SYS_ADMIN
>=1: Disallow CPU event access by users without CAP_SYS_ADMIN
>=2: Disallow kernel profiling by users without CAP_SYS_ADMIN
```

Da bismo izmenili trenutnu vrednost u datoteci `perf_event_paranoid`, potrebne su root privilegije:

```
$ echo -1 | sudo tee /proc/sys/kernel/perf_event_paranoid
```

Dodatno, poželjno je podesiti i `kptr_restrict` podešavanje:

```
kptr_restrict

This toggle indicates whether restrictions are placed on exposing kernel addresses via /proc and
↪  other interfaces.

When kptr_restrict is set to 0 (the default) the address is hashed before printing. (This is the
↪  equivalent to %p.)

When kptr_restrict is set to 1, kernel pointers printed using the %pK format specifier will be
↪  replaced with 0s unless the user has CAP_SYSLOG and effective user and group ids are equal to the
↪  real ids. This is because %pK checks are done at read() time rather than open() time, so if
↪  permissions are elevated between the open() and the read() (e.g via a setuid binary) then %pK
↪  will not leak kernel pointers to unprivileged users. Note, this is a temporary solution only. The
↪  correct long-term solution is to do the permission checks at open() time. Consider removing world
↪  read permissions from files that use %pK, and using dmesg_restrict to protect against uses of %pK
↪  in dmesg(8) if leaking kernel pointer values to unprivileged users is a concern.

When kptr_restrict is set to 2, kernel pointers printed using %pK will be replaced with 0s regardless
↪  of privileges.
```

Da bismo izmenili trenutnu vrednost u datoteci `kptr_restrict`, potrebne su root privilegije:

```
$ echo 0 | sudo tee /proc/sys/kernel/kptr_restrict
```

8.2.1 Događaji

Perf može da meri događaje iz raznih izvora i oni se dele u kategorije:

- softverski (npr. promena konteksta) - njihov izvor su brojači u kernelu
- hardverski (npr. broj ciklusa, promašaji L1 keša) - njihov izvor su procesor i PMU (skr. *Performance Monitoring Unit*)
- *tracepoint* događaji - implementirani od strane `ftrace` infrastrukture kernela

Lista podržanih događaja se može dobiti komandom `list`:

```
$ perf list
```

```
List of pre-defined events (to be used in -e):

 cpu-cycles OR cycles                       [Hardware event]
 instructions                               [Hardware event]
 cache-references                           [Hardware event]
```

```
cache-misses                              [Hardware event]
branch-instructions OR branches           [Hardware event]
branch-misses                             [Hardware event]
bus-cycles                                [Hardware event]

cpu-clock                                 [Software event]
task-clock                                [Software event]
page-faults OR faults                     [Software event]
minor-faults                              [Software event]
major-faults                              [Software event]
context-switches OR cs                    [Software event]
cpu-migrations OR migrations              [Software event]
alignment-faults                          [Software event]
emulation-faults                          [Software event]

L1-dcache-loads                           [Hardware cache event]
L1-dcache-load-misses                     [Hardware cache event]
L1-dcache-stores                          [Hardware cache event]
L1-dcache-store-misses                    [Hardware cache event]
L1-dcache-prefetches                      [Hardware cache event]
L1-dcache-prefetch-misses                 [Hardware cache event]
L1-icache-loads                           [Hardware cache event]
L1-icache-load-misses                     [Hardware cache event]
L1-icache-prefetches                      [Hardware cache event]
L1-icache-prefetch-misses                 [Hardware cache event]
LLC-loads                                 [Hardware cache event]
LLC-load-misses                           [Hardware cache event]
LLC-stores                                [Hardware cache event]
LLC-store-misses                          [Hardware cache event]

LLC-prefetch-misses                       [Hardware cache event]
dTLB-loads                                [Hardware cache event]
dTLB-load-misses                          [Hardware cache event]
dTLB-stores                               [Hardware cache event]
dTLB-store-misses                         [Hardware cache event]
dTLB-prefetches                           [Hardware cache event]
dTLB-prefetch-misses                      [Hardware cache event]
iTLB-loads                                [Hardware cache event]
iTLB-load-misses                          [Hardware cache event]
branch-loads                              [Hardware cache event]
branch-load-misses                        [Hardware cache event]

rNNN (see 'perf list --help' on how to encode it) [Raw hardware event descriptor]

mem:<addr>[:access]                       [Hardware breakpoint]

kvmmmu:kvm_mmu_pagetable_walk             [Tracepoint event]

[...]

sched:sched_stat_runtime                  [Tracepoint event]
sched:sched_pi_setprio                    [Tracepoint event]
syscalls:sys_enter_socket                 [Tracepoint event]
syscalls:sys_exit_socket                  [Tracepoint event]
```

```
[...]
```

8.2.2 Brojanje događaja

Bez navedenog spiska događaja, perf podrazumevano sakuplja navedene informacije:

```
$ perf stat -B dd if=/dev/zero of=/dev/null count=1000000
```

```
1000000+0 records in
1000000+0 records out
512000000 bytes (512 MB) copied, 0.956217 s, 535 MB/s

 Performance counter stats for 'dd if=/dev/zero of=/dev/null count=1000000':

          5,099 cache-misses             #      0.005 M/sec (scaled from 66.58%)
        235,384 cache-references         #      0.246 M/sec (scaled from 66.56%)
      9,281,660 branch-misses            #      3.858 %     (scaled from 33.50%)
    240,609,766 branches                 #    251.559 M/sec (scaled from 33.66%)
  1,403,561,257 instructions             #      0.679 IPC   (scaled from 50.23%)
  2,066,201,729 cycles                   #   2160.227 M/sec (scaled from 66.67%)
            217 page-faults              #      0.000 M/sec
              3 CPU-migrations           #      0.000 M/sec
             83 context-switches         #      0.000 M/sec
     956.474238 task-clock-msecs         #      0.999 CPUs

     0.957617512  seconds time elapsed
```

Konkretni događaji se mogu meriti uz pomoć `-e` opcije:

```
$ perf stat -e cycles dd if=/dev/zero of=/dev/null count=100000
```

Moguće je navesti sufiks `:` sa jednim ili više modifikatora koji korisniku omogućavaju da specifikuje kada će se događaji brojati. Na primer, modifikator `u` posmatra samo nivoe privilegija veće od `0` (`user`) dok modifikator `k` posmatra nivo privilegija `0` (`kernel`):

```
$ perf stat -e instructions:u dd if=/dev/zero of=/dev/null count=100000
```

Moguće je u jednom pozivu meriti više događaja:

```
$ perf stat -e cycles,instructions,cache-misses ...
```

Perf je moguće zakačiti za pokrenut proces opcijom `-p`. Na taj način se prate sve niti procesa. Ukoliko se ne proslede dodatni argumenti, perf će posmatrati proces dok ne dobije signal za prekid. Alternativno, moguće je proslediti ime komande čiji završetak će zaustaviti profajliranje:

```
$ perf stat -e cycles -p <id> sleep 2
```

8.2.3 Uzorkovanje

Perf se može koristiti za prikupljanje profila (*sampling*) na nivou niti, procesa ili CPU kroz komande `record`, `report` i `annotate`. Profili će podrazumevano biti sačuvani u datoteci `perf.data` i mogu se analizirati komandama `report` i `annotate`.

Primer:

```
$ sudo perf record -F 99 -a -g -- sleep 20
```

Opcije:

- `-F 99` - frekvencija sampliranja u Hz (visoke frekvencije koštaju više, 99Hz ne bi trebalo da utiče na izvršavanje)
- `-a` - sampliranje na svim CPU jezgrima
- `-g` - uključivanje informacija o stekovima (*stack traces*); opcija `-g` prima opcioni argument koji predstavlja metod razvijanja steka
- `--` - preskače opcioni metod opcije `-g`

```
$ sudo perf report -n --stdio
```

```
[...]
# Overhead  Samples     Command       Shared Object                              Symbol
# ........ ..........  ........  ...................  .................................
#
    20.97%        208       fio  [kernel.kallsyms]   [k] hypercall_page
                |
                --- hypercall_page
                    check_events
                   |
                   |--63.94%-- 0x7fff695c398f
                   |
                   |--18.27%-- 0x7f0c5b72bd2d
                   |
                    --17.79%-- 0x7f0c5b72c46d

    14.21%        141       fio  [kernel.kallsyms]   [k] copy_user_generic_string
                |
                --- copy_user_generic_string
                    do_generic_file_read.constprop.33
                    generic_file_aio_read
                    do_sync_read
                    vfs_read
                    sys_read
                    system_call_fastpath
                    0x7f0c5b72bd2d

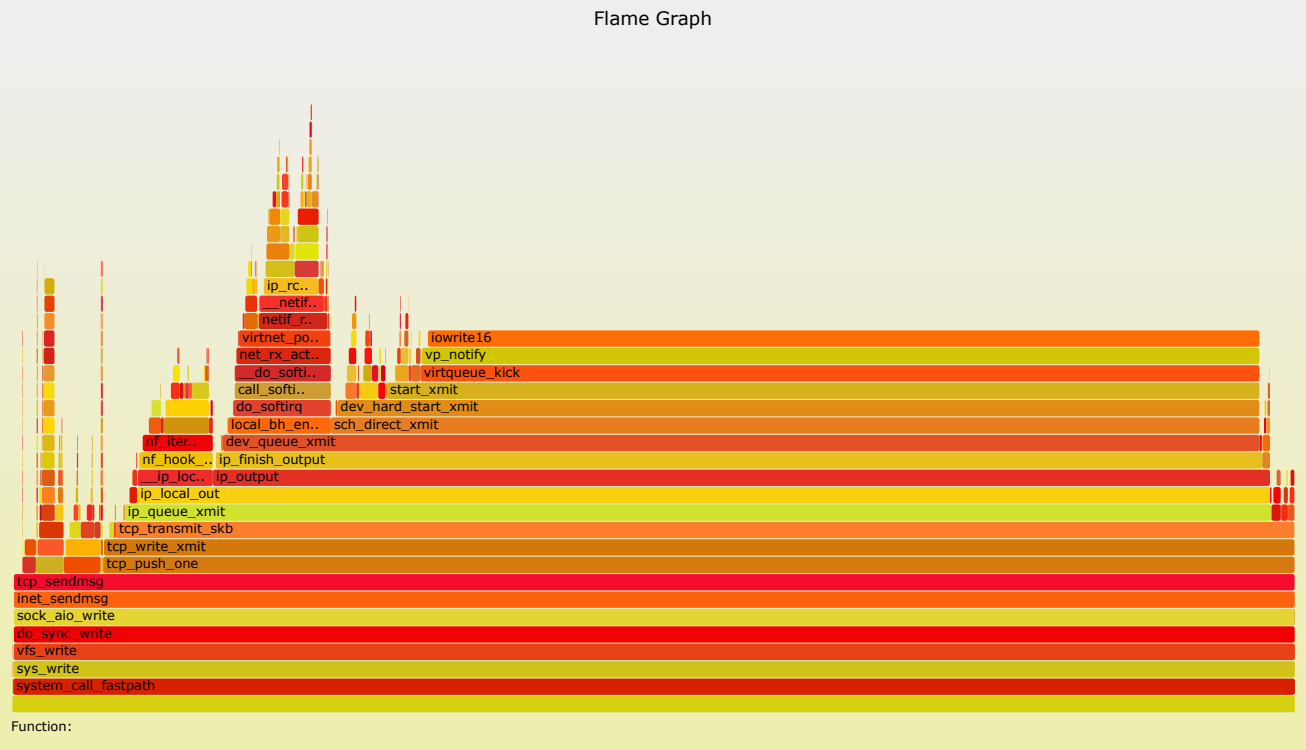
    10.79%        107       fio  [vdso]              [.] 0x7fff695c398f
                |
                --- 0x7fff695c398f
                    clock_gettime
[...]
```

`perf report` podrazumevano prikazuje interaktivni interfejs, ali i tekstualni putem `--stdio` opcije. Opcija `-n` ispisuje broj uzoraka.

Izveštaje je moguće vizuelizovati kroz vatrene grafike (*flame graphs*). Oni pokazuju populaciju uzoraka na x osi, a dubinu steka na y osi. Svaka funkcija je jedan pravougaonik, širine relativne broju uzoraka. Vatreni grafici se mogu generisati pomoću komande `perf script`:

```
$ git clone https://github.com/brendangregg/FlameGraph
$ cd FlameGraph
$ sudo perf record -F 99 -ag -- sleep 60
$ perf script | ./stackcollapse-perf.pl > out.perf-folded
$ cat out.perf-folded | ./flamegraph.pl > perf-kernel.svg
#-- ako ne želimo da keširamo najsporiji korak (pisanje u datoteku)
# perf script | ./stackcollapse-perf.pl | ./flamegraph.pl > perf-kernel.svg
```

Primer jednog vatrenog grafika:



8.3 Intel VTune Profiler

Intel VTune Profiler je profajler za analizu performansi programa na nivou procesora, niti i memorijskog podsistema. Koristi se kada jednostavan izveštaj o vremenu po funkcijama nije dovoljan i kada treba razumeti zašto program troši vreme: čekanje niti, promašaji keša, propusnost memorije, loša iskorišćenost jezgara ili mikroarhitekturna ograničenja.

VTune se može koristiti iz grafičkog interfejsa i iz komandne linije. U ovim materijalima prikazujemo komandne šablone, jer su pogodniji za ponavljanje vežbi i za dokumentovanje merenja. Primeri nisu lokalno provereni u repozitorijumu, pošto VTune zahteva zasebnu instalaciju i često dodatne dozvole za hardversko uzorkovanje.

8.3.1 Česte analize

- **Hotspots**: pronalazi funkcije i pozive u kojima program troši najviše CPU vremena. Ovo je početna analiza za većinu programa.
- **Microarchitecture Exploration**: koristi hardverske brojače da opiše zašto CPU ne izvršava instrukcije očekivanom brzinom, npr. zbog čekanja na memoriju ili loše predikcije grananja.
- **Memory Access**: analizira pristup memoriji, promašaje keša, propusnost memorije i NUMA efekte.
- **Threading**: prikazuje neravnotežu između niti, čekanje na zaključavanja, previše niti i delove izvršavanja u kojima jezgra ostaju neiskorišćena.

8.3.2 Komandni tok rada

Program prvo treba prevesti u konfiguraciji koja odgovara pitanju koje merimo. Za traženje uskih grla koristi se optimizovana verzija, ali sa simbolima za debagovanje, jer simboli omogućavaju da se uzorci vežu za funkcije i linije koda.

Osnovno sakupljanje Hotspots profila:

```
$ vtune -collect hotspots -result-dir vtune-hotspots -- ./program arg1 arg2
```

Tekstualni izveštaj iz postojećeg direktorijuma rezultata:

```
$ vtune -report hotspots -result-dir vtune-hotspots
```

Analiza rada niti:

```
$ vtune -collect threading -result-dir vtune-threading -- ./program
$ vtune -report summary -result-dir vtune-threading
```

Analiza pristupa memoriji:

```
$ vtune -collect memory-access -result-dir vtune-memory -- ./program
$ vtune -report summary -result-dir vtune-memory
```

Mikroarhitekturna analiza je korisna tek kada je već poznato da program značajno vreme provodi na CPU-u:

```
$ vtune -collect uarch-exploration -result-dir vtune-uarch -- ./program
$ vtune -report summary -result-dir vtune-uarch
```

Direktorijumi `vtune-hotspots`, `vtune-threading`, `vtune-memory` i `vtune-uarch` su izlazni artefakti. Ne treba ih dodavati u repozitorijum.

8.3.3 Kako čitati rezultat

VTune izveštaj prvo treba čitati od nivoa procesa i niti ka funkcijama. Ako je program uglavnom u stanju čekanja, optimizacija petlje koja troši malo aktivnog CPU vremena neće promeniti ukupno vreme. Ako je program CPU-bound, Hotspots i Microarchitecture Exploration pomažu da se izdvoje funkcije koje stvarno zaslužuju dalju analizu.

Kod višedretvenih programa prvo proveriti da li sve niti rade koristan posao. Čest nalaz je da program ima mnogo niti, ali malo paralelnog rada zbog zaključavanja, redova čekanja ili serijskih faza. Tada povećanje broja niti može pogoršati performanse.

Kod memorijski intenzivnih programa obratiti pažnju na lokalnost podataka. Memory Access analiza može da pokaže da program troši vreme na čekanje memorije, ali ne predlaže automatski bezbednu promenu strukture podataka. Predloženu optimizaciju treba posebno testirati i izmeriti.

8.3.4 Ograničenja i zamke

- VTune koristi uzorkovanje i hardverske brojače. Kratka izvršavanja i nereprezentativni ulazi daju šum.
- Hardversko uzorkovanje može zahtevati drajvere, kernel podešavanja ili administratorske dozvole, u zavisnosti od operativnog sistema.
- Izveštaj zavisi od simbola. Bez simbola za debagovanje rezultat se često svodi na adrese ili nejasne okvire poziva.
- Kod upravljanih jezika, JIT kompajler i runtime utiču na pripisivanje vremena metodama. Potrebno je proveriti da li VTune vidi simbole za konkretan runtime.
- VTune identifikuje verovatna uska grla. Ne dokazuje ispravnost programa i ne garantuje da je optimizacija bezbedna.

8.3.5 Odnos prema alatu `perf`

`perf` je dobar prvi izbor na Linux sistemima kada treba brzo sakupiti profil ili proveriti hardverske događaje bez teške instalacije. VTune ima bogatije grupisanje nalaza, grafički interfejs i unapred definisane analize za niti, memoriju i mikroarhitekturu. U praksi je koristan tok rada da se `perf` koristi za brzu potvrdu problema, a VTune za detaljniju analizu kada je usko grlo dovoljno važno.

8.4 Java async-profiler

async-profiler je uzorkujući profajler za HotSpot JVM. Može da meri CPU vreme, alokacije, zaključavanja, wall-clock vreme i da izveze rezultat kao flame graph ili JFR snimak. Koristan je kada `perf` vidi JVM proces, ali ne daje dovoljno dobar prikaz Java metoda, JIT-ovanog koda i runtime aktivnosti.

Primer je namerno jednostavan Java program bez Maven ili Gradle projekta. Jedna nit troši CPU tražeći proste brojeve, druga pravi alokacije, a dve niti se nadmeću oko istog zaključavanja:

```
import java.util.ArrayList;
import java.util.List;

public class ProfilerTarget {
```

```
    private static final Object LOCK = new Object();
    private static volatile boolean running = true;
    private static volatile long sink = 0;

    private static boolean isPrime(int value) {
        if (value < 2) {
            return false;
        }

        for (int i = 2; i * i <= value; i++) {
            if (value % i == 0) {
                return false;
            }
        }

        return true;
    }

    private static void cpuWorker() {
        int candidate = 10_000;
        while (running) {
            if (isPrime(candidate)) {
                sink += candidate;
            }
            candidate++;
            if (candidate > 30_000) {
                candidate = 10_000;
            }
        }
    }

    private static void allocationWorker() {
        while (running) {
            List<byte[]> batch = new ArrayList<>();
            for (int i = 0; i < 200; i++) {
                batch.add(new byte[4096]);
            }
            sink += batch.size();
        }
    }

    private static void lockWorker() {
        while (running) {
            synchronized (LOCK) {
                for (int i = 0; i < 50_000; i++) {
                    sink += i & 7;
                }
            }
            Thread.yield();
        }
    }

    public static void main(String[] args) throws InterruptedException {
        int seconds = args.length > 0 ? Integer.parseInt(args[0]) : 120;

        System.out.printf("pid=%d duration=%ds%n", ProcessHandle.current().pid(),
```

```
                        seconds);
        System.out.println("Attach async-profiler from another terminal.");

        Thread cpu = new Thread(ProfilerTarget::cpuWorker, "cpu-worker");
        Thread alloc = new Thread(ProfilerTarget::allocationWorker,
                                  "allocation-worker");
        Thread lockA = new Thread(ProfilerTarget::lockWorker, "lock-worker-a");
        Thread lockB = new Thread(ProfilerTarget::lockWorker, "lock-worker-b");

        cpu.start();
        alloc.start();
        lockA.start();
        lockB.start();

        Thread.sleep(seconds * 1000L);
        running = false;

        cpu.join();
        alloc.join();
        lockA.join();
        lockB.join();

        System.out.println("sink=" + sink);
    }
}
```

Prevedimo program:

```
$ make
```

Pokrenemo ga u jednom terminalu:

```
$ java ProfilerTarget 120
pid=12345 duration=120s
Attach async-profiler from another terminal.
```

U drugom terminalu koristimo PID koji je program ispisao. Naziv komande može biti `asprof` ili putanja do skripte iz raspakovane async-profiler arhive.

CPU profil CPU profil je početna analiza kada nas zanima gde JVM proces aktivno troši procesorsko vreme:

```
$ asprof -e cpu -d 30 -f async-cpu.html 12345
```

U flame graph-u očekujemo da se izdvoje `cpuWorker` i `isPrime`. Širina okvira predstavlja broj uzoraka, ne trajanje jednog poziva. Širok okvir znači da je metoda često bila na steku u trenucima uzorkovanja.

Alokacije Alokacioni profil pokazuje gde nastaju objekti i koliko bajtova prolazi kroz alokator:

```
$ asprof -e alloc -d 30 -f async-alloc.html 12345
```

U ovom primeru treba da se vidi `allocationWorker`. Ako CPU profil ne pokazuje ovu metodu kao dominantnu, a alokacioni profil je pokazuje, problem nije nužno CPU petlja nego pritisak na GC i memorijski podsistem.

Zaključavanja Profil zaključavanja meri vreme izgubljeno na kontenciju:

```
$ asprof -e lock -d 30 -f async-lock.html 12345
```

Ovde očekujemo trag kroz `lockWorker`, jer dve niti ulaze u isti `synchronized` blok. Ovakav nalaz treba povezati sa dizajnom deljenog stanja: optimizacija tela petlje neće pomoći ako niti većinu vremena čekaju na isti monitor.

Wall-clock i JFR izlaz Wall-clock profil uzorkuje proteklo vreme, pa može pokazati i čekanja:

```
$ asprof -e wall -d 30 -f async-wall.html 12345
```

Za analizu u Java Mission Control-u ili arhiviranje merenja možemo napraviti JFR datoteku:

```
$ asprof -e cpu -d 30 -f async-cpu.jfr 12345
```

Zamke

- Profilisati reprezentativan ulaz i dovoljno dugo izvršavanje; kratki profili su često šum.
- Proveriti da li se meri CPU, alokacija, lock ili wall-clock događaj. Različiti događaji odgovaraju na različita pitanja.
- U kontejnerima i restriktivnim sistemima mogu biti potrebna dodatna podešavanja za `perf_event_paranoid`, capability-je ili pristup procesu.
- Flame graph pokazuje gde je program bio tokom uzorkovanja. Ne dokazuje sam po sebi da je promena koda bezbedna.

9 eBPF

eBPF je mehanizam u Linux kernelu koji omogućava da se mali programi bezbedno učitaju u kernel i izvršavaju na definisanim tačkama: mrežni događaji, sistemski pozivi, funkcije kernela, tracepoint-i, perf događaji i druge lokacije. U ovom poglavlju eBPF tretiramo kao alat za posmatranje sistema i profajliranje, a ne kao zaseban projekat za pisanje kernel ekstenzija.

Za razliku od prethodnih laboratorijskih vežbi, primeri u ovom poglavlju su komandni obrasci i očekivani oblici izlaza. Ne oslanjaju se na lokalne build skripte, jer eBPF zavisi od verzije kernela, BTF informacija, capability-ja, distribucije i ograničenja kontejnera.

9.1 Model izvršavanja

Tipičan eBPF tok ima dva dela:

- **Kernel program**: mali program koji se izvršava kada se dogodi određeni događaj.
- **Korisnički program**: alat koji učitava eBPF program, čita mape, prikazuje rezultate i gasi merenje.

Pre učitavanja, kernel proverava eBPF program verifier-om. Verifier odbija programe koji mogu da pristupe nevažećoj memoriji, imaju neograničene petlje, koriste nedozvoljene helper funkcije ili ne mogu da dokažu bezbednost pristupa. Zbog toga je eBPF moćan, ali nije proizvoljno izvršavanje koda u kernelu.

Podaci se najčešće razmenjuju kroz *mape*. Mapa može biti brojač, histogram, tabela po PID-u, tabela po steku ili ring buffer za događaje. Ovaj model je važan za tumačenje rezultata: eBPF program obično radi minimalan posao na događaju, a korisnički alat kasnije prikazuje agregirani rezultat.

9.2 bpftrace

bpftrace je visok nivo za brze eBPF skripte i jednolinijske provere. Dobar je kada pitanje može da se izrazi kao: kada se dogodi ovaj događaj, uvećaj brojač, zabeleži argument, napravi histogram ili ispiši proces.

Brojanje poziva sistemskog poziva `openat` po procesu:

```
$ sudo bpftrace -e 'tracepoint:syscalls:sys_enter_openat { @[comm] = count(); }'
```

Očekivani oblik izlaza posle prekida kombinacijom `Ctrl-C`:

```
@[bash]: 3
@[python3]: 18
@[java]: 42
```

Histogram veličina pročitanih blokova:

```
$ sudo bpftrace -e 'tracepoint:syscalls:sys_exit_read /args->ret > 0/ { @bytes = hist(args->ret); }'
```

Praćenje ulaza u funkciju kernela:

```
$ sudo bpftrace -e 'kprobe:vfs_read { @[comm] = count(); }'
```

Praćenje korisničke funkcije preko uprobe-a zahteva putanju do biblioteke ili izvršne datoteke. Primer za `malloc` iz `libc`:

```
$ sudo bpftrace -e 'uprobe:/lib/x86_64-linux-gnu/libc.so.6:malloc { @[ustack] = count(); }'
```

Putanja do `libc` zavisi od distribucije. Pre upotrebe proveriti je komandom `ldd` ili `ldconfig -p`.

9.3 BCC

BCC je skup alata i biblioteka za pisanje eBPF programa iz Python-a i C-sličnog eBPF koda. U nastavi je koristan iz dva razloga: već dolazi sa praktičnim alatima, a zatim se može pokazati kako izgleda mali prilagođeni tracer.

Primeri gotovih alata:

```
$ sudo execsnoop
$ sudo opensnoop
$ sudo biolatency
$ sudo runqlat
```

`execsnoop` prikazuje pokretanje procesa, `opensnoop` otvaranje datoteka, `biolatency` latenciju blok uređaja, a `runqlat` čekanje zadataka u scheduler redu.

Minimalni oblik BCC skripte je:

```
from bcc import BPF

program = r"""
int trace_exec(void *ctx) {
    bpf_trace_printk("exec event\n");
    return 0;
}
"""

b = BPF(text=program)
b.attach_kprobe(event="do_execveat_common", fn_name="trace_exec")
b.trace_print()
```

Ovaj primer je namerno pojednostavljen. U realnom materijalu treba koristiti stabilne tracepoint-e kada postoje, jer se imena internih kernel funkcija mogu menjati između verzija kernela.

9.4 Profajliranje pomoću eBPF-a

eBPF se često koristi za sistemsko profajliranje, naročito kada treba povezati korisničke i kernel stekove, off-CPU čekanje ili događaje koji nisu dobro pokriveni običnim aplikativnim profajlerom.

CPU profil preko `profile` probe u `bpftrace`-u:

```
$ sudo bpftrace -e 'profile:hz:99 { @[ustack] = count(); }'
```

Ovakav profil uzorkuje korisničke stekove oko 99 puta u sekundi. Za kernel stekove koristi se `kstack`, a za kombinovanje oba pogleda često se koriste gotovi alati koji izvoz pretvaraju u flame graph.

Odnos prema `perf`-u je praktičan:

- `perf` je prvi izbor za lokalni CPU profil, hardverske događaje i standardni Linux tok rada.
- eBPF je jači kada pitanje zahteva dinamičko filtriranje događaja, histograme u kernelu, povezivanje sa PID-ovima, datotekama, mrežom ili scheduler događajima.
- Flame graph je format prikaza, ne konkretan alat. Može nastati iz `perf`, async-profiler-a, eBPF alata ili drugih izvora stekova.

9.5 Dozvole i zamke

eBPF vežbe često ne rade iz prve na studentskim mašinama. To nije nužno greška u skripti. Proveriti sledeće:

- Kernel mora podržavati potrebne eBPF mogućnosti i BTF informacije za neke alate.
- Potrebne su privilegije: često `root`, `CAP_BPF`, `CAP_PERFMON` ili slična podešavanja, u zavisnosti od kernela.
- `perf_event_paranoid`, `kptr_restrict` i slična podešavanja mogu sakriti događaje ili simbole.
- Kontejneri često nemaju potrebne capability-je ili pristup `/sys/kernel`.
- Kprobe na internu kernel funkciju je manje stabilan od tracepoint-a.
- Uprobe zavisi od putanje biblioteke, simbola i optimizacija korisničkog programa.

Kada eBPF program ne može da se učita, prvo pročitati poruku verifier-a. Ona često pokazuje da pristup memoriji nije dokazan kao bezbedan, da helper nije dozvoljen za taj tip programa ili da je tip podataka pogrešan za izabrani događaj.

10 Simboličko izvršavanje (alat KLEE)

KLEE je javno dostupan alat koji se distribuira pod licencom NCSA otvorenog koda Univerziteta Ilinois. Služi za simboličko izvršavanje programa i za automatsko generisanje test primera. KLEE vrši analizu nad LLVM međureprezentacijom i koristi SMT rešavač STP za proveravanje uslova ispravnosti koje generiše. KLEE koristi nekoliko optimizacija i heuristika za poboljšavanje pokrivenosti koda prilikom simboličkog izvršavanja. KLEE se koristi i kao sastavni deo raznih platformi za razvijanje novih alata za analizu programa. KLEE ima dva cilja:

1. da pokrije svaku liniju izvornog koda u programu
2. da detektuje svaku opasnu operaciju ako postoji ijedna ulazna vrednost koja može da prouzrokuje grešku

Ukoliko prilikom generisanja test primera za neku od putanja, KLEE pronađe test primer koji izaziva grešku, onda će pored tog test primera biti generisane još dve datoteke:

- `test<N>.pc` - uslovi ograničenja koji važe za tu putanju i koji su prosleđeni solveru u `KQuery` sintaksi.
- `test<N>.<TYPE>.err` - dodatne informacije o grešci

`<TYPE>` označava tip detektovane greške. Neke od grešaka koje KLEE može da detektuje su:

- **ptr**: Čitanje ili pisanje u nevalidnu memorijsku lokaciju;
- **free**: Dupli ili nekorektan poziv funkcije `free()`;
- **abort**: Program je pozvao funkciju `abort()`;
- **assert**: Uslov proveravan funkcijom `assert` nije tačan;
- **div**: Deljenje nulom ili računanje po modulu 0;
- **user**: Problem je u vezi sa ulazom (neispravan poziv `klee intrinsic`) ili pogrešan način upotrebe KLEE-a.
- **exec**: Dogodio se problem koji je KLEE sprečio u izvršavanju programa, npr. nepoznata instrukcija, poziv preko neispravnog pokazivača na funkciju ili inline funkcije.
- **model**: KLEE nije mogao da ispuni kompletnu preciznost i ispituje samo deo svih stanja programa. Na primer, simboličke veličine prosleđene `malloc` funkciji nisu trenutno podržane i u takvim situacijama KLEE konkretizuje taj argument.

10.1 Primer upotrebe KLEE alata

Naredni primer (`01_intro`) će nas sprovesti kroz osnovne korake pripreme izvornog koda za korišćenje alata. Razmotrimo kratak program u kome se testira funkcija koja proverava da li je učitan ceo broj paran:

```
#include <stdio.h>

int even(int x) {
    if (x % 2 == 0)
        return 1;
    if (x % 2)
        return 0;
```

```
    return 1;
}

int main(int argc, char *argv[]) {
    int x;
    scanf("%d", &x);
    printf("%d", even(x));
    return 0;
}
```

Možemo primetiti da je poslednja naredba u definiciji funkcije `even` nedostižna, odnosno da će se uvek izvršiti prvi ili drugi `return` u funkciji.

Priprema programa za simboličko izvršavanje:

1. uključiti zaglavlje `klee/klee.h`
2. umesto učitavanja vrednosti promenljive koja nas zanima, označiti je simboličkom pomoću funkcije `klee_make_symbolic`
3. umesto korišćenja eksterne funkcije za ispis koju KLEE ne može da proveri, povratnu vrednost funkcije `even` ćemo vratiti preko povratne vrednosti funkcije `main`

Argumenti funkcije `klee_make_symbolic`, redom:

1. adresa promenljive
2. veličina promenljive
3. naziv promenljive (proizvoljna niska karaktera)

Opciono: primetili smo da je poslednja linija funkcije `even` nedostižna, ali to možemo i potvrditi pozivom funkcije `klee_assert(0)`, što će signalizitati grešku alatu ukoliko bude postojala putanja kojom se dolazi do poziva ove funkcije. U slučaju da takva putanja ne postoji, neće biti signalizirana greška i potvrdiće se pretpostavka da ne postoji ulaz za koji je ta naredba dostižna.

Napomena: Korišćenje funkcije `klee_assert` zahteva uključivanje zaglavlja `assert.h`.

```
#include <assert.h>
#include <klee/klee.h>

int even(int x) {
    if (x % 2 == 0)
        return 1;
    if (x % 2)
        return 0;
    klee_assert(0);
    return 1;
}

int main(int argc, char* argv[]) {
    int x;
    klee_make_symbolic(&x, sizeof(x), "x");
    return even(x);
}
```

Pomoću Clang kompilatora generišemo LLVM bitkod koji će biti ulaz za KLEE (Clang će kreirati datoteku sa nazivom `even.bc`):

```
$ clang -emit-llvm -c -g even.c
$ klee even.bc
```

```
KLEE: output directory is "..."
Using STP solver backend

KLEE: done: total instructions = 42
KLEE: done: completed paths = 2
KLEE: done: generated tests = 2
```

Na osnovu dobijenog izlaza, postoje dve putanje u našem programu, kao što smo i očekivali, u zavisnosti od toga da li je argument paran ili neparan po vrednosti. KLEE je generisao direktorijum pod nazivom `klee-out-0` sa test primerom za svaku eksploatisanu putanju. Pri sledećem pokretanju generisao bi se direktorijum `klee-out-1`, tj. u opštem slučaju direktorijum pod nazivom `klee-out-N` za prvo slobodno `N`. Takođe, napravljen je i simbolički link `klee-last` koji uvek ukazuje na poslednji `klee-out` direktorijum. Ukoliko izlistamo direktorijum `klee-out-0`, između ostalog, primetićemo datoteke sa ekstenzijom `.ktest`. To su binarne datoteke koje sadrže test primere za svaku putanju zasebno i koje se mogu pročitati korišćenjem alata `ktest-tool`:

```
$ ktest-tool --write-ints test000001.ktest
```

```
ktest file : 'test000001.ktest'
args       : ['even.bc']
num objects: 2
object    0: name: 'model_version'
object    0: size: 4
object    0: data: 1
object    1: name: 'x'
object    1: size: 4
object    1: data: 0
```

```
$ ktest-tool --write-ints test000002.ktest
```

```
ktest file : 'test000002.ktest'
args       : ['even.bc']
num objects: 2
object    0: name: 'model_version'
object    0: size: 4
object    0: data: 1
object    1: name: 'x'
object    1: size: 4
object    1: data: 1
```

Primećujemo da je KLEE generisao ulaz `0` i `1` za pokrivanje dve otkrivene putanje, tj. jednu parnu i jednu neparnu ulaznu vrednost.

Da bismo proverili da li naš program daje očekivane izlaze za generisane test primere, možemo testirati polazni program korišćenjem vrednosti `data` iz pročitanih testova ili koristiti simbolički program zahvaljujući posebnoj biblioteci `libkleeRuntest` i koja pozive funkcije `klee_make_symbolic` zamenjuje pozivom funkcije koja promenljivoj dodeljuje vrednost zapisanu u odgovarajućoj test datoteci.

Testiranje pomoću `libkleeRuntest` biblioteke:

```
$ export LD_LIBRARY_PATH=/putanja/do/klee/Release+Asserts/lib
$ gcc \
    -I /putanja/do/klee/include/ \
    -L /putanja/do/klee/Release+Asserts/lib even.c \
    -lkleeRuntest
$ KTEST_FILE=klee-last/test000001.ktest ./a.out
$ echo $?
1
$ KTEST_FILE=klee-last/test000002.ktest ./a.out
$ echo $?
0
```

Očekivano, nismo dobili poruku od `klee_assert` što potvrđuje pretpostavku da je označen deo koda nedostižan za bilo koji ulaz.

Statističke informacije u vezi sa izvršavanjem KLEE alata nad nekim bitkodom, kao što su broj izvršenih instrukcija, pokrivenost instrukcija bitkoda, pokrivenost grana bitkoda i slično, možemo dobiti pomoću Python skripta `klee-stats`. Informacije se prikazuju u tabelarnoj formi. Ovaj alat se poziva nad direktorijumom koji je generisao KLEE. Da bismo bili sigurni da je to najnovija verzija direktorijuma, najjednostavnije je alat pozvati sa argumentom `klee-last`.

```
$ klee-stats klee-last
```

```
------------------------------------------------------------------------
|  Path   |  Instrs|  Time(s)|  ICov(%)|  BCov(%)|  ICount|  TSolver(%)|
------------------------------------------------------------------------
|klee-last|      42|     0.06|    12.16|     5.71|     296|       90.36|
------------------------------------------------------------------------
```

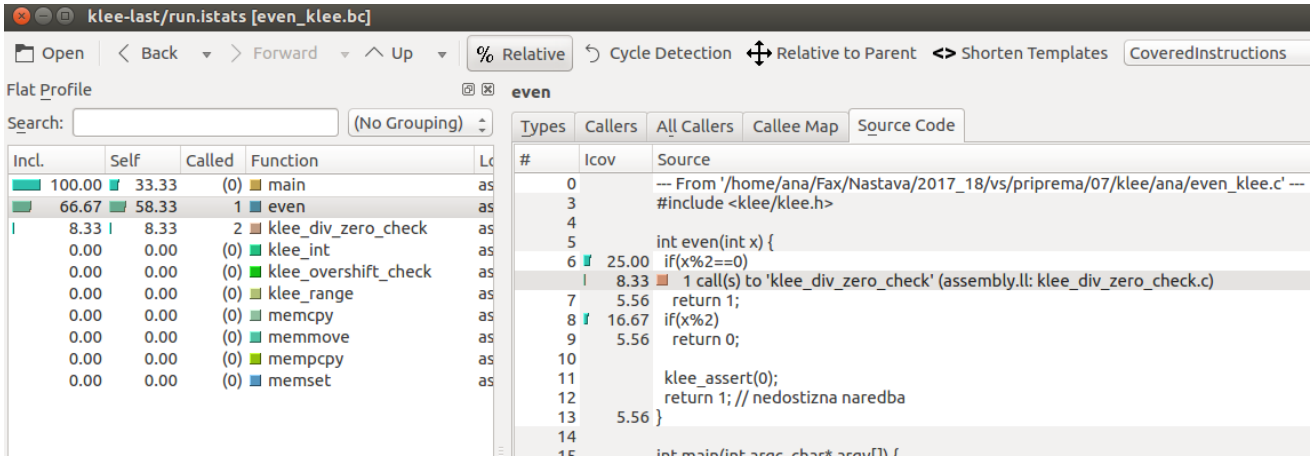
Legendu za tabelu možemo dobiti pomoću opcije `--help`.

```
LEGEND
--------  --------------------------------------------
Instrs    number of executed instructions
Time      total wall time (s)
ICov      instruction coverage in the LLVM bitcode (%)
BCov      branch coverage in the LLVM bitcode (%)
ICount    total static instructions in the LLVM bitcode
TSolver   time spent in the constraint solver
```

Vizuelni prikaz statistike o simboličkom izvršavanju možemo dobiti korišćenjem alata *KCachegrind*:

```
$ kcachegrind klee-last/run.istats
```

Izborom `CoveredInstructions` ili `UncoveredInstructions` i izborom kartice `Source Code` možemo tačno videti koje linije jesu, odnosno nisu pokrivene test primerima.



Primetimo da su brojevi koje sada vidimo daleko veći od brojeva koje je prikazao `klee-stats`. Razlog je to što gcov razmatra samo linije u jednoj datoteci, a ne u celoj aplikaciji, dok sa KCachegrind-om možemo čitati izlaz gcov-a o pokrivenosti koda i ispratiti koje su linije pokrivene, a koje ne.

10.2 Primer semantičke greške

Pogledajmo primer neispravne funkcije za proveru parnosti celog broja:

```
int bad_even(int x) {
    if (x % 2 == 0)
        return 1;
    if (x % 2 == 1)
        return 0;
    return 1;
}
```

Ovaj primer, za razliku od prethodnog ima grešku prilikom provere neparnih negativnih brojeva jer je za njih ostatak pri deljenju jednak `-1` pa nisu obuhvaćeni drugom `if` naredbom.

Pokušaćemo da korišćenjem KLEE alata otkrijemo ovu grešku i dobijemo test primer za koji je dostižna poslednja `return` naredba. Kao i malopre, u glavnom programu označavamo simboličku promenljivu i dodajemo poziv funkcije `klee_assert(0)` koja će signalizirati grešku pri eksploatisanju putanje u kojoj je malopređašnja nedostižna naredba.

Nakon dobijanja LLVM bitkoda `bad_even.bc` i pokretanja KLEE alata:

```
KLEE: output directory is "klee-out-0"
Using STP solver backend
KLEE: ERROR: bad_even.c:12: ASSERTION FAIL: 0
KLEE: NOTE: now ignoring this error at this location

KLEE: done: total instructions = 39
KLEE: done: completed paths = 3
KLEE: done: generated tests = 3
```

Primetimo da su sada pronađene tri putanje i generisani odgovarajući test primeri, ali je i prijavljena greška koju je signalizirala funkcija `klee_assert`. Ukoliko pogledamo sadržaj direktorijuma sa test primerima, primetićemo da je napravljena datoteka `test000002.assert.err` koja nam ukazuje da je gore pomenuta greška signalizirana pri drugom test primeru.

Pogledajmo kakav je ulaz u pitanju:

```
$ ktest-tool --write-ints test000002.ktest
```

```
ktest file : 'test000002.ktest'
args       : ['bad_even.bc']
num objects: 1
object    0: name: 'x'
object    0: size: 4
object    0: data: -2147483647
```

KLEE je pronašao da je sporna vrednost ulaza `-2147483647`. Upotrebom debagera i korišćenjem sporne ulazne vrednosti može se uočiti mesto problema, odnosno da je zaboravljeno da je ostatak pri deljenju sa `2` u slučaju negativnih brojeva `-1`, a ne `1`. Očekivano, vreme izvršavanja koje je potrošeno na solver je veće nego u slučaju ispravnog programa.

```
$ klee-stats klee-last
```

```
------------------------------------------------------------------------
|  Path   |  Instrs|  Time(s)|  ICov(%)|  BCov(%)|  ICount|  TSolver(%)|
------------------------------------------------------------------------
|klee-last|      39|     0.17|    11.30|     7.14|     292|       93.78|
------------------------------------------------------------------------
```

Generalno, prikazanom upotrebom funkcije `klee_assert` možemo označiti putanju za koju smo zainteresovani, bilo da bismo proverili da li je ta putanja izvodljiva ili da bismo imali na izlazu označen ulaz kojim se ta putanja dostiže.

10.3 Primer greške u radu sa pokazivačima

U slučaju da postoji putanja programa na kojoj može doći do nepravilnog pristupa memorijskim lokacijama, što je čest slučaj pri radu sa pokazivačima, `KLEE` će sam prijaviti grešku i generisati

ulaz na osnovu koga možemo popraviti svoj program. Primer ovakvog programa se može videti u primeru `03_err_pointer`.

Označićemo simbolički ulaz i uključiti potrebno zaglavlje i zatim sačuvati kod u datoteku `pointer_error_sym.c`. Na osnovu nje ćemo napraviti odgovarajući bitkod i pokrenuti alat KLEE. KLEE je pronašao `8` putanja i generisao `5` test primera, a pritom je i javio `memory error: out of bound pointer`. Test primer koji dovodi do prijavljene greške će biti označen posebnom datotekom sa ekstenzijom `.ptr.err`. Primetimo da je broj test primera manji od broja istraženih putanja.

BITNO: `KLEE` neće generisati sve moguće ulaze koji dovode do iste greške. Ukoliko je tako nešto potrebno, prilikom pokretanja dodati opciju `--emit-all-errors`:

```
$ klee --emit-all-errors pointer_error_sym.bc
```

Ako uvedemo pomenutu opciju prilikom pokretanja alata KLEE, biće generisano `8` test primera, od čega `4` označena greškom za memoriju. Čitanjem dobijenih test primera, možemo ispraviti sve putanje koje su dovele do problema sa pokazivačima.

Pokrenuti `klee-stats` nad direktorijumom koji je generisan u prvom slučaju i sa dodavanjem opcije za emitovanje svih grešaka. Primetiti da se jedino razlikuje vreme potrošeno na solver dok je pokrivenost koda i grana identična.

10.4 Korišćenje C bibliotečkih funkcija i simboličkog okruženja

Kada se KLEE koristi nad programom koji poziva C bibliotečke funkcije, potrebno je navesti opcije `--libc=uclibc --posix-runtime`. One omogućavaju ispravno linkovanje i rad sa simboličkim okruženjem. Opcija `--allow-external-sym-calls` dozvoljava pozivanje spoljnih funkcija sa simboličkim vrednostima.

Dodatne opcije za rad sa simboličkim okruženjem:

1. `--sym-arg <N>` - simbolički argument dužine najviše N
2. `--sym-args <MIN> <MAX> <N>` - najmanje MIN, a najviše MAX simboličkih argumenata komandne linije, svi maksimalne dužine N
3. `--sym-files <NUM> <N>` - NUM simboličkih datoteka veličine N (ne uključujući stdin)
4. `--sym-stdin <N>` - simbolički standardni ulaz veličine N
5. `--sym-stdout` - simbolički standardni izlaz

Korisna je i opcija `--optimize`, koja uklanja mrtav kod tokom istraživanja putanja i može da ubrza izvršavanje. Svaki poziv eksterne funkcije usložnjava pronalaženje putanja i generisanje test primera.

10.4.1 Primer sa simboličkim argumentom komandne linije

Primer `04_libc` sadrži program za validaciju lozinke jednog sistema koja se zadaje kao argument komandne linije i za koju jedino znamo da je maksimalne dužine `100`. Pretpostaviti da postoji samo jedna valjana kombinacija za lozinku. Zadatak je pronaći lozinku bez rešavanja zadatih jednačina koje su postavljene u programu.

Zadatak ćemo rešiti označavanjem putanje koja nas zanima pozivom funkcije `klee_assert(0)`, a to je upravo nakon ispisa poruke o uspešno pronađenoj lozinki (datoteka `password_sym.c`). Nakon što se generiše bitkod `password_sym.bc`, dodajemo potrebne opcije prilikom poziva KLEE alata:

```
$ klee --optimize --libc=uclibc --posix-runtime password_sym.bc -sym-arg 100
```

Nakon što je prošlo par sekundi, može se primetiti da je generisana samo jedna poruka o grešci i čak 102 putanje. Da bismo dobili onu koja nas zanima, izvući ćemo naziv datoteke ekstenzije .err i pročitati odgovarajuću .ktest datoteku:

```
$ ls klee-last/*err
klee-last/test000026.assert.err
$ ktest-tool test000026.ktest
```

```
ktest file : 'test000026.ktest'
args       : ['password_sym.bc', '-sym-arg', '100']
num objects: 2
object    0: name: 'arg0'
object    0: size: 101
object    0: data: 'Pandi_panda\x00\xff\xff\xff\xff
\xff\xff\xff\xff\xff\xff\xff\xff\xff\xff\xff\xff\xff
\xff\xff\xff\xff\xff\xff\xff\xff\xff\xff\xff\xff\xff
\xff\xff\xff\xff\xff\xff\xff\xff\xff\xff\xff\xff\xff
\xff\xff\xff\xff\xff\xff\xff\xff\xff\xff\xff\xff\xff
\xff\xff\xff\xff\xff\xff\xff\xff\xff\xff\xff\xff\xff
\xff\xff\xff\xff\xff\xff\xff\xff\xff\xff\xff\xff\xff
\xff\xff\xff\xff\xff\xff\xff'
object    1: name: 'model_version'
object    1: size: 4
object    1: data: 1
```

Dobili smo da je ulaz Pandi_panda koji prati terminirajuća nula ispravna lozinka.

10.5 KLEE heuristike

KLEE je primenjen na svih 89 programa u verziji 6.10 paketa GNU Coreutils, koji obuhvata oko 80,000 linija koda biblioteka i 61,000 linija iz konkretnih programa. Ti programi interaguju sa okruženjem da bi omogućili različite funkcionalnosti, kao što je manipulisanje sistemom datoteka (npr. ls, dd, chmod), prikazivanje i konfigurisanje svojstava sistema (npr. hostname, logname, printenv), kontrolu poziva komandi (npr. nohup, nice, env), obrada tekstualnih datoteka (npr. sort, od, patch), itd.

Svaki program iz paketa Coreutils dolazi sa velikim skupom ručno pisanih testova, koji se proširuje sa svakom novo otkrivenom greškom ili dodatom opcijom. KLEE je nadmašio ručno pisani skup testova u pogledu ukupne pokrivenosti linija (84.5% nasuprot 67.7%) i prosečne pokrivenosti po alatu (90.9% nasuprot 68.4%). Postigao je 100% pokrivenost na 16 programa i preko 90% na 56 programa dok ručno pisani testovi imaju 100% pokrivenost samo na jednom programu (true) i preko 90% samo na 7 programa. U suštini, za 89h, tj. 1h po programu, KLEE je za 16.8% premašio pokrivenost skupa ručno pisanih testova u periodu od 15 godina.

KLEE je pronašao 10 grešaka u Coreutils (uglavnom grešaka sa memorijom). U tabeli ispod je dat spisak poziva koje je KLEE generisao za pronađene greške. Prve tri greške postoje bar od 1992. godine, dok su ostale novije. U trenutku objavljivanja rezultata greška za seq je već bila

ispravljena u neobjavljenoj verziji, a ostale greške su potvrđene i ispravljene u toku od 2 dana od prijavljivanja. Dodatno, od tada se u zvaničnom skupu testova nalaze i KLEE-om generisani testovi za ove greške.

```
paste -d\\ abcdefghijklmnopqrstuvwxyz
pr -e t2.txt
tac -r t3.txt t3.txt
mkdir -Z a b
mkfifo -Z a b
mknod -Z a b p
md5sum -c t1.txt
ptx -F abcdefghijklmnopqrstuvwxyz
ptx x t4.txt
seq -f %0 1
```

```
t1.txt: "\t \tMD5("
t2.txt: "\b\b\b\b\b\b\b\t"
t3.txt: "\n"
t4.txt: "a"
```

10.5.1 Korišćenje različitih heuristika za pretragu

KLEE koristi sledeće heuristike za pretragu putanja:

- Depth-First Search (DFS)
- Random State Search
- Random Path Selection
- Non Uniform Random Search (NURS)

Heuristika se može izabrati KLEE-ovom opcijom `-search`, npr. `-search=dfs`. Za *NURS* se mora dodati opcija za heuristiku kojom se dodeljuju težine pojedinim putanjama i tako odlučuje koja će biti naredna praćena:

- `nurs:covnew`: koristi *Coverage-New* heuristiku kojom se daje veća težina putanji koja će pokriti nepokriven deo koda;
- `nurs:md2u`: koristi *Min-Dist-to-Uncovered* heuristiku kojom se bira putanja najbliža nepokrivenoj instrukciji;
- `nurs:depth`: koristi `2^{depth}` heuristiku
- `nurs:icnt`: koristi *Instr-Count* heuristiku
- `nurs:cpicnt`: koristi *CallPath-Instr-Count* heuristiku
- `nurs:qc`: koristi *Query-Cost* heuristiku

KLEE-u se može zadati da koristi više heuristika na *round-robin* način, da bi se sprečilo da pojedine heuristike zaglave u lokalnom maksimumu.

Podrazumevana je *random-path* ukomponovan sa `nurs:covnew`.

Moguće je zadati željenu heuristiku korišćenjem opcije `--search`. Detaljnije informacije u vezi sa parametrima za ovu opciju možete dobiti pozivom KLEE-a sa opcijom `--help`.

10.5.2 Testiranje heuristika

Testiraćemo izbor putanja različitih heuristika na primerima koji imaju `N` simboličkih promenljivih i za svaku od njih imamo `if-else` grane. Nakon tih provera imamo i računanje količnika. Zbog toga što je nedozvoljeno deliti nulom, KLEE implicitno za tu naredbu dodaje proveru. Takvo ponašanje se javlja kod svih kritičnih mesta u kôdu. Za svaki broj simboličkih promenljivih imamo 3 verzije programa. Verzije sa sufiksom `-1.c` i `-2.c` imaju deljenje nulom samo na različitim putanjama, dok verzija sa sufiksom `-v.c` nema spornu putanju.

Ukoliko želimo da vidimo za svaki generisani test primer putanju i upit koji je slan solveru, prilikom poziva KLEE-a treba zadati opciju `--write-paths`.

Za potrebe testiranja dodaćemo i opciju `-exit-on-error` da se prekine sa simboličkim izvršavanjem čim se dođe do sporne putanje.

Merenje je automatizovano skriptom `measure_time.sh`.

Pokrećemo sa `./measure_time.sh dfs yes 15` da se dfs heuristika koristi za izbor putanje, da nam se generišu sve putanje i upiti za solver i da ne pokrećemo programe sa više od 15 simboličkih promenljivih.

Za verzije `*-1.c`, ako se koristi opcija `--search=dfs`, odmah se nailazi na spornu putanju. Stoga se test primer za nju generiše uvek kao `test1`. Nije potrebno čekati kraj cele pretrage jer se greška odmah prijavljuje.

Za verzije `*-2.c`, ako se koristi opcija `--search=dfs`, sporna putanja je poslednja koja se obilazi. Do nje se neće doći brzo; vreme zavisi od broja putanja koje joj prethode.

Sa opcijom `--search=random_path` primenjenom na programe sa većim brojem promenljivih, KLEE možda neće pronaći spornu putanju. Uzrok je vrlo razgranato stablo izvršavanja, tj. veliki broj putanja.

Ukoliko otvorimo generisanu datoteku sa izveštajem `report_dfs.txt` videćemo da su verzije `*-1.c` uvek ranije dolazile do greške i prekidale dalje simboličko izvršavanje. U odgovarajućem direktorijumu naći ćemo testove. Za test1 nas interesuje putanja. Primećujemo da je uvek birana `else` grana i da smo tako odmah došli do sporne putanje. Verzija `*-2.c` je tako napisana da `dfs` heuristikom tek na kraju obilazimo spornu putanju. Kod složenijih stabala izvršavanja, tj. kod programa sa većim brojem simboličkih promenljivih može se dogoditi i da je ne pronađe pre nego što potroši resurse.

Ako imamo `N` promenljivih i za svaku od njih po grananje imamo 2^N putanja. Verovatnoća da heuristika `random-path` ubode spornu putanju je `1/{2^N}`. To se može dogoditi ranije ili kasnije, ali se može desiti da se i ne isprati ta putanja pre prekoračenja memorijskog prostora, zbog prevelike razgranatosti stabla.

Problem sa memorijom nastaje jer je KLEE program koji se izvršava u okviru jednog procesa i celo stablo mora biti u svakom trenutku u memoriji. Bolje performanse se postižu paralelizacijom. *Cloud9* je program koji dinamički distribuira delove stabla između POSIX niti koje izvršavaju simboličko izvršavanje programa bazirano na KLEE-u. Time postiže bolju pokrivenost za kraće vreme.

Ipak, ukoliko se iskoristi KLEE-ova opcija `-optimize` primenjuju se optimizacije kompajlera i izvršavanje je višestruko brže. Samim tim pronalaze se sporne putanje i na programima sa vrlo razgranatim stablom.

10.5.3 Kreiranje nove heuristike za KLEE

Kôd vezan za heuristike u KLEE-u se nalazi u datotekama `/build/klee/lib/Core/Searcher.h` i `/build/klee/lib/Core/Searcher.cpp`. Nova heuristika treba da bude pisana u klasi koja će naslediti klasu `Searcher`. Kôd nove heuristike uneti u pomenute datoteke. Navesti ime nove heuristike u `CoreSearchType` i dopuniti metode klase `UserSearcher` sa informacijama o novoj heuristici. Klasa `Executor` svojom metodom `run` kreira stablo izvršavanja, kreira objekat klase `UserSearcher` i prati i izvršava stanja programa, tj. objekata klase `ExecutionState`.

Nakon izmena koda bilo potrebno i ponovo izgraditi KLEE. Zato se treba pozicionirati u KLEE-ov direktorijum.

```
$ cd /klee/dir
$ LLVM_VERSION=6.0 SANITIZER_BUILD= BASE="/libc++/path/" \
    ./scripts/build/build.sh libcxx
$ mkdir build
$ cd build
$ cmake \
    -DENABLE_SOLVER_STP=ON \
    -DENABLE_SOLVER_Z3=ON  \
    -DENABLE_POSIX_RUNTIME=ON \
    -DENABLE_KLEE_UCLIBC=ON \
    -DKLEE_UCLIBC_PATH="/klee-uclibc/path/" \
    -DUSE_CMAKE_FIND_PACKAGE_LLVM=OFF  \
    -DLLVM_CONFIG_BINARY="/bin/llvm-config" \
    -DLLVMCC="/bin/clang" \
    -DLLVMCXX="/bin/clang++" \
    -DENABLE_DOCS=ON \
    -DENABLE_DOXYGEN=ON \
    -DENABLE_KLEE_LIBCXX=ON \
    -DLIT_ARGS="-v" \
    -DLIT_TOOL="/usr/bin/"   \
    -DENABLE_UNIT_TESTS=OFF \
    -DENABLE_SYSTEM_TESTS=ON \
    -DCMAKE_BUILD_TYPE="Debug" \
    ../

$ make
$ sudo make install
```

11 Ograničena provera modela (alat CBMC)

Provera modela je tehnika verifikacije softvera kojom se određuju sve moguće putanje kroz program i čuvaju u vidu konačnog automata. Zatim se prati svaka putanja i vrše se provere. Većina alata otkriva osnovne greške: deljenje nulom, mrtve petlje, trku za podacima itd.

Kod provere modela, program koji treba da se verifikuje je modelovan u vidu konačnog automata i specifikacija je zadata temporalnom logikom. Glavna ideja ograničene provere modela (engl. *bounded model checking*) jeste da pronađe kontraprimer u izvršavanjima čija je dužina ograničena celim brojem `k`. Ako greška nije prijavljena, možemo povećavati `k` sve dok greška ne bude nađena, problem više ne može da se prati ili se dostigne neko drugo ranije definisano gornje ograničenje modela. Problem provere modela se efikasno može svesti na problem zadovoljivosti iskazne logike i kao takav može biti rešen SAT rešavačem. Moderni SAT rešavači rešavaju probleme i sa više stotina hiljada promenljivih.

Ograničena provera modela ima dve karakteristike:

- korisnik obezbeđuje ograničenje broja ciklusa koje treba istražiti, samim tim tehnika je nepotpuna ako je ograničenje nedovoljno visoko
- koristi SAT tehnike.

Proverom modela automatski se pronalaze sve putanje koje se mogu izvršiti i čuvaju u internom modelu. Zatim se izvršava svaka putanja internog modela i proverava pojavljuju li se osnovne greške i da li su ispunjena svojstva modela. Svojstva modela se mogu zadati temporalnom logikom. Neki od operatora temporalne logike, za iskaze `p` i `q`:

- `Fp` - `p` važi neko vreme u budućnosti
- `Gp` - `p` važi uvek u budućnosti
- `Xp` - `p` važi sledeći put
- `p` $\cup$ `q` - `p` važi dok `q` važi

Primer upotrebe operatora bi mogao da bude: Automobil ne kreće dok je menjač u poziciji *Parking*.

```
(~start) U gear_park
```

11.1 Primer sa mikrotalasnom rernom

Uzmimo primer programa koji simulira rad mikrotalasne rerne. Svojstva koja moraju biti ispunjena:

- ako su vrata zatvorena, mogu biti otvorena.
- ako su vrata otvorena, mogu buti zatvorena.
- ako je grejač isključen, može biti uključen, ali samo ako su vrata zatvorena.
- ako je grejač uključen, može biti isključen.

Ove zahteve možemo predstaviti programom:

```java
public class Microwave {
    boolean doorOpen = false;
    boolean heatOn = false;

    public Microwave() {}

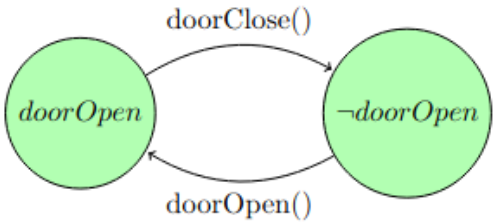
    public void openDoor() {
        if (!doorOpen) doorOpen = true;
    }

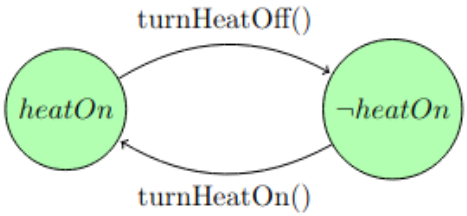
    public void closeDoor() {
        if (doorOpen) doorOpen = false;
    }

    public void turnHeatOn() {
        if (!heatOn && !orOpen) heatOn = true;
    }

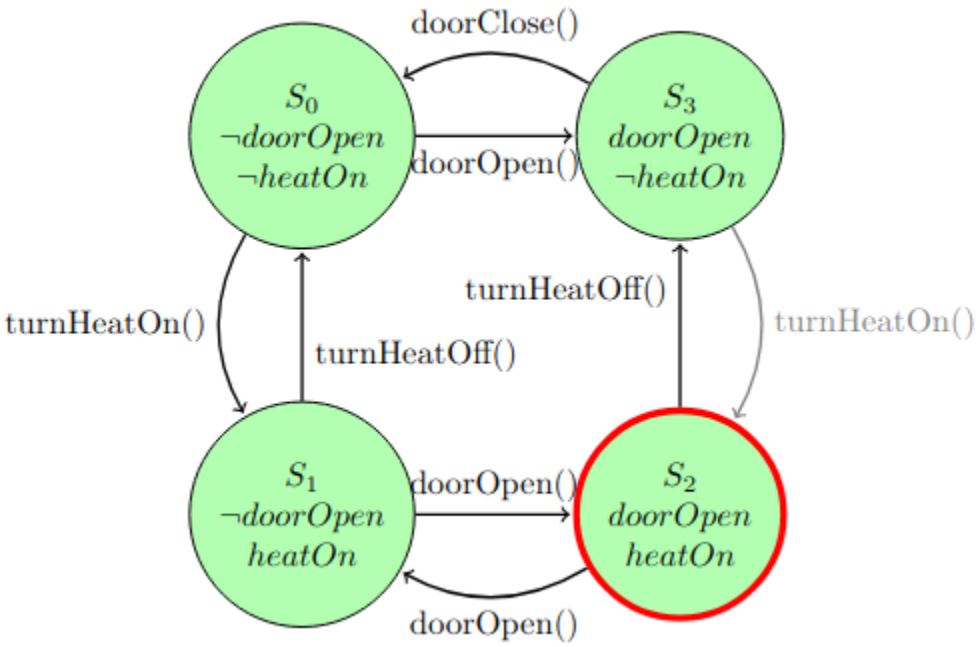
    public void turnHeatOff() {
        if (heatOn) heatOn = false;
    }
}
```

Program dozvoljava otvaranje i zatvaranje vrata kao i paljenje i gašenje grejača. Kao zaštitno svojstvo, postoji provera da li su vrata zatvorena pre nego što se grejač uključi, inače ga neće uključiti. Kreirajmo konačni automat za ove funkcionalnosti:





Dijagram na slici prikazuje kako se stanje program menja, tj. vrednosti promenljivih `doorOpen` i `heatOn`. Sastavimo kompletan dijagram:



Na slici vidimo četiri stanja sa različitim vrednostima dve logičke promenljive u obliku krugova. Sistem može menjati stanje korišćenjem nekog od metoda klase. Imamo opasno stanje `S2` u kom su vrata otvorena i grejač uključen. U ovo stanje nije moguće doći pozivom metode `turnHeatOn()` iz stanja S3 jer je metoda tako napisana da ne uključuje grejač ako su vrata otvorena. Ipak, sistem može doći u sporno stanje `S2`, tako što prvo dođe u stanje `S1` i zatim pozove metod `doorOpen()` dok je grejač već uključen. Ta putanja možda nije uočljiva iz koda ili sa prvog automata. Međutim, postaje jasno vidljiva kada se napravi dijagram svih stanja.

Kada se kreira model programa i lista svojstava, moguće je ispratiti svaku moguću putanju i proveriti ispravnost svih svojstava.

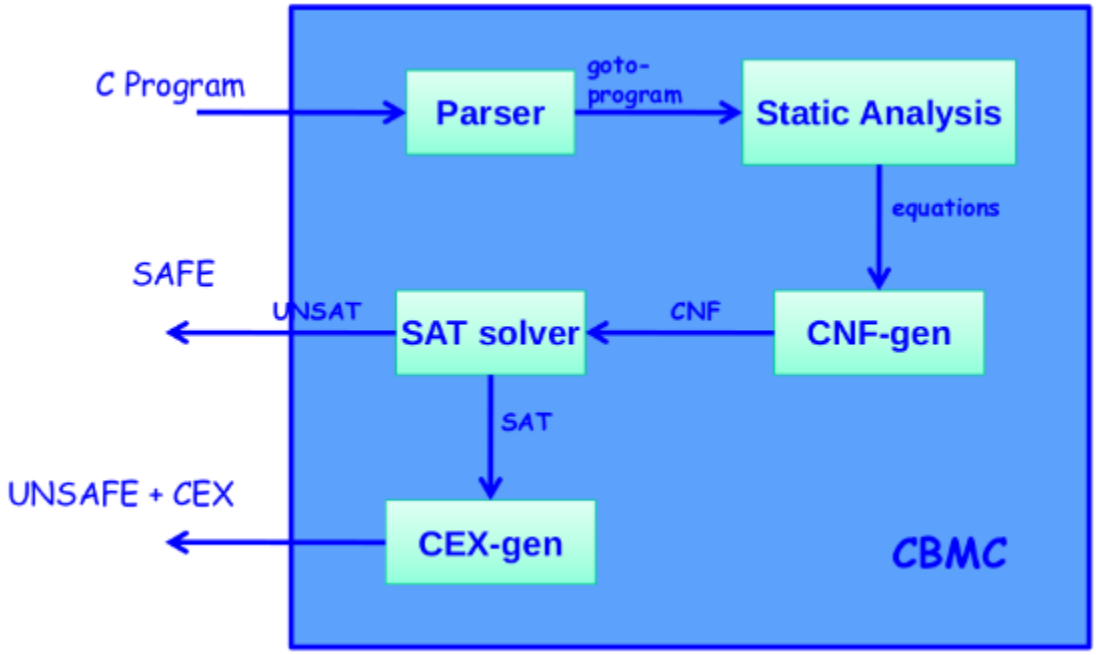
U slučaju da promenljive programa mogu uzeti više vrednosti, bilo bi mnogo više stanja u modelu. Tada se koristi mnogo više memorije, pa možda i sva memorija na računaru. Ova pojava se zove eksplozija prostora stanja i to je i dalje aktivan problem za proveru modela.

Neki od alata za proveru modela shodno programskom jeziku:

- Java: NASA's PathFinder, BANDERA, CBMC, DBRover
- C/C++: BLAST, CBMC, PCAchecker, DBRover, DSVerifier, ESBMC, SATABS

11.2 CBMC

Alat CBMC, kao i kompajleri, traži `.c` datoteku kao argument komandne linije. Prevodi program i objedinjuje ga sa definicijama drugih funkcija iz više `.c` datoteka, poput linkera. Ipak, umesto da proizvede izvršivu verziju programa, CBMC sprovodi simboličko izvršavanje programa.



11.2.1 Primer sa argumentima komandne linije

Kao primer, razmotrimo program `01_file1.c`:

```
int puts(const char *s) { }

int main(int argc, char **argv) {
    puts(argv[2]);
}
```

Očigledno je da program ima grešku, jer niz `argv` može imati i manje od 3 elementa i da se sa `argv[2]` izlazi izvan okvira niza. Pokrenimo CBMC sledećom komandom:

```
$ cbmc 01_file1.c --show-properties --bounds-check --pointer-check
```

Opcije `--bounds-check` i `--pointer-check` navode CBMC da traži greške u vezi pokazivača ili granica nizova. Zbog opcije `--show-properties` CBMC će štampati listu svih svojstava koje proverava. Prikazuje, između ostalog, i svojstvo `"object bounds in argv"` zajedno sa lokacijom pogrešnog pristupa elementu niza. CBMC uglavnom određuje svojstva koja treba da proveri. To se postiže preliminarnom statičkom analizom, koju vrši generator tvrđenja (engl. *assertion generator*) *goto-instrument* da označi deo koda koji potencijalno sadrži grešku.

Generator tvrđenja generiše tvrđenja za verifikaciju sledećih svojstava. Komande su izdvojene da bi se opcije mogle direktno kopirati u terminal:

- **Prekoračenje bafera** - Za svaki pristup elementu niza, proverava se da li je narušena donja ili gornja granica.

  ```
  --bounds-check
  ```

- **Upotreba pokazivača** - Traži dereferenciranje `NULL` ili neispravnih pokazivača.

```
--pointer-check
```

- **Deljenje nulom** - Proverava da li u programu postoji deljenje nulom.

```
--div-by-zero-check
```

- **Aritmetička potkoračenja i prekoračenja** - Proverava da li je došlo do prekoračenja ili potkoračenja prilikom aritmetičkih operacija nad označenim, neoznačenim ili realnim brojevima.

```
--signed-overflow-check
--unsigned-overflow-check
--float-overflow-check
```

- **Nije broj** - Proverava da li račun sa realnim brojevima može da rezultuje `NaN` vrednošću.

```
--nan-check
```

- **Neinicijalizovana promenljiva** - Proverava da li program koristi lokalnu promenljivu pre nego što je inicijalizovana.
- **Trka za podacima** - Proverava da li dve niti programa pristupaju istoj deljenoj promenljivoj u isto vreme.

Daljom analizom je potrebno utvrditi da li su to zaista greške. CBMC sprovodi analizu simboličkom simulacijom, koja odgovara prevođenju programa u formulu. Ta formula se kombinuje sa svojstvom. Generisanu formulu možemo videti navođenjem opcije `--show-vcc` (engl. *Verification Condition*).

```
$ cbmc 01_file1.c --show-vcc --pointer-check
```

Prilikom analize generiše se KNF formula koja opisuje program, zajedno sa svojstvima. To je izraz koji zadovoljava uslove problema i negaciju tvrđenja. Formula se prosleđuje SAT rešavaču koji odlučuje da li je formula zadovoljiva. Ako jeste, CBMC pronalazi kontraprimer. Ako nije, tvrđenje važi za izvršenje sa ograničenjem `k`.

Formula, odnosno uslovi verifikacije, treba tek da budu provereni da bi se proverilo da li važi navedeno svojstvo. Pokrećemo sa:

```
$ cbmc 01_file1.c --pointer-check
```

Korišćenjem SAT rešavača, CBMC detektuje da ne važi uslov za granice niza `argv` i štampa sledeću poruku:

```
Violated property:
  file 01_file1.c line 7 function main
  dereference failure: object bounds in argv[(signed long int)2]
```

Da bismo razumeli problem i zašto svojstvo ne važi, CBMC može da nam generiše trag izvršavanja kontraprimera korišćenjem opcije `--trace`:

```
$ cbmc 01_file1.c --pointer-check --trace
```

Na ekranu nam se prikazuje kontraprimer, u vidu toka izvršavanja programa počevši od `main` funkcije do stanja koje narušava svojstvo. U ovom primeru svojstvo je narušeno u neispravnom indeksiranju elementa niza. Takođe, prikazuje nam i vrednosti koje ulazne promenljive treba da imaju da bi se izazvala greška. Prikazuje nam i uslov čijom proverom se došlo do zaključka.

```
CBMC version 5.10 (cbmc-5.10) 64-bit x86_64 linux
Parsing 01_file1.c
Converting
Type-checking 01_file1
Generating GOTO Program
Adding CPROVER library (x86_64)
Removal of function pointers and virtual functions
Generic Property Instrumentation
Running with 8 object bits, 56 offset bits (default)
Starting Bounded Model Checking
size of program expression: 43 steps
simple slicing removed 10 assignments
Generated 7 VCC(s), 1 remaining after simplification
Passing problem to propositional reduction
converting SSA
Running propositional reduction
Post-processing
Solving with MiniSAT 2.2.1 with simplifier
462 variables, 1031 clauses
SAT checker: instance is SATISFIABLE
Solving with MiniSAT 2.2.1 with simplifier
462 variables, 0 clauses
SAT checker inconsistent: instance is UNSATISFIABLE
Runtime decision procedure: 0.00274368s

** Results:
[main.pointer_dereference.1] dereference failure: pointer NULL in argv[(signed long int)2]: SUCCESS
[main.pointer_dereference.2] dereference failure: pointer invalid in argv[(signed long int)2]:
↪  SUCCESS
[main.pointer_dereference.3] dereference failure: deallocated dynamic object in argv[(signed long
↪  int)2]: SUCCESS
[main.pointer_dereference.4] dereference failure: dead object in argv[(signed long int)2]: SUCCESS
[main.pointer_dereference.5] dereference failure: pointer outside dynamic object bounds in
↪  argv[(signed long int)2]: SUCCESS
[main.pointer_dereference.6] dereference failure: pointer outside object bounds in argv[(signed long
↪  int)2]: FAILURE
[main.pointer_dereference.7] dereference failure: invalid integer address in argv[(signed long
↪  int)2]: SUCCESS

Trace for main.pointer_dereference.6:

State 17  thread 0
---------------------------------------------------
```

```
    INPUT argc: 1 (00000000 00000000 00000000 00000001)

State 18  thread 0
------------------------------------------------------
    argv'[1]=((char *)NULL) (00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000)

State 21 file 01_file1.c line 3 thread 0
------------------------------------------------------
    argc=1 (00000000 00000000 00000000 00000001)

State 22 file 01_file1.c line 3 thread 0
------------------------------------------------------
    argv=argv' (00000010 00000000 00000000 00000000 00000000 00000000 00000000 00000000)

Violated property:
    file 01_file1.c line 5 function main
    dereference failure: pointer outside object bounds in argv[(signed long int)2]
    16l + POINTER_OFFSET(argv) >= 0l && OBJECT_SIZE(argv) >= 24ul + (unsigned long
    ↪  int)POINTER_OFFSET(argv) || DYNAMIC_OBJECT(argv)


** 1 of 7 failed (2 iterations)
VERIFICATION FAILED
```

U ovom primeru, ako je `argc == 1` dogodiće se prekoračenje granica niza `argv`. Ako dodamo proveru uslova da je `argc >= 3`, greška je otklonjena i CBMC-ov izveštaj dokazuje da su sva svojstva zadovoljena.

11.2.2 Primer odvojenog testiranja funkcija

CBMC je pisan sa namerom da se koristi i za ugrađen softver, a takvi programi obično imaju drugačije ulazne tačke od `main` funkcije. Pored toga CBMC je koristan za verifikaciju modula programa, poput ovog primera.

```c
int array[10];

int sum() {
    unsigned i, sum = 0;
    for (i = 0; i < 10; i++)
        sum += array[i];
}
```

Potrebno je CBMC-u da naglasimo koja je ulazna tačka, a to je ovde funkcija `sum`. Pozivamo CBMC sa sledećim opcijama:

```
$ cbmc 02_file2.c --function sum --bounds-check
```

Često je potrebno napraviti odgovarajuće propratne alate (*eng. harness*) za funkciju da bi se podesilo odgovarajuće okruženje.

U ovom primeru možemo da primetimo i da CBMC odmotava petlju iz programa. Pošto CBMC formulu šalje SAT rešavaču, formula mora biti konačna. Zbog toga CBMC izvršava ograničenu proveru modela. Stoga se petljama broj iteracija mora ograničiti odozgo da bi se garantovalo pronalaženje grešaka. CBMC može proveriti da li je izvršeno dovoljno odmotavanja.

Svaka druga petlja (`for`, `do-while`) biće prezapisana u `while` petlju.

Treba znati da su globalne promenljive inicijalizovane pre izvršavanja date funkcije, što može dovesti do drugačijih stanja od onih prilikom realnog izvršavanja funkcije. CBMC ne pokušava da odredi preduslove za argumente funkcije.

11.2.3 Primer neuspelog odmotavanja petlji

```
int binsearch(int x) {
    int a[16];
    signed low = 0, high = 16;

    while (low < high) {
        signed middle = low + ((high - low) >> 1);

        if (a[middle] < x)
            high = middle;
        else if (a[middle] > x)
            low = middle + 1;
        else // a[middle] == x
            return middle;
    }

    return -1;
}
```

Pokrenimo CBMC:

```
$ cbmc 03_binsearch.c --function binsearch --bounds-check
```

Primetićemo da se odmotavanje petlje ne zaustavlja samo od sebe jer CBMC ne uspeva da otkrije gornje ograničenje za broj iteracija ove petlje. U tom slučaju gornje ograničenje za odmotavanje petlje mora da se zada preko opcije `--unwind n`, gde `n` predstavlja broj provera uslova ostanka u petlji:

```
$ cbmc 03_binsearch.c --function binsearch --unwind 6  --bounds-check
```

CBMC potvrđuje da su svi pristupi elementima niza u okviru granica, iako to u ovom slučaju zavisi od šiftovanja udesno. Ako se upotrebi opcija `--unwinding-assertions`, proveravaće da li je petlja dovoljno odmotana. Za kraća odmotavanja od 6 bili bi ispisani koraci izvršavanja koji zahtevaju više odmotavanja petlje. Ako je pozvan i sa opcijom `--trace`, generisaće nam i kontraprimer.

```
$ cbmc 03_binsearch.c --function binsearch --unwind 5 --unwinding-assertions  --bounds-check --trace
```

```
[binsearch.unwind.0] unwinding assertion loop 0: FAILURE

** 1 of 5 failed (2 iterations)
VERIFICATION FAILED
```

11.2.4 Primer sa neograničenim petljama

CBMC se može primeniti i na program sa neograničenim petljama. U takvim situacijama, CBMC služi samo za traženje grešaka i ne pokušava da pronađe sve. Program `04_lock-example.c` je primer programa sa svojstvom koje je korisnik definisao, navodeći `assert` naredbe u kodu.

`while` petlja nema konkretnu gornju granicu broja iteracija. Stoga, mora se naglasiti koliko odmotavanja petlje treba CBMC da izvrši. Postoje dva načina da se tako nešto postigne:

- opcijom `--unwind` da se ograniči koliko puta su petlje odmotane.
- opcijom `--depth` da se ograniči broj koraka programa koje treba pratiti.

Ukoliko navedemo i opciju `--unwinding-assertions`, CBMC će proveriti da li je argument uz opciju `--unwind` dovoljno veliki da pokrije sve putanje kroz program. Ukoliko je premali, detektovaće nam da nije izvršeno dovoljno odmotavanja.

Vratimo se na primer i pokrenimo CBMC:

```
$ cbmc 04_lock-example.c  --unwind 1 --unwinding-assertions
```

Ne prijavljuje nam grešku, osim nedovoljnog odmotavanja. Ukoliko se izostavi poslednja opcija, neće ni to prijaviti. Već sa ograničenjem `2`, CBMC detektuje grešku. Upotrebom opcije `--trace` generisaće se kontraprimer koji narušava važenje tvrđenja (*eng. assertion*).

```
$ cbmc 04_lock-example.c  --unwind 1 --trace
```

Bez prijave o nedovoljnom odmotavanju petlje ili korišćenjem opcije `--depth`, CBMC ne dokazuje korektnost programa, ali je i dalje koristan za nalaženje grešaka.

Pokrećemo CBMC ponovo sa ciljem da proverimo ima li program grešaka.

```
$ cbmc 04_lock-example.c  --unwind 1
```

Jedinu grešku koju dobijamo je da smatra da petlja nije odmotana do kraja. Zato pokrećemo ponovo sa većim brojem iteracija.

```
$ cbmc 04_lock-example.c  --unwind 2
```

Dobijamo grešku da korisnikovo tvrđenje ne važi:

```
** Results:
[lock.assertion.1] assertion !(LOCK != FALSE): SUCCESS
[unlock.assertion.1] assertion LOCK: FAILURE

** 1 of 2 failed (2 iterations)
VERIFICATION FAILED
```

Generišimo i kontraprimer pozivom :

```
$ cbmc 04_lock-example.c  --unwind 2 --trace
```

```
Counterexample:

State 19 file lock-example.c line 19 function main thread 0
------------------------------------------------------
    got_lock=0 (00000000000000000000000000000000)

State 20 file lock-example.c line 19 function main thread 0
------------------------------------------------------
    got_lock=0 (00000000000000000000000000000000)

State 21 file lock-example.c line 20 function main thread 0
------------------------------------------------------
    times=1713006976 (01100110000110100110100110000000)

State 35 file lock-example.c line 32 function main thread 0
------------------------------------------------------
    got_lock=4294967295 (11111111111111111111111111111111)

State 36 file lock-example.c line 32 function main thread 0
------------------------------------------------------
    times=1713006975 (01100110000110100110100101111111)
```

Shvatamo da se u toj jednoj iteraciji naše petlje dogodilo da je izvršena linija `32` i smanjena vrednost od `get_lock`, a s obzirom da je tip `unsigned int`, dobila je vrednost najvećeg neoznačenog celog broja. To ne sme da se dogodi, jer je smisao te promenljive da se poveća kada se ostvari zaključavanje i da se smanji na `0` kada se otključa. Izmenimo kod i primetimo da više neće prijavljivati problem.

11.2.5 Odmotavanje zasnovano na broju iteracija

Pogledajmo primer `05_loop1.c`

```
int main(int argc, char **argv) {
    int i = 1;
    while (i < 5) {
        i = i + 1;
    }
    return 0;
}
```

Ovo bi bio primer programa u kome je dovoljno da imamo 5 odmotavanja petlje. Transformisan program bez petlje izgleda ovako: `05_loop1_unwinded.c`

```c
#include <assert.h>

int main(int argc, char **argv) {
    int i = 1;
    if (i < 5) {
        i = i + 1;
        if (i < 5) {
            i = i + 1;
            if (i < 5) {
                i = i + 1;
                if (i < 5) {
                    i = i + 1;
                    assert(!(i < 5));
                }
            }
        }
    }
}
```

Program odgovara toku izvršavanja petlje sa 4 iteracije, a uslov ostanka u petlji se 5 puta proverava. Odgovara pozivu:

```
$ cbmc 05_loop1.c  --unwind 5 --unwinding-assertions
```

Za svaki manji broj odmotavanja, manji od 5, prijavio bi nedovoljno odmotavanje. Transformisan kod za 3 odmotavanja bi izgledao ovako: `05_loop1_unwinded_insufficently.c`

```c
#include <assert.h>

int main(int argc, char **argv) {
    int i = 1;
    if (i < 5) {
        i = i + 1;
        if (i < 5) {
            i = i + 1;
            assert(!(i < 5));
        }
    }
}
```

Odgovara pozivu:

```
$ cbmc 05_loop1.c  --unwind 2 --unwinding-assertions
```

Ukoliko ograničimo odmotavanje petlje, i pri tom ne navedemo opciju `--unwinding-assertions`, posle određenog broja iteracija pretpostaviće se da više ne važi uslov ostanka u petlji i nastaviće se dalje. Dakle, sve preostale putanje kroz petlju biće ignorisane.

```c
#include <assert.h>

int main(int argc, char **argv) {
    int i = 1;
    if (i < 5) {
        i = i + 1;
        if (i < 5) {
            i = i + 1;
            assume(!(i < 5));
        }
    }
}
```

U mnogim slučajevima CBMC može da automatski odredi gornju granicu iteracija petlje, pa i u slučajevima kada broj odmotavanja petlje nije konstanta. Primer `06_loop2.c` to prikazuje:

```c
#include <assert.h>

_Bool f(int i) { return (i + 1) % 23 == 0; }

int main() {
    int i;
    for (i = 0; i < 100; i++) {
        if (f(i))
            break;
    }

    assert(i < 100);
}
```

`while` petlja ima očigledno gornje ograničenje za broj iteracija, ali iz petlje se može i ranije izaći u zavisnosti od povratne vrednosti funkcije `f()`. CBMC može automatski da odmota petlju do završetka. To automatsko detektovanje granice odmotavanja petlji može biti neuspešno ukoliko broj iteracija petlje zavisi od podataka. Takođe, broj iteracija može biti prevelik ili neograničen. U tim slučajevima je bolje koristiti opciju komandne linije `--unwind B`, gde `B` označava broj provera uslova ostanka u petlji, odnosno skokova na početak. Tada se telo petlje izvršava `B-1` put.

11.2.6 Zadavanje ograničenja odmotavanja za pojedinačne petlje

Upotrebom opcije `--unwind` ograničenje za odmotavanje se namešta globalno, za sve petlje u programu. Pogledajmo primer `07_loop3.c`:

```c
#include <assert.h>

_Bool f(int i) { return (i + 1) % 23 == 0; }

int main() {
    int i;
    for (i = 0; i < 100; i++) {
```

```
        if (f(i))
            break;
    }

    for (i = 0; i < 100; i++) {
        if (f(i * 2))
            break;
    }

    assert(i < 100);
}
```

Da bismo zadali ograničenje posebno za svaku petlju, prvo pokrenimo program sa opcijom `--show-loops` da bismo dobili listu svih petlji u programu i njihove identifikatore.

```
$ cbmc 07_loop3.c --show-loops
```

```
CBMC version 5.10 (cbmc-5.10) 64-bit x86_64 linux
Parsing 07_loop3.c
Converting
Type-checking 07_loop3
Generating GOTO Program
Adding CPROVER library (x86_64)
Removal of function pointers and virtual functions
Generic Property Instrumentation
Loop main.0:
    file 07_loop3.c line 9 function main

Loop main.1:
    file 07_loop3.c line 13 function main
```

Za program sa 2 petlje zadajemo ograničenja odmotavanja za petlje redom, 10, odnosno 20, opcijom `--unwindset`.

```
$ cbmc 07_loop3.c --unwindset main.0:10,main.1:20 --unwinding-assertions
```

```
CBMC version 5.10 (cbmc-5.10) 64-bit x86_64 linux
Parsing 07_loop3.c
Converting
Type-checking 07_loop3
Generating GOTO Program
Adding CPROVER library (x86_64)
Removal of function pointers and virtual functions
Generic Property Instrumentation
Running with 8 object bits, 56 offset bits (default)
Starting Bounded Model Checking
Unwinding loop main.0 iteration 1 file 07_loop3.c line 9 function main thread 0
Unwinding loop main.0 iteration 2 file 07_loop3.c line 9 function main thread 0
Unwinding loop main.0 iteration 3 file 07_loop3.c line 9 function main thread 0
Unwinding loop main.0 iteration 4 file 07_loop3.c line 9 function main thread 0
```

```
Unwinding loop main.0 iteration 5 file 07_loop3.c line 9 function main thread 0
Unwinding loop main.0 iteration 6 file 07_loop3.c line 9 function main thread 0
Unwinding loop main.0 iteration 7 file 07_loop3.c line 9 function main thread 0
Unwinding loop main.0 iteration 8 file 07_loop3.c line 9 function main thread 0
Unwinding loop main.0 iteration 9 file 07_loop3.c line 9 function main thread 0
Not unwinding loop main.0 iteration 10 file 07_loop3.c line 9 function main thread 0
size of program expression: 140 steps
simple slicing removed 2 assignments
Generated 1 VCC(s), 1 remaining after simplification
Passing problem to propositional reduction
converting SSA
Running propositional reduction
Post-processing
Solving with MiniSAT 2.2.1 with simplifier
120 variables, 11 clauses
SAT checker: instance is SATISFIABLE
Solving with MiniSAT 2.2.1 with simplifier
120 variables, 0 clauses
SAT checker inconsistent: instance is UNSATISFIABLE
Runtime decision procedure: 0.00124571s

** Results:
[main.assertion.1] assertion i<100: SUCCESS
[main.unwind.0] unwinding assertion loop 0: FAILURE

** 1 of 2 failed (2 iterations)
VERIFICATION FAILED
```

11.2.7 Odsecanje petlji

U nekim slučajevima je poželjno da odsečemo veoma duboke petlje u cilju dostizanja koda koji dolazi posle petlje. Pogledajmo primer `08_loop4.c`:

```c
#include <assert.h>
#include <stdio.h>

int main() {
    int i;
    for (i = 0; i < 10000; i++) {
        printf("i = %d\n", i);
    }

    assert(0);
}
```

U ovom primeru za malo ograničenje odmotavanja petlje provera posle petlje neće biti izvršena. Ako se smatra da neka petlja nije bitna za kod koji sledi posle nje, možemo koristiti opciju `--partial-loops`. Ova opcija dozvoljava da se delimično izvrše putanje kroz petlju i time i za mala ograničenja odmotavanja pronalazi kontraprimer za proveru koja se izvršava posle petlje.

```
$ cbmc 08_loop4.c  --unwind 1 --partial-loops
```

Problem sa delimičnim izvršavanjem petlje je što putanja koja na kraju bude izvršena ne mora biti jedna od validnih putanja kroz petlju u originalnom programu. Opcijom `--partial-loops` stavlja se `break` naredba na mestu gde stoji `assert`, odnosno `assume`, prilikom ograničenog odmotavanja petlje sa opcijom `--unwinding-assertions`, odnosno bez nje.

Pozivom:

```
$ cbmc 08_loop4.c  --unwind 10
```

se ne dolazi do provere `assert` i ne detektuje se greška. Potrebno je da se petlja razmota 10001 put, da bi se stiglo do ostatka programa.

11.2.8 Odmotavanje zasnovano na dubini (Depth-based Unwinding)

Ograničenje za broj odmotavanja petlje nekada nije dovoljno. U situacijama kada je teško kontrolisati veličinu generisane formule samo opcijom `--unwind`, opcijom `--depth D` zadajemo da je odmotavanje petlje ograničeno brojem instrukcija koje su izvršene na toj putanji, bez obzira na redni broj iteracije petlje. CBMC koristi broj instrukcija u grafu kontrole toka (*eng. control flow graph*), a ne broj instrukcija u izvornom kodu.

11.2.9 Dekompozicija koraka analize

Instrumentacija:

```
$ cbmc 09_abs.c --function abs --signed-overflow-check --show-goto-functions
```

Svojstva:

```
$ cbmc 09_abs.c --function abs --signed-overflow-check --show-properties
```

Static Single Assign (SSA):

```
$ cbmc 09_abs.c --function abs --signed-overflow-check --program-only
```

Verification condition (VCC):

```
$ cbmc 09_abs.c --function abs --signed-overflow-check --show-vcc
```

Verify:

```
$ cbmc 09_abs.c --function abs --signed-overflow-check
```

11.2.10 Uprošćavanje verifikacione formule

```
$ cbmc 10_escapefunction.c --function escape --pointer-check --bounds-check \
    --slice-formula --unwind 4
```

Opcija `--slice-formula` uklanja nepotrebne dodele, tj. dodele koje nisu u vezi sa svojstvima koje proveravamo. Na taj način uprošćava formulu koja se šalje SAT solveru i ubrzava proveru modela. Ispravimo prijavljene greške i proverimo ponovo. Nije moguće rešiti problem što `s` može biti bilo kakav pokazivač.

U programu `11_escapefunctionharness.c` imamo `main` funkciju koja će adekvatno pripremiti sve argumente potrebne za funkciju `escape`.

```
$ cbmc 11_escapefunctionharness.c --pointer-check --bounds-check \
    --slice-formula --unwind 4
```

Probati sa različitim ograničenjem za odmotavanje. Uočiti greške i ispraviti ih. Zatim povećati ograničenje radi pretrage drugih grešaka (`>20`).

11.2.11 Binarna reprezentacija grafa kontrole toka

`goto-cc` je CBMC-ova zamena za kompajler i linker, (`goto-cl` verzija za MS VisualStudio). Komandom

```
$ goto-cc 12_simple.c -o simple.goto
```

dobijamo binarnu reprezentaciju grafa kontrole toka.

Da bismo generisali instrumentovan C kod, izvršavamo komandu:

```
$ goto-instrument --dump-c simple.goto out.c
```

`goto-instrument` čita binarnu reprezentaciju grafa kontrole toka i primenjuje odgovarajuću statičku analizu. Kod je dopunjen proverama na mestu gde je uočena potencijalna greška. Zatim se čuva dopunjena binarna reprezentacija. Komanda:

```
$ goto-instrument simple.goto --show-properties
```

daje sva uočena svojstva prilikom instrumentacije. Instrumentovan kod analiziramo CBMC-om, izvršavanjem:

```
$ cbmc out.c
```

ili:

```
$ cbmc simple.goto
```

11.2.12 Modelovanje problema

CBMC obezbeđuje primitive koje nisu pisane na jeziku C, a koje modeluju nedeterminističke funkcije, tj. funkcije koje mogu uzastopno vratiti mnogo različitih vrednosti. Upotreba:

```
int nondeter_int();
```

deklariše nedeterminističku funkciju koja vraća podatak tipa `int`. Ovakve funkcije su dostupne za razne tipove. Nedeterminističke funkcije su korisne prilikom modelovanja ulaznih podataka, nepoznatog okruženja, bibliotečkih funkcija i praktično bilo čega što se može različito ponašati prilikom izvršavanja u različitim okruženjima. Modelovanje nedeterminističkim funkcijama je praktično, ali može biti skupo. Sve moguće putanje izvršavanja moraju biti razmotane i istražene, čak i one koje nisu moguće u praksi.

CBMC omogućava eksplicitno zadavanje korisnikovih preduslova i postuslova i time omogućava da se analiza vrši po funkciji, čime se značajno smanjuje opterećenje analizatora. Da bismo proverili da li određeni poziv funkcije `foo` važi svojstvo `P`, delimo odlučivanje na dva dela:

- Pretpostavljajući da određeni predefinisani preduslov `C` važi u početnom stanju funkcije `foo`, i pretpostavljajući da svojstvo `P` važi na ulazu, da li svojstvo `P` važi i po izlasku iz funkcije?
- Kad god je funkcija `foo` pozvana, da li garantovano važi preduslov `C` na ulazu?

Kada oba uslova važe, analiza će garantovano biti korektna. Na ovaj način se proveravaju samo preduslovi i postuslovi, ali ne i implementacija. Ovo olakšava testiranje, ali otvara prostor za sve potencijalne greške programera. Ponekad ima smisla kombinovati ovaj pristup sa klasičnom analizom. Na primer, u nekom programu se nedeterminističke funkcije koriste za modelovanje sistemskih poziva, dok se pretpostavke i garancije važenja uslova koriste na bibliotečkim funkcijama da bi osigurale njihovo izvršavanje i u isto vreme svele analizu kôda na manje, lokalne funkcije.

Tehnika postavljanja uslova i garancija važenja se postiže specifikacijom:

```
__CPROVER_assert(uslov,poruka)
```

Ovo je osnovni oblik korisnički definisanih specifikacija koje CBMC podržava. Kada se pokrene provera modela, proverava se važenje uslova navedenog kao prvi argument na svim putanjama do definisane granice. Ako granica nije data u komandnoj liniji, CBMC će kao i uvek pokušati da pronađe gornju granicu dubine izvršavanja programa i verifikovati ga nakon odmotavanja. Drugi argument je niska za dijagnostikovanje, jer će biti prikazana u rezultatu analize ako CBMC pronađe kontraprimer za uslov.

Pretpostavke okruženja se zadaju specifikacijom:

```
__CPROVER_assume(uslov)
```

CBMC interpretira ovaj poziv malo drugačije od prethodnog. Bilo koja putanja koja ne zadovoljava uslov naveden kao argument biće odbačena iz analize. Čak i ako takva putanja negde kasnije sadrži neku grešku, ona neće biti prijavljena u rezultatu. Ovo je korisno u slučajevima

modelovanja okruženja, ukoliko verujemo da će argumenti prosleđeni funkciji uvek zadovoljavati navedene uslove. Ove dve specifikacije, zajedno sa nedeterminističkim funkcijama i programiranjem, omogućavaju modelovanje zanimljivih tvrđenja.

U primeru `05_assume.c`, ne postoji izračunavanje za koje istovremeno može biti ispunjen uslov iz `if ( x>0 )` i pretpostavka iz `__CPROVER_assume (x < 0)`. Zbog svojstva `__CPROVER_assume` sve grane stabla izvršavanja od te pretpostavke koje ne zadovoljavaju uslov biće odbačene. Iz tog razloga CBMC neće prijaviti grešku, šta god uslov bio.

U primeru `06_postcondition.c`, CBMC ne prijavljuje grešku iako za `N = -1` i `x = 0` ne bi važio uslov. To se događa jer `N` i `x` nisu inicijalizovane i CBMC pretpostavlja da su im vrednosti `0`. Da bismo uklonili tu pretpostavku moramo dodati nedeterminističke vrednosti za `N` i `x`.

11.2.13 Višenitni programi

U primeru `10_increment.c`, kontraprimer koji nam CBMC prijavljuje nam govori da je `x` dva puta postavljeno na 1. Problem je u pristupu kritičnoj sekciji od strane obe niti. Ograđivanjem kritične sekcije `mutex`-om rešavamo taj problem.

U primeru `11_incrementLoop.c` problem je što imamo petlju u kojoj se čita vrednost deljene promenljive i potom ažurira. CBMC ne uspeva sam da odredi gornju granicu odmotavanja petlji, pa je neophodno da je zadamo.

U primeru `12_mutex.c` postoje 3 niti, jedna glavna u kojoj se izvršava `main` funkcija, jedna koja je proizvođač (engl. *producer*) i jedna koja je potrošač (engl. *consumer*). Kako se program izvršava, instrukcije koje niti izvršavaju mogu se ukrstiti u bilo kom mogućem redosledu jer je raspoređivanje prepušteno operativnom sistemu. CBMC ovakve situacije modelira odmotavanjem kôda koji izvršava svaka nit u svim mogućim redosledima, tretirajući izbor rasporeda nedeterminističkim. Svaka nit ima svoj lokalni stek okvir, i on je dostupan samo odgovarajućoj niti. Globalne promenljive su dostupne svim nitima. Problem nastaje kada više niti pristupa deljenim podacima.

U ovom kodu `producer` i `consumer` pokušavaju da postignu međusobno isključenje preko 3 globalne promenljive: `try_producer`, `try_consumer` i `turn`. Kada bilo koja osim glavne niti želi da pristupi kritičnoj sekciji, ona signalizira svoju nameru postavljajući promenljivu `try_producer`, odnosno `try_consumer`, na 1. Potom daje priliku drugoj niti da uđe u kritičnu sekciju, postavljajući promenljivu `turn` na vrednost koja odgovara drugoj niti i čekajući u `while` petljama. Kada nit uđe u kritičnu sekciju, ona obavi svoj posao nad deljenim podacima i vraća vrednost promenljive `try_producer`, odnosno `try_consumer`, na 0.

Prvo ćemo ispraviti greške vezane za memoriju. Neki problemi sa memorijom mogu biti izazvani prilikom pristupanja deljenim podacima ako međusobno isključenje dve niti nije ispunjeno.

CBMC ne može naći gornju granicu odmotavanja petlji, naročito zato što imamo petlje u kojima niti čekaju na ulazak u kritičnu sekciju.

```
$ cbmc 12_mutex.c --unwind 7 --no-unwinding-assertions \
    --slice-formula --bounds-check --pointer-check
```

Pronalazi nam kontraprimer u kom se vidi da nit koja odgovara funkciji `consumer` ulazi prva u kritičnu sekciju i svojom akcijom smanjuje `num_waiting` na -1, a potom se poziva funkcija `process` da ispiše element na tom indeksu.

Ponavljamo prethodni poziv i iz podužeg kontraprimera zaključujemo da u poslednjoj liniji funkcije `next_data` ne vraćamo vrednost koju smo upisali u niz, već narednu. Popravljamo funkciju tako da vrati vrednost na indeksu `data_pos-1`.

Opcija komandne linije `--unwind 7` utiče na međusobno isključenje niti, jer utiče na petlju u kojoj se čeka na dozvolu za ulazak u kritičnu sekciju. Petlja se izvršava kao da smo dodali `break` nakon 6. iteracije. Zbog toga se mogu dogoditi nejasne greške, na primer da `num_waiting` bude smanjen na 0 u niti `consumer`, a potom vraćen na 2 u niti `producer`, što se ne bi smelo dogoditi.

Uveriti se da i `producer` i `consumer` nit završavaju i da su svi podaci iz `data_source` obrađeni od strane niti `consumer`. Biće potrebno modifikovati kod da bi se ovo ostvarilo. Savet je da se koriste CBMC-ove korisničke specifikacije.

12 Clang alati za analizu C/C++ programa

Clang nije samo prevodilac. U ovom poglavlju koristimo nekoliko alata iz Clang/LLVM ekosistema koji pokrivaju različite oblike analize C i C++ programa:

- **upozorenja prevodioca** proveravaju lokalne obrasce tokom prevođenja;
- **Clang statički analizator** simbolički prolazi kroz putanje programa i prijavljuje greške bez pokretanja programa;
- **`clang-tidy`** kombinuje provere stila, održivosti, modernizacije C++ koda i neke statičke provere;
- **sanitizeri** instrumentišu program pri prevođenju i prijavljuju greške tokom izvršavanja.

Ove tehnike se dopunjuju. Statički analizator i `clang-tidy` mogu pronaći greške pre pokretanja programa, dok sanitizeri proveravaju konkretno izvršavanje i često daju precizan stek poziva do mesta greške.

12.1 Kada koristiti koji Clang alat

Alat	Najbolji slučaj upotrebe	Šta proveriti u izveštaju
Upozorenja prevodioca	Brza provera lokalnih grešaka tokom svakog prevođenja.	Da li upozorenje ukazuje na stvarnu grešku ili na nejasan kod koji treba pojednostaviti.
Clang statički analizator	Putanje kroz funkcije, neinicijalizovane vrednosti, pogrešni API pozivi i memorijske greške bez pokretanja programa.	Putanju koju analizator navodi i pretpostavku zbog koje je greška moguća.
`clang-tidy`	Održavanje C++ koda, modernizacija, stil i izabrane statičke provere u projektu.	Da li provera odgovara pravilima projekta i da li automatska izmena menja semantiku.
Sanitizeri	Greške koje se vide tokom konkretnog izvršavanja: memorija, nedefinisano ponašanje, curenje memorije i trke za podacima.	Prvi relevantan okvir steka, vrstu greške i ulaz koji je grešku izazvao.

12.2 Clang statički analizator

Clang je kompilator otvorenog koda za C familiju jezika. Koristi LLVM optimizator i generator koda. Više informacija zajedno sa uputstvima je moguće pronaći u priručniku za Clang kompilator. Clang statički analizator je deo Clang projekta.

Clang statički analizator radi nad izvornim kodom i ne pokreće program. Analizator gradi model mogućih putanja kroz funkcije, prati vrednosti promenljivih i proverava da li postoji putanja koja vodi do greške. Zbog toga može da pronađe greške koje obična upozorenja prevodioca ne vide, na primer dereferenciranje NULL pokazivača posle uslovnog grananja, curenje resursa ili pogrešno korišćenje POSIX API-ja.

Analizator nije zamena za testove. On proverava model programa koji uspe da izgradi. Ako nema dovoljno informacija o ulazima, bibliotekama ili invarijantama programa, može propustiti grešku ili prijaviti lažno pozitivan nalaz. Zbog toga se nalazi uvek čitaju kroz prikaz putanje koju analizator prijavljuje.

Clang statički analizator koristi razne implementacije proveravača (engl. *checkers*) prilikom analize. Proveravači su kategorisani u familije - *podrazumevani* i *eksperimentalni (alpha)*. Podrazumevani proveravači izvršavaju bezbednosne provere, prate korišćenje API funkcija, traže mrtav kod i ostale logičke greške. Eksperimentalni (alpha) proveravači nisu podrazumevano uključeni pošto često daju lažno pozitivne rezultate. Uputstvo za implementaciju proveravača se može naći na sledećem linku.

Neke kategorije podrazumevanih proveravača:

- Core Checkers vrše provere opšte namene kao što su deljenje nulom, dereferenciranje NULL pokazivača, korišćenje neinicijalizovanih vrednosti itd.
- C++ Checkers vrše provere specifične za C++ programski jezik
- Dead Code Checkers traže mrtav kod
- Nullability Checkers proveravaju dodele ili prosleđivanje NULL pokazivača u kontekstu gde se ne očekuje NULL pokazivač
- Optin Checkers prate neinicijalizovane objekte i virtualne pozive (C++)
- OS X Checkers vrše provere specifične za Objective-C i proveravaju upotrebu Apple SDK-a (OS X i iOS)
- Security Checkers proveravaju nebezbedno korišćenje API-ja i vrše provere na osnovu CERT standarda bezbednog kodiranja
- Unix Checkers proveravaju upotrebu Unix i POSIX programerskih interfejsa

Spisak proveravača možemo dobiti opcijom `-Xanalyzer -analyzer-list-enabled-checkers`. Proveravače možemo uključiti zadavanjem opcija Clang analizatoru (analizator pozivamo zadavanjem opcije `--analyze`):

```
$ clang \
    -Xanalyzer -analyzer-checker=alpha \
    --analyze divzero.c
```

Alternativno, analizator možemo pokrenuti na jednostavniji način korišćenjem alata koji dolaze uz Clang. Ti alati su posebno korisni u situacijama kada koristimo sisteme za izgradnju projekata.

scan-build je alat koji dozvoljava korisnicima da pokrenu statički analizator nad izvornim kodom projekta kao deo regularnog procesa izgradnje projekta preko komandne linije. Prilikom izgradnje projekta, kako se izvorne datoteke prevode, tako se i analiziraju statičkim analizatorom. Nakon završetka procesa izgradnje projekta, rezultati analize se mogu videti kao HTML stranice.

`scan-build` radi tako što menja odgovarajuće promenljive okruženja (engl. *environment variables*), npr. `CC` i `CXX`, sa ciljem da se pokrene "lažni" prevodilac koji prvo pokreće `gcc` ili `clang`, a zatim pokreće i statičku analizu. Ovaj proces stoga radi nezavisno od korišćenog sistema za izgradnju projekta (make, CMake, itd.), sve dok se koriste odgovarajuće promenljive okruženja.

```
$ scan-build
USAGE: scan-build [options] <build command> [build options]
```

Primeri pokretanja `scan-build` alata:

```
$ scan-build make
$ scan-build make -j4
$ scan-build xcodebuild
```

Moguće je analizirati pojedinačne datoteke:

```
$ scan-build gcc -c 1.c 2.c
$ scan-build -internal-stats -stats \
    -o tmp \
    -enable-checker security,deadcode,core,alpha \
    clang -cc1 \
    -analyze -analyzer-display-progress \
    -Werror \
    divzero.c
```

Česte opcije koje se prosleđuju `scan-build` alatu:

Option	Description
`-o`	Target directory for HTML report files. Subdirectories will be created as needed to represent separate "runs" of the analyzer. If this option is not specified, a directory is created in `/tmp` to store the reports.
`-h` (or no arguments)	Display all scan-build options.
`-k --keep-going`	Add a "keep on going" option to the specified build command. This option currently supports `make` and `xcodebuild`. This is a convenience option; one can specify this behavior directly using build options.
`-v`	Verbose output from scan-build and the analyzer. A second and third `-v` increase verbosity.
`-V`	Open results in a web browser after the build is completed.
`--use-analyzer Xcode`	scan-build uses the 'clang' executable relative to itself for static analysis. One can override this behavior with this
(`--use-analyzer [clang path]`)	option by using the `clang` packaged with Xcode (on OS X) or from the PATH.

Izlaz `scan-build` alata je skup HTML datoteka od kojih svaka predstavlja zaseban izveštaj. Zbirna datoteka `index.html` se kreira za lak pristup svim izveštajima. Opcija `-o` se može iskoristiti za promenu putanje gde će se sačuvati izveštaji. Ako opcija `-o` nije navedena, `scan-build` će izveštaje sačuvati u `/tmp` direktorijumu. Putanja do izveštaja biće ispisana na kraju rada alata. Moguće je automatski otvoriti izveštaje nakon završetka analize zadavanjem opcije `-V`. Alternativno, može se koristiti alat `scan-view` za prikazivanje izveštaja `scan-build` alata.

Kod svakog nalaza treba proveriti tri stvari:

- **putanju**: uslovi i grane kojima analizator dolazi do greške;

- **pretpostavku**: vrednost ili stanje koje analizator smatra mogućim;
- **posledicu**: tačan pristup memoriji, API poziv ili logički uslov koji pravi problem.

Ako je pretpostavka nemoguća u realnom programu, nalaz je kandidat za lažno pozitivan rezultat. Tada treba razmotriti da li kodu nedostaje `assert`, provera ulaza ili jasniji ugovor funkcije. Ako je pretpostavka moguća, nalaz treba tretirati kao grešku čak i kada se teško reprodukuje testom.

Preporuke za korišćenje `scan-build` alata:

- Projekat **uvek** treba analizirati u *Debug* konfiguraciji. Pretpostavke i asertacije će biti od koristi analizatoru da smanji prostor pretrage i da značajno smanji broj lažno-pozitivnih upozorenja.
- Koristiti *verbose* konfiguraciju prilikom debagovanje `scan-build` alata. Opcija `-v` tera `scan-build` da ispisuje dodatne informacije.
- Ako projekat koristi `configure` skriptu generisanu od strane `autoconf` alata, pokrenuti uvek prvo `scan-build` za `configure` skript [5], pa tek onda za regularnu komandu izgradnje projekta.

Ako želimo deo koda da isključimo iz analize, to možemo uraditi ograđivanjem koda na sledeći način:

```
#ifndef __clang_analyzer__
// Kod koji ne treba da se analizira
#endif
```

12.2.1 `clang-tidy`

`clang-tidy` je alat za statičke provere koje se pokreću nad pojedinačnim prevodilačkim jedinicama. Za razliku od `scan-build` toka, `clang-tidy` se često koristi za održavanje kvaliteta koda: modernizaciju C++ konstrukcija, proveru čitljivosti, otkrivanje čestih bug obrazaca i sprovođenje pravila iz skupova kao što su `bugprone`, `performance`, `readability`, `modernize` i `clang-analyzer`.

Za projekte koji koriste CMake najčešći ulaz je `compile_commands.json`. Ta datoteka govori alatu kojim opcijama se svaka izvorna datoteka zaista prevodi:

```
$ cmake -S . -B build -DCMAKE_EXPORT_COMPILE_COMMANDS=ON
$ clang-tidy -p build src/file.cpp
```

Za manji primer bez baze prevodilačkih komandi mogu se navesti opcije posle `--`:

```
$ clang-tidy example.cpp -- -std=c++17 -Iinclude
```

Izbor provera treba držati usko vezan za cilj. Za traženje bug-ova korisne su grupe `bugprone-*` i `clang-analyzer-*`. Za refaktorisanje i modernizaciju C++ koda korisne su `modernize-*` i `readability-*`, ali te provere često daju mnogo nalaza koji nisu greške.

[5] `configure` skripte obično generišu `make` datoteke u kojima su hardkodirane putanje do kompilatora, odnosno nisu navedene preko promenljivih okruženja, što znači da scan-build u tim situacijama ne može da ubaci "lažni" prevodilac u proces izgradnje projekta.

12.2.2 Primeri

U direktorijumu `examples_individual` ovog poglavlja nalaze se pojedinačne izvorne datoteke koje se mogu analizirati. Primeri se pokreću pomoću skripte `analyze`, dostupne u istom direktorijumu:

```
$ cd 11_clang/01_static_analyzer/examples_individual
$ ./analyze 01_struct_access.c core,security,alpha -v
```

Direktorijum `examples_make` sadrži veći primer projekta. Primer koristi `make` sistem za izgradnju.

12.3 Sanitizeri u Clang-u

Sanitizeri su instrumentacije koje kompilator ubacuje u program. Program se zatim pokreće kao obična izvršna datoteka, ali dodatni kod proverava memorijske pristupe, aritmetiku, curenje memorije ili pristup deljenim podacima. Zbog toga su sanitizeri pogodni za lokalni razvoj i CI provere. U odnosu na Valgrind, zahtevaju posebno prevođenje programa, ali se često izvršavaju brže i daju precizan stek poziva za konkretno izvršavanje.

Primeri u ovom direktorijumu koriste Clang:

```
$ make
```

ASan AddressSanitizer (ASan) detektuje neispravne pristupe memoriji kao što su prekoračenje hip bloka, prekoračenje steka, upotreba memorije posle oslobađanja i neispravno oslobađanje memorije. Primer ispod alocira prostor za string bez završnog nul karaktera, a zatim kopira i taj bajt.

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(void) {
    const char *source = "sanitizer";
    size_t length = strlen(source);
    char *copy = malloc(length);
    if (copy == NULL) {
        return 1;
    }

    memcpy(copy, source, length + 1);
    puts(copy);
    free(copy);
    return 0;
}
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

```c
int main(void) {
    const char *source = "sanitizer";
    size_t length = strlen(source);
    char *copy = malloc(length + 1);
    if (copy == NULL) {
        return 1;
    }

    memcpy(copy, source, length + 1);
    puts(copy);
    free(copy);
    return 0;
}
```

```
$ ./asan_bad.out
$ ./asan_fixed.out
```

U izveštaju loše verzije tražiti red `heap-buffer-overflow`. Stek poziva pokazuje gde je upis van granica nastao, a deo izveštaja sa opisom alociranog regiona pokazuje koliko bajtova je zaista rezervisano.

UBSan UndefinedBehaviorSanitizer (UBSan) prijavljuje ponašanje koje C i C++ standard ne definišu. To obuhvata potpisano celobrojno prekoračenje, neispravne pomeraje bitova, dereferenciranje neporavnatih pokazivača i slične greške. U ovom primeru množenje prelazi opseg tipa `int`.

```c
#include <limits.h>
#include <stdio.h>

static int bonus_points(int solved_tasks) { return solved_tasks * 1000000; }

int main(void) {
    int solved_tasks = INT_MAX / 1000000 + 1;
    printf("bonus=%d\n", bonus_points(solved_tasks));
    return 0;
}
```

```c
#include <limits.h>
#include <stdbool.h>
#include <stdio.h>

static bool bonus_points(int solved_tasks, int *result) {
    if (solved_tasks < 0 || solved_tasks > INT_MAX / 1000000) {
        return false;
    }

    *result = solved_tasks * 1000000;
    return true;
}
```

```c
int main(void) {
    int solved_tasks = INT_MAX / 1000000 + 1;
    int result = 0;

    if (!bonus_points(solved_tasks, &result)) {
        puts("broj zadataka je van opsega");
        return 0;
    }

    printf("bonus=%d\n", result);
    return 0;
}
```

```
$ ./ubsan_bad.out
$ ./ubsan_fixed.out
```

Makefile koristi opciju `-fno-sanitize-recover=undefined` da se program zaustavi posle prijave. U CI proverama je to korisno jer greška dobija nenulti izlazni status.

LSan LeakSanitizer (LSan) proverava curenje memorije na kraju procesa. Može da se koristi samostalno opcijom `-fsanitize=leak`, a često se koristi i zajedno sa ASan instrumentacijom. Loša verzija učitava tri linije i gubi pokazivače na alocirane kopije.

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

static void load_line(const char *text) {
    char *line = malloc(strlen(text) + 1);
    if (line == NULL) {
        exit(1);
    }

    strcpy(line, text);
    printf("%c\n", line[0]);
}

int main(void) {
    load_line("verification");
    load_line("sanitizer");
    load_line("leak");
    return 0;
}
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

static void load_line(const char *text) {
    char *line = malloc(strlen(text) + 1);
    if (line == NULL) {
```

```
        exit(1);
    }

    strcpy(line, text);
    printf("%c\n", line[0]);
    free(line);
}

int main(void) {
    load_line("verification");
    load_line("sanitizer");
    load_line("leak");
    return 0;
}
```

```
$ ./lsan_bad.out
$ ./lsan_fixed.out
```

Kod loše verzije očekuje se izveštaj sa tekstom `detected memory leaks`. Popravljena verzija oslobađa svaku kopiju pre izlaska iz funkcije.

TSan ThreadSanitizer (TSan) detektuje trke za podacima u programima sa nitima. Primer ima dve niti koje uvećavaju istu globalnu promenljivu bez sinhronizacije.

```
#include <pthread.h>
#include <stdio.h>

enum { ITERATIONS = 100000 };

static int counter = 0;

static void *worker(void *arg) {
    (void)arg;

    for (int i = 0; i < ITERATIONS; i++) {
        counter++;
    }

    return NULL;
}

int main(void) {
    pthread_t first;
    pthread_t second;

    pthread_create(&first, NULL, worker, NULL);
    pthread_create(&second, NULL, worker, NULL);
    pthread_join(first, NULL);
    pthread_join(second, NULL);

    printf("counter=%d\n", counter);
    return 0;
}
```

```c
#include <pthread.h>
#include <stdio.h>

enum { ITERATIONS = 100000 };

static int counter = 0;
static pthread_mutex_t counter_mutex = PTHREAD_MUTEX_INITIALIZER;

static void *worker(void *arg) {
    (void)arg;

    for (int i = 0; i < ITERATIONS; i++) {
        pthread_mutex_lock(&counter_mutex);
        counter++;
        pthread_mutex_unlock(&counter_mutex);
    }

    return NULL;
}

int main(void) {
    pthread_t first;
    pthread_t second;

    pthread_create(&first, NULL, worker, NULL);
    pthread_create(&second, NULL, worker, NULL);
    pthread_join(first, NULL);
    pthread_join(second, NULL);

    printf("counter=%d\n", counter);
    return 0;
}
```

```
$ ./tsan_bad.out
$ ./tsan_fixed.out
```

U lošoj verziji očekuje se `data race` izveštaj. Popravljena verzija štiti deljeni brojač muteksom, pa svaki pristup promenljivoj `counter` ima istu sinhronizacionu disciplinu.

MemorySanitizer kao napredna vežba MemorySanitizer (MSan) detektuje čitanje neinicijalizovane memorije. U praksi često zahteva da su i biblioteke koje program koristi prevedene sa MSan instrumentacijom. Zbog toga ga ne koristimo kao osnovnu lokalnu vežbu u ovom poglavlju. Kao naprednu vežbu, izolovati mali program bez spoljašnjih zavisnosti i prevesti ga komandom:

```
$ clang -g -O1 -fsanitize=memory -fno-omit-frame-pointer example.c -o example
```

Mesta za proširenje Sledeće teme su dobar nastavak nakon Clang sanitizer vežbi:

1. **Kombinovanje sanitizera i fuzz testiranja**: ASan i UBSan se često uključuju tokom fuzz testiranja kako bi se padovi programa pretvorili u precizne izveštaje.

2. **LLVM pokrivenost izvornog koda**: `llvm-profdata` i `llvm-cov` mogu povezati izvršene testove sa pokrivenošću izvornog koda.
3. **Podešavanje**: opcije kao što su `ASAN_OPTIONS` i `TSAN_OPTIONS` kontrolišu izveštaje, supresije i ponašanje pri grešci.
4. **CI konfiguracija**: sanitizer build treba držati odvojen od produkcionog build-a i pokretati ga nad testovima koji imaju dovoljno pokrivenosti.

13 Jezici za programiranje i formalnu verifikaciju (Dafny)

Dafny je programski jezik sa podrškom za pisanje specifikacija i formalno verifikovanje programa, opremljen statičkim verifikatorom programa. Uklapanjem sofisticiranog automatizovanog rezonovanja sa poznatim programerskim idiomima i alatima, Dafny omogućava programerima da pišu kod koji se može formalno verifikovati uzimajući u obzir navedene specifikacije. Dafny programe je moguće prevesti u C#, Java, JavaScript i Go, i tako integrisati Dafny u postojeće projekte.

13.1 Pisanje Dafny programa

Dafny program (ekstenzije `.dfy`) predstavlja skup deklaracija. Deklaracije uvode *tipove*, *metode* i *funkcije* (redosled deklaracija je nebitan). Tipovi mogu biti *klase* i *induktivni tipovi*. Klase takođe mogu sadržati skup deklaracija *polja*, *metoda* i *funkcija*. Ukoliko program sadrži definiciju metoda `Main` bez parametara, onda će to biti ulazna tačka programa, međutim nije neophodno da program ima `Main` metod da bi bio verifikovan.

13.1.1 Polja

Unutar klase, polje se definiše na sledeći način:

```
var x: T
```

Tip `T` je neophodno navesti (neće biti dedukovan). Polja je dodatno moguće kvalifikovati ključnom rečju `ghost` koja označava da se to polje koristi samo unutar specifikacija.

Tipovi podataka u Dafny jeziku:

- `bool` - istinitosni tip
- `int` - neograničeni ceo broj
- `nat` - podskup tipa `int`, predstavlja nenegativne cele brojeve
- `string` - niske
- `set<T>` - konačni imutabilni set vrednosti tipa `T`
- `seq<T>` - imutabilna sekvenca vrednosti tipa `T`
- `array<T>`, `array2<T>`, `array3<T>` ... `arrayN<T>` - nizovni tipovi dimenzije `N`
- `object` - nadtip svih klasnih tipova

13.1.2 Metode

Deklaracija metoda ima sledeći oblik:

```
method M(a: A, b: B, c: C) returns (x: X, y: Y, z: Y)
  requires Pre
  modifies Frame
  ensures Post
  decreases TerminationMetric
{
  Body
}
```

gde su:

- `a`, `b` i `c` ulazni parametri metoda (metodi unutar klase imaju implicitni ulazni parametar `this`)
- `x`, `y` i `z` izlazni parametri metoda
- `Pre` istinitosni izraz koji predstavlja preduslov metoda
- `Frame` skup objekata čija polja mogu biti ažurirana pozivom metoda; može biti lista izraza gde svaki izraz može biti objekat ili skup objekata (uključujući i `this` ukoliko je metod deklarisan unutar klase) - `Frame` metoda je unija takvih listi i objekata alociranih unutar tela metoda
- `Post` istinitosni izraz koji predstavlja postuslov metoda
- `TerminationMetric` funkcija varijanta metoda
- `Body` implementacija metoda

Podrazumevano, uslovi imaju vrednost `true` dok je `Frame` prazan skup. Ukoliko nije navedena, Dafny će pokušati da dedukuje funkciju varijantu metoda.

Metode se mogu koristiti kao leme navođenjem ključne reči `lemma` umesto `method`.

13.1.3 Funkcije

Primer definicije funkcije:

```
function F(a: A, b: B, c: C): T
  requires Pre
  reads Frame
  ensures Post
  decreases TerminationMetric
{
  Body
}
```

Za razliku od metoda, funkcije mogu imati samo jedan **izraz** (ne naredbu) u telu (`Body`), dok metode mogu imati više izraza ili drugih naredbi u svom telu.

Preduslov omogućava funkciji da bude definisana uslovno - Dafny će verifikovati da preduslov važi prilikom svakog poziva funkcije. Postuslov može biti dobro mesto za navođenje svojstava funkcije koja zahtevaju induktivni dokaz.

Na primer, definicija ispod navodi postuslov da je rezultat rada funkcije uvek pozitivan (funkcija referiše na samu sebe u postuslovu):

```
function Factorial(n: int): int
    requires 0 <= n
    ensures 1 <= Factorial(n)
{
    if n == 0 then 1 else Factorial(n-1) * n
}
```

Podrazumevano, funkcije su duhovi (`ghost`), dakle ne mogu se pozvati iz koda koji se izvršava. Ovo ponašanje se može pregaziti definisanjem funkcije kao `function method`. Specijalno, funkcije koje imaju povratnu vrednost `bool` se mogu deklarisati navođenjem ključne reči `predicate` umesto `function`.

13.1.4 Induktivni tipovi

Induktivni tipovi su tipovi čije se vrednosti kreiraju iz fiksiranog skupa konstruktora. Na primer:

```
datatype Tree = Leaf | Node(Tree, int, Tree)

// moguce je definisati imena parametara konstruktora
datatype Tree = Leaf | Node(left: Tree, data: int, right: Tree)
```

Za svaki konstruktor `Ct` se implicitno deklariše i član `Ct?` koji vraća `true` ako je objekat konstruisan korišćenjem konstruktora `Ct`. Na primer:

```
var t0 := Leaf;
var t1 := Node(t0, 5, t0);
t1.Node?    // true
t0.Node?    // false
```

13.1.5 Naredbe

```
var LocalVariables := ExprList;

Lvalues := ExprList;

assert BoolExpr;

print ExprList;

if BoolExpr0 {
    Stmts0
} else if BoolExpr1 {
    Stmts1
} else {
    Stmts2
}

while BoolExpr
    invariant Inv
    modifies Frame
    decreases Rank
{
    Stmts
}

match Expr {
    case Empty => Stmts0
    case Node(l, d, r) => Stmts1
}

break;

return;
```

13.1.6 Izrazi

Izrazi u Dafny jeziku su slični izrazima u C sintaksi, sa malim izmenama/dodacima:

- `==>` operator implikacije
- `<==>` ako-i-samo-ako operator
- poređenja se mogu ulančavati: `0 <= i < j <= a.Length == N`
- `%` uvek vraća nenegativan broj
- prisutni su kvantifikatori:
  - `forall x :: Expr`
  - `exists x :: Expr`
- operacije nad skupovima:
  - `+` (unija)
  - `*` (presek)
  - `-` (razlika)
  - `<` (pravi podskup)
  - `<=` (podskup)
  - `>` (pravi nadskup)
  - `>=` (nadskup)
  - `!!` (razdvojenost skupova)
  - `in` i `!in` ((ne)pripadnost skupu, npr. `x !in S`)
  - `{}` (konstrukcija skupa, npr: `{x,y}`)
- operacije nad sekvencama:
  - `+` (konkatenacija)
  - `<` (pravi prefiks)
  - `<` (prefiks)
  - `in` i `!in` ((ne)pripadnost sekvenci, npr. `x !in S`)
  - `||` (dužina sekvence, npr. `|S|`)
  - `[]` (`0`-indeksiranje sekvence, npr. `S[i]`)
  - `[..]` (sečenje sekvence, npr. `S[1..10]`, desni kraj se ne uključuje)

13.2 Primer: Faktorijel

Primer `factorial.dfy` prikazuje rekurzivnu funkciju sa preduslovom i postuslovom. Preduslov `requires 0 <= n` ograničava domen funkcije na nenegativne brojeve. Postuslov `ensures 1 <= Factorial(n)` navodi svojstvo koje Dafny mora da dokaže za svaku dozvoljenu vrednost argumenta.

```
function Factorial(n: int): int
    requires 0 <= n
    ensures 1 <= Factorial(n)
{
    if n == 0 then 1 else Factorial(n-1) * n
}
```

U telu funkcije postoje dva slučaja. Za `n == 0` rezultat je `1`. Za veće vrednosti rezultat se računa rekurzivno pozivom `Factorial(n-1) * n`. Preduslov je potreban jer bi se bez njega rekurzija mogla nastaviti ka negativnim vrednostima.

Primer se može proveriti komandom:

```
$ dafny verify factorial.dfy
```

Ako se ukloni preduslov ili se promeni rekurzivni poziv tako da se argument ne smanjuje, verifikator prijavljuje da ne može da dokaže korektnost ili terminaciju funkcije.

13.3 Primer: Maksimum sekvence

Primer `maximum.dfy` prikazuje metod koji pronalazi najveći element neprazne sekvence. Preduslov `requires values != []` isključuje prazan ulaz, jer za praznu sekvencu ne postoji element koji bi mogao da bude maksimum.

```
method Maximum(values: seq<int>) returns (max: int)
    requires values != []
    ensures max in values
    ensures forall i | 0 <= i < |values| :: values[i] <= max
{
    max := values[0];
    for idx := 0 to |values|
        invariant max in values
        invariant forall j | 0 <= j < idx :: values[j] <= max
    {
        if max < values[idx] {
            max := values[idx];
        }
    }
}

lemma MaximumIsUnique(values: seq<int>, m1: int, m2: int)
    requires m1 in values && forall i | 0 <= i < |values| :: values[i] <= m1
    requires m2 in values && forall i | 0 <= i < |values| :: values[i] <= m2
    ensures m1 == m2
{
    // This lemma does not need a body: Dafny is able to automatically prove its correctness.
}
```

Postuslovi opisuju rezultat bez oslanjanja na konkretnu implementaciju:

- `max in values` zahteva da rezultat bude jedan od elemenata ulazne sekvence;
- kvantifikator `forall` zahteva da nijedan element sekvence nije veći od rezultata.

Petlja održava iste dve ideje kroz invarijante. Prva invarijanta kaže da trenutni kandidat za maksimum i dalje dolazi iz ulazne sekvence. Druga kaže da su svi do sada obrađeni elementi manji ili jednaki kandidatu. Na kraju petlje obrađeni su svi elementi, pa invarijante postaju dovoljne za dokaz postuslova.

Lema `MaximumIsUnique` formalizuje da dva broja koja zadovoljavaju uslove maksimuma nad istom sekvencom moraju biti jednaka. Takve leme se koriste kada je pomoćno svojstvo jasno matematički, ali ga je korisno imenovati da bi se koristilo u drugim dokazima.

Primer se proverava komandom:

```
$ dafny verify maximum.dfy
```

14 Instalacije

14.1 Alati za debagovanje i razvojna okruženja

14.1.1 gdb

Za većinu Linux distribucija je dostupan paket `gdb`. `gdb` je za neke distribucije deo paketa za razvoj (npr. `build-essential` za Ubuntu).

Za vežbe sa core dump-ovima proveriti i sistemsku konfiguraciju za čuvanje core datoteka. Na distribucijama sa `systemd-coredump` koristi se komanda `coredumpctl`; na drugim sistemima može biti dovoljno `ulimit -c unlimited`.

Za prikačivanje na postojeći proces mogu biti potrebna podešavanja vezana za `ptrace`, naročito `/proc/sys/kernel/yama/ptrace_scope`.

Popularne modifikacije i front-end prikazi informacija koje gdb pruža:

- gdb-dashboard
- gdb-frontend (repozitorijum)

14.1.2 QtCreator

Instalirati QtCreator sa zvanične stranice. Alternativno, moguće je i instalirati ceo Qt radni okvir koji uključuje i QtCreator.

Za neke Linux distribucije je dostupan paket `qt<VERZIJA>-creator`.

14.1.3 Java agenti

Za pokretanje primera su dovoljni JDK i Maven. Na primer, za Ubuntu OS, dovoljno je preuzeti OpenJDK i Maven:

```
$ apt-get install default-jdk    # JDK 8, 11, 17, 21 su LTS
$ apt-get install openjdk-17-jdk # alternativno, specificna verzija
$ java -version                  # proveriti instalaciju
$ apt-get install maven
$ mvn -version
```

Za neke alate i skripte je neophodno postaviti `JAVA_HOME` promenljivu okruženja na direktorijum JDK instalacije:

```
$ export JAVA_HOME=/path/to/jdk
$ $JAVA_HOME/bin/java -version
```

Obično se `$JAVA_HOME/bin` postavi na `PATH` kako bi se `java*` alati lakše pokretali:

```
$ export PATH=$PATH:$JAVA_HOME/bin
$ which java
/path/to/jdk/bin/java
$ java -version
```

14.2 Alati/Biblioteke za testiranje jedinica koda i pokrivenosti koda

14.2.1 gcov, lcov

`gcov` dolazi podrazumevano uz `gcc` kompajler. Alat `lcov` je obično dostupan u okviru paketa sa istim imenom. Instalacija na Ubuntu distribuciji bi, na primer, izgledala ovako: `sudo apt-get install lcov`

14.2.2 Gradle

Da bi se Gradle instalirao, neophodno je na sistemu imati verziju JDK-a veću od 8. Gradle se potom instalira kroz paket `gradle` za većinu popularnih Linux distribucija. Alternativno, moguće je preuzeti unapred spremne Gradle artefakte i ručno instalirati Gradle. Primeri pretpostavljaju da je izvršiva datoteka (ili alias) `gradle` dostupna na `PATH`-u i pokreće Gradle alat.

Neobavezno za ovaj primer, za laku organizaciju u okruženju sa više različitih JDK verzija, može se koristiti alat SDKMAN. Gradle se može instalirati korišćenjem SDKMAN-a:

```
$ sdk install gradle <verzija>
```

Na primer:

```
$ sdk install gradle 7.5.1
```

14.2.3 xUnit / NUnit

`xUnit` i `NUnit` se jednostavno instaliraju sa NuGet repozitorijuma (xUnit, NUnit) ili uz pomoć IDE-a, ili kroz komande:

```
$ dotnet add package xunit --version 2.4.2
$ dotnet add package NUnit --version 3.13.3
```

Alati koji se isporučuju kao lokalni .NET alati obnavljaju se iz direktorijuma koji sadrži manifest `.config/dotnet-tools.json`. Primer za pokrivenost koda koristi `dotnet-coverage`, pa je za tu vežbu dovoljno pokrenuti:

```
$ dotnet tool restore
```

14.3 Alati/Biblioteke za Mock testiranje

14.3.1 Moq

`Moq` se jednostavno instalira sa NuGet repozitorijuma ili uz pomoć IDE-a, ili kroz komandu:

```
$ dotnet add package Moq --version 4.18.2
```

14.4 Profajliranje instrumentacijom

14.4.1 Valgrind

Valgrind se na većini Linux distribucija može instalirati kroz paket `valgrind`. Npr., za Ubuntu:

```
$ sudo apt-get install valgrind
```

Za grafički prikaz izveštaja nekih Valgrind-ovih alata može se koristiti program *KCachegrind*. Instalacija, npr., za Ubuntu:

```
$ sudo apt-get install kcachegrind
```

14.4.2 gprof

`gprof` je deo GNU binutils paketa na većini Linux distribucija. Za prevođenje primera potreban je i GCC.

```
$ gcc --version
$ gprof --version
```

Na Debian/Ubuntu sistemima tipično je dovoljno instalirati razvojne alate:

```
$ sudo apt-get install build-essential binutils
```

14.4.3 LLVM XRay

Za LLVM XRay potrebni su Clang, XRay runtime i alat `llvm-xray`. Nazivi paketa zavise od distribucije i verzije LLVM-a.

```
$ clang++ --version
$ llvm-xray --version
```

Na Debian/Ubuntu sistemima paketi su obično vezani za verziju LLVM-a, na primer `clang`, `llvm` i `compiler-rt`. Ako linkovanje primera prijavi da XRay runtime nije dostupan, instalirati odgovarajući `compiler-rt` paket za istu LLVM verziju.

14.5 Profajliranje uzorkovanjem

14.5.1 Perf

Perf se na većini Linux distribucija može instalirati kroz paket `perf`. Npr., za Ubuntu:

```
$ sudo apt-get install perf
```

14.5.2 Intel VTune Profiler

Intel VTune Profiler se instalira kroz Intel oneAPI pakete ili kao zaseban VTune Profiler paket, u zavisnosti od operativnog sistema. Posle instalacije proveriti da li je komanda `vtune` dostupna:

```
$ vtune --version
```

Ako komanda nije u putanji, potrebno je učitati okruženje koje instalacija isporučuje. Kod oneAPI instalacije to je najčešće skripta `setvars.sh`:

```
$ source /opt/intel/oneapi/setvars.sh
```

Za hardversko uzorkovanje mogu biti potrebna dodatna podešavanja drajvera, kernela ili korisničkih dozvola. Ta podešavanja zavise od sistema, pa ih treba proveriti u zvaničnom instalacionom uputstvu za korišćenu verziju VTune-a.

14.5.3 async-profiler

Za vežbu je potreban JDK i async-profiler. Na Linux-u se async-profiler najčešće preuzima kao arhiva sa stranice izdanja projekta, a zatim se koristi komanda `asprof` iz raspakovanog direktorijuma.

```
$ java -version
$ javac -version
$ asprof --version
```

Ako `asprof` nije u putanji, koristiti punu putanju do skripte, na primer:

```
$ /opt/async-profiler/bin/asprof --version
```

Za attach na JVM proces mogu biti potrebne iste korisničke dozvole kao za proces koji se profiliše. Za neke događaje i okruženja potrebna su i Linux podešavanja vezana za `perf`.

14.6 eBPF alati

Za osnovne eBPF vežbe korisni su `bpftrace`, BCC alati, simboli za debagovanje i kernel zaglavlja koja odgovaraju pokrenutom kernelu. Nazivi paketa zavise od distribucije.

Primer za Ubuntu/Debian sisteme:

```
$ sudo apt-get install bpftrace bpfcc-tools linux-headers-$(uname -r)
```

Primer za Arch Linux:

```
$ sudo pacman -S bpftrace bcc
```

Provera osnovne instalacije:

```
$ bpftrace --version
$ sudo bpftrace -l 'tracepoint:syscalls:sys_enter_*' | head
```

Ako alati prijavljuju greške sa dozvolama, proveriti capability-je, `perf_event_paranoid`, `kptr_restrict`, dostupnost BTF-a i to da li se vežba pokreće unutar kontejnera.

14.7 Simboličko izvršavanje

14.7.1 KLEE

KLEE se može instalirati prateći uputstva na zvaničnoj stranici. Preporučeno je koristiti Docker kontejner:

```
$ docker pull klee/klee:3.0
$ docker run --rm -ti --ulimit='stack=-1:-1' klee/klee:3.0
```

14.8 Provera modela

14.8.1 CBMC

CBMC se na većini Linux distribucija može instalirati kroz paket `cbmc`. Npr., za Ubuntu:

```
$ sudo apt-get install cbmc
```

14.9 Clang alati

14.9.1 Clang

Clang se na većini Linux distribucija može instalirati kroz paket `clang`. Npr., za Ubuntu:

```
$ sudo apt-get install clang
```

scan-build alat za statičku analizu i drugi Clang alati koje koristimo kroz interfejs komandne linije na većini Linux distribucija dolaze zajedno sa Clang-om. Za starije verzije moguće je da je neophodno instalirati paket `clang-tools`.

Sanitizer primeri u Clang poglavlju koriste Clang runtime biblioteke. Na većini Linux distribucija dolaze uz paket `clang`. Ako povezivanje ne uspe za neku `-fsanitize` opciju, instalirati i paket koji distribucija koristi za LLVM runtime biblioteke. Taj paket se često zove `compiler-rt`; na Debian/Ubuntu sistemima naziv često počinje sa `libclang-rt`.

Napomena za alat scan-build iz clang-tools skupa alata: Korisnici Windows operativnog sistema treba da instaliraju Perl kako bi scan-build alat funkcionisao. Na Windows sistemima scan-build se isporučuje kao Batch skript `scan-build.bat`. Više informacija o specifičnostima za Windows OS u kontekstu korišćenja scan-build alata je moguće naći u scan-build priručniku.

14.10 Jezici za programiranje i formalnu verifikaciju

14.10.1 Dafny

Uputstvo za installaciju Dafny jezika se može naći unutar Dafny repozitorijuma.